DATAWORKS

Users Guide

How to set up and use each of the platform's features

Release 0.42.1

November 9, 2026

About this guide

How to set up and use each of the platform's features, one at a time. Each section says what a feature is for, who may use it, how to set it up (on the platform's pages, in a project's spec, and through the API), how it behaves, the install settings that change it, and its limits.

This guide is the reference. The lab guide is the other way in: one company's quarter, told in 22 labs that build on each other, each with something to check. Where a lab shows a feature at work, the chapter here links to it.

The examples use the lab's company, Harbor Outfitters, its projects `retail` and `marketing`, and the example people: **alice**, an analyst; **bob**, an engineer who also administers the platform; and **carol**, who has no access. An install for real users only (`examples.users: false`) has none of them: read their names as people in the same groups.

Where else to look

- **This guide as one PDF:** DataWorks-Users-Guide.pdf, rebuilt with each release; each release's is also in the GitLab project's package registry (`dataworks-users-guide`), and the latest is the `users-guide` job's artifact on the default branch. Build it yourself with `python docs/users-guide/pdf/build.py . guide.pdf` (after `pip install -r docs/users-guide/pdf/requirements.txt`).
- The lab guide: the platform end to end, in order, with checks.
- The handoff: how the platform is built and installed, and where the code is.
- The API reference: the account menu's **API reference** on the platform, or `/api/openapi.json`.
- The Python package: the API from Python, signed in as a person or a service account.
- The changelog: what each release changed, and what an upgrade needs.

Contents

About this guide . . . 2

Where else to look . . . 2

1. Concepts and first sign-in . . . 11

The platform in one page . . . 11

People and groups . . . 12

Signing in . . . 12

Projects . . . 14

The tools . . . 15

The home page and the top bar . . . 16

A project's tabs . . . 16

What each person sees . . . 17

The example people . . . 18

Settings that change this chapter . . . 19

Limits and gotchas . . . 19

2. Projects . . . 20

Making a project . . . 20

The spec at a glance . . . 23

Members and roles . . . 23

Service accounts: applications as members . . . 24

Tables . . . 26

Files . . . 27

Pipelines . . . 28

Where the project is told of trouble (notify) . . . 29

Limits . . . 30

Changing a project, and its status . . . 31

Deleting a project . . . 31

Project templates . . . 32

Shared libraries, on the same page . . . 37

3. Getting data in . . . 38

Which way in . . . 38

Uploads and the landing area . . . 39
Imports: what they have in common . . . 40
A files import . . . 42
A REST API import . . . 43
A database import (sql_database) . . . 44
A Salesforce import . . . 45
A Gong import . . . 46
A HighSpot import . . . 47
An SAP S/4HANA import . . . 48
Schedules and runs . . . 50
Reset: copy everything again . . . 51
Removing an import . . . 51
Sources: a database queried in place . . . 51
Change-capture copies . . . 53
Adopted tables: another platform's Iceberg tables, where they are . . . 54
Connections . . . 57
What the audit log records . . . 59

4. Access and protection . . . 60

Who sees what . . . 60
Tables readers may read . . . 61
Row filters for readers . . . 62
Column masks for readers . . . 63
Mask types . . . 64
People's data (tables.people) . . . 64
Protection tags and the classifier . . . 65
Confirming and removing a tag . . . 67
The install's protection rules . . . 68
Seeing the rules: the Access tab and others . . . 69
What the audit log records . . . 69

5. Data quality . . . 70

What the platform watches . . . 70
Checks . . . 70
The eight kinds of check . . . 72
Freshness . . . 73
Profiling: what a table holds . . . 74

The Quality view . . . 74
Project alerts . . . 74
Where alerts go . . . 75
Muting . . . 76
Settings . . . 77
What the audit log records . . . 77

6. Metrics and business terms . . . 78

The semantic model . . . 78
Asking for metrics . . . 81
Tests . . . 83
Sharing a model, and building on another project's . . . 84
Business terms . . . 85
Settings . . . 87
Errors you may meet . . . 87

7. Dashboards, apps and assistants . . . 89

Dashboards (Superset) . . . 89
Superset's datasets made from the models . . . 90
Apps . . . 91
Assistants over MCP . . . 93
Registering an assistant . . . 95
The API reference . . . 96
Settings . . . 97

8. Flows . . . 98

What a flow is . . . 98
Building a flow on the canvas . . . 99
The step kinds . . . 100
The prepare step and its processors . . . 102
The model step (Ask a model) . . . 104
Previews and what they cost . . . 106
Saving a flow as a merge request . . . 107
Code steps: changing a flow by hand . . . 108
The flows API . . . 108
Settings . . . 109
Limits and gotchas . . . 109

9. Pipelines and the project's repo . . . 111

Connecting the project's repo . . . 111
The project's Airflow . . . 113
DAGs in the repo . . . 114
Running pipelines and reading run logs . . . 115
dbt in a pipeline . . . 115
Merge-request reviews on data branches . . . 117
Scheduled notebooks . . . 118
The company's shared libraries . . . 119
The pipelines API . . . 120
Settings . . . 121
Limits and gotchas . . . 121

10. Notebooks and machine learning . . . 123

JupyterLab and VS Code . . . 123
buckets_lake: the notebook signed in as you . . . 124
Distributed Spark . . . 126
MLflow, a workspace per project . . . 127
Registering and approving model versions . . . 129
Model cards . . . 130
Scoring on a schedule, and drift . . . 130
Serving a model as an API . . . 132
Models in the catalog's lineage . . . 134
Scheduled notebooks . . . 135
The notebook templates: tables, churn, propensity, forecast . . . 136
Settings . . . 138
Errors you may meet . . . 139

11. Data products . . . 140

Publishing a data product . . . 140
The contract, and broken promises . . . 143
Asking for a product . . . 143
Approving, declining and removing consumers . . . 144
Reading a product in place . . . 145
Certifying . . . 146
Changing and removing a product . . . 147

Where products show . . . 147
Settings . . . 148
Errors you may meet . . . 148

12. Deliveries to partners . . . 149

What a delivery is for . . . 149
Who can do what . . . 149
Targets: where files go . . . 150
PGP: encrypting for the recipient . . . 151
Declaring a delivery . . . 151
Formats and file names . . . 154
Checks before sending . . . 154
Approvals . . . 154
Runs and manifests . . . 156
The delivery log . . . 156
Settings . . . 157
Limits and gotchas . . . 157

13. Privacy . . . 159

The legal basis and the deadline . . . 159
Who can do what . . . 159
Where a person's data is . . . 160
What was sent about a person . . . 161
Erasure . . . 162
The erasure record . . . 163
What history keeps . . . 164
Retention . . . 164
Settings . . . 165
Limits and gotchas . . . 166

14. AI . . . 167

The LLM gateway . . . 167
Budgets and keys . . . 169
The guardrail and forbidden terms . . . 170
Model calls . . . 171
Asking a model on the project's key . . . 172
The project's gateway as OpenAI's API . . . 172

Agents . . . 173
Agents with a notebook . . . 176
Agents with a prompt . . . 177
The project's own tools, and agents asking agents . . . 178
Knowledge banks . . . 179
Settings . . . 181
Errors you may meet . . . 182

15. History and releases . . . 183

A table's history . . . 183
Reading the tables as they were (As of) . . . 184
Tags: moments kept, such as a close . . . 185
Nightly maintenance and what history is kept . . . 186
The project as code . . . 187
Planning and applying a release . . . 188
Rolling back . . . 190
The API . . . 190
Settings . . . 191
Limits and gotchas . . . 191

16. Roles . . . 192

Two kinds of role . . . 192
The built-in project roles . . . 192
The project permissions . . . 193
Defining a project role . . . 194
Giving a project role . . . 195
How a project role is applied . . . 196
Reading tables: readers' rules, read-all and editors . . . 196
Platform roles . . . 197
Changing and removing a role . . . 199
Settings . . . 199
Limits and gotchas . . . 200

17. Operating the platform . . . 201

Installing . . . 201
Checking an install: storscale test . . . 204
Health and the administrators' alerts . . . 204

The audit log . . . 206
Governance . . . 207
Usage and cost . . . 207
Limits . . . 208
Table maintenance . . . 209
External stores . . . 210
The platform's files on a company's store . . . 211
Backups and restore . . . 213
Upgrades . . . 214
Buckets capacity . . . 214
The install's settings . . . 215
The Python package and the API . . . 218

18. Troubleshooting . . . 220

Request IDs . . . 220
The run log . . . 221
The project's status: Applied to . . . 221
Signing in and access . . . 223
Projects and their spec . . . 223
Connections . . . 224
Uploads and imports . . . 224
Adopted tables . . . 225
Pipelines . . . 225
SQL, metrics and limits . . . 225
Access and protection . . . 226
The table catalog (Nessie) . . . 226
History, tags and maintenance . . . 227
Releases . . . 227
Flows . . . 228
Data products . . . 228
Deliveries . . . 228
Privacy: search and erasure . . . 229
AI: the gateway, agents, knowledge banks, apps . . . 230
Spark in notebooks . . . 231
Models . . . 232
Roles and templates . . . 232
Audit, usage, governance and health . . . 232

Installs and upgrades 233

1. Concepts and first sign-in

This chapter is for everyone who uses the platform: analysts, engineers and the administrators who run it. It says what the platform is made of, how people and Keycloak groups decide who gets in and what they may do, how signing in works, what a project is, which tools sit behind the platform's pages, and what each of the example people (alice, bob and carol) sees on their first sign-in. The other chapters build on the words defined here.

In this chapter:

- The platform in one page
- People and groups
- Signing in
- Projects
- The tools
- The home page and the top bar
- A project's tabs
- What each person sees
- The example people
- Settings that change this chapter
- Limits and gotchas

The platform in one page

The platform is a lakehouse with a web app in front of it. Data lives as Iceberg tables in Buckets (S3-compatible storage), catalogued by Nessie and queried through Trino. Apache Ranger decides, query by query, who may read what in Trino. Nessie answers the platform's own services alone: people reach the tables through Trino, or through a notebook's Spark for the tables they read whole (Access and protection). Keycloak signs everyone in, once, for every tool. Around that core are the tools people work in: JupyterHub (notebooks), Superset (SQL Lab and dashboards), Airflow (pipelines), OpenMetadata (the catalog), Grafana (monitoring), MLflow (models) and an LLM gateway (models through a budget).

Two programs of the platform's own hold it together:

Program	What it does
platformd	The web app and its API (`/api/...`). It signs people in with Keycloak, shows each person their projects and tools, and checks every change they ask for.
The platform operator	Applies each project to the systems it lives in: Keycloak groups, Ranger policies, Buckets buckets and policies, Nessie namespaces, Airflow. It looks every few seconds for a changed project, and goes over every project again each minute, so a change made outside the platform (someone added to a Keycloak group) is picked up within a minute.

A **project** is what the operator applies (see Projects below). A person never configures Ranger, Buckets or Keycloak groups by hand for a project: the project says what it wants, and the operator keeps the systems that way.

People and groups

Everyone is a user in Keycloak, in the realm `lakehouse`. What a person may do comes from the Keycloak groups they're in:

Groups	Setting (platformd)	Default	What membership gives
The platform's groups	`PLATFORM_GROUPS`	`analysts,engineers`	May use the platform at all. Someone in none of these is shown "No access".
The administrators' groups	`PLATFORM_ADMIN_GROUPS`	`engineers`	Administration, every project, every permission in every project, and the platform roles' permissions.
A project's member groups	the project's `members`	none	Membership in that project, with a role (see Projects).
A platform role's groups	Administration → Roles	see below	A platform-wide permission: every project's usage, the Governance page, certifying data products, protecting people's data, adopting another platform's tables. See Roles.

The install's own platform roles take their groups from four settings, the first three defaulting to the administrators' groups:

Platform role	Gives	Setting	Chart value
`finance`	`see-all-usage`	`USAGE_VIEWER_GROUPS`	`usage.viewerGroups` (a list; `[]` by default)
`governance`	`certify-products`, `see-governance`	`PRODUCT_CERTIFIER_GROUPS`	`products.certifierGroups` (comma-separated; `""` by default)
`data-protection`	`protect-personal`, `see-governance`	`DELIVERY_APPROVER_GROUPS`	`deliveries.approverGroups` (comma-separated; `""` by default)
`table-adoption`	`adopt-tables`	`ADOPTION_GROUPS`	`adoption.groups` (a list; `[data-migration]` by default, a group setup makes and leaves empty)

Groups beat names: a project that names the group `analysts` as its readers has every analyst as a reader, the day they join the group, and loses them the day they leave. Name a person (`{"user": "alice", ...}`) only when they need something their group doesn't give.

Signing in

What it's for

One sign-in for the platform and every tool behind it, so no one shares passwords and no tool keeps accounts of its own.

Who can do it

Anyone with a Keycloak account. Only people in one of the platform's groups (`PLATFORM_GROUPS`) get past the first page.

How it works

1. Open the platform's address (on a local install, `http://storscale.localhost:8800`). The web app asks platformd who's signed in (`GET /api/session`); with no session, the browser goes to `/auth/login` and on to Keycloak's sign-in page (Keycloak is at `/sso` on the platform's address).
2. Sign in with Keycloak's form. Keycloak sends the browser back to `/auth/callback`; platformd checks the token (OpenID Connect, authorization code with PKCE) and starts a session: a cookie, `storscale_session`, good for 10 hours.
3. Open any tool from the **Tools** menu: each one signs in with Keycloak on the way, without a form, because the person already has a Keycloak session.
4. **Sign out** (the account menu, top right) ends the platform's session and Keycloak's too, so the tools are signed out with it.

With the company's account (Entra ID)

An install can let people sign in with their Microsoft Entra ID account (`sso.entra`, Operating the platform). Keycloak's page then has a button for it (*Microsoft Entra ID*, or the install's name for it) beside the form, and the rest is as above:

- **Their username** is their Entra login name's local part: `jane.doe@example.com` signs in as `jane.doe`. It's the name projects, the audit log and the tools know them by.
- **Their name and email** are Entra's `given_name`, `family_name` and `email` claims, updated at every sign-in. The app registration must add them to its ID token (*Token configuration*, optional claims); without them Keycloak splits the display name ("Jane Doe (Acme)" becomes first name "Jane Doe", last name "(Acme)") and asks the person to correct their profile.
- **Their groups** come from the app roles Entra gives them on the platform's app registration, mapped to the platform's groups: by default `Analyst` puts them in `analysts`, and `Engineer` and `Administrator` in `engineers` (the administrators). They're set again at every sign-in, so a role an Entra administrator takes away takes its groups away at the next one. Someone with no role signs in and gets "No access".
- **Projects** still say who's a member, by username or by group, as for anyone else.

Sign-ins and sign-outs are in the audit log (`signed.in`, `signed.out`). A sign-in by someone in none of the platform's groups is recorded as refused, with "in none of the platform's groups". See Operating the platform for the audit log.

From a script

The API takes a Keycloak access token of the realm's in place of the cookie: `Authorization: Bearer <token>`. platformd reads the person and their groups from the token, so a script can do what its person may do on the pages and nothing more. A change (anything but a `GET`) also needs the header `X-Platform-Request: 1`, which a cross-site form can't send. The account menu's **API reference** (and `GET /api/openapi.json`) lists every route. Operating the platform covers the Python package, which handles tokens for you.

A token for a script, as yourself, without a password (an Entra ID sign-in has none to give it): the realm's public client `storscale-agent` takes a device code (OAuth's device authorization grant). The script asks for one, you open its link in your own browser, sign in and grant it access (Keycloak's page asks), and the script gets your token: the Python package does all of it, `storscale.Platform(url)`.

```
curl -s https://<platform>/sso/realms/lakehouse/protocol/openid-connect/auth/device \
  -d client_id=storscale-agent -d scope=openid -d code_challenge=$CHALLENGE -d
    code_challenge_method=S256
# open its verification_uri_complete; then, every interval seconds until it's approved:
curl -s https://<platform>/sso/realms/lakehouse/protocol/openid-connect/token -d
  client_id=storscale-agent \
  -d grant_type=urn:ietf:params:oauth:grant-type:device_code -d device_code=$DEVICE_CODE -d
    code_verifier=$VERIFIER
```

Limits and gotchas

- **"No access".** Signed in, but in none of the platform's groups: the page says "Your account isn't in a group that may use DataWorks (analysts, engineers)." with the install's groups, and offers **Sign out**. An administrator adds the person to a group in Keycloak; then they sign out and in again.
- **Group changes need a new sign-in for the platform's own checks.** Whether someone is an administrator, may use the platform, or has a platform role is read from the groups in their token as they signed in. Project membership is different: the operator reads Keycloak's groups each minute, so a person added to a project's group is a member within a minute, without signing in again.
- **Sessions live in platformd's memory.** When platformd restarts, everyone signs in again; Keycloak's own session makes that silent (no form).
- **An old sign-in link** (more than 10 minutes, or used once already) starts the sign-in again rather than failing.
- **"Keycloak refused the sign-in: ..."** is Keycloak's own refusal, passed on with its error code.

See Lab 1 in the lab guide.

Projects

A project is a team's tables, files and pipelines, and who may do what with them. It's the unit of ownership: everything else in this guide (imports, checks, metrics, flows, agents, deliveries, models, apps) belongs to a project and follows its rules.

A project is written as a spec, JSON, which an administrator makes on Administration → Projects (see Projects):

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "description": "Harbor Outfitters: orders, customers, campaigns",
    "members": [
      {"group": "engineers", "role": "editor"},
      {"group": "analysts", "role": "reader"}
    ],
    "tables": {},
    "files": {},
    "pipelines": {"serviceAccount": "airflow-pipelines"}
  }
}
```

What each part becomes:

Part	In the systems	Named
The project's members, by role	Keycloak groups, one per role, and the same groups in Ranger and Buckets	`retail-editors`, `retail-readers`, `retail-role-<role>` for a role of the company's own

Part	In the systems	Named
`tables`	A namespace in Nessie's `iceberg` catalog, queried through Trino, with Ranger's policies on it	`iceberg.retail` (dashes in the name become underscores)
`files`	A bucket in Buckets, with a policy per role	`retail`
`pipelines`	A Keycloak service account that runs the project's pipelines: it reads and writes the tables, and the files under `landing/`	`retail-pipelines` (its group)

Each project has **members**, each a Keycloak group or a person, with a **role**:

Role	In short
`editor`	Makes and changes the project's tables, files, pipelines, metrics and models; everything the project has.
`reader`	Reads what the project shows readers (the tables it lists, through its row filters and masks), its files, and calls its models.
A role of the company's own	What an administrator defined on Administration → Roles, from the platform's permissions. See Roles.

Two more kinds of member are the platform's, never given by hand: `pipelines` (the project's service account) and its agents' and served models' accounts, which read as its readers do. An application (a CRM, a system that reads the tables) is a member through a **service account** of the project's, in a role, registered by its editors and approved by an administrator; it may also act for the people signed in to it, under their own rules. See Projects.

The operator records, for each project, what it last applied: a **phase** (`Ready`, `Applying`, `Error`, `Deleting`; the web app shows "Waiting for the operator" before the first pass), a line per system it applied (`keycloak`, `ranger`, `buckets`, `nessie`, and others as the project uses them), and who's in it through which role. A project's Overview shows these under **Applied to** and **People**.

The tools

The **Tools** menu (top bar) lists the tools behind the platform; Home lists the same under **Also on the platform**. Each tool checks for itself what a person may do: the menu is what to show, not what's allowed.

Tool	What it is	Where it opens
Notebooks	JupyterLab, in a notebook server of your own, with your own Buckets credentials (JupyterHub); on Kubernetes, Distributed Spark beside it, and the notebook templates in `work/templates/`	In the platform's frame, `/notebooks/`
SQL	Superset's SQL Lab: query the lakehouse in Trino, as yourself	In the platform's frame, `/dashboards/sqllab/`
Dashboards	Superset's dashboards and charts	In the platform's frame, `/dashboards/`
Pipelines	Airflow: pipelines, their runs and their logs	In the platform's frame, `/pipelines/`
Catalog	OpenMetadata: every table and column, their lineage and their checks	Its own tab, at `catalog.` under the platform's address
Monitoring	Grafana: the storage's dashboards and alerts	In the platform's frame, `/monitoring/`
Access policies	Apache Ranger: who may query what. Administrators only, signed in with Ranger's `admin` account	Its own tab, at `access.` under the platform's address

Trino itself takes JDBC and CLI clients at its own port (on a local install, `https://localhost:8443`), signed in with a Keycloak token. Every query, from any tool, runs in Trino as the person who asked, so Ranger's rules hold the same in SQL Lab, a dashboard, a notebook and an assistant.

Most of the time, a project's own tabs do the work without opening a tool: the SQL tab, the Data tab, the Pipelines tab. The tools are for what the platform's pages don't cover.

The home page and the top bar

Home

Home greets the person by first name ("Good morning, Alice") and shows:

- **Your projects**: a card for each project they're a member of (every project, for an administrator), with their role, the project's description, its phase when it isn't Ready, how many tables and metrics it has, failing checks or "checks passing", and when its pipeline last ran. "You're not in any project yet. Ask an administrator to add you." when there's none.
- **Continue in <project>**: back to the project they were last in.
- **Apps**: the apps of their projects, when there are any (see Dashboards, apps and assistants).
- **Recent pipeline runs**: the six latest runs across their projects, who started them (or "schedule").
- **Needs attention**: every failing check and every pipeline whose last run failed, each linking to it; "Nothing: every check is passing and every last run succeeded." otherwise.
- **Also on the platform**: the tools.
- A notice from the Health page when part of the platform is down.

Home refreshes itself every 30 seconds.

The top bar

Part	What it does
The brand (top left)	Home.
Project	Switch project: "Your projects" with your role in each, then Home, and for an administrator **Manage projects**.
Search (press `/`)	In a project: its business terms, data products, tables, columns, metrics and pipelines. Outside one: business terms, data products and projects. Terms come first, then certified products.
Business terms	The company's terms (when the semantic layer is installed). See Metrics and business terms.
Data products	Every project's data products. See Data products.
Tools	The tools above.
Your name	The account menu: who you're signed in as and your groups; **Administration** (administrators); **Usage and cost** and **Governance** (for those with the platform roles that give them, who aren't administrators); **API reference**; **Sign out**.

A project's tabs

Open a project (`/p/<project>`) and its header shows its name, your role in it, its phase when it isn't Ready, and its description; below, the tabs. A tab shows only when the project has what it needs, and some only for some people:

Tab	Shown when	What it's for	Chapter
Overview	always	Tiles, how data flows, what each role can do, its people, its alerts, where it's applied; Edit and Delete for administrators	Projects

Tab	Shown when	What it's for	Chapter
Data	it has `tables` (and the catalog is installed)	Each table: its columns, preview, quality, history	Data quality
Metrics	`tables` (and the semantic layer)	The semantic model and its metrics	Metrics
SQL	`tables` (and the semantic layer)	Queries in Trino, as yourself	Metrics
Notebooks	the Notebooks tool is there	JupyterHub, in the frame	Notebooks and models
Pipelines	it has `pipelines` (and Airflow)	Its pipelines, runs and logs	Pipelines and git
Flows	`tables` (and the semantic layer)	The canvas	Flows
Agents	`tables` (and the semantic layer)	Its agents	AI
Apps	`tables` (and the semantic layer)	Its apps	Dashboards, apps and assistants
Models	always	MLflow's models, approvals, scoring, serving	Notebooks and models
Dashboards	the Dashboards tool is there	Superset, in the frame	Dashboards, apps and assistants
Tags	`tables`	Moments of the tables kept	History and releases
Deliveries	it has at least one delivery	Files to partners	Deliveries
Releases	its pipelines have a repo	Plans and releases of the project as code	History and releases
Imports	`tables`	Copying data in on a schedule	Getting data in
Connections	always	Its credentials	Getting data in
Access	always	Who's in it, through which group, what each role gives, what readers don't see	Access and protection
Audit	you have `see-audit` in it (editors, administrators)	Its audit log	Operating the platform
Usage	you have `see-usage` in it, or `see-all-usage` across the platform	Its usage and cost	Operating the platform

The project's Edit page (`/p/<project>/edit`) is for administrators, opened from **Edit project** on the Overview or **Change members** on the Access tab; it isn't a tab.

What each person sees

What a person sees follows from two things: their groups (which give the platform and its Administration) and their role in each project.

	Someone in none of the platform's groups	A project's reader	A project's editor	An administrator
Gets in	No: "No access"	Yes	Yes	Yes

	Someone in none of the platform's groups	A project's reader	A project's editor	An administrator
Projects on Home and in the switcher	none	those they're a member of	those they're a member of	every project
A project's Overview "What you can do here"		"You're a reader."	"You're an editor."	"You administer the platform; you're not a member of this project." (unless also a member)
Tables		those the project shows readers, through its row filters and masks	every table as it is	what the groups they're in are given in Trino (see below)
Audit and Usage tabs		no	yes	yes
Run pipelines, change metrics, build flows, apps and agents, give connections		no	yes	yes
Edit or delete the project, make projects		no	no	yes
Administration (account menu)		no	no	yes
Access policies (Ranger) in Tools		no	no	yes

A role of the company's own sits in between: the Overview says "Your role here is <role>.", and what the role's permissions give decides each tab and button. See Roles.

An administrator's power in a project is the platform's: on the project's pages and API they may do everything. Ranger gives administrators nothing of their own: what they read in Trino is what the groups they're in are given, so an administrator who isn't a member of a project reads none of its tables (the install's `engineers` are editors of the example projects, which is why bob reads them).

The example people

A local install comes with three people in Keycloak, and their password is `LAKEHOUSE_USER_PASSWORD` in `stack/.env`. On Kubernetes they come too, unless the install is for real users only (`examples.users: false`, set before the first install); their password is generated on install and kept in the Secret `storscale-env`:

```
kubectl -n storscale get secret storscale-env -o jsonpath='{.data.LAKEHOUSE_USER_PASSWORD}' | base64
  -d
```

Person	Keycloak group	Gets in	Administrator	In the sample project `sales`
alice (Alice Analyst), an analyst	`analysts`	yes	no	reader (through `analysts`): the EU's orders, card numbers showing their last four digits
bob (Bob Engineer), an engineer	`engineers`	yes	yes	editor (through `engineers`): every order, whole
carol (Carol Contractor)	none	no: "No access"	no	none

The install also comes with a sample project, `sales` (orders and payroll, for the integration guides), whose editors are `engineers` and readers `analysts`. The lab guide makes `retail` and `marketing` for Harbor Outfitters.

With the defaults, bob is in every one of the install's platform roles (their groups default to the administrators'). On Compose, the data-protection approvers are `engineers,analysts` (`DELIVERY_APPROVER_GROUPS` in `stack/compose.yaml`), so the example has an approver besides bob: alice has `protect-personal` and `see-governance`, and sees **Governance** in her account menu. On Kubernetes, `deliveries.approverGroups` is empty, so only the administrators are.

See Lab 1 in the lab guide, which signs in as each of them.

Settings that change this chapter

What	platformd's environment (Compose: `stack/compose.yaml`)	Chart (`values.yaml`)	Default
Who may use the platform	`PLATFORM_GROUPS`	not a value: platformd's default	`analysts,engineers`
Who administers it	`PLATFORM_ADMIN_GROUPS`	not a value: platformd's default	`engineers`
Every project's usage (finance)	`USAGE_VIEWER_GROUPS`	`usage.viewerGroups`	the administrators' groups
Certifying products, Governance	`PRODUCT_CERTIFIER_GROUPS`	`products.certifierGroups`	the administrators' groups
Data-protection approvers	`DELIVERY_APPROVER_GROUPS` (Compose sets `engineers,analysts`)	`deliveries.approverGroups`	the administrators' groups
Adopting, taking over and giving back tables	`ADOPTION_GROUPS`	`adoption.groups`	`data-migration`
The platform's address	`STORSCALE_DOMAIN`, `STORSCALE_PORT` in `stack/.env`	`global.domain`, `global.port`, `global.scheme`	`storscale.localhost`, `8800`, `http`
The brand in the top bar and the tab	`BRAND_NAME`, `BRAND_TITLE`, `BRAND_LOGO`, `BRAND_ICON`	`brand.name`, `brand.title`, `brand.logo`, `brand.icon`	none: the tab reads *DataWorks*
The example people, and the sales project	the realm (`stack/keycloak/realm.json`)	`examples.users` (set before the first install)	`true`
Sign-in with Entra ID	setup's `ENTRA_AUTHORITY`, `ENTRA_CLIENT_ID`, `ENTRA_CLIENT_SECRET`	`sso.entra` (Operating the platform)	off

Limits and gotchas

- `examples.users: false` **changes only a new install's realm.** On an install made with the example people, it stops making the sales project, but alice, bob and carol stay in Keycloak until an administrator removes them there.
- **Signed in, but no projects.** Home says "You're not in any project yet. Ask an administrator to add you." An administrator adds the person (or a group they're in) to a project's members.
- **A tab is missing.** Tabs show only when the project has what they need (Pipelines needs `pipelines`; Deliveries needs a delivery; Releases needs a repo) and when you have the permission (Audit, Usage). The table in A project's tabs says which.
- **The Trino address on Home is fixed text** (`https://localhost:8443`), right for a local install; on another install, Trino's port is wherever the chart's `trino.service` puts it.

2. Projects

A project is a team's tables, files and pipelines, and who may do what with them. This chapter is for the platform's administrators, who make and change projects, and for the engineers who need to know exactly what a project's spec says and what the platform does with each part of it. It covers making a project on Administration → Projects (the form and its JSON box) and through the API, its members and their roles, its tables, files and pipelines, where it's told of trouble, its limits, changing and deleting it, and making a project from one of the company's templates.

In this chapter:

- Making a project
- The spec at a glance
- Members and roles
- Service accounts: applications as members
- Tables
- Files
- Pipelines
- Where the project is told of trouble (notify)
- Limits
- Changing a project, and its status
- Deleting a project
- Project templates
- Shared libraries, on the same page

Making a project

What it's for

A project gives a team a home for its data: a namespace of tables, a bucket of files, a service account for its pipelines, and its members in their roles, all written down once. The platform applies it to Keycloak, Ranger, Buckets, Nessie and Airflow and keeps them that way, so no one sets up access tool by tool.

Who can do it

The platform's administrators (the `PLATFORM_ADMIN_GROUPS`, `engineers` by default). Anyone else who tries is refused with "only administrators change projects". A project's editors change parts of it from its own tabs (its imports, connections, agents, apps and more, each in its chapter), never the whole spec.

Set up: on the page

1. Open **Administration** (the account menu, top right) → **Projects** → **New project**. (**From a template** beside it opens Project templates.)
2. **Name**: lowercase letters, digits and dashes, 3 to 40 characters, starting with a letter: `retail`. The project's groups, bucket and namespace are named after it, and it can't be changed later.
3. **Description**: a line saying what it's for: "Harbor Outfitters: orders, customers, campaigns".

4. **Members**: a new project starts with the group `engineers` as editors. For each member pick **Keycloak group** or **Person**, type the group's or the person's username, and pick a role (editor, reader, or a role defined on Administration → Roles). **Add a member** adds a row; **Remove** takes one out. See Members and roles.
5. **Tables** (ticked by default): the project's namespace, and what its readers may read. See Tables.
6. **Files** (ticked by default): a bucket of its own. See Files.
7. **Pipelines**: tick it and give the Keycloak client whose service account runs them, `airflow-pipelines` on the example install. See Pipelines.
8. **Alerts**: a Teams or Slack channel (the name of a connection of the project's), email addresses, and the tables that must change in time. See Where the project is told of trouble.
9. The form also has **Imports**, **Sources** and **Table maintenance**: see Getting data in and History and releases.
10. Read **What each role will get**, below the form: a card per role (Editors, Readers, Pipelines) saying in words what each group gets, as the platform will apply it.
11. Press **Save**. The browser goes to the new project's Overview, which says **Waiting for the operator**, then **Applying**, then **Ready**, within a minute.

Form and **JSON** (top right) switch between the form and the whole project as JSON. The JSON box takes every field of the spec, including those the form doesn't show (limits, checks, deliveries, products and the rest); what you type there is what's saved. **Form** is greyed out while the JSON doesn't parse.

Set up: the spec

The same project as JSON, as the JSON box shows it and the API takes it:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "description": "Harbor Outfitters: orders, customers, campaigns",
    "members": [
      {"group": "engineers", "role": "editor"},
      {"group": "analysts", "role": "reader"}
    ],
    "tables": {
      "readers": {
        "rowFilters": [{"table": "uploads_orders", "filter": "region = 'EU'"}]
      },
      "people": [
        {"table": "uploads_customers", "column": "customer_id"},
        {"table": "uploads_orders", "column": "customer_id"}
      ],
      "freshness": [{"table": "uploads_orders", "within": "24h"}]
    },
    "files": {},
    "pipelines": {"serviceAccount": "airflow-pipelines"},
    "notify": {"teams": "retail-alerts", "email": ["retail-data@example.com"]},
    "limits": {"queries": 4, "queued": 40, "queryMemory": "4GB", "storage": "500Gi"}
  }
}
```

`apiVersion` and `kind` may be left out (they're filled in); `metadata.name` and `spec.members` may not. An empty object (`"tables": {}`, `"files": {}`) asks for the part with its defaults; leaving the key out means the project doesn't have it.

Set up: the API

Method and path	What it does	Who
`GET /api/projects`	The projects the caller may see (an administrator's, every one), each with its spec, status, the caller's role and permissions	anyone signed in
`GET /api/projects/{name}`	One of them	anyone signed in, for a project they may see
`PUT /api/projects/{name}`	Make or change a project: the body is the whole project, as above	administrators
`DELETE /api/projects/{name}`	Delete it (see Deleting a project)	administrators
`GET /api/access`	Who has which role in which project	administrators

A change needs the header `X-Platform-Request: 1`:

```
curl -X PUT "$PLATFORM/api/projects/retail" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" \
  -H "Content-Type: application/json" --data @retail.json
```

How it behaves

- platformd reads the spec strictly (an unknown field is refused), fills in its defaults, checks every part at once, and answers with every problem it found, joined by "; ", so a spec is fixed in one go. On Kubernetes, projects are `Project` resources and Kubernetes checks them against its schema too; its refusal is passed on.
- A member given a role nobody defined is refused: "role steward: reader, editor, or a role defined on Administration → Roles (none is called that)".
- Saved, the project is in the project store, and the operator applies it at once (platformd asks it to; it also looks every few seconds). Each system it applies is a line under **Applied to** on the Overview: `keycloak` (the groups and who's in them), `ranger` (the groups, and the policies on the namespace, named `project:<name>:...`), `buckets` (the bucket, its size limit and its policies), `nessie` (the namespace), and others as the project uses them (`airflow`, `imports`, `trino`, `copy`, `llm`, `serving`, `deliveries`, `agents`, `reviews`, `roles`).
- The audit log records `project.saved` with who did it and what changed: "created", "changed: tables.readers, limits", or "saved, unchanged".

Limits and gotchas

- **The name is fixed.** A project can't be renamed: make a new one. A `PUT` whose body names another project is refused: "the project's name must be retail".
- **The JSON is checked strictly.** A misspelt field is refused as `not a project: json: unknown field "membres"`.
- **"apiVersion and kind must be platform.storscale.io/v1alpha1 and Project"**: a body with other values there.
- **"a project needs at least one member"**.
- The form doesn't show limits, checks, deliveries, products, agents, apps or the rest: use the JSON box, or the tab each belongs to.

See Lab 2 in the lab guide.

The spec at a glance

Every field of `spec`, and where it's covered:

Field	What it is	Chapter
`description`	A line saying what the project is for	here
`members`	Who's in it, with which role	here, Roles
`tables`	Its namespace of Iceberg tables	here
`tables.readers`	What readers may read: tables, row filters, masks	Access and protection
`tables.people`	Tables of people's data, and the column identifying a person	Access and protection, Privacy
`tables.checks`, `tables.freshness`	Data-quality checks; how recently tables must change	Data quality
`tables.maintenance`	The nightly upkeep (`paused`)	History and releases
`tables.tags`	Moments of the tables kept	History and releases
`tables.retention`	How long rows are kept	Privacy
`files`	Its bucket	here
`pipelines`	Its service account, and its repo	here, Pipelines and git
`notify`	Where it's told of trouble	here, Data quality
`limits`	Its share of Trino and its bucket's size	here, Operating
`sources`, `imports`	Databases queried in place; data copied in	Getting data in
`deliveries`	Files to partners	Deliveries
`semantic`	Its model shared, or built on others'	Metrics and business terms
`products`	Its data products	Data products
`llm`, `agents`, `knowledge`, `tools`	The gateway budget, agents, knowledge banks, its tools	AI
`apps`	Its apps	Dashboards, apps and assistants
`scoring`, `serving`	Models scoring tables, and answering as an API	Notebooks and models
`cost`	Its monthly budget	Operating the platform

Members and roles

What it's for

Members say who may do what in the project. Name groups wherever you can: someone who joins the group in Keycloak joins the project, and someone who leaves it leaves.

Who can do it

Administrators, on the Edit page or through `PUT /api/projects/{name}`.

Set up

Each member is a Keycloak group or one person, never both, with a role:

```
"members": [
  {"group": "engineers", "role": "editor"},
  {"group": "analysts", "role": "reader"},
  {"user": "alice", "role": "editor"}
]
```

Field	Value
`group`	A Keycloak group's name (letters, digits, `.`, `_`, `@`, `-`; up to 100 characters)
`user`	A Keycloak username
`role`	`editor`, `reader`, or a project role defined on Administration → Roles (see Roles)

On the Edit page: **Members**, a row each (**Keycloak group** or **Person**, the name, the role), **Add a member**, **Remove**. The Access tab's **Change members** (administrators) opens the same page.

How it behaves

- The operator finds each member in Keycloak (every person in a group) and puts each person in one Keycloak group of the project's, by role: `<project>-editors`, `<project>-readers`, or `<project>-role-<role>`. The same groups are in Ranger (its policies name them) and in Buckets (a policy per group).
- **One role each: the one that gives the most.** Someone who's a member twice (alice as a reader through `analysts`, and as an editor by name) has the editor's role. An editor's role is above every other; a reader's below every other; between them, the role that gives more permissions wins. So the readers' row filters and masks never hold someone another role lifts them from.
- The project's status lists each person with their role, the group they came through ("via analysts") or "named", and what their role lets them do. The Overview's **People** card and the **Access** tab show it; Administration → **Access** shows every person across every project.
- Keycloak's groups are read again each minute: someone added to `analysts` is a reader of every project that names `analysts`, within a minute, with nothing saved.

Limits and gotchas

- **"member 2: give a group or a user, not both"**: a member with both, or neither.
- **"no Keycloak group "retail-team""** under **Applied to** (`keycloak`), phase Error: the group doesn't exist in Keycloak. Make it there, or fix the name; the operator tries again within seconds.
- **"no Keycloak user "dana""**: the same, for a person.
- **"no role steward: an administrator defines it (Administration → Roles); its members have no access meanwhile"**: a role that was defined when the project was saved and has since gone. Define it again, or change the members.
- `pipelines`, `agent`, `serving`, `admin`, `editors` and `readers` aren't roles a member can be given: they're the platform's own.

Service accounts: applications as members

What it's for. An application that works with the project as a member would, signed in as itself rather than as a person: a CRM that keeps the project's sales metrics (its semantic model) and its import, a company system that reads its tables or queries its metrics on a schedule. A service account is that application's identity in the project: a member in a role you choose, here and nowhere else, with a secret of its own that can be rotated and revoked, and every call audited under its name.

Who can do it. The project's editors register and revoke them. An **administrator other than the one who registered it** approves each and rotates its secret: a service account may be an editor, and only administrators make people members.

Set up

On the page. The project's **Access** tab → **Service accounts** → **Register one**: a name (lowercase letters, digits and dashes), its role (reader, editor, or one of the company's roles), and what it's for. It waits for approval. An administrator presses **Approve**: the page shows its client and secret **once**; give them to the application. **New secret** makes another (the old one stops working within a minute); **Revoke** takes it away at once.

Through the API:

Method and path	Who	What it does
`GET /api/projects/{name}/serviceaccounts`	members	The service accounts, each with its client, the name its tokens carry, and whether it's approved
`PUT /api/projects/{name}/serviceaccounts/{account}`	editors	Registers one, or changes it: `{"role": "editor", "description": "...", "owner": "alice"}`. A changed role or owner needs approval again
`POST /api/projects/{name}/serviceaccounts/{account}/approve`	an administrator, not its registrant	Approves it: the answer has `clientId` and `secret` (once) and `tokenUrl`
`POST /api/projects/{name}/serviceaccounts/{account}/rotate`	an administrator	A new secret, shown once
`DELETE /api/projects/{name}/serviceaccounts/{account}`	editors	Revokes it

The application signs in with OAuth's client credentials, and calls the API with the token, as any member does:

```
TOKEN=$(curl -s https://<platform>/sso/realms/lakehouse/protocol/openid-connect/token \
  -d grant_type=client_credentials -d client_id=svc-crm-provisioner -d client_secret=$SECRET | jq -r
    .access_token)
curl -s -X PUT https://<platform>/api/projects/crm/semantic -H "Authorization: Bearer $TOKEN" \
  -H "X-Platform-Request: 1" -H "Content-Type: application/json" -d @model.json
```

In Python: `storscale.Platform(url, client_id="svc-crm-provisioner", client_secret=secret)`.

How it behaves

- Its client is `svc-<project>-<name>` (confidential, client credentials only), made by the operator within a minute of approval; its tokens name it `service-account-svc-<project>-<name>`, which is how the project's **Members** list, the audit log and Ranger's audit show it.
- It's a member of the project in its role: what that role may do on the pages' API, read in Trino (Ranger) and change, it may; nothing in any other project, and never an administrator's tools.
- Its secret is kept as the project's connection `<project>-svc-<name>`, which only the platform reads.
- The platform's audit log records `serviceaccount.registered`, `.approved`, `.rotated` and `.revoked`, and refusals (approving one's own, approving without being an administrator).

Acting for the people signed in to it (onBehalfOf)

An application people sign in to through Entra ID (the CRM) can also ask for the project's data **as each of them**, under their own rules, in place of its own: a forecast panel showing a seller their own pipeline, a manager their team's.

- **Set up, once per install:** in Entra, the platform's app registration exposes a scope (`access_as_user`) and issues v2 access tokens, and the application's own app registration is granted delegated permission to it; the chart's `sso.entra.onBehalfOf` turns Keycloak's side on (Operating).
- **Per project:** the service account is registered with **It may also act for the people who sign in to it through Entra ID** (`"onBehalfOf": true`), and approved by an administrator as any service account is (a change to it needs approval again).
- **How the application does it:** for the person signed in to it, it asks Entra for a token for the platform's app on their behalf (OAuth's on-behalf-of, scope `api://<the platform's app>/access_as_user`), then gives that to Keycloak with its own client ID and secret:

```
curl -s https://<platform>/sso/realms/lakehouse/protocol/openid-connect/token \
  -d grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer -d assertion=$ENTRA_ON_BEHALF_OF_TOKEN \
  -d client_id=svc-crm-crm -d client_secret=$SECRET -d scope=openid
```

The answer is a token **for that person** (their username and groups), which the API takes as theirs: their membership, their readers' rules, their row filters and masks.

- **For this project alone.** platformd refuses such a token (its `azp` the service account's client) anywhere but the project's routes (`/api/projects/<project>/...`) and `/api/session`, and while the account is approved to act for people: "this token was got on alice's behalf by svc-crm-crm, a project's service account: it's for that project alone". The audit log records each refusal (`onbehalf.refused`).
- **People must have signed in to the platform once:** Keycloak finds them by their Entra link, made then. Until they have, the exchange answers "User not found": the application can tell them to open the platform once.
- The token lasts as long as the platform's do (minutes), with no refresh token: the application asks again.

Limits and gotchas

- "a service account is approved by an administrator other than the one who registered it"; "... only administrators make: an administrator approves it" (an editor approving).
- A service account doesn't register service accounts.
- Tokens it already has last until they expire (minutes) after a revocation or a rotation; new ones are refused.
- A role nobody defined gives it nothing, as for people: the project's status says so.

Tables

What it's for

`tables` gives the project a namespace of Iceberg tables in the lakehouse, which its people query through Trino and its pipelines write, under Ranger's policies.

Who can do it

Administrators turn it on. Editors (and roles with `write-tables`) then make, change and drop tables in it.

Set up

On the Edit page, tick **Tables**. **Namespace** may be left empty for the default. The readers' rules (**Tables readers may read**, **Row filters for readers**, **Column masks for readers**) are in Access and protection.

```
"tables": {
  "namespace": "retail",
  "readers": {"tables": ["uploads_orders", "uploads_customers"]},
  "maintenance": {"paused": false}
}
```

Field	Default	What it is
`catalog`	`iceberg`	Trino's catalog, on Nessie
`namespace`	the project's name, dashes as underscores (`sales-eu` → `sales_eu`)	The schema in that catalog
`readers`	every table, as it is	What readers see: see Access and protection
`checks`, `freshness`	none	See Data quality
`maintenance`	on	`{"paused": true}` stops the nightly compaction. See History and releases
`tags`, `people`, `retention`	none	See History and releases, Access and protection, Privacy

How it behaves

- The operator makes the namespace in Nessie and keeps Ranger's policies on it: editors (and roles with `write-tables`) may do anything in it; readers read the tables the project shows readers, under its row filters and masks; the pipelines' service account reads, inserts, deletes, creates, drops and alters tables in it (dbt's models, imports, flows), and nothing outside it. A table's past (`orders@close-2026-09`, `orders#<commit>`) is under the same rules.
- Tables are made by people (`CREATE TABLE iceberg.retail.notes ...` on the SQL tab), by imports and uploads (Getting data in), by flows and dbt.
- The catalog (OpenMetadata) finds the project's tables within a minute or so; the Overview says "The catalog hasn't seen this project's tables yet. It looks every minute." until it has.

Limits and gotchas

- **"tables: catalog "x" and namespace "Sales" must be lowercase identifiers"**: letters, digits and underscores, starting with a letter or underscore.
- Imports, sources' copies, deliveries and knowledge banks need `tables`: without it they're refused ("its tables are the project's, and it has none (tables)").

Files

What it's for

`files` gives the project a bucket of its own in Buckets: uploads land there, pipelines read and write its `landing/` folder, and notebooks read it with the person's own credentials.

Who can do it

Administrators turn it on. Who may read and write it follows the roles: editors read and write; readers read.

Set up

Tick **Files** on the Edit page; **Bucket** may be left empty for the default.

```
"files": {"bucket": "retail"}
```

Field	Default	Rule
`bucket`	the project's name	3 to 40 lowercase letters, digits and dashes

How it behaves

- The operator makes the bucket, sets its size limit (the project's `limits.storage`, else the install's `limits.project.storage`, else none) and keeps a Buckets policy for each of the project's groups: those whose role has `write-files` (editors) may read, write and delete; those with `read-files` (readers) may list and read.
- The pipelines' service account may read and write under `landing/` only. Uploads go to `landing/uploads/`, which a `files` import reads (Getting data in).
- At 80% of its size limit the project is alerted ("the bucket retail is 80% full (...)"), and again when it's full ("the bucket retail is full (...): new files are refused"): Buckets refuses new files once it's full.

Limits and gotchas

- **"files: bucket "Retail": 3 to 40 lowercase letters, digits and dashes"**.
- Give each project a bucket of its own: a bucket's policies are its project's groups', and the operator doesn't check that another project names the same one.

Pipelines

What it's for

`pipelines` gives the project a service account that runs its pipelines: its imports, deliveries, scorings, scheduled notebooks and agents, its dbt models and its own DAGs. Whoever starts a run, it runs as this account, not as the person.

Who can do it

Administrators turn it on. Starting and clearing runs takes `run-pipelines` (editors).

Set up

Tick **Pipelines** on the Edit page and fill in **Keycloak client whose service account runs them**: `airflow-pipelines`, the example realm's.

```
"pipelines": {"serviceAccount": "airflow-pipelines"}
```

Field	What it is
`serviceAccount`	The Keycloak client (with a service account) that runs them. Required.
`repo`, `path`, `ref`, `credentials`, `memory`, `reviews`	The project's own git repo, its Airflow and merge-request reviews: see Pipelines and the project's repo

An import gives a project pipelines if it has none. An import added on the **Imports** tab (or with `PUT /api/projects/{name}/imports/{import}`) runs as the project's pipelines, so when the project has no

`pipelines`, the platform adds `{"serviceAccount": "airflow-pipelines"}` as it saves the import, and says so in the audit log ("the project runs pipelines now (airflow-pipelines), as imports need"). See Getting data in.

How it behaves

- The operator puts the client's service account in the project's group `<project>-pipelines`, which Ranger and Buckets name: it reads and writes the namespace's tables and `landing/` in the bucket.
- Without a repo, the project's pipelines are the platform's (imports, deliveries, and so on) in the shared Airflow. With a repo, its own DAGs come from it; on Kubernetes the project then gets an Airflow of its own. See Pipelines and the project's repo.
- The **Pipelines** tab appears.

Limits and gotchas

- **"Keycloak client "x" has no service account"** under **Applied to**: the client doesn't exist, or has no service account. Make it in Keycloak, or name `airflow-pipelines`.
- **"pipelines: path, ref, credentials, memory and reviews need a repo"**: those fields without `repo`.
- Taking `pipelines` away removes the pipelines' group: imports and deliveries then can't run.

Where the project is told of trouble (notify)

What it's for

The project's people hear of a failed pipeline run, a failing check or a table not changed in time where they work: a Teams or Slack channel, or email. What the alerts are and how to mute them is in Data quality.

Who can do it

Administrators set where (the Edit page). Editors give the channel's webhook as a connection (Connections tab, which takes `connections`).

Set up

1. On the project's **Connections** tab, add a connection of kind **A Teams channel** (key: the Workflows webhook URL) or **A Slack channel** (key: the incoming webhook URL). Its name starts with the project's and a dash: `retail-alerts`.
2. On the Edit page, under **Alerts**: **Teams channel** (the connection's name), **Slack channel** (the connection's name), **Email** (comma-separated addresses).

```
"notify": {
  "teams": "retail-alerts",
  "slack": "retail-alerts-slack",
  "email": ["retail-data@example.com", "bob@example.com"]
}
```

How it behaves

The platform looks at each project's issues every minute and tells the channels when an issue starts, and when a check or a table comes right. Email goes through the install's mail server, as set for the platform's own alerts (the Secret `storscale-alerts`; see Operating the platform). The Overview's **Alerts** card says where the project is told, or "Not told anywhere yet: the platform's administrators set where, when they edit the project."

Limits and gotchas

- **"notify: teams "Alerts" isn't a connection's name"**; **"notify: "bob" isn't an email address"**.
- A `teams` or `slack` that names a connection the project doesn't have isn't refused when saved; the project just isn't told there. Add the connection.
- No mail server set for the install: email isn't sent.
- The Overview's **Alerts** card names only a Teams channel and email: a project told only in Slack reads "Not told anywhere yet", though its Slack channel is told.

Limits

What it's for

Limits cap the project's share of Trino and the size of its bucket, so one project can't slow or fill the platform for the others.

Who can do it

Administrators, in the JSON box (the form has no fields for them) or through the API.

Set up

```
"limits": {"queries": 8, "queued": 80, "queryMemory": "30%", "storage": "2Ti"}
```

Field	Meaning	Rule	Install default (`limits.project.*`)
`queries`	Trino queries running at once	1 to 100	`4`
`queued`	Queries waiting; more are refused	up to 1000	`40`
`queryMemory`	Trino memory for its queries	`4GB`, `512MB`, or a share of Trino's: `50%`	`50%`
`storage`	Its bucket's size	`500Gi`, `2Ti` (needs `files`)	none

A field left out takes the install's default.

How it behaves

The operator keeps a Trino resource group for each project with these limits. A query counts against the project when a member names the project (the SQL page and metrics tag their queries `project:<name>`) and when the project's pipelines run it; the rest count against the person. Notebooks are a person's, with a person's limits. More in Operating the platform.

Settings

`limits.project.queries`, `queued`, `queryMemory` and `storage` in the chart (Compose: the operator's `LIMITS_PROJECT_QUERIES`, `LIMITS_PROJECT_QUEUED`, `LIMITS_PROJECT_QUERY_MEMORY`, `LIMITS_PROJECT_STORAGE`, with the same defaults when unset).

Limits and gotchas

- **"limits: queries 200: from 1 to 100 at once"**, **"limits: queryMemory "4G": as 4GB, 512MB or 50%"**, **"limits: storage "500GB": as 500Gi or 2Ti"**, **"limits: storage needs files (a bucket)"**.
- The storage alert's own words say to raise the limit with "Edit, Limits": that's the JSON box.

Changing a project, and its status

Set up

On the project's **Overview**, an administrator's **Administration** card has **Edit project**; the Access tab has **Change members**. Both open `/p/<project>/edit`: the same form and JSON box, the name fixed. **Save** sends the whole project (`PUT /api/projects/{name}`).

How it behaves

- A save replaces the spec. What only the platform sets is kept as it was, whatever the body says: a delivery's approval, a data product's consumers and certification, an agent's approval.
- The operator applies the change within seconds. It also goes over every project again each minute, and tries a failed one again after 10 seconds.
- The project's phase: **Waiting for the operator** (never applied yet), **Applying**, **Ready**, **Error** (a part failed: its line under **Applied to** says why), **Deleting**. **Refresh status** (on the Administration card) reloads it.

Limits and gotchas

- A project in **Error** still has what applied: a failed part is tried again, and the rest stay as they were.
- A change to a role a project gives (on Administration → Roles) applies the project again too, without its spec changing. See Roles.

Deleting a project

What it's for

A team or a trial that's finished: its access and credentials go, its data stays for someone to remove on purpose.

Who can do it

Administrators.

Set up

On the project's Overview, the **Administration** card: **Delete**, then, at "Delete retail? Its access and the credentials it keeps go; its data stays.", **Delete** again. Or:

```
curl -X DELETE "$PLATFORM/api/projects/retail" -H "Authorization: Bearer $TOKEN" -H
  "X-Platform-Request: 1"
```

It answers 204.

How it behaves

The project's phase is **Deleting** while the operator undoes it:

Goes	Stays
Its Keycloak groups, Ranger groups and policies, Buckets policies	Its tables (the namespace in Nessie)
Its sources' Trino catalogs and their copies' pods	Its bucket and its files
Its imports' and deliveries' Airflow connections	The copies' tables
Its gateway keys, its agents' and served models' clients	
Its merge requests' data branches, its own Airflow (with its run history)	
Its Nessie tags (the history they kept goes after the catalog's next collection)	
Its connections: every credential it kept	
Its MLflow workspace: experiments, runs, models and their artifacts	

The audit log records `project.deleted`.

Limits and gotchas

- **"no such project"** (404): the name is wrong, or it's gone.
- The tables and bucket keep their names: a project made again with the same name has the same namespace and bucket, with what was left in them.

Project templates

What it's for

A new team asks for a project. An administrator makes it from one of the company's templates, so it works on day one: the company's checks, its limits, where it's told of trouble, its pipelines' settings, and a copy of the template's repo (a sample pipeline, a dbt model) as the team's own, in the company's GitLab. The team changes what it needs from there.

Who can do it

- Anyone signed in sees the templates (what a project would start with) on Administration → Templates and `GET /api/templates`.
- Administrators define, change and remove templates, and make projects from them. Anyone else is refused: "only administrators change projects".

Set up: the install's GitLab

A template's repo, and the project's copy of it, live in the install's GitLab. Without one, templates are a spec only (no repo), and there are no libraries.

On Kubernetes, put a token in a Secret of the release's namespace (key `token`) and give the chart its GitLab:

```
kubectl -n storscale create secret generic storscale-gitlab --from-literal=token=<token>
storscale upgrade --context <cluster> --set templates.gitlab.url=https://gitlab.example.com \
  --set templates.gitlab.group=data/teams --set templates.gitlab.tokenSecret=storscale-gitlab
```

Chart value	Compose (`stack/.env`)	What it is
`templates.gitlab.url`	`GITLAB_URL`	GitLab's address, `https://gitlab.example.com`. Empty: templates without repos, no libraries
`templates.gitlab.group`	`GITLAB_GROUP`	The group new projects' repos are made in, by its full path: `data/teams`
`templates.gitlab.tokenSecret`	`GITLAB_TOKEN` (the token itself)	The Secret holding the token (key `token`)

The token (a group's or a service account's, scope `api`) must be allowed to make projects in the group and to read the templates' and libraries' repos. On Kubernetes platformd reads it from the Secret at each use, so a new token needs no restart. Compose's `stack/compose.yaml` defaults these to the `templates` test suite's stand-in GitLab (`http://test:9480`, group `teams`), which answers only while that suite runs: set all three in `stack/.env` for a real one.

The Templates page says where copies go ("... as its own, in https://gitlab.example.com/data/teams"), or "the install's GitLab (none is set: templates.gitlab)".

Set up: defining a template

1. Make the template's repo in GitLab: a folder of DAGs (`dags/`), a dbt project, a README. Write `{project}` wherever the new project's name belongs (a DAG's `dbt.build(project="{project}", ...)`, a dbt profile, the README's title).
2. Open **Administration** → **Templates** → **Define a template**.
3. **Name**: lowercase letters, digits and dashes (`starter`); fixed once defined.
4. **What it's for**: a line; required.
5. **Its repo in GitLab**: the repo's path, `data/templates/starter`. Empty: no repo.
6. **The project's spec**: the `spec` part of a project, as the Edit page's JSON box shows it, without `members` and without `pipelines.repo` (those come when a project is made). It starts with a sample:

```
{
  "description": "{project}: a new team's",
  "tables": {},
  "files": {},
  "pipelines": {"serviceAccount": "airflow-pipelines", "path": "dags"},
  "limits": {"storage": "100Gi"}
}
```

7. Press **Define it**. The card lists what the template gives: "A copy of the repo data/templates/starter, its pipelines in dags/", "Tables, 1 freshness targets, 1 checks", "A bucket of its own", "Limits: storage 50Gi", "Told of trouble at data-team@example.com".

Change opens the same form (the name fixed); **Remove** asks "Remove the template starter? Projects made from it stay as they are."

A complete template, through the API (`PUT /api/templates/starter`, with `X-Platform-Request: 1`):

```
{
  "description": "A new team's project: a sample pipeline and dbt model, the company's check, where
    it's told of trouble",
  "repo": "data/templates/starter",
  "spec": {
    "description": "{project}: made from the starter template",
    "tables": {
      "checks": [{"table": "orders", "column": "order_id", "check": "unique"}],
      "freshness": [{"table": "orders", "within": "24h"}]
    },
    "files": {},
    "pipelines": {"serviceAccount": "airflow-pipelines", "path": "dags", "memory": "2Gi", "reviews":
      {"dbt": "dags/dbt"}},
    "notify": {"email": ["data-team@example.com"]},
    "limits": {"storage": "50Gi"}
  }
}
```

Field	What it is
`description`	What the template is for. Required.
`repo`	Its repo in the install's GitLab, `group/repo`. Optional.
`spec`	A project's spec, without `members` or `pipelines.repo`. `{project}` anywhere in it becomes the project's name.

The name comes from the path; `changedBy` and `changedAt` are the platform's.

Set up: making a project from a template

1. On **Administration** → **Templates** (or Administration → Projects → **From a template**), on the template's card, press **Make a project**.
2. **Name**: the project's name, `growth`.
3. **Editors**: Keycloak groups, or people as `@name`, comma-separated: `growth-team, @dana`. At least one.
4. **Readers**: the same, optional: `analysts`.
5. **Told of trouble at**: email addresses, comma-separated. Empty: the template's.
6. Press **Make it**. With a repo it says "Copying its repo..." (it can take up to two minutes); then the browser opens the new project.

Through the API, `POST /api/templates/{template}/projects` (administrators, with `X-Platform-Request: 1`):

```
{
  "name": "growth",
  "description": "The growth team's campaigns and experiments",
  "members": [
    {"group": "growth-team", "role": "editor"},
    {"user": "dana", "role": "editor"},
    {"group": "analysts", "role": "reader"}
  ],
  "notify": {"email": ["growth-team@example.com"]}
}
```

Field	What it is
`name`	The project's name. Required.
`members`	As a project's (any role, including the company's own). Required.
`description`	Replaces the template's, when given (the page doesn't send one: the template's stands).
`notify`	Replaces the template's `notify` whole, when given.

It answers 201 with `{"name": ..., "spec": <the project>, "repo": <the new repo's web address>}`.

How it behaves

Making a project from a template, in order:

1. **Checked first.** The project is made from the template on paper (with a stand-in repo) and checked as any project is, and the name is checked to be free, so nothing is made for a project that can't be.
2. **The repo copied** (a template with a repo). The template's repo is copied into the install's group as a repo of the project's own, named as the project (`data/teams/growth`), described "The growth project's pipelines, from the starter template". The platform waits for GitLab to finish copying (at most two minutes), then unlinks the copy from the template's: it's the team's, not a fork.
3. `{project}` **filled in.** In every text file of the copy's default branch, `{project}` becomes the project's name, in one commit: "The growth project's, from the starter template". A file without `{project}` is left as it is.
4. **A token of the repo's own.** GitLab makes a project access token for the new repo: named `storscale-growth`, the Developer role, scopes `api` and `read_repository`, valid for a year (GitLab's most). The platform keeps it as the project's connection `growth-repo` (kind `git`: user `storscale-growth`, the token as its password), which is never shown again. The template's token, and the install's, are never given to the project.
5. **The spec.** The template's spec, with `{project}` replaced by the name, the members (and description and notify, when given), and, with a repo, its pipelines naming the copy: `pipelines.repo` the copy's https address, `pipelines.credentials.secretRef` `growth-repo`. A template with a repo but no `pipelines` gets `{"serviceAccount": "airflow-pipelines"}`. A template whose pipelines have `reviews` gets them pointed at the install's GitLab and the copy's path, so merge requests on the new repo are reviewed on data branches from day one.
6. **Saved and applied.** The project is saved and the operator applies it at once. On Kubernetes, a project with a repo gets an Airflow of its own, which clones the copy with its own token; in Compose, the repo is a DAG bundle in the shared Airflow. See Pipelines and the project's repo.

If any step fails, what was made is taken back: the connection removed, the copied repo deleted.

The project is the team's from then on. Changing or removing the template changes no project made from it; each keeps what it was made with.

The audit log records `template.saved` ("added" or "changed", and the repo), `template.removed`, and, for the project, `project.saved` with "from the template starter; its repo data/teams/growth (a copy of data/templates/starter)".

Settings

Chart value	Compose	Default
`templates.gitlab.url`	`GITLAB_URL`	`""` (chart); the test suite's stand-in (Compose)
`templates.gitlab.group`	`GITLAB_GROUP`	`""` (chart); `teams` (Compose)
`templates.gitlab.tokenSecret`	`GITLAB_TOKEN`	`""` (chart); the stand-in's token (Compose)

Templates and libraries are kept in the platform's bucket (`templates/templates.json`, `templates/libraries.json`); without the platform's bucket (`STORE_URL`), they can't be changed: "the platform keeps no files here (no STORE_URL)".

Limits and gotchas

What's refused when defining a template (422 unless said):

Message	Why
"a template's name: lower-case letters, digits and dashes"	The name
"say what the template is for (its description)"	No description
"repo: the template repo's path in GitLab, as group/repo"	`repo` isn't a path
"a template has no members: they're given when a project is made from it"	`spec.members` given
"pipelines.repo: a project made from the template gets its repo (a copy of the template's); leave it out"	`spec.pipelines.repo` given
"a project made from it: ..."	The spec as a project would have it is wrong; the rest of the message says what, as for any project
"the install has no GitLab for templates' repos (GITLAB_URL, GITLAB_GROUP, GITLAB_TOKEN): leave repo out"	A repo, and no GitLab
"the template's repo data/templates/starter: ..."	GitLab can't find the repo with the install's token
"a template, as JSON: its description, repo and spec: ..." (400)	The body isn't JSON, or has a field a template doesn't

- **A template without a repo can't set the repo's settings.** `pipelines.path`, `ref`, `memory` and `reviews` need a repo, so the form's sample spec (which has `"path": "dags"`) is refused without one: "a project made from it: pipelines: path, ref, credentials, memory and reviews need a repo". Take `path` out, or give the template a repo.
- **The page's spec box isn't checked as JSON until you save:** "The spec isn't JSON: ...".

What's refused when making a project:

Message	Status	Why
"no template starter"	404	The name
"there's a project growth already"	409	The name is taken
(a project's own problems, as for any project)	422	The name, the members, or the template's spec with them
"the install has no GitLab for templates' repos (GITLAB_URL, GITLAB_GROUP, GITLAB_TOKEN)"	503	The template has a repo, the install no GitLab
"copying the template's repo: ..."	502	GitLab refused the copy: a repo of that name in the group already, or the token may not make projects there
"the project's repo: gitlab: the template's repo wasn't copied within 2m0s (still ...)"	502	GitLab was slow; nothing is left behind: try again
"the project's repo: ..."	502	Filling in `{project}`, or the token, failed (GitLab's own words follow)
"a connection growth-repo is another's: remove it first"	502	A Secret of that name exists that isn't the project's

- **The repo token lasts a year.** Renew it before then: make a new project access token on the repo in GitLab and paste it as the password of the connection `<project>-repo` on the project's Connections tab.
- `{project}` **is filled in once,** when the project is made, in files of the default branch only (the first thousand entries of the repo's tree). Later changes to the template's repo don't reach the copy.
- **On Compose with the defaults,** `GITLAB_URL` is the test suite's stand-in: a template with a repo fails unless that suite is running. Set a real GitLab in `stack/.env`.

Shared libraries, on the same page

Below the templates, **Shared libraries** lists the company's code that projects use without copying: a repo in the install's GitLab at a ref (a tag is best: it doesn't move). The platform reads it with the install's token for whoever the realm signed in, so no project holds a token for it. In a dbt project's `packages.yml`, name it by its address (the table's **In packages.yml** column, `- git: "https://gitlab.example.com/data/libraries/company-dbt.git"`); in a pipeline, `storscale.library.use("company-py")`; in a notebook, `buckets_lake.library("company-py")`.

Administrators **Add a library** (name, what it's for, its repo in GitLab, ref), **Change** it (a new ref reaches every project at its next run) or **Remove** it ("Projects naming it can't fetch it any more."). The ref is checked to be in the repo when it's saved.

Method and path	What it does	Who
`GET /api/templates`	The templates and libraries, whether the caller may change them, and the install's GitLab and group	anyone signed in
`PUT /api/templates/{template}`	Define or change a template	administrators
`DELETE /api/templates/{template}`	Remove it	administrators
`POST /api/templates/{template}/projects`	Make a project from it	administrators
`GET /api/libraries`	The libraries, each with its repo's address	anyone the realm signed in (a person, a project's pipelines, an agent)
`GET /api/libraries/{library}`	A library, with the commit its ref names now	the same
`GET /api/libraries/{library}/archive`	Its files at a commit (default its ref's), as `.tar.gz`	the same
`PUT /api/libraries/{library}`	Define one, or move it to another ref: `{"description", "repo", "ref"}`	administrators
`DELETE /api/libraries/{library}`	Remove it	administrators

How pipelines, dbt and notebooks use libraries is in Pipelines and the project's repo.

3. Getting data in

This chapter is for the people who bring a project's data onto the platform: an engineer adding a feed from an application or a database, an analyst uploading a spreadsheet, an administrator wiring up a project's sources. It covers every way in: files uploaded to the project's landing area and imported as tables; imports from a REST API, a database, Salesforce, Gong, HighSpot or SAP S/4HANA, each on a schedule; a database queried where it is, with no copy; a database copied by change capture; another platform's Iceberg tables adopted where they are, taken over and given back; and the connections that hold the credentials all of these sign in with. Who may read what arrives is chapter 4; checking that it's right is chapter 5.

In this chapter:

- Which way in
- Uploads and the landing area
- Imports: what they have in common
- A files import
- A REST API import
- A database import (sql_database)
- A Salesforce import
- A Gong import
- A HighSpot import
- An SAP S/4HANA import
- Schedules and runs
- Reset: copy everything again
- Removing an import
- Sources: a database queried in place
- Change-capture copies
- Adopted tables: another platform's Iceberg tables, where they are
- Connections
- What the audit log records

Which way in

You have	Use	Lands as	Fresh
A CSV, Excel, JSON-lines or Parquet file	An upload, then a **files import**	`uploads_<name>` (or `<import>_<name>`)	Each scheduled run reads the file again
An application with a REST API	A **rest_api import**	`<import>_<resource>`	On its schedule; only what's past the cursor
A PostgreSQL or MySQL table with a column that grows	A **sql_database import**	`<import>_<table>`	On its schedule; only what's past the cursor (deletes aren't seen)
Salesforce	A **salesforce import**, from the Imports tab's form	`<import>_<object>`	On its schedule; some objects by their changes

You have	Use	Lands as	Fresh
Gong (calls, transcripts, scorecards)	A **gong import**, from the Imports tab's form	`<import>_<object>`	On its schedule; calls and answered scorecards by what's new
HighSpot (spots, items, users)	A **highspot import**, from the Imports tab's form	`<import>_<object>`	On its schedule; all of it each run
SAP S/4HANA (partners, products, orders, documents)	An **sap import**, from the Imports tab's form	`<import>_<object>`	On its schedule; most objects by their last change
A database the company runs, needed in queries today	A **source**	A Trino catalog `<project>_<source>`, nothing copied	Always: queried live
A database whose every change should reach the lakehouse	A source with **copy** (Kubernetes only)	`<source>_<table>`	Within `copy.commitEvery` seconds (30)
Iceberg tables another platform keeps (a Hive Metastore and its object store)	An **adopted table** (`tables.adopt`; an administrator sets up the store)	The name you give it, its files left where they are	Within a minute of the other platform's change

Everything that lands is one of the project's Iceberg tables, in its namespace, under its rules (chapter 4), in the catalog with its lineage. See Lab 3 in the lab guide, which does the first and the fifth, and shows where the others start.

Uploads and the landing area

A project's landing area is `landing/uploads/` in its bucket. An upload puts a file there, as the person who uploads it, and shows its first rows with the types the platform reads in them. A file in the landing area is nothing until a files import makes it a table; the canvas (chapter 8) then takes that table as an input.

Who can do it: uploading and removing an upload takes the permission `write-files` (editors have it). The **Uploads** panel's buttons show to those with `build` (editors). Listing and previewing take membership of the project; the bucket's own policy decides what each person may read (`read-files`: readers have it). **Import as a table** takes `connections` (it adds an import).

Set up

1. The project needs **Files** (a bucket) and **Tables**. Both are ticked on the project's Edit page (chapter 2). Leave the bucket's name empty: uploads go to the bucket named after the project.
2. Open the project → **Flows**. The **Uploads** panel is at the bottom.
3. Press **Upload a file** and pick a `.csv`, `.xlsx`, `.xls`, `.jsonl` or `.parquet` file. The panel shows the file's name, its size and, for a CSV, a preview: each column's name and type, and the first 20 rows.
4. Press **Import as a table** beside the file and accept or change the table's name (the prompt reads "The table's name (uploads_<name>):"). That's the files import.

Through the API, a multipart form with the field `file` (and optionally `name`, the file's name in the landing area):

```
curl -X POST "$PLATFORM/api/projects/retail/uploads" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" \
  -F file=@customers.csv
```

The answer holds the upload (`name`, `size`, `at`, `format`), its `preview` and the `table` name the platform suggests (`customers`).

Route	What
`GET /api/projects/{name}/uploads`	The landing area's files, newest first
`POST /api/projects/{name}/uploads`	Upload one (multipart, field `file`)
`GET /api/projects/{name}/uploads/{file}`	A file's preview, read again
`DELETE /api/projects/{name}/uploads/{file}`	Remove it

How it behaves

- The platform writes the file as you: your Keycloak token is exchanged with Buckets' STS, so Buckets' policy for the project decides. A reader's upload is refused by the platform first ("the project's editors upload files to it"); a write the bucket refuses says "Buckets refused it: the bucket's policy doesn't let you write it".
- The preview reads the first 64 KB of a CSV. A header row names the columns (lower case, other characters as underscores; an empty one becomes `column_<n>`). A first line with more semicolons than commas is read as semicolon-separated. Each column's type is the narrowest every value fits: `bigint`, `double`, `boolean`, `date` (`2026-05-03`), `timestamp` (`2026-05-03T10:00:00Z` or `2026-05-03 10:00:00`), else `varchar`. An Excel, JSON-lines or Parquet file says "previewed once imported"; any other file says "not a file an import reads (csv, xlsx, jsonl, parquet)".
- The suggested table name is the file's name without its extension, lower case, other characters as underscores, at most 60 characters, and `file_` in front when it would start with a digit.
- The audit log records `upload` (with the file's name and size) and `upload.removed`.

Limits and gotchas

- A file is at most 256 MB. Its name is letters, digits, dots, dashes and underscores, up to 121 characters, starting with a letter or digit ("a file's name is letters, digits, dots, dashes and underscores").
- Uploading a file of a name that's there already replaces it. The next run of its import reads the new one.
- Removing an upload doesn't remove its import or its table.
- Uploads and files imports use the bucket named after the project, whatever `files.bucket` says. Leave `files.bucket` empty, or uploads and imports look for a bucket that isn't there.
- The landing area is a folder of the bucket: anyone who may read the project's files (its readers) may read what's uploaded, whatever the readers' rules say of the table made from it.

Imports: what they have in common

An import copies data into the project's tables on a schedule. It's dlt, run in the platform's Airflow as one of the project's pipelines: one Airflow DAG per import, `import_<project>_<import>` (dashes in the project's name become underscores), tagged `project:<name>` and `import`. Each of its resources lands in the project's namespace as `<import>_<resource>`: merged on its primary key when it has one, otherwise appended to (a files import replaces its table instead). Where a resource names a cursor, each run copies only what's past it; dlt keeps where it got to in the project's namespace.

Who can do it: adding, changing, removing and resetting an import takes the permission `connections` (editors have it); refused, it says "the project's editors change its imports (it takes the permission connections)". Running one now takes `run-pipelines`. An administrator may also write imports into the spec on the Edit page.

The spec

`spec.imports` is a list; each import has:

Field	What
`name`	1 to 20 lowercase letters, digits and underscores, starting with a letter. Its tables are `<name>_<resource>`
`source`	`files`, `rest_api`, `sql_database`, `salesforce`, `gong`, `highspot`, `sap` or `files`
`connection`	One of the project's connections: `rest_api` an API token, `sql_database` a database user, `salesforce` a Salesforce org, `gong` a Gong API key, `highspot` a HighSpot API key, `sap` an SAP communication user. A files import has none
`schedule`	Cron's five fields, in UTC (`0 * * * *`). Empty: hourly
`files` / `api` / `database` / `salesforce` / `gong` / `highspot` / `sap`	What it copies, by its source (below); only the one its `source` names

Three ways to add one

- **From a form.** Files: the Uploads panel's **Import as a table** (above). Salesforce, Gong, HighSpot and SAP: Imports → **Add Salesforce**, **Add Gong**, **Add HighSpot**, **Add SAP S/4HANA** (below). REST APIs and databases: the project's Edit page, section **Imports** → **Add an import** (administrators).
- **In the spec**, on the Edit page's **JSON** (administrators).
- **Through the API**, one import at a time, the rest of the spec as it is:

```
curl -X PUT "$PLATFORM/api/projects/retail/imports/crm" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json"
    \
  -d '{"source": "rest_api", "connection": "retail-crm-token", "schedule": "0 * * * *",
       "api": {"baseUrl": "https://crm.example.com/v1", "resources": [{"name": "leads", "path":
         "leads", "dataSelector": "data", "primaryKey": "id"}]}}'
```

  The import's name is the path's; a body that names one is ignored for it. `DELETE /api/projects/{name}/imports/{import}` removes it.

How it behaves

- An import runs as the project's pipelines: they read its landing area and write its tables. An import added through the API or a form to a project without pipelines gives it pipelines (the shared Airflow's service account `airflow-pipelines`), and the audit log says "the project runs pipelines now (airflow-pipelines), as imports need".
- A project with an Airflow of its own (on Kubernetes, one with a pipelines repo: chapter 9) still runs its imports in the platform's Airflow, signed in as that project's own pipelines (the platform's Airflow holds their sign-in as the connection `pipelines_<project>`). Its Pipelines tab lists them beside its own.
- The import's credentials go into the platform's Airflow as a connection of its own (`import_<project>_<import>`), handed to the run in its environment, never in its arguments or the log.
- Within a minute of the save the operator applies the project; Airflow then reads the import's DAG at its next parse (allow a few minutes) and it appears on the **Pipelines** tab.
- The import adds two columns of its own to every table it writes: `_dlt_load_id` (the run that wrote the row) and `_dlt_id` (the row's own ID). They're never empty: a row you insert by hand needs them too (Lab 5 inserts `'by-hand'` and `'by-hand-307'`). The catalog never classifies them. dlt also keeps tables of its own in the namespace (`_dlt_loads`, `_dlt_pipeline_state`, `_dlt_version`).
- The catalog draws each import's pipeline upstream of its tables (lineage).

- A run that fails is one of the project's alerts (chapter 5).

Limits and gotchas

- An import needs the project to have tables: "imports: crm: its tables are the project's, and it has none (tables)".
- A resource with neither a primary key nor a cursor is appended to whole on every run: its rows pile up. Give it a primary key (rows merged), a cursor (only what's new), or both.
- Two resources that would land in the same table are refused: "imports: crm: two resources would both be crm_leads".
- An import written into the spec on the Edit page doesn't get pipelines by itself. Tick **Pipelines** too, or the run is refused its landing area (`Forbidden`).
- A connection named in the spec that the project doesn't have is told on the Overview, in the `imports` part of the status ("import crm: no connection "retail-crm" among the project's"). Through the API it's refused at once: "no connection retail-crm among the project's (its Connections tab)".

A files import

A files import reads files of the landing area into tables: CSV, Excel (every sheet, stacked), JSON lines and Parquet, by the file's extension. It needs no connection.

Set up

From the Uploads panel, **Import as a table** adds the file to the import called `uploads` (made the first time, hourly, `0 * * * *`), as the table `uploads_<name>`. Each file imported the same way joins it.

In the spec (`spec.imports`):

```
{
  "name": "uploads",
  "source": "files",
  "schedule": "0 * * * *",
  "files": {
    "tables": [
      {"name": "customers", "file": "customers.csv", "primaryKey": "customer_id"},
      {"name": "orders", "file": "orders.csv"},
      {"name": "prices", "file": "prices-*.csv"}
    ]
  }
}
```

Field	What
`files.tables[].name`	The table: `<import>_<name>`
`files.tables[].file`	A file of `landing/uploads/` (no folders), or a glob (`prices-*.csv`), ending in `.csv`, `.xlsx`, `.xls`, `.jsonl` or `.parquet`
`files.tables[].primaryKey`	Optional: rows merged on it

How it behaves

- Each run reads the files again as the project's pipelines (their Keycloak client's token, exchanged with Buckets' STS: the bucket's policy for `<project>-pipelines` lets them read and write `landing/` and nothing else of the bucket).
- Without a primary key, a run replaces the table with what the files hold now. With one, rows are merged on it: a row whose key is in the table is updated, a new key added, and a row gone from the file stays.
- An ISO date (`2026-05-03`) or timestamp (`2026-05-03T10:00`, `2026-05-03 10:00:00`, `2026-05-03T10:00:00.123456`) in a CSV, Excel or JSON-lines file becomes a `date` or a `timestamp`, as the preview said. A value that looks like one but isn't (`2026-02-30`) stays text. Parquet keeps its own types.

Limits and gotchas

- A timestamp with a zone (`2026-05-03T10:00:00Z`, `+02:00`) stays text in the table, though the preview calls it a timestamp.
- A table imported before 0.31.0 has its dates as text. It stays so until its import is reset.
- "imports: uploads: orders: a file of landing/uploads/ (no folders), ending in one of csv, xlsx, jsonl, parquet": a folder in the name, or an extension it doesn't read.
- "imports: uploads: a files import needs no connection: it reads the project's landing area as its pipelines".

A REST API import

dlt's `rest_api` source: each resource is a path under the API's base address answering a list of objects; dlt follows the pages itself.

Set up

1. On **Connections**, add one of kind **An API token** (`token`), named `<project>-<something>`. It's sent as a Bearer token. An API that needs none needs no connection.
2. On the Edit page (administrators), **Imports → Add an import**: the name, **a REST API**, the connection, the schedule (empty: hourly), the base URL, and for each resource its name, path, where in the answer the list is ("data selector"), its primary key, and a cursor (its field, its query parameter, where it starts). **Add a resource** for more. Or `PUT /api/projects/{name}/imports/{import}`, as above.

```
{
  "name": "crm",
  "source": "rest_api",
  "connection": "retail-crm-token",
  "schedule": "0 * * * *",
  "api": {
    "baseUrl": "https://crm.example.com/v1",
    "resources": [
      {"name": "leads", "path": "leads", "dataSelector": "data", "primaryKey": "id",
       "cursor": "updated_at", "cursorParam": "since", "cursorStart": "2026-01-01T00:00:00Z",
       "paging": {"type": "cursor", "next": "nextCursor", "param": "cursor"}}
    ]
  }
}
```

Field	What
`api.baseUrl`	`http://` or `https://`

Field	What
`resources[].name`	An identifier: the table `<import>_<name>`
`resources[].path`	Under `baseUrl` (`leads`), never a full address
`resources[].dataSelector`	A JSONPath to the list in the answer (`data`), when it isn't the answer itself
`resources[].primaryKey`	Merged on it
`resources[].cursor`, `cursorParam`, `cursorStart`	The objects' field that grows (`updated_at`, an ID); the query parameter the API takes it in (`since`); the first value asked for. All three or none
`resources[].paging`	How the answer comes in pages. `{"type": "cursor", "next": "nextCursor", "param": "cursor"}`: the next page's cursor is in the answer at `next` (a JSONPath), and goes back as the query parameter `param`; the last page's is null. `{"type": "link", "next": "next"}`: the next page's URL is in the answer at `next`. `{"type": "single"}`: one page. None: as dlt finds them, which it does for a `Link` header or a `next` URL, **not for a cursor**: such an API, unpaged here, gives its first page alone

How it behaves

The first run asks for `?since=<cursorStart>`; each run after asks from the largest `updated_at` it saw. With paging, each page is asked for with the cursor the last one gave (`?since=…&cursor=2`), until it gives none. The imports suite shows it: a lead changed and one added reach `crm_leads` merged, none twice.

Limits and gotchas

- "imports: crm: a rest_api import has an api: its baseUrl (http or https) and its resources".
- "imports: crm: leads: a cursor names its field, its query parameter and where it starts".
- "imports: crm: leads: cursor paging names where the next page's cursor is in the answer (next: nextCursor) and the query parameter it goes back in (param: cursor)"; "… the paging's parameter since is the cursor's too: one is the page, the other where the copy got to".
- The token's connection must be of kind `api` with a `token`; the operator otherwise says "import crm: its connection has no token".

A database import (sql_database)

A PostgreSQL or MySQL table, read by a cursor column. It's for a table that only grows, or whose rows carry when they changed. Deletes in the database aren't seen; for those, use a change-capture copy.

Set up

1. On **Connections**, add one of kind **A database user**: host, port (empty: 5432 or 3306), database, user and password. An import needs the host: a JDBC URL alone won't do.
2. On the Edit page, **Imports → Add an import → a database, by a cursor**, its kind, and each table as `schema.table` with its cursor and primary key columns. Or in the spec:

```
{
  "name": "erp",
  "source": "sql_database",
  "connection": "retail-erp-db",
  "schedule": "30 * * * *",
  "database": {
    "kind": "postgresql",
    "tables": [
      {"table": "public.invoices", "cursor": "updated_at", "primaryKey": "invoice_id"},
      {"table": "public.regions", "primaryKey": "region_id"}
    ]
  }
}
```

The tables land as `erp_invoices` and `erp_regions` (the schema is dropped from the name).

Limits and gotchas

- `database.kind` is `postgresql` or `mysql`: "imports: erp: a sql_database import has a database (its kind, postgresql or mysql, and its tables) and a connection".
- Tables of the same name in two schemas would land in the same table, and are refused.
- The connection needs a host, a user and a password: "import erp: its connection needs a host, username and password".
- The account needs only to read the tables. Give it a read-only account of the database's own.

A Salesforce import

dlt's Salesforce source, from a form. Pick the objects; those that carry their changes are copied by them and merged on their `Id`, the others replaced on each run.

Set up

1. On **Connections**, add one of kind **A Salesforce org** (fields below): a user, password and security token; or a user and password with a connected app's consumer key and secret; or an instance address and a session.
2. Open **Imports** → **Add Salesforce**. Name (default `crm`), **Sign in with** (the project's Salesforce connections; none yet: the form says to add one), the objects (those marked ↻ copy only what changed), **When (cron, UTC)** (default `0 * * * *`) and **History from** (a date; empty: all of it). Press **Add**.

```
{
  "name": "crm",
  "source": "salesforce",
  "connection": "retail-salesforce",
  "schedule": "0 2 * * *",
  "salesforce": {"objects": ["account", "opportunity", "contact"], "since": "2024-01-01"}
}
```

Objects copied by their changes (↻), merged on `Id`	Objects replaced each run
`account`, `campaign_member`, `event`, `opportunity`, `opportunity_contact_role`, `opportunity_line_item`, `task`	`campaign`, `contact`, `lead`, `pricebook_2`, `pricebook_entry`, `product_2`, `sf_user`, `user_role`

`GET /api/connectors` lists the connectors the forms offer (Salesforce, Gong, HighSpot, SAP S/4HANA), each with its objects and those copied by their changes.

How it behaves

- `since` applies to the objects copied by their changes: the first run starts at that date.
- `domain` on the connection picks `login` (production) or `test` (a sandbox). `caCertificate` is trusted beside the usual authorities, for a proxy that inspects TLS.
- Saved, the audit log's `import.saved` names the objects.

Limits and gotchas

- "imports: crm: a salesforce import names its objects (salesforce.objects) and a connection (kind salesforce)".
- "imports: crm: "orders" isn't one of Salesforce's objects: account, campaign, ...".
- `since` is a date: "imports: crm: since "1/1/24": a date, as 2024-01-01".
- The connection needs a user's name and password, or an instance's address and a session; otherwise the operator says so on the Overview.
- An SAP connector: not yet.

A Gong import

Gong's calls, their transcripts and its scorecards, read from Gong's v2 API with an API key, from a form.

Set up

1. In Gong, a technical administrator makes the API key: **Company settings** → **Ecosystem** → **API** → **Create**. It's an access key and its secret. Gong may give the company its own API address (shown on that page); otherwise it's `https://api.gong.io`.
2. On **Connections**, add one of kind **A Gong account's API key**: **Access key**, **Access key secret**, and **API address** only if Gong gave you one.
3. Open **Imports** → **Add Gong**. Name (default `gong`), **Sign in with**, the objects (all of them ticked to start), **When (cron, UTC)** and **History from** (the calls and answered scorecards from that date; empty: all of them). Press **Add**.

```
{
  "name": "gong",
  "source": "gong",
  "connection": "sales-gong",
  "schedule": "0 3 * * *",
  "gong": {"objects": ["users", "calls", "call_transcripts"], "since": "2026-01-01"}
}
```

Object	Table	How
`calls` ↻	`<import>_calls`	Calls started since the last run's newest (from `since` the first time), merged on `id`. Private calls are left out
`call_transcripts` ↻	`<import>_call_transcripts`	The transcript of each call that run copied (100 calls a request), merged on `call_id`; who said what is in `<import>_call_transcripts__transcript` and its `__sentences`
`answered_scorecards` ↻	`<import>_answered_scorecards`	Scorecards answered on calls, from the last run's newest review, merged on `answered_scorecard_id`
`users`	`<import>_users`	Gong's users, replaced each run
`scorecards`	`<import>_scorecards`	The scorecards' definitions, replaced each run

How it behaves

- It signs in with the key and secret (HTTP Basic), pages through each list by Gong's cursor, and when Gong says it's sending too many requests (429), waits as long as Gong asks and tries again (up to 8 times).
- `call_transcripts` without `calls` still reads the calls (to know which transcripts to ask for), but only the transcripts are kept.
- No call since the last run is not an error: the run copies nothing new.
- Transcripts are what people said on calls: give the tables protection tags or leave them out of readers' rules as you would any personal data, and see chapter 13 for erasure.

Limits and gotchas

- Gong allows about 3 requests a second and 10,000 a day for a company; a large first copy of transcripts can take a while, and may use a good part of the day's allowance.
- A wrong key or secret fails the run with Gong's 401 in its log, and the run is one of the project's alerts.
- "imports: gong: "deals" isn't one of Gong's objects: users, calls, call_transcripts, scorecards, answered_scorecards".

A HighSpot import

HighSpot's spots, the items in them and its users, read from HighSpot's v1.0 API with an API key, from a form.

Set up

1. HighSpot's API needs its **Platform Plus** add-on. A HighSpot administrator makes an API key and its secret in HighSpot's settings, and notes the API's address for the company's region, as `https://api-su2.highspot.com` (HighSpot says which).
2. On **Connections**, add one of kind **A HighSpot API key**: **API key**, **API secret** and **API address**.
3. Open **Imports** → **Add HighSpot**. Name (default `highspot`), **Sign in with**, the objects, **When (cron, UTC)**. Press **Add**. (There's no **History from**: each run copies all of it.)

```
{
  "name": "highspot",
  "source": "highspot",
  "connection": "sales-hs",
  "schedule": "0 4 * * *",
  "highspot": {"objects": ["spots", "items", "users"]}
}
```

Object	Table	How
`spots`	`<import>_spots`	The spots the key may view
`items`	`<import>_items`	Every item in each of those spots, with its spot in `spot_id`
`users`	`<import>_users`	HighSpot's users

Each is replaced on every run.

How it behaves

- It signs in with the key and secret (HTTP Basic), reads each list 100 at a time, and waits out HighSpot's 429s as Gong's are.
- What the key may see is what's copied: a spot the key can't view, and its items, aren't.

Limits and gotchas

- No copy by changes: a large HighSpot is read whole on each run, so schedule it daily, not hourly.
- Items are the content's details (title, type, links), not the files themselves.

An SAP S/4HANA import

S/4HANA's business partners, customers, suppliers, products, sales orders, billing documents and purchase orders, with their items, read from its released OData APIs (the ones listed on SAP's Business Accelerator Hub, api.sap.com) as a communication user, from a form. It's the platform's own connector: no SAP software runs on the platform.

Set up

1. In S/4HANA, an SAP administrator makes a **communication arrangement** for the APIs below (in S/4HANA Cloud, from the communication scenarios that include them: `SAP_COM_0008` for business partners, customers and suppliers, `SAP_COM_0009` for products, `SAP_COM_0109` for sales orders, `SAP_COM_0120` for billing documents, `SAP_COM_0053` for purchase orders; on premises, with the services activated in the gateway, `/IWFND/MAINT_SERVICE`), with a **communication user** signed in by user and password, read access being enough. The platform must reach the gateway's address over HTTPS (ask the network team; the open question in use case 8.4).
2. On **Connections**, add one of kind **An SAP S/4HANA communication user**: the API's address, the user, its password, the **SAP client** if the system has several (`100`), and, if the gateway's certificate is signed by the company's own CA, that CA's certificate.
3. Open **Imports** → **Add SAP S/4HANA**. Name (default `erp`), **Sign in with**, the objects (business partners, sales orders and their items ticked to start), **When (cron, UTC)** and **History from** (the objects copied by their changes from that date; empty: all of it). Press **Add**.

```
{
  "name": "erp",
  "source": "sap",
  "connection": "retail-sap",
  "schedule": "0 2 * * *",
  "sap": {"objects": ["business_partners", "sales_orders", "sales_order_items", "products"], "since":
    "2026-01-01"}
}
```

Object	API and entity set	Merged on	How
`business_partners` ↻	`API_BUSINESS_PARTNER` `A_BusinessPartner`	`BusinessPartner`	Changed since the last run, by `LastChangeDate` (a day: that day is read again, and merged)
`customers`	`API_BUSINESS_PARTNER` `A_Customer`	`Customer`	Replaced each run (SAP stamps no change on it)
`suppliers`	`API_BUSINESS_PARTNER` `A_Supplier`	`Supplier`	Replaced each run

Object	API and entity set	Merged on	How
products ↻	API_PRODUCT_SRV A_Product	Product	By LastChangeDateTime
sales_orders ↻	API_SALES_ORDER_SRV A_SalesOrder	SalesOrder	By LastChangeDateTime
sales_order_items ↻	API_SALES_ORDER_SRV A_SalesOrderItem	SalesOrder, SalesOrderItem	The items of the orders a run copies
billing_documents ↻, billing_document_items ↻	API_BILLING_DOCUMENT_SRV A_BillingDocument, A_BillingDocumentItem	BillingDocument (and BillingDocumentItem)	As sales orders
purchase_orders ↻, purchase_order_items ↻	API_PURCHASEORDER_PROCESS_SRV A_PurchaseOrder, A_PurchaseOrderItem	PurchaseOrder (and PurchaseOrderItem)	As sales orders

Each lands as `<import>_<object>`, its columns SAP's field names in lower case with underscores (`SalesOrder` becomes `sales_order`, `TotalNetAmount` `total_net_amount`).

How it behaves

- It reads each API's `$metadata` once a run, so the columns have their types: amounts and quantities are decimals, dates and times timestamps, though OData V2 sends them as text and `/Date(…)/`.
- It reads a thousand rows a request, follows the gateway's own paging (`__next`) when the gateway pages, and waits out a 429 or 503 (the gateway busy) as long as it's told, up to eight tries.
- Items are asked for forty orders or documents at a time (`SalesOrder eq '…' or …`).
- Choosing items without their orders still reads the orders (to know which items to ask for), but keeps only the items.
- Navigation properties (`to_Item`, `to_Partner`) aren't followed: an object is its entity set's own fields.

ODP extractions: CDS views and extractors, by SAP's own delta

The released APIs stamp changes on a few objects and never record deletions. SAP's Operational Data Provisioning (ODP) does both: the gateway serves a CDS view (SAP's `I_*` and `C_*` views, or the company's own) or a classic extractor (`2LIS_11_VAITM`) as OData, and keeps a **delta queue** for each subscriber, so each read after the first returns what was inserted, changed or deleted since. It's the way SAP allows other tools to extract data (SAP Note 3255746 rules out ODP over RFC for them, not over OData).

Set up in SAP (the SAP team, once per extraction):

1. The CDS view must be delta-enabled: annotated `@Analytics.dataExtraction.enabled: true` with change data capture (`@Analytics.dataExtraction.delta.changeDataCapture.automatic: true`, or its mapping). Classic extractors are delta-enabled already.
2. In transaction `SEGW`, make a project, then on its Data Model **Redefine → ODP Extraction**: the context **ABAP Core Data Services** (a CDS view, by its SQL view name) or **DataSources/Extractors**. Generate it, and activate the service in `/IWFND/MAINT_SERVICE`. Its entity set is named `EntityOf<view or extractor>` (or `FactsOf…`, `AttrOf…`).
3. Give the platform's communication user access to the service.

On the platform: on the SAP import's form, **Add an ODP extraction**: a name of your own (its table is `<import>_<name>`), the **service** (`ZSALES_ODP_SRV`), the **entity set** (`EntityOfZCDS_SALES_ITEM`) and the **key fields** its rows are merged on (`SalesOrder, SalesOrderItem`, as SAP names them). An SAP import may have ODP extractions alone, with no objects ticked. In the spec:

```
"sap": {"objects": [], "odp": [{"name": "sales_items", "service": "ZSALES_ODP_SRV",
                                "entitySet": "EntityOfZCDS_SALES_ITEM", "primaryKey": ["SalesOrder",
                                  "SalesOrderItem"]}]}
```

How it behaves:

- The first run subscribes (`Prefer: odata.track-changes`) and copies every row. SAP answers with a delta link, which the platform keeps with the import's own record of where it got to (dlt's state, saved only when a run has landed).
- Each run after reads the delta since: rows SAP marks deleted (`ODQ_CHANGEMODE D`) are deleted from the table by their key once the run has landed; an extractor's before images (`ODQ_ENTITYCNTR -1`) are left out; the rest are merged. `ODQ_CHANGEMODE` and `ODQ_ENTITYCNTR` aren't kept as columns.
- **Copy all again** drops the table and the kept delta link, so the next run subscribes again and copies everything. The old subscription stays in SAP's delta queue until the SAP team ends it (transaction `ODQMON`).

Limits and gotchas:

- **One subscription per user and service.** SAP's delta queue knows a subscriber by its user and service, so two imports reading the same extraction with the same communication user take each other's deltas. Give each import its own user, or each extraction its own service.
- **A delta not landed is read again.** A run that fails before it lands keeps the old delta link, and SAP repeats that delta. A run that lands and then fails to delete (a message in its log) leaves those rows until the next reset.
- An entity set that gives no delta link isn't an ODP extraction's: the run copies it whole each time, and says so in its log.
- "imports: erp: odp sales_items: the fields its rows are merged on (primaryKey), as SAP names them".

Limits and gotchas

- **Deleted in SAP stays here.** A deleted order, or an item taken off an order, isn't seen (the APIs' change stamps don't record deletions): **Copy all again** (a reset) brings the tables back to what SAP has. An ODP extraction (below) sees deletions.
- Customers and suppliers are read whole each run; on a large system, schedule them daily.
- The communication user sees what its arrangement allows: an API not in the arrangement fails the run with SAP's 403 in its log; a wrong password, 401; a wrong client, SAP's own message.
- Other entity sets of the released APIs aren't offered; a CDS view of SAP's or the company's comes in as an ODP extraction (below).
- "imports: erp: "A_SalesOrder" isn't one of SAP S/4HANA's objects: business_partners, customers, ...": use the object's name from the table above, not the entity set's.

Schedules and runs

Who can do it: the schedule is part of the import (`connections`); **Run now** takes `run-pipelines`.

- `schedule` is cron's five fields in UTC: `0 * * * *` on the hour, `30 6 * * 1-5` at 06:30 on weekdays. Empty means hourly. Anything else is refused: "imports: crm: schedule "hourly": five cron fields, as 0 * * * *".
- One run of an import at a time; a missed schedule isn't caught up.
- To run one now: **Pipelines** → pick `import_<project>_<import>` → **Run now**. Or `POST /api/projects/{name}/pipelines/{pipeline}/runs`. The audit log records `pipeline.started`.

- Each run's log is beside it on the Pipelines tab ("log", or "why it failed" for a failed run): `GET /api/projects/{name}/pipelines/{pipeline}/runs/{run}/log`. See chapter 9.
- A failed latest run is an alert to the project (chapter 5).

Reset: copy everything again

A reset drops the import's tables and where its cursors got to, then copies everything into new tables. Use it when a table's columns must change type (a table imported before 0.31.0, whose dates are text), when a cursor has gone wrong, or when the source was rewritten.

Who can do it: `connections` (editors). Refused: "the project's editors reset its imports".

Set up

- **Imports** tab → the import's **Copy all again**; or on the Edit page, the import's **Reset** (it then reads "Copying all again...").
- `POST /api/projects/{name}/imports/{import}/reset`. The answer is the run's ID: `{"run": "reset-20261024T091500"}`.

How it behaves

- The run drops every table of the namespace whose name starts with `<import>_`, and dlt's record of them, then copies from the start: from each cursor's `cursorStart`, from Salesforce's, Gong's or SAP's `since`, every file again.
- The tables are made afresh, with their columns' types as the data says now. **Their history before the reset goes with them**: the History tab and reading as it was (chapter 15) start again at the reset.
- The audit log records `import.reset`: "its tables and where it got to dropped, all copied again".

Limits and gotchas

- Any table of the namespace named `<import>_…` goes, whoever made it. Don't name your own tables after an import (`uploads_summary` beside the import `uploads` would be dropped).
- Until the run ends the tables are missing: queries, dashboards and checks on them fail meanwhile.

Removing an import

Imports → **Remove** ("Remove crm? Its tables stay."), the Edit page's **Remove** beside it, or `DELETE /api/projects/{name}/imports/{import}`. The import's DAG and its Airflow connection go. Its tables stay, as data does: drop them on the SQL tab when they're no longer wanted. The audit log records `import.removed` ("its tables stay"). A missing one answers "no such import".

Sources: a database queried in place

A source is one of the company's databases, queried where it is: a Trino catalog of the project's own, `<project>_<source>` (dashes in the project's name become underscores). Nothing is copied. Its tables join with the lakehouse's in the same query.

Who can do it: sources are in the spec, which administrators change (the Edit page). The connection is given by anyone with `connections`.

Set up

1. Ask the database's owner for a read-only account. On **Connections**, add one of kind **A database user**: host, port, database (for Oracle, its service name), user, password; or a JDBC URL in place of host, port and database.
2. On the Edit page, **Sources** → **Add a source**: its name, kind, the connection, the schemas it may read (empty: all), and **Readers** if the project's readers may read it too. With Readers ticked, **Add a row filter** and **Add a mask** say what they see, each table as `schema.table`. Or in the spec:

```
{
  "sources": [
    {
      "name": "finance",
      "kind": "sqlserver",
      "connection": "retail-finance-db",
      "schemas": ["budget"],
      "readers": true,
      "rowFilters": [{"table": "budget.lines", "filter": "region = 'EU'"}],
      "masks": [{"table": "budget.lines", "column": "approver", "type": "MASK"}]
    }
  ]
}
```

3. Save. Within a minute (on Kubernetes, up to a minute more while Trino sees the connection's file) the catalog `retail_finance` answers on the SQL tab:

```
SELECT l.region, sum(l.amount) AS budget, sum(o.amount) AS revenue
FROM retail_finance.budget.lines l
JOIN uploads_orders o ON o.region = l.region
GROUP BY l.region
```

Field	What
`name`	1 to 20 lowercase letters, digits and underscores, starting with a letter
`kind`	`postgresql`, `mysql`, `sqlserver` or `oracle`
`connection`	A database user of the project's
`readers`	`true`: its readers may read it too (default: editors and pipelines only)
`schemas`	Only these (default all)
`rowFilters`, `masks`	What its readers see; tables as `schema.table`. Mask types as for the project's tables (chapter 4)
`copy`	Copy its tables into the project's: below

How it behaves

- The operator makes the catalog in Trino while it runs. The user and password go into a credentials file Trino reads, never into the statement Trino logs and Ranger audits; only the address is in the statement.
- The address is made from the connection: PostgreSQL `host:5432/database`, MySQL the whole server (each database a schema), SQL Server `host:1433` with the database and `encrypt=true`, Oracle `host:1521/service`. A `url` given is used as it is.
- Ranger lets the project's editors, read-all roles and pipelines read it, and its readers when `readers` is true. Nobody may write it.

- The catalog (OpenMetadata) describes its schemas, tables and columns as its own Trino account, which may read none of its rows.
- Removing a source drops its catalog.

Settings

None of the chart's.

Limits and gotchas

- "sources: finance: masks and row filters are what readers see, and its readers can't read it (readers: true)".
- A connection without a user and password: "source finance: its connection needs a username and a password".
- A `url` that isn't `jdbc:…`, or holds the word password: "its url holds a password (or a line break): the password goes in password, which Trino never logs".
- PostgreSQL, SQL Server and Oracle need the database: "a PostgreSQL source names its database".
- The connection's `sslmode` and `trustServerCertificate` are read by copies only. For a source queried in place, put such options in a JDBC URL.
- Heavy queries land on the company's database. When that's too much, copy it instead.

Change-capture copies

A copy brings a source's tables into the project's own by change capture: a first copy of each, then every insert, update and delete. Each lands as `<source>_<table>` in the project's namespace, under its rules like any of its tables. Kubernetes only.

Who can do it: administrators (the spec); the database's administrator grants the account what change capture needs.

Set up

1. The account needs the database's change log: PostgreSQL `REPLICATION` and `wal_level=logical`; SQL Server CDC on the tables; MySQL the row binlog.
2. On the Edit page, in the source's row, **copy: schema.table, …** (MySQL: `database.table`). Or `copy` on the source:

```
{
  "sources": [
    {"name": "dashboards", "kind": "postgresql", "connection": "retail-dashboards-db",
     "copy": {"tables": ["public.dashboards", "public.slices"]}}
  ]
}
```

The tables land as `dashboards_dashboards` and `dashboards_slices`.

How it behaves

- The operator runs a Deployment per copying source, `copy-<project>-<source>`, with Debezium Server and its Iceberg sink. It reads the change log and writes the tables itself through Nessie, keyed on each table's primary key: inserts and updates upserted, deletes applied, new columns added as they come, decimals kept exact.
- A change is committed to the table within `copy.commitEvery` seconds.

- On PostgreSQL it makes its own replication slot and publication, both named `storscale_<catalog>` (`storscale_retail_dashboards`).
- Where it got to is kept in Iceberg tables of the platform's own namespace, `storscale_cdc`.
- The project's **Overview** shows a **Copies** card: each copied table and when its last change was committed. `GET /api/projects/{name}/copies` says the same.
- A copy that isn't running for 10 minutes is an alert to the project: "the copy of dashboards isn't running (since 09:12 UTC)".
- The catalog draws each source table to its copy, column by column.

Settings

Chart value	Default	What
`copy.commitEvery`	`30`	Most seconds between a change and its commit (at least 5)
`copy.memory`	`1Gi`	Each copy's pod
`images.debezium`	`ghcr.io/memiiso/debezium-server-iceberg:1.2.0.Final`	The copier
`postgres.logicalReplication`	`false`	The platform's own Postgres with `wal_level=logical`, so a source on it can be copied (tests)

Limits and gotchas

- Compose can't run it: the project's status says "copying a source needs Kubernetes (Compose can't start its pods)".
- Oracle can't be copied yet: "sources: erp: a oracle database can't be copied yet (postgresql, mysql, sqlserver can)".
- The project needs tables: "sources: dashboards: its tables are copied into the project's, and it has none (tables)".
- Two source tables of the same name in different schemas would land in one table, and are refused.
- The connection needs a host, user and password: "copying needs its connection's host, username and password (a url alone won't do)".
- Removing `copy` stops the copy; its tables stay. A PostgreSQL replication slot stays too, and the database keeps its log for it until its administrator drops the slot.

Adopted tables: another platform's Iceberg tables, where they are

What it's for. Moving to the platform from another one with Iceberg tables (a Hive Metastore, a MinIO): copying terabytes takes days, and the other platform's jobs keep writing meanwhile. An adopted table is the other platform's table, taken into a project without copying it: the project's namespace has it, by a name you give it, from the files where they are. Each time the other platform changes it, the project sees the change within a minute. It's read and governed like any of the project's tables (the Data page, the catalog, SQL, notebooks, dbt, dashboards, the readers' rules), and it's read-only here until it's handed over.

Who can do it. An administrator sets up the store once (the chart's `externalStores`: Operating). Then adopting, taking over, giving back and stopping take two things: being an editor of the project (one who may write its tables), and the platform's **table-adoption** role (the Keycloak group `data-migration` unless the install names others; the chart's `adoption.groups`, or more on **Administration → Roles**: Roles). Administrators have both. Editors without the role see what's adopted and where from, but not the stores, nor the buttons; the platform answers them 403,

naming the group, and records the refusal in the audit log.

Set up

On the project's **Data** page, **Adopt tables** (its editors in the table-adoption role): choose the store, then one of its schemas, and its tables are listed with their rows, size and when each was last written. Tick those to adopt, give each its name here if not the store's, and **Adopt**. Those already adopted are marked; views and tables whose metadata can't be read say so. The operator registers them within a minute, and the catalog lists them a little after (until then they show as *coming*). A table's page then says where it's adopted from, with **Take over…** and **Stop adopting** (below).

Or in the project's spec, `tables.adopt`: each table's name in the project, and where it is, as `<store>.<schema>.<table>`:

```
"tables": {
  "adopt": [
    {"table": "orders", "from": "lakehouse.sales.orders"},
    {"table": "customers", "from": "lakehouse.sales.customers"}
  ]
}
```

Field	What it is
`table`	Its name in the project's namespace: lowercase letters, digits and underscores
`from`	Where it is: the store's name (as the administrator set it up), the schema and the table in the store's own catalog

How it behaves

- **Nothing is copied.** The operator asks the store's catalog for the table's current metadata file and registers the table from it in the project's namespace (Nessie's Iceberg REST API). Its data files stay in the store's buckets, which Trino reads there.
- **It follows the other platform.** Each minute the operator looks again; when the store's table has a newer metadata file, the table is registered again from it (for a moment between the two, the table isn't there). The project's history (Nessie's commits) records each step.
- **Read-only here.** Its Ranger policy lets the project's editors, readers and pipelines read it, and no one write, alter or drop it: the other platform writes it, and a write from here would split its history from theirs. The nightly maintenance leaves it alone, and the store's key is a read-only one.
- **Readers' rules hold.** Masks, row filters, `readers.tables` and data products apply to it as to the project's other tables.
- **A table of the project's own by the same name is never replaced:** the project's status shows "the project has a table orders of its own …", and nothing is registered until one of them is renamed.
- **Taken out of the spec,** it's unregistered: gone from the project, its files untouched in the store. A deleted project's adopted tables are unregistered the same way.

Taking a table over

While the other platform writes a table, it stays adopted. Once it stops, a project editor in the table-adoption role takes it over: it's then the project's own, where its files are. Nothing is copied. On the table's page, **Take over…**, then **Take it over** once you've read what it means; or through the API:

```
curl -X POST -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" \
  https://<platform>/api/projects/<project>/tables/<table>/takeover
```

- **Its store must take takeovers.** The administrator gives it a key that may also write its buckets (the chart's `externalStores[].takeover.secret`, Operating); without one the takeover is refused, naming the store.
- **The other platform must have stopped writing it.** Its last change there must be a day old (`TAKEOVER_QUIET`, platformd's setting); before that the takeover is refused, saying when the table last changed. Stop the jobs writing it first.
- **From its newest metadata file:** the project's table is registered from it if the operator hadn't yet, then recorded as taken over (`takenOver`, `takenOverBy` on its `tables.adopt` entry, which only the takeover sets and every save keeps) and in the audit log (`table.takenover`).
- **Then it's one of the project's tables:** its editors and pipelines write it under the project's roles, the nightly maintenance keeps it in shape (compaction, old snapshots and their files expired, in its store's bucket), erasure deletes a person's rows in it, and a retention may be set on it. The operator no longer follows the store.
- **Its writers there are stopped,** where the platform may write the store's own catalog (`externalStores[].catalog.write`): the takeover also takes the table out of it (Iceberg's `unregister_table`, which moves no file), so the other platform's jobs no longer find it. If that fails, nothing is taken over. Without it, stopping them is the other platform's to do.
- **A second writer is an error.** If the other platform writes the table after all (or makes one by its name again), the project's status says so ("the other platform has written it since it was taken over ..."): its change isn't here, and two writers lose each other's changes. Stop its writer there, and copy across what it wrote.
- **It stays.** Taken out of `tables.adopt`, or with the project deleted, it stays as the project's other tables do.
- **Its files stay in the store's bucket.** Copying the table into the platform's own warehouse (`CREATE TABLE … AS SELECT`, then a rename) moves them, if wanted.

Giving a table back

A table taken over can be given back to the platform it came from, where the platform may write that store's catalog: on its page, **Give back...**, then **Give it back** (or `POST /api/projects/<project>/tables/<table>/giveback`). Its project's editors in the table-adoption role.

- **Its current state goes back:** its newest metadata file, everything written here included, is registered in the store's own catalog under its old name (Iceberg's `register_table`; nothing copied), for the other platform's jobs to write again.
- **This project must have stopped writing it:** its last change here must be a day old (`TAKEOVER_QUIET`), and the store must have no table by its name again; otherwise it's refused, saying which.
- **Adopted again:** the project's entry is as before the takeover (no `takenOver`): read-only here, followed, the nightly maintenance off. Stop adopting it if the project doesn't need it any more. In the audit log: `table.givenback`.

Limits and gotchas

- **Read-only, all of it, until it's taken over.** Retention can't be set on an adopted table (the spec says so), and dbt models, flows and pipelines may read it but not replace it.
- **One table per entry;** a whole schema is a list of entries.
- **Erasure** (Privacy) counts a person's rows in an adopted table listed in `tables.people` but can't delete them here: the erasure's record says so, and they're erased on the platform the table is adopted from (gone here within a minute of that). In a table taken over, they're deleted here, as in the project's other tables.
- **The store's credentials are the administrator's**, set once for the install; a project names the store, never a key.

Connections

A connection is a credential the project's editors give the platform once: a token, a database account, a key. The spec names it; members see its name, kind, who added it and when, its values that aren't secret, and where it's used; nobody sees a secret value again.

Who can do it: adding, changing and removing take the permission `connections` (editors). Any member sees the list. Refused: "the project's editors change its connections (it takes the permission connections)".

Set up

1. Open the project → **Connections** → **Add a connection**.
2. **Name**: it must start with the project's name and a dash (`retail-crm-token`), in lower-case letters, digits and dashes, up to 63 characters.
3. **Kind**: pick one; its fields appear (required ones starred). Press **Add**.
4. To rotate a secret, press **New values** on the connection and paste the new one; a secret left empty stays as it was. **Save new values**.
5. **Remove** is greyed out while the spec names it ("Used by ...").

Through the API:

```
curl -X POST "$PLATFORM/api/projects/retail/connections" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"name": "retail-erp-db", "kind": "database",
       "values": {"host": "erp.example.com", "port": "5432", "database": "erp", "username":
         "lake_reader", "password": "…"}}'
```

Route	What
`GET /api/projects/{name}/connections`	Its connections (`values` not secret, `set` the secret keys that have a value, `usedBy`), and every kind's fields
`POST /api/projects/{name}/connections`	Add one: `{name, kind, values}`
`PUT /api/projects/{name}/connections/{connection}`	New values: `{values}` (and `kind`, for one made outside the platform)
`DELETE /api/projects/{name}/connections/{connection}`	Remove one the spec doesn't name

Every kind and its fields

Secret fields (marked †) are never shown again. "One of" means at least one of those keys has a value.

Kind (`kind`)	Label	For	Fields
`git`	A git token	The pipelines' repo (https)	`username` (the token's name); `password`† (token; GitLab: the `api` scope, the Developer role), required; `webhookToken`† (GitLab webhook's token)
`ssh`	An SSH key	The pipelines' repo (ssh)	`sshPrivateKey`†, required, multi-line; `sshPrivateKeyPassphrase`†
`database`	A database user	Sources, copies and sql_database imports	`host`, `port`, `database`, `username` (required), `password`† (required), `url` (a JDBC URL, in place of host, port and database), `sslmode` (PostgreSQL), `trustServerCertificate` (SQL Server: `true`). One of `host`, `url`

Kind (`kind`)	Label	For	Fields
`salesforce`	A Salesforce org	Salesforce imports	`username`, `password`†, `securityToken`†, `consumerKey`†, `consumerSecret`†, `organizationId` (sign-in from trusted IP ranges), `domain` (`login` or `test`), `instanceUrl` (with a session, in place of a user), `sessionId`†, `caCertificate` (multi-line). One of `password`, `sessionId`
`gong`	A Gong account's API key	Gong imports	`accessKey` (required), `accessKeySecret`† (required), `baseUrl` (default `https://api.gong.io`; the company's own, if Gong gave one), `caCertificate` (multi-line)
`sap`	An SAP S/4HANA communication user	SAP imports	`baseUrl` (required: the gateway's address, as `https://sap-gw.example.com:44300`, or S/4HANA Cloud's API address, as `https://my123456-api.s4hana.cloud.sap`), `username` (required), `password`† (required), `client` (the SAP client; empty: the system's default), `caCertificate` (multi-line)
`highspot`	A HighSpot API key	HighSpot imports	`apiKey` (required), `apiSecret`† (required), `baseUrl` (the region's API address, as `https://api-su2.highspot.com`; required), `caCertificate` (multi-line)
`api`	An API token	rest_api imports, and the project's tools (sent as a Bearer)	`token`†, required
`sftp`	An SFTP server	Deliveries' targets	`host` (required), `port`, `username` (required), `password`†, `sshPrivateKey`† (multi-line), `hostKey` (as in known_hosts, required), `path` (folder). One of `password`, `sshPrivateKey`
`s3`	An S3 bucket	Deliveries' targets	`endpoint` (empty: AWS), `region`, `bucket` (required), `prefix`, `accessKey` (required), `secretKey`† (required)
`pgp`	A recipient's PGP key	Deliveries' encryption	`pgpPublicKey` (armored, required, multi-line)
`agent`	An agent's secret	An agent of the project's (made when it's approved)	`clientId`, `secret`† (required)
`teams`	A Teams channel	The project's alerts (`notify.teams`)	`webhook`† (Workflows webhook URL), required
`slack`	A Slack channel	The project's alerts (`notify.slack`)	`webhook`† (incoming webhook URL), required

Deliveries' kinds are chapter 12; the git and ssh kinds chapter 9; the agent's chapter 14.

How it behaves

- On Kubernetes a connection is a Secret in the platform's namespace, labelled `platform.storscale.io/connection=<project>`; in Compose, a file in the platform's bucket (`connections/<project>/<name>.json`). A project names only its own: a Secret of the platform's or another project's is refused ("that name is taken by something that isn't one of the project's connections").
- A change asks the operator to apply the project at once: Airflow's copies of the credentials, Trino's sources' catalogs and the copies' pods follow within a minute.
- The audit log records `connection.saved` (its kind, "added" or "changed", and which keys: never their values) and `connection.removed`.
- Deleting the project removes its connections.

Limits and gotchas

- The name rule: "a connection's name starts with the project's and a dash (retail-dags), in lower-case letters, digits and dashes". (Lab 3 of the lab guide names its database connection `dashboards-db`; on the project `retail` it must be `retail-dashboards-db`.)
- A name that's taken: "the project has a connection retail-erp-db already: give it new values".
- A key the kind hasn't: "token: no such key for A database user". A missing required field: "Password is needed". Neither of a one-of: "one of host, url is needed".
- A value is at most 64 KB.
- Removing one the spec names is refused, naming its uses: "still used by source finance, import erp". The spec's uses counted are the pipelines' repo, `notify.teams`, sources, imports, deliveries (target and PGP key) and tools. A Slack connection named by `notify.slack` isn't counted: removing it is allowed, and the project's Slack alerts then fail.
- A connection made outside the platform shows as "made outside the platform"; give it its kind with **New values**.

What the audit log records

Action	When	Detail
`upload`	A file uploaded	Its name and size
`upload.removed`	An upload removed	Its name
`import.saved`	An import added or changed (through the API or a form)	"added: files", the objects for Salesforce, Gong, HighSpot and SAP, and whether pipelines were given
`import.removed`	An import removed	"its tables stay"
`import.reset`	A reset asked for	"its tables and where it got to dropped, all copied again"
`pipeline.started`	**Run now**	The run's ID
`connection.saved`	A connection added or given new values	Its kind and the keys changed
`connection.removed`	A connection removed	
`project.saved`	The spec saved on the Edit page	Which parts changed

The project's **Audit** tab shows them (permission `see-audit`); see chapter 17.

4. Access and protection

This chapter is for whoever decides who sees what of a project's tables: the project's owner, the administrator who edits its spec, and the privacy officer who sets what a tag of personal data means across the install. It covers the readers' rules (which tables readers may read, which rows, and what of each column), the declaration of which tables hold people, the protection tags the catalog suggests and people confirm or remove, the install's rules for those tags, and what each kind of person ends up seeing: editors, readers, roles of the company's own, the project's pipelines and the projects that consume its data products. Every rule here is applied by Apache Ranger in Trino, so it holds wherever a person's query goes through Trino: the SQL tab, the Data tab's preview, dashboards, notebooks, the canvas's previews, apps and assistants. Roles themselves are chapter 16; erasure and the rest of privacy, chapter 13.

In this chapter:

- Who sees what
- Tables readers may read
- Row filters for readers
- Column masks for readers
- Mask types
- People's data (tables.people)
- Protection tags and the classifier
- Confirming and removing a tag
- The install's protection rules
- Seeing the rules: the Access tab and others
- What the audit log records

Who sees what

A project's members are Keycloak groups (or named people), each with a role (chapter 2). The operator puts each role's people in a Keycloak group of the project's (`<project>-editors`, `<project>-readers`, `<project>-role-<name>`), and Ranger's policies name those groups. How a group reads the tables depends on its role's permissions: `write-tables` writes them and reads them all as they are; `read-all` reads them all as they are; `read` reads them under the readers' rules.

Who	Tables	Readers' row filters and masks	Protection tags' masks
Editors, and roles with `write-tables`	All, and may make, change and drop them	None	None: they see the column as it is
Roles with `read-all` (and not `write-tables`)	All, read only	None	None
Readers, and roles with `read` (and not `read-all`)	Those readers may read (default all)	Applied	Masked, unless the install excepts one of their groups
The project's **pipelines** (`<project>-pipelines`): its imports, deliveries, scorings and DAGs	All, read and write	None	None

Who	Tables	Readers' row filters and masks	Protection tags' masks
A project **consuming** one of its data products: that project's readers, editors, pipelines and roles that read	The product's table only	Applied, as for readers	Masked, unless excepted
The catalog's own Trino account (`openmetadata`): its checks, profiles and samples	All, read only	None	As they are (tagged columns aren't profiled)
Anyone else	Nothing of the project's		

An **editor is never filtered or masked by the readers' rules, whatever other groups they're in.** alice can be an editor of `retail` and also an analyst, in a group that reads `retail` as a reader or that consumes its data product: in the readers' filter and mask policies the project's editors and read-all roles come first, unfiltered and unmasked, and Ranger takes the first item a person matches. Readers and consuming projects get the rules.

Someone given several roles in one project (a group as reader, and themselves as editor) has the one that gives the most.

Administrators may do every platform action in every project (the Edit page, confirming tags, muting alerts). What they read in Trino is what their memberships give, like anyone else.

The rules reach a table read as it was, too: `uploads_orders@close-2026-09` and `uploads_orders#<commit>` are under the same policies as `uploads_orders` (chapter 15).

There's no way around them to the tables themselves. Nobody but the platform's engines reads the warehouse's files, and the table catalog (Nessie) answers the platform's own services alone (Trino, platformd, the operator, the copies, the imports): not people, their notebooks or the pipelines' code. A notebook's Distributed Spark gets the catalog through platformd, for the tables its person reads whole and nothing else (chapter 10).

Tables readers may read

By default a project's readers may read every table in its namespace. Name some, and they may read only those; the rest are the editors' alone.

Who can do it: administrators (the spec, on the Edit page).

Set up

On the project's Edit page (Overview → **Edit project**), under **Tables**, **Tables readers may read**: a comma-separated list, empty for "all of them". In the spec it's `tables.readers.tables`:

```
{
  "tables": {
    "readers": {
      "tables": ["uploads_orders", "uploads_customers", "uploads_campaigns"]
    }
  }
}
```

How it behaves

- Ranger's policy on the namespace lets readers see it; a policy per listed table (`project:<name>:read:<table>`) lets them read it. A table not listed is refused in Trino: on the SQL tab the query fails with Trino's "Access Denied".

- A new table made later is the editors' alone until it's listed.
- On the Data tab every member sees every table's name; a table readers can't read has a small "editors" beside it, and opened, the banner says "Only editors can read it." (to a reader: "You see this table as a reader.").
- A data product's table is readable by its consumers whether or not it's listed (chapter 11).

Limits and gotchas

- Each name must be a table's: "readers: table "Orders" isn't an identifier".
- The list is of names; it doesn't check that the tables exist. A name misspelt shows nothing to readers.

See Lab 13 in the lab guide.

Row filters for readers

A row filter is a SQL condition on a table: readers see only the rows it holds for.

Who can do it: administrators (the spec).

Set up

On the Edit page, **Row filters for readers** → **Add a row filter**: the table, and the condition (`region = 'EU'`). In the spec, `tables.readers.rowFilters`:

```
{
  "tables": {
    "readers": {
      "rowFilters": [
        {"table": "uploads_orders", "filter": "region = 'EU'"},
        {"table": "uploads_campaign_responses", "filter": "responded_at >= DATE '2026-01-01'"}
      ]
    }
  }
}
```

How it behaves

- Ranger adds the condition to every reader's query of the table, in Trino: `SELECT count(*) FROM uploads_orders` gives alice the EU's rows, about a third of bob's 305.
- The project's editors and read-all roles have an item of their own in the same policy with no filter at all (an empty one): Trino refuses to update a table under any row filter, even `true`, so an editor can still insert, update and delete.
- Consuming projects of a data product on the table get the same filter.
- The project's pipelines aren't named in it: they read every row.

Limits and gotchas

- One filter per table: Ranger keeps one row-filter policy per table. Combine conditions with `AND` and `OR`.
- The condition is Trino SQL on the table's own columns. A condition that doesn't compile fails every reader's query of the table, not the save.
- "readers: row filter on "uploads_orders" needs a table and a condition".

See Lab 2 and Lab 13.

Column masks for readers

A mask shows readers less of a column: its last four characters, a hash, the year of a date, nothing.

Who can do it: administrators (the spec).

Set up

On the Edit page, **Column masks for readers** → **Add a mask**: the table, the column, and the type. In the spec, `tables.readers.masks`:

```
{
  "tables": {
    "readers": {
      "masks": [
        {"table": "uploads_customers", "column": "signed_up", "type": "MASK_DATE_SHOW_YEAR"},
        {"table": "uploads_orders", "column": "card_number", "type": "MASK_SHOW_LAST_4"}
      ]
    }
  }
}
```

The three parts together, as they sit in a project's spec:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "description": "Harbor Outfitters: orders, customers, campaigns",
    "members": [
      {"group": "engineers", "role": "editor"},
      {"group": "analysts", "role": "reader"}
    ],
    "tables": {
      "readers": {
        "tables": ["uploads_orders", "uploads_customers", "uploads_campaigns"],
        "rowFilters": [{"table": "uploads_orders", "filter": "region = 'EU'"}],
        "masks": [{"table": "uploads_customers", "column": "signed_up", "type":
          "MASK_DATE_SHOW_YEAR"}]
      },
      "people": [
        {"table": "uploads_customers", "column": "customer_id"},
        {"table": "uploads_orders", "column": "customer_id"}
      ]
    },
    "files": {},
    "pipelines": {"serviceAccount": "airflow-pipelines"}
  }
}
```

How it behaves

- Ranger masks the column for readers (and consuming projects) in every query. The Data tab's **Readers see** column says what they get ("year only", "last 4 only"), and a reader's **Preview** shows it: `2026-01-01` for every `signed_up`.
- Editors and read-all roles come first in the mask's policy with no mask (`MASK_NONE`).

- A project's own mask and the install's protection tags add up: on a column the project masks and the catalog tags, the project's readers (and consuming projects) get the project's mask (the last four digits of a card, say), everyone else the tag's, and the tag's excepted groups see it as it is. Other tagged columns keep the tags' masks.

Limits and gotchas

- One mask per column.
- **A project's own masks are written for a text column.** `MASK`, `MASK_SHOW_LAST_4`, `MASK_SHOW_FIRST_4` and `MASK_HASH` give text back, and Trino refuses a mask that changes a number's or a date's type ("Invalid column mask"), for every reader's query of the column. On a number or a date use `MASK_NULL` (or `MASK_DATE_SHOW_YEAR` on a date), or let a protection tag do it: a tag's mask knows the column's type (below).
- **A project's own mask can't lift a tag's.** A readers' mask of `MASK_NONE` on a column the catalog tagged personal gives readers the tag's hash, not the values. Any other mask of the project's stands for its readers, even one that shows more than the tag's (`MASK_SHOW_LAST_4` on a card number the catalog tagged financial): it's the project's choice for its own readers, and the Governance page lists each project's masks. The Data tab's **Readers see** describes the project's own mask, and the **Protection** column the tag.
- "readers: mask type "HASH" must be one of MASK, MASK_SHOW_LAST_4, MASK_SHOW_FIRST_4, MASK_HASH, MASK_NULL, MASK_NONE, MASK_DATE_SHOW_YEAR".

Mask types

Type	The Data tab says	What a reader gets, as a project's mask	As a protection tag's mask (it knows the column's type)
`MASK`	masked	Ranger's redaction of the text	Text: redacted. Any other type: null
`MASK_SHOW_LAST_4`	last 4 only	The last four characters; the rest redacted	Text: so. Any other type: null
`MASK_SHOW_FIRST_4`	first 4 only	The first four characters; the rest redacted	Text: so. Any other type: null
`MASK_HASH`	hashed	SHA-256 of the text, as 64 hex characters	Text: the same. `bigint` and `integer`: a number of the same type, hashed. Any other type (a date, a decimal): null
`MASK_NULL`	hidden	Null	Null
`MASK_NONE`	shown	The value as it is	The value as it is
`MASK_DATE_SHOW_YEAR`	year only	The date's year (January the first of it)	The same

A number hashed to a number keeps its nulls, its duplicates and its joins: `customer_id` hashed in `uploads_customers` and in `uploads_orders` still joins, and the catalog's checks on it (not null, unique, references) still answer truly. Its values don't show. Before 0.31.0 such a column was nulled.

People's data (tables.people)

`tables.people` says which tables hold people's data and which column identifies a person in each. That column is personal from the moment it's declared: the catalog tags it `personal`, confirmed. The same declaration is what a delivery of those tables needs a data-protection approver for (chapter 12), what a search of what was sent about a person looks in, and what an erasure reaches (chapter 13).

Who can do it: administrators (the spec, in the Edit page's JSON: the form has no field for it).

Set up

```
{
  "tables": {
    "people": [
      {"table": "uploads_customers", "column": "customer_id"},
      {"table": "uploads_orders", "column": "customer_id"}
    ]
  }
}
```

How it behaves

- At its next round (every two minutes) the classifier tags each declared column `personal`, confirmed, in the catalog. The Data tab's **Protection** column shows it as `personal` with no **remove** beside it ("declared by the project").
- From then on the column is masked as the install's rule for `personal` says: on a local install hashed, for everyone but the engineers. A `bigint` ID is hashed to a `bigint`.

Limits and gotchas

- One entry per table: "tables: people: uploads_orders twice".
- "tables: people: "uploads_customers".": a table and its column identifying a person".
- To take a declaration back, remove it from the spec. Removing the tag on the Data page doesn't: the next round tags it again.

See Lab 2 and Lab 4.

Protection tags and the classifier

Nobody writes a policy for a column of personal data that arrives in a new import. The catalog finds it, tags it, and the platform masks it at once; the project's owner then confirms the tag or removes it.

The catalog (OpenMetadata) has a classification called **Protection** with three tags:

Tag	What it's for
`personal`	Identifies a person, or is about one: names, contact details, identifiers, dates of birth
`financial`	A person's money: salaries, balances, card and account numbers' amounts
`health`	A person's health: diagnoses, prescriptions, conditions

A tag on a column is **suggested** (found by the catalog) or **confirmed** (by a person, or declared by the project).

Who can do it: the classifier runs by itself. Confirming and removing take the permission `classify` (editors; administrators).

How the classifier decides

Every two minutes catalog-sync looks at every project's tables, as the catalog knows them:

1. **Declared:** each column in `tables.people` is tagged `personal`, confirmed.
2. **By name:** a column (of any type) whose name matches is tagged, suggested.
   - `personal`: `email`, `e_mail`, `phone`, `mobile`, `telephone`, `ssn`, `social_security`, `passport`, `national_id`, `tax_id`, `iban`, `card_number`, `cc_number`, `credit_card`, `date_of_birth`, `dob`, `birth_date`, `birthdate`, `first_name`, `last_name`, `surname`, `full_name`, `home_address`, `street`, `postal_code`, `zip_code`
   - `financial`: `salary`, `wage`, `income`, `net_worth`, `credit_score`, `account_balance`, `balance`
   - `health`: `diagnosis` (and words starting `diagnos`), `icd10`, `icd_10`, `medical`, `prescription`, `health_condition`, `condition`

   The word may be the whole name or a part between underscores: `customer_email`, `phone_2`.
3. **By value:** each text column not yet tagged is sampled: its first 200 rows, read from Trino as the catalog's own account. When at least 3 values are there and 60% of them look like one of these, it's tagged `personal`, suggested:

Looks like	Pattern
An email address	`name@domain.tld`
A card number	13 to 19 digits (spaces or dashes allowed) passing the Luhn check
A national ID	`123-45-6789`
A bank account	An IBAN
A phone number	9 to 20 characters of digits, spaces, `( ) . -`, an optional leading `+`, at least 8 digits; **not** a date or a time written as text (`2026-05-03`, `2026-05-03 10:00`) and not a plain decimal number (`1792780188.63`)

4. **Never:** the import's own columns, `_dlt_load_id`, `_dlt_id` and the rest starting `_dlt_`, are never tagged. A column that already has a Protection tag isn't looked at again. A suggestion someone removed isn't made again.
5. What's tagged now (declared, suggested, confirmed, with each column's type) is written to the platform's bucket, `classification/<project>.json`. The operator reads it at its next pass over the project (each project at least once a minute) and makes a Ranger mask policy for each tagged column: `project:<name>:protect:<table>.<column>`.

How it behaves

- **A suggested tag protects at once**, before anyone confirms it (the install's `applySuggested`, true by default).
- The project's Overview lists each suggested tag as an alert, "uploads_customers.email looks personal: the catalog tagged it, and it's masked", with "Confirm the tag, or remove it if the column isn't personal, on the Data page." It goes to the project's channels like any alert (chapter 5) and clears once the tag is confirmed or removed.
- The Data tab's **Protection** column shows each column's tags: `personal (suggested)` in blue, `personal` (confirmed) in amber; hovering says "declared by the project", "found by the catalog" or "tagged in the catalog".
- A column with two tags is masked by the strongest: `health`, then `financial`, then `personal`.
- The mask follows the column's type as the catalog has it (see Mask types).
- Who sees a tagged column as it is: the project's editors, its read-all roles and its pipelines (they own and write the data, and Trino refuses a write to a table the writer is masked on), the platform's maintenance account (erasure, retention), and the groups the install's rule excepts. Everyone else who can read the table sees it masked: readers, roles with `read`, consuming projects, and the catalog's own account.

- The tag's mask reaches every tool that reads through Trino as the person: the SQL tab, previews, dashboards, notebooks, apps, assistants. The project's pipelines read the column as it is, so a delivery of it sends it as it is: that's why a delivery of personal columns needs a data-protection approver (chapter 12).

Settings

The classifier's timings are catalog-sync's own environment, not chart values: it ingests the catalog's metadata every 60 seconds (`METADATA_EVERY`) and classifies every 120 (`CLASSIFY_EVERY`). A new table is in the catalog within a minute or so and classified within two more; its mask follows within a minute of that. What the tags mean is the install's: below.

Limits and gotchas

- Only `personal` is found by value. `financial` and `health` are found by a column's name only.
- Free text that mentions a person (a review signed with an email address) isn't found unless 60% of the sampled values look like one: the classifier tags columns, not sentences.
- The sample is the table's first 200 rows, not a random 200.
- The catalog's own account sees a tagged column as it is, so the checks test its values (chapter 5); the profiler leaves tagged columns out, so their values' ranges aren't in the catalog.
- catalog-sync reads up to 200 tables of a project's namespace each round.

See Lab 4 in the lab guide.

Confirming and removing a tag

Who can do it: the permission `classify` (editors; administrators). Refused: "the project's editors confirm or remove a column's tags".

Set up

On **Data**, open the table. In the **Protection** column:

- **confirm** beside a suggested tag: the tag stays, whatever the catalog finds later.
- **remove** beside a tag that isn't declared: the tag goes, and the catalog doesn't suggest it again. Within a minute the column is read as it is.

The catalog's own page for the table (Tools → Catalog) shows the same tags under **Protection**, and is another place to confirm or correct them.

Through the API:

```
curl -X POST "$PLATFORM/api/projects/retail/classification" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"table": "uploads_customers", "column": "email", "tag": "personal", "action": "confirm"}'
```

The answer is `{"state": "confirmed"}` or `{"state": "removed"}` (`"absent"` when the tag to remove wasn't there). `confirm` on a tag the column doesn't have adds it, confirmed: that's how a person tags a column the classifier missed. `GET /api/projects/{name}/classification` lists the project's tagged columns (`table`, `column`, `tag`, `state`, `type`, `declared`, `automated`), with when catalog-sync last wrote them (`at`) and the tags there are.

How it behaves

- The platform changes the tag in the catalog (as its own account), and the project's view at once; catalog-sync's next round writes the file again, and the operator's next pass changes Ranger's policy.
- The audit log records `classification.confirmed` or `classification.removed`, with the table, column and tag; a refused attempt is recorded too.

Limits and gotchas

- "the catalog has no column uploads_customers.mail": the table or column isn't in the catalog yet, or is misspelt.
- "a table, a column and a tag, each an identifier"; "the action is confirm or remove".
- A declared column has no **remove** on the Data page; removing its tag through the API lasts only until the next round. Take it out of `tables.people` instead.
- A tag must be one of the Protection classification's: `personal`, `financial`, `health`.

The install's protection rules

What each tag means is set once for the install: the mask, and the Keycloak groups that see the column as it is. Project owners don't write masks for personal data.

Who can do it: whoever installs the platform (the chart's values, or Compose's environment).

Set up

On Kubernetes, the chart's `protection` values:

```
protection:
  applySuggested: true
  tags:
    personal: {mask: MASK_HASH, except: [data-protection]}
    financial: {mask: MASK, except: []}
    health: {mask: MASK_NULL, except: []}
```

In Compose, `PROTECTION_TAGS` on both `platformd` and `platform-operator` in `stack/compose.yaml`, as JSON; a local install excepts the engineers:

```
PROTECTION_TAGS: '{"personal": {"mask": "MASK_HASH", "except": ["engineers"]}, "financial": {"mask":
  "MASK", "except": ["engineers"]}, "health": {"mask": "MASK_NULL", "except": ["engineers"]}}'
```

`PROTECTION_APPLY_SUGGESTED` (default `true`; `false`: confirmed tags only) is the chart's `applySuggested`.

Setting	Default	What
`protection.tags.personal`	`{mask: MASK_HASH, except: []}`	Hashed, for everyone
`protection.tags.financial`	`{mask: MASK, except: []}`	Redacted, for everyone
`protection.tags.health`	`{mask: MASK_NULL, except: []}`	Null, for everyone
`protection.applySuggested`	`true`	A suggested tag protects before anyone confirms it

A tag given replaces the default's rule for it; a tag left out keeps the default; a rule without a mask is `MASK`.

How it behaves

- The operator reads the rules and makes each tagged column's mask policy from them; platformd shows them on the Governance page ("What readers see": each tag, its mask and who's excepted, and whether suggested tags count).
- Changing them takes a restart of both (`storscale upgrade` with the new values), then each project's next pass.

Limits and gotchas

- A bad value stops platformd and the operator starting: "PROTECTION_TAGS: "Personal" isn't a tag's name", "PROTECTION_TAGS: personal: mask "HASH" isn't one of MASK, MASK_SHOW_LAST_4, ...".
- An excepted group sees the column as it is only where it may read the table at all.
- The catalog has three tags. A rule for a tag of another name has nothing in the catalog to apply to.

Seeing the rules: the Access tab and others

Who can do it: any member sees a project's Access tab; administrators see Administration → Access.

- **Access** (a project's tab): **Members**, each person, their role and through which Keycloak group (or "named in the project"), with **Change members** for administrators; "Someone who joins one of those groups in Keycloak joins the project within a minute." **What readers don't see**: the tables they may read, each row filter ("Only rows where ...") and each mask. Beside them, a card per role saying in words what it gets in each system ("Read uploads_orders, uploads_customers, uploads_campaigns in iceberg.retail", "uploads_orders: only rows where region = 'EU'"), your own role marked "you", and a card for each role of the company's own the project gives.
- **Data**: above a table's columns, a banner when readers see less: "Readers see less of this table." to an editor, "You see this table as a reader." to a reader, with each filter and mask. The **Readers see** column says it per column; **Protection** shows the tags.
- **The Edit page** shows "What each role will get" before you save.
- **Administration → Access** (administrators; `GET /api/access`): every person, and their role in each project.
- **Governance** (administrators, product certifiers and data-protection approvers): every project's readers' rules, declared people's tables and tagged columns, and the install's tag rules (chapter 17).
- **Ranger**: each project's policies are named `project:<name>:…` and say "From the project <name>: the platform operator keeps it as the project says." The operator puts back anything changed by hand.

What the audit log records

Action	When
`project.saved`	The spec saved (readers' rules and people's tables are in it), with which parts changed
`classification.confirmed`, `classification.removed`	A tag confirmed or removed, with the table, column and tag
A refusal of `classification.changed`	Someone without `classify` tried

Every query Trino allowed or refused is in the audit log too, with the Ranger policy that decided it: the project's **Audit** tab shows the accesses to its tables (chapter 17). See Lab 21.

5. Data quality

This chapter is for a table's owner: the engineer or analyst who wants a table to say when it's wrong or late before a dashboard reader notices. It covers the checks a project declares on its tables (all eight kinds), freshness targets, the profile the catalog keeps of every table, where the results show (the Quality view, the Overview, the catalog), the project's alerts and the channels they go to (Teams, Slack, email), and muting one for a while. Where the data comes from is chapter 3; a data product's contract, which promises the same checks and freshness to its consumers, is chapter 11.

In this chapter:

- What the platform watches
- Checks
- The eight kinds of check
- Freshness
- Profiling: what a table holds
- The Quality view
- Project alerts
- Where alerts go
- Muting
- Settings
- What the audit log records

What the platform watches

What	Declared in	Run by	How often	Shows on
Checks	`tables.checks`	catalog-sync, as OpenMetadata tests	Every round of catalog-sync (about every 20 seconds plus the round's work)	Data → Quality; Overview; the catalog's table page
Freshness	`tables.freshness`	platformd	Every minute	Overview's Alerts
Profile	Nothing: every table	catalog-sync, OpenMetadata's profiler	A table with no profile, or one over an hour old	Data → Columns; the catalog's table page
Project alerts	`notify` says where	platformd	Every minute	Overview's Alerts; Teams, Slack, email

See Lab 5 in the lab guide.

Checks

A check is a test on one of the project's tables: a column with no nulls, a row count in a range, values from a list, a query that must return nothing. The catalog runs each one, keeps its result on the table, and the project hears when one fails and when it passes again.

Who can do it: checks are in the spec, which administrators change (the Edit page's **JSON**: checks have no form of their own yet). Anyone who may see the project sees the results.

Set up

On the project's Edit page (Overview → **Edit project**), press **JSON**, and under `spec.tables` add `checks`:

```
{
  "tables": {
    "checks": [
      {"table": "uploads_orders", "check": "notNull", "column": "customer_id"},
      {"table": "uploads_orders", "check": "rowCount", "min": 300},
      {"table": "uploads_customers", "check": "unique", "column": "email"},
      {"table": "uploads_orders", "check": "acceptedValues", "column": "region", "values": ["EU",
        "US", "APAC"]},
      {"table": "uploads_orders", "check": "range", "column": "amount", "min": 0, "max": 10000},
      {"table": "uploads_customers", "check": "regex", "column": "region", "pattern":
        "^(EU|US|APAC)$"},
      {"table": "uploads_orders", "check": "references", "column": "customer_id", "refTable":
        "uploads_customers", "refColumn": "customer_id"},
      {"table": "uploads_orders", "check": "sql", "name": "no_refund_without_amount", "sql": "SELECT
        order_id FROM uploads_orders WHERE refunded AND amount <= 0"}
    ]
  }
}
```

Press **Save**. Through the API the whole project goes: `PUT /api/projects/{name}` with the project's JSON (as `GET /api/projects/{name}` gives its `spec`).

Each check has:

Field	What
`table`	One of the project's tables
`check`	`unique`, `notNull`, `rowCount`, `acceptedValues`, `range`, `regex`, `references` or `sql`
`column`	The column, for all but `rowCount` and `sql`
`min`, `max`	`rowCount`: rows. `range`: the column's values. Whole numbers; one or both
`values`	`acceptedValues`: what the column may hold
`pattern`	`regex`: what every value matches
`refTable`, `refColumn`	`references`: the table and column every value is found in
`sql`	`sql`: a query that returns no rows when all is well
`name`	Optional; a `sql` check needs one. Lowercase letters, digits and underscores

How it behaves

- catalog-sync turns each check into an OpenMetadata test case on the table and runs the table's tests every round. The first result comes within a minute or two of the save, once the catalog has the table.
- Each test is named `<project>_<table>_<name or column>_<kind>` (a `regex` check ends in `_matches`): `retail_uploads_orders_customer_id_notNull`, `retail_uploads_orders_rowCount`, `retail_uploads_orders_no_refund_without_amount_sql`. The platform's pages drop the `<project>_<table>_` part.
- A check taken out of the spec is deleted from the catalog, with its results, at the next round.
- Checks run as the catalog's own Trino account: every row, every column as it is, tagged ones too (chapter 4).

- A failing check is an issue on the project (below) until it passes; its channels hear when it starts failing and when it passes again.

Limits and gotchas

- Checks run on the table as it is now; they don't stop a pipeline's run or a write. A dbt model's own tests, which do, are chapter 9.
- Two checks of the same kind on the same column of a table get the same name; give one a `name`.
- A check's failing rows aren't shown on the platform's pages, only its counts; the catalog's own page for the test case has what OpenMetadata keeps.
- `min` and `max` are whole numbers: a `range` with `9.99` is refused when the spec is read ("not a project: json: cannot unmarshal number 9.99 ...").
- The refused words, by kind:
  - "check 3: unique needs a column"
  - "check 2: rowCount needs a min, a max, or both"
  - "check 4: acceptedValues needs the values"
  - "check 6: regex needs a pattern"
  - "check 7: references needs refTable and refColumn (the table and column every value is found in)"
  - "check 8: sql needs the query (one that returns no rows when all is well)"
  - "check 8: a sql check needs a name (lowercase letters, digits and underscores)"
  - "check 1: "notnull" must be one of unique, notNull, rowCount, acceptedValues, range, regex, references, sql"

The eight kinds of check

Kind	Passes when	OpenMetadata's test
`unique`	No value of `column` appears twice	`columnValuesToBeUnique`
`notNull`	`column` has no nulls. A failure says how many: "nullCount=1"	`columnValuesToBeNotNull`
`rowCount`	The table's rows are at least `min` and at most `max`	`tableRowCountToBeBetween`
`acceptedValues`	Every value of `column` is one of `values`	`columnValuesToBeInSet`
`range`	Every value of `column` is at least `min` and at most `max`	`columnValuesToBeBetween`
`regex`	Every value of `column` that isn't null matches `pattern` (Trino's `regexp_like`, on the value as text)	A query of the values that don't match, which must return none
`references`	Every value of `column` that isn't null is found in `refTable.refColumn`	A query of the values the other table lacks, which must return none
`sql`	Your `sql` returns no rows	Your query, which must return none

A `regex` pattern is matched anywhere in the value unless anchored: `^(EU|US|APAC)$` for the whole of it.

A `references` check's other table is one of the same project's tables.

A `sql` check's query may name the project's tables bare, as on the SQL tab: `FROM uploads_orders` and `JOIN uploads_customers` are read as the project's. Write it to return the rows that are wrong:

```
SELECT o.order_id
FROM uploads_orders o
JOIN uploads_customers c ON c.customer_id = o.customer_id
WHERE o.ordered_on < c.signed_up
```

Only names after `FROM` and `JOIN` are qualified; a table named elsewhere (after a comma, say) must be written in full, `iceberg.retail.uploads_customers`.

Freshness

A freshness target says how recently a table must have changed. A table older than that is stale, and the project hears of it.

Who can do it: administrators (the Edit page).

Set up

On the Edit page's form, under **Alerts**, **Tables that must change in time** → **Add a table**: the table, and the window (`24h`). In the spec, `tables.freshness`:

```
{
  "tables": {
    "freshness": [
      {"table": "uploads_orders", "within": "24h"},
      {"table": "crm_leads", "within": "2h"}
    ]
  }
}
```

`within` is a number and a unit: `30m`, `24h`, `7d` (`m` minutes, `h` hours, `d` days).

How it behaves

- Every minute platformd asks Nessie when each table's current snapshot was made. Past the window, the project has an issue: "uploads_orders is stale: not changed within 24h", with "last changed 2026-10-23 06:00 UTC" (or "never changed", for a table with no snapshot yet). It shows amber on the Overview, and the channels hear of it as a warning.
- When the table changes, the issue clears, and the channels are told it's resolved.
- A table that can't be looked at (missing, or Nessie not answering) shows on the Overview's Alerts as "Can't tell when uploads_orders last changed: ...".

Limits and gotchas

- Anything that commits a new snapshot counts as a change, whether or not the rows differ: the nightly compaction when it rewrites the table's files, and an import that replaces its table on every run (a files import without a primary key) can keep a table looking fresh. Set the window to what the business expects, and check the source too.
- "tables: freshness: within "1 day": as 30m, 24h or 7d".
- Freshness isn't shown on the Data tab or in the catalog yet: it's an alert.
- A data product's contract promises a freshness of its own to its consumers (chapter 11).

Profiling: what a table holds

The catalog profiles every table of every project: its row count, and each column's share of nulls, how many distinct values, the smallest and the largest.

Who can do it: nothing to set up. Every member sees the profile.

How it behaves

- catalog-sync profiles a table that has no profile, or one over an hour old, with OpenMetadata's profiler, on the whole table, as the catalog's Trino account. It takes a few of a project's tables each round (three), so after a restart the profiles fill in over a few minutes.
- **Data** → a table → **Columns**: above the columns, "305 rows, profiled 12 minutes ago."; the **Holds** column says what each holds: "no nulls · 3 distinct · 2 to 4 characters", "5% null · 280 distinct · 3.5 to 9999". For a text column the smallest and largest are its values' lengths.
- The catalog's own page for the table shows the same.

Limits and gotchas

- The profile is the whole table's, unfiltered, so the Data tab leaves it out for someone who reads under the readers' rules where the rules hide something: no row count or column figures for a table with a row filter, none for a column with a mask. A tagged column isn't profiled at all. The catalog's own pages show the profile to whoever the catalog lets see the table.
- A table just made has no profile until the next round reaches it ("–" in **Holds**).

The Quality view

Who can do it: any member.

- **Data** → a table → **Quality**: each check, its status (**passing**, **failing**, **aborted**, or **not run yet**), its last result ("nullCount=1") and when it ran. The tab shows a red count of the failing ones; the list of tables shows a red dot beside a table with one.
- **Overview**: a red banner while any check fails ("uploads_orders: customer_id_notNull is failing." with its result, and **See the check**); the **Checks** tile counts them ("6/7", "1 failing", "all passing", "none yet").
- **The catalog** (Tools → Catalog): the table's page shows the same tests and their results.
- `GET /api/projects/{name}/flow` gives every table's checks, with `status` (`Success`, `Failed`, `Aborted`, or empty: not run yet), `result` and `when`.

Project alerts

platformd looks at every project each minute and keeps a list of what's wrong in it now: its issues. The Overview's **Alerts** card lists them; the project's channels are told as each starts and as it comes right.

Who can do it: every member sees the list; the channels are set by administrators (the Edit page); muting takes `mute-alerts`.

Kind	The issue says	Told as	Resolved told
A pipeline's latest run failed	"import_retail_uploads's latest run failed", the run's ID	down	No: the next good run ends it quietly
A check failing	"uploads_orders: the check retail_uploads_orders_customer_id_notNull is failing", its result	down	Yes
A stale table	"uploads_orders is stale: not changed within 24h"	warning	Yes
A column the catalog tagged	"uploads_customers.email looks personal: the catalog tagged it, and it's masked" (chapter 4)	down	Yes
A table's maintenance failed	"uploads_orders: its maintenance failed", the error (chapter 15)	warning	Yes
A copy not running for 10 minutes	"the copy of dashboards isn't running (since 09:12 UTC)" (chapter 3)	down	Yes
The bucket filling	"the bucket retail is 80% full (...)", then "... is full (...): new files are refused" (chapter 17)	warning, then down	Yes
A data product off its contract	"the product orders breaks its contract: ..." (chapter 11)	down	Yes
The LLM budget, the monthly cost budget, a scoring's drift	As chapter 14, chapter 17 and chapter 10 say		Yes

Each issue links to where to look (the pipeline's runs, the table's Quality tab). `GET /api/projects/{name}/alerts` gives the list (`issues`, each with `key`, `kind`, `title`, `detail`, `link` and `mutedUntil`), where the project is told (`notify`), and what couldn't be looked at (`unknown`). An agent of the project reads the same list at `GET /api/agent/projects/{name}/alerts` (chapter 14).

How it behaves

- An issue is told once, when it starts. While it lasts it isn't told again.
- A check that passes again, a table that changes, and the rest of the kinds marked above are told as **resolved**.
- When platformd starts, what's wrong already isn't told: its first look only learns the list.
- When the project's `notify` changes (a channel named, an address added), everything wrong now is told at once to all its channels.
- A pipeline's issue is its latest run: a new run that succeeds ends it.

Where alerts go

Who can do it: administrators (the Edit page). The Teams and Slack connections are given by anyone with `connections` (chapter 3).

Set up

1. On the project's **Connections**, add one of kind **A Teams channel** (`webhook`: the channel's Workflows webhook URL), or **A Slack channel** (`webhook`: an incoming webhook URL), named `<project>-...`.
2. On the Edit page's form, under **Alerts**: **Teams channel** (the connection's name), **Slack channel** (the connection's name), **Email** (addresses, comma-separated). In the spec, `notify`:

```
{
  "notify": {
    "teams": "retail-alerts",
    "slack": "retail-alerts-slack",
    "email": ["retail-data@harbor.example"]
  }
}
```

3 Save. The Overview's Alerts card ends "Told to its Teams channel and retail-data@harbor.example."; with nothing set, "Not told anywhere yet: the platform's administrators set where, when they edit the project."

How it behaves

- Teams gets an Adaptive Card; Slack a message with the title in bold, each fact on a line and the link; email a message through the install's mail server. Each says "<project>: <issue>" (or "<project>: resolved: <issue>"), the project and the detail, and links to the platform ("Open it in the platform").
- Email goes through the install's mail server, set in the platform's Secret `storscale-alerts` (`smtpServer`, `smtpFrom`, `smtpUser`, `smtpPassword`; chapter 17). Without it, the addresses get nothing.
- An alert that can't be sent (a webhook refused, a connection missing) is in platformd's log ("project alert not sent"), not on the page.

Limits and gotchas

- `notify.teams` and `notify.slack` must be a connection's name ("notify: teams "Retail Alerts" isn't a connection's name"); each address an address ("notify: "retail-data" isn't an email address"). The spec doesn't check the connection exists: a misspelt one fails when there's something to tell.
- A connection named by `notify.teams` or `notify.slack` can't be removed while named: "still used by alerts (notify.slack)".
- Not yet: the failing task and its error in a pipeline's alert (the run is named and linked), and a check's failing rows.

Muting

Muting stops one issue's alerts for a while: while it's being fixed, or while a source is known to be down.

Who can do it: the permission `mute-alerts` (editors; administrators). Refused: "the project's editors mute its alerts".

Set up

On the Overview's **Alerts** card, **Mute** beside an issue mutes it for a day ("No alerts about this for a day"); it then says "Muted until ..." and offers **Unmute**. Through the API, for up to a week:

```
curl -X POST "$PLATFORM/api/projects/retail/alerts/mute" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"key": "stale/uploads_orders", "hours": 72}'
```

`key` is the issue's (from `GET /api/projects/{name}/alerts`: `check/<table>/<check>`, `stale/<table>`, `pipeline/<pipeline>/<run>`, `protect/<table>/<column>`, ...); `hours` from 0 (unmute) to 168. The answer is the project's alerts, as the GET gives them.

How it behaves

- A muted issue still shows on the Overview; it isn't told when it starts or when it's resolved.
- The audit log records `alert.muted`, with the issue and the hours.

Limits and gotchas

- Mutes are kept in the platform's bucket (`alerts/mutes.json`), so a restart of platformd keeps them; the audit log keeps who muted what.
- A pipeline's issue is keyed by its run: muting it mutes that run's failure, not the next run's.
- "want {key, hours}: up to a week".

Settings

Setting	Where	Default	What
The install's mail server	Secret `storscale-alerts`: `smtpServer`, `smtpFrom`, `smtpUser`, `smtpPassword`	None	Projects' email alerts (chapter 17)
catalog-sync's round	catalog-sync's environment, `SYNC_EVERY`	20 seconds	How often checks run (not a chart value)
Profiles' age	catalog-sync's environment, `PROFILE_EVERY`	3600 seconds	A table's profile older than this is made again (not a chart value)
Tables profiled per round	catalog-sync's environment, `PROFILE_PER_ROUND`	3	Per project (not a chart value)

What the audit log records

Action	When
`project.saved`	The spec saved: checks, freshness and `notify` are in it, with which parts changed
`alert.muted`	An issue muted or unmuted, for how many hours
`connection.saved`	A Teams or Slack channel's connection added or changed

See Lab 21 and chapter 17.

6. Metrics and business terms

A project's metrics are defined once, in its semantic model, and every tool asks that one definition: the Metrics tab, Superset's datasets, apps, assistants over MCP, flows and notebooks. This chapter is for the people who write those definitions (a project's editors, and anyone given `change-metrics`), for the analysts who ask them, and for the administrators who make a project's model the company's, so that other projects build on it and its business terms (Revenue, Customer) are everyone's words.

In this chapter:

- The semantic model
- Asking for metrics
- Tests
- Sharing a model, and building on another project's
- Business terms
- Settings
- Errors you may meet

The semantic model

What it's for. Finance, marketing and the web team each wrote their own SQL for revenue and got three numbers. A semantic model says once what revenue is (which column, which rows count, how it's added up) and by what it can be broken down (region, channel, month). People and tools then ask for `revenue` by `order__region`, not for SQL, and get the same answer to the cent, each under their own access rules.

Who can do it. Every member of the project reads the model and asks its metrics. Changing it takes the permission `change-metrics`: a project's editors have it, and so does any custom role given it (Roles). Readers see the definitions but can't change them.

The model is MetricFlow's YAML, read by the platform's semantic layer (`semanticd`) without dbt. Each project has at most one model, kept in the platform's store as `semantic/<project>.yaml`. The **Metrics** tab appears on a project that has tables (`tables` in its spec) on an install with the semantic layer.

Set up

On the page. Open the project → **Metrics**. The **Definitions** card at the bottom has **Edit** (for those who may change the model; **Show** for everyone else). Paste or write the YAML in the box and press **Save**. While it saves the button says **Checking...**; **Discard** throws away an unsaved draft.

As a file in the project's repo. A project kept as code has the model in `storscale/semantic.yaml`, and a release applies it (History and releases). There is no field for the model in the project's spec: the spec only says whether the model is shared, or builds on shared ones (see Sharing).

Through the API. The model is read and written whole:

Method and path	What it does	Who
`GET /api/projects/{name}/semantic`	The model's YAML, and what it offers: `metrics` (name, description, type, dimensions), `tables`, `terms`, `tests`, `builtOn`	The project's members

Method and path	What it does	Who
`PUT /api/projects/{name}/semantic`	Saves a new model, body `{"yaml": "..."}`, after compiling it	`change-metrics`

```
# A token as bob (the public client storscale-agent; the Python package does this for you)
TOKEN=$(curl -s -d grant_type=password -d client_id=storscale-agent -d username=bob -d
  "password=$PASSWORD" \
  http://storscale.localhost:8800/sso/realms/lakehouse/protocol/openid-connect/token | jq -r
    .access_token)

# Save the model in orders.yaml as retail's
jq -Rs '{yaml: .}' orders.yaml | curl -s -X PUT
  http://storscale.localhost:8800/api/projects/retail/semantic \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" -d
    @-
```

Every change through the API (anything but `GET`) sends `X-Platform-Request: 1`, or it's refused with "missing X-Platform-Request".

The documents of a model

A model is a YAML file of several documents, separated by `---`. Each document is a mapping with one of these keys:

Document	What it is	Read by
`semantic_model`	A table, its entities (keys), dimensions (what to group by) and measures (what to add up)	MetricFlow
`metric`	A metric made from measures, or from other metrics	MetricFlow
`project_configuration`	MetricFlow's own settings, such as the time spine (optional: see Limits)	MetricFlow
`terms`	The business's words for what the model defines: see Business terms	The platform
`tests`	Known answers to known questions: see Tests	The platform

`terms` and `tests` are the platform's; it takes them out before MetricFlow reads the rest. A document that isn't a mapping is refused: "each document of a model is a mapping (semantic_model, metric, terms, tests)".

Inside `semantic_model` and `metric` the fields are MetricFlow's own (`node_relation`, `defaults.agg_time_dimension`, `entities`, `dimensions`, `measures`; a metric's `type` and `type_params`); MetricFlow's documentation describes them. The platform adds nothing to them.

A complete example

`retail`'s model from Lab 6, over the table `uploads_orders`:

```
semantic_model:
  name: orders
  description: Every order, one row each; test rows and refunds excluded by the metrics.
  node_relation: {database: iceberg, schema_name: retail, alias: uploads_orders}
  defaults: {agg_time_dimension: ordered_on}
  entities:
    - {name: order, type: primary, expr: order_id}
    - {name: customer, type: foreign, expr: customer_id}
  dimensions:
    - {name: region, type: categorical, description: "EU, US or APAC"}
    - {name: channel, type: categorical, description: "web, store or partner"}
    - {name: ordered_on, type: time, type_params: {time_granularity: day}}
  measures:
    - {name: amount_sum, expr: "case when not test and not refunded then amount end", agg: sum,
      description: "The orders' amounts, net of refunds and test rows"}
    - {name: orders, expr: "case when not test then 1 end", agg: sum}
    - {name: refunds, expr: "case when refunded then 1 end", agg: sum}
    - {name: customers, expr: customer_id, agg: count_distinct}
---
metric:
  name: revenue
  description: What the orders brought in, net of refunds, test rows excluded.
  type: simple
  type_params: {measure: amount_sum}
---
metric:
  name: order_count
  description: How many real orders there were.
  type: simple
  type_params: {measure: orders}
---
metric:
  name: customer_count
  description: How many customers ordered.
  type: simple
  type_params: {measure: customers}
---
metric:
  name: average_order_value
  description: Revenue per order.
  type: ratio
  type_params: {numerator: {name: revenue}, denominator: {name: order_count}}
---
terms:
  - name: revenue
    owner: bob
    definition: What the orders brought in, net of refunds, in US dollars. Test rows are never
      counted.
    synonyms: [sales, turnover]
---
tests:
  - name: three_regions
    metrics: [revenue]
    group_by: [order__region]
    expect: {rows: 3}
```

`node_relation` names the table in full: `database` is the Trino catalog (`iceberg`), `schema_name` the project's namespace (the project's name with dashes made underscores, unless `tables.namespace` says otherwise) and `alias` the table.

Dimensions are asked for by their entity and name, joined by two underscores: `order__region`, `order__ordered_on`. Every metric can also be grouped by `metric_time`, at a grain: `metric_time__month`. Through a foreign entity, a model reaches the dimensions of the model whose primary entity it is: with a `customers` model whose primary entity is `customer`, `orders` can be grouped by `customer__segment`.

How it behaves

- **Saving compiles first.** `semanticd` compiles the whole model with MetricFlow (with the shared models it builds on, if any) and checks its terms and tests. Anything wrong is refused with the reason, and nothing is saved: "not YAML: ...", "MetricFlow can't read it: ...", MetricFlow's validation messages, a term or a test naming something the model lacks.
- **A saved model applies to the next question anyone asks.** There's nothing to deploy: the Metrics tab, apps, Superset's datasets and assistants all read the saved model. Superset's dataset follows within a minute (Dashboards).
- **The page shows what it offers. The metrics** card lists each metric with its type and what it can be grouped by, and the tables the model reads ("over iceberg.retail.uploads_orders"). If the saved model no longer compiles (a shared model it builds on changed, say), a red banner says "The definitions don't compile:" and why.
- **Where it's recorded.** Each save is in the platform's audit log as `semantic.saved` on `project:<name>`, with the metrics it added, removed or changed ("metric changed: revenue"), or "saved" when it changed none; `semanticd`'s log has it too. The questions asked of it are in Ranger's audit log, which the project's **Audit** tab shows (Asking for metrics). A model applied by a release is in the platform's audit log as the release (History and releases).

Limits and gotchas

- **One model per project**, as one file; it may hold any number of semantic models and metrics.
- **The time spine.** MetricFlow wants a time spine declared even when no question uses one. When the model declares none (no `project_configuration` document), the platform declares one: a table `time_spine_day` in the project's namespace, with a column `date_day`. It declares it; it doesn't make it. Plain questions don't read it; a metric that needs one (cumulative metrics, for instance) fails until that table exists. Make it in the project (a dbt model, or SQL), or declare your own `project_configuration`.
- **The rest is MetricFlow's.** Its own checks (names, references between measures and metrics, entity types) are passed on as it words them.
- **An administrator who isn't a member** of the project is refused the model ("you're not a member of retail"): the semantic layer answers members only.
- **A model that names a table you can't read compiles**, and the questions fail at Trino: compiling doesn't read the data.

Asking for metrics

What it's for. Getting a number without writing SQL: pick the metrics, pick what to group them by, and get the governed answer, with the SQL it ran shown.

Who can do it. The project's members. Each question runs in Trino as the person asking, so a reader under a row filter gets only the rows their rules allow (alice, a reader of `retail`, gets the EU only), and masked columns stay masked.

Set up

On the project → **Metrics**, the **Ask for metrics** card:

1. Under **Metrics**, press the chips of the metrics you want (hover one for its description).
2. Under **Group by**, press the dimensions to break them down by. Only those all the picked metrics share are offered.

3 Press **Ask**. The **Answer** card shows the rows (and bars, when there's one label and one number and at most 20 rows). **The SQL it ran, as you** opens the compiled SQL.

The top bar's search finds a project's metrics too; picking one opens the Metrics tab with it chosen.

Through the API: `POST /api/projects/{name}/semantic/query`, as a member:

```
{
  "metrics": ["revenue", "order_count"],
  "group_by": ["order__region"],
  "where": ["{{ Dimension('order__channel') }} = 'web'"],
  "order_by": ["-revenue"],
  "limit": 100
}
```

Field	What it is
`metrics`	The metrics, by name: at least one ("ask for at least one metric")
`group_by`	Dimensions, as `entity__dimension`, or `metric_time__day` (`__week`, `__month`, ...)
`where`	Conditions in MetricFlow's template form: `{{ Dimension('order__region') }} = 'EU'`, `{{ TimeDimension('metric_time', 'day') }} >= '2026-09-01'`
`order_by`	Metric or dimension names; a leading `-` for descending
`limit`	Rows, at most 1000
`explain`	`true`: the SQL only, not run

The answer has `columns`, `types`, `rows` and the `sql` it ran. From Python, the same route by its operation ID: `p.call("post_projects_name_semantic_query", {"metrics": ["revenue"], "group_by": ["order__region"]}, name="retail")`.

How it behaves

- MetricFlow compiles the question to Trino SQL; `semanticd` runs it in Trino **as the person**, with their own Keycloak token, in the project's namespace, under the project's limits (the Trino resource group the client tag `project:<name>` selects).
- The SQL starts with a comment naming who asked and through what: `-- storscale: agent=<client> user=<person> project=<project> metrics=<metrics>`. Ranger's audit log keeps it, and the project's **Audit** tab shows it, so a question asked through an assistant names both the assistant and the person.
- An answer has at most 1000 rows; the Metrics tab asks for 100.

Limits and gotchas

- A question MetricFlow can't compile (a dimension the metric doesn't have, a misspelt name) is refused as "MetricFlow can't answer that: ...".
- A table the person may not read is Trino's refusal, passed on as a 403 with Ranger's words ("Access Denied: ...").
- The **Group by** chips are the dimensions every picked metric has; to group by `metric_time` at a grain, use the API, or an app (Apps).

Tests

What it's for. A changed definition shouldn't quietly change last month's revenue. A test pins an answer to a question (revenue by region has three rows; the quarter had 300 real orders), so a change that moves it fails before anyone relies on it. A test may also be a question in words with the metric call it should become, so a renamed metric or a lost synonym fails before an assistant meets it.

Who can do it. Writing tests is changing the model (`change-metrics`). Any member may run them; they run as whoever runs them.

Set up

Add a `tests` document to the model:

```
tests:
  - name: three_regions
    metrics: [revenue]
    group_by: [order__region]
    expect: {rows: 3}
  - name: web_orders
    metrics: [order_count]
    where: ["{{ Dimension('order__channel') }} = 'web'"]
    expect: {rows: 1}
  - name: all_orders
    metrics: [order_count]
    expect: {values: [[300]]}
  - name: turnover_in_words
    question: What was turnover by region?
    metrics: [revenue]
    group_by: [order__region]
```

Field	What it is
`name`	Required, unique in the model ("each test has a name", "two tests named ...")
`metrics`	Required: at least one metric the model has ("test ...: the model has no metric ...")
`group_by`, `where`	Lists, as in a question ("test ...: group_by is a list")
`expect.rows`	How many rows the answer has: a number
`expect.values`	The rows themselves: a list of rows, each a list. Compared in any order; numbers as numbers, to six decimal places
`question`	A question in words. The model is asked which metric call answers it; the test passes when the call is `metrics` by `group_by` (in any order), and, if `expect` is given too, when its answer matches

A test without `question` needs `expect` with `rows` or `values`.

To run them: on **Metrics**, the **Definitions** card has **Run the N tests** when the model has tests. Each shows ✓ or with why: "2 rows, expected 3", "the rows differ from the expected: ...", "the model asked for [...] by [...], expected [...] by [...]".

Through the API: `POST /api/projects/{name}/semantic/tests`, as a member, body `{}` for the saved model, or `{"yaml": "..."}` to test a model before saving it. The answer lists each test with `passed`, `why`, its `rows`, and counts `passed`, `failed` and `skipped`.

How it behaves

- **Checked on save.** A model whose tests name a metric it doesn't have is refused, so renaming a metric a test uses fails the save, saying which test.
- **Run as the person.** A test's answer is the person's: as alice, under the EU row filter, `three_regions` fails with one row. Run tests as someone who reads the table as it is (an editor).
- **Questions in words** go to a model through the LLM gateway, on the project's key, with the model's metrics (their descriptions, and the terms' synonyms) and dimensions. A project without a gateway key (no `llm.budget`) skips them: "not asked: the project has no gateway key (llm.budget)". Each question is in the platform's audit log as `llm.asked`, with what it spent (AI).
- **In merge requests.** With a GitLab repo, a merge request's review runs the tests on its data branch and compares each answer with production's (Pipelines and the project's repo).

Limits and gotchas

- `expect.values`: a value that reads as a number is compared as a number, to six decimal places, so `22168.4` and `"22168.40"` (Trino returns decimals as text) match; anything else is compared as text, and `null` as null.
- A question test spends the project's LLM budget each time it runs.

Sharing a model, and building on another project's

What it's for. The company's words shouldn't be defined again in each project. One project's model (`retail`'s, with revenue and customers) is made the company's: its terms appear on **Business terms** and in the catalog, and other projects build their own metrics on its definitions instead of copying them.

Who can do it. Both settings are in the project's spec, which administrators change (Projects).

Set up

On the project → **Edit** (administrators), in the JSON box:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "members": [{"group": "engineers", "role": "editor"}, {"group": "analysts", "role": "reader"}],
    "tables": {},
    "semantic": {"shared": true}
  }
}
```

and for a project that builds on it:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "marketing"},
  "spec": {
    "members": [{"group": "analysts", "role": "editor"}],
    "tables": {},
    "semantic": {"uses": ["retail"]}
  }
}
```

Field	What it is
`semantic.shared`	`true`: this project's model is the company's, for others to build on; its terms are the company's
`semantic.uses`	The shared models this project's model builds on, by project name ("semantic: uses: ... isn't a project's name"; "a project's model doesn't build on itself")

The same through the API is the whole spec: `PUT /api/projects/{name}` (Projects).

Then `marketing`'s model uses `retail`'s metrics by name, without defining them:

```
metric:
  name: revenue_per_customer
  description: Revenue per customer, on retail's definitions of both.
  type: ratio
  type_params: {numerator: {name: revenue}, denominator: {name: customer_count}}
```

How it behaves

- **Compiled together.** A project's model is compiled with the shared models it uses first, then its own. `marketing`'s Metrics tab lists retail's metrics too, and can ask them by `marketing`'s dimensions and retail's.
- **Not defined twice.** A project that builds on a shared model may not define its semantic models' or metrics' names again: "the company's model has these already, so build on them rather than defining them again: revenue (the metric of retail)".
- **A shared model can't break its users.** Saving a shared model compiles every project that builds on it against the new text; if one would no longer compile, the save is refused naming it: "it would break the models of marketing (...)".
- **Answers cite their source.** An assistant's answer names each metric's definition and the project it's defined in (Assistants).
- **Reading still takes access.** Building on retail's metrics gives `marketing` the definitions, not the rows: the SQL reads `iceberg.retail.uploads_orders`, and runs as the person asking. They need to read that table: as a member of `retail`, or as a consumer of a data product of it (Data products).

See Lab 6 (sharing) and Lab 12 (building on it).

Limits and gotchas

- `uses` naming a project whose model isn't shared is accepted in the spec and ignored: nothing is built on it. Mark that project `shared` first.
- Each project keeps its own time spine; a shared model's `project_configuration` isn't carried over.
- There's no review of a shared model's changes beyond the compile and its users' compile; keep the shared project as code with merge requests for that (Pipelines and the project's repo).

Business terms

What it's for. "Customer", "revenue", "basket size": the words people use, each with one definition, one owner and the synonyms people say instead, tied to what the model defines. People find data by those words before they find tables: the **Business terms** page, the top bar's search, the catalog's glossary, and assistants all start from them.

Who can do it. Terms are part of a model: writing them takes `change-metrics` on that project. They are the company's only when the project's model is shared (`"semantic": {"shared": true}`). Everyone signed in sees them.

Set up

Add a `terms` document to the shared project's model, and save it on **Metrics** → **Edit**:

```
terms:
  - name: revenue
    owner: bob
    definition: What the orders brought in, net of refunds, in US dollars. Test rows are never
      counted.
    synonyms: [sales, turnover]
  - name: customer
    owner: bob
    definition: A person or company that has placed at least one order.
    synonyms: [client, account]
  - name: average_order_value
    owner: bob
    definition: "Revenue per real order, in US dollars: what a customer spends on one order, on
      average."
    synonyms: [basket size, AOV]
```

Field	What it is
`name`	Required: an entity, a metric or a dimension the model defines (`customer` is the entity above, `revenue` the metric)
`definition`	The meaning, in plain words
`owner`	Who answers for it: a user name
`synonyms`	Other words for it, which search and assistants match
`label`	Optional: the name shown (default: the name with underscores as spaces, capitalised: "Average order value")
`kind`	Optional: `entity`, `metric` or `dimension`, when one name is more than one of them

Quote a definition that has a colon in it. YAML reads `definition: Revenue per real order, in US dollars: what…` as a mapping inside a mapping and the save is refused ("not YAML: mapping values are not allowed here"). Put the whole definition in double quotes, as above. The same goes for any value with `: `, or one starting with `[`, `{`, `*`, `&` or `#`.

How it behaves

- **Checked on save.** Each term must name an entity, a metric or a dimension of this model; otherwise the save is refused, naming it: "terms: basket_size isn't an entity, metric or dimension of this model". With `kind`, the name must be one of that kind.
- **On the Business terms page.** The top bar's **Business terms** lists every term of every shared model: its label and kind, its definition, owner, and the projects using it. A term's own page (`/terms/<name>`) shows its definition, owner and synonyms ("Also called"), the shared model and project it's defined in, **Metrics on it** (for an entity: the metrics over the same tables), **Where it comes from** (the tables), and **Projects using it** (projects building on the shared model whose own model names it).
- **In search.** The top bar's search lists terms first, matching their label, definition and synonyms: "turnover" finds Revenue.
- **In the catalog.** catalog-sync keeps OpenMetadata's glossary **Business terms** in step with the shared models' terms (each with its definition, synonyms, and its owner when the catalog knows that user), tags the tables each term comes from with it, and removes terms no longer in any shared model. It runs every `METADATA_EVERY` seconds (60), so a new term is in the catalog within a few minutes.
- **For assistants.** The MCP tools `search_terms`, `describe_term` and `list_terms` answer from them, and every answer cites the terms behind what it used (Assistants). A question test's prompt lists the terms' synonyms beside the metrics.

- **For data products.** The publish form lists the company's terms to tick as those the product holds (Data products).

Through the API: `GET /api/terms`, to anyone signed in: each term with `name`, `label`, `kind`, `definition`, `owner`, `synonyms`, `project`, `tables`, `metrics` and `usedBy`.

See Lab 6, steps 5 and 6.

Limits and gotchas

- **Only shared models' terms are the company's.** A project whose model isn't shared may have a `terms` document; it's checked on save, and appears nowhere: not on Business terms, not in the catalog, not to assistants. Mark the project `"semantic": {"shared": true}` to publish its terms.
- **A term names what its own model defines.** In a project that builds on a shared model, a term naming the shared model's metric is refused: the term belongs in the shared model, with the metric.
- **A term isn't a free word.** "Basket size" can't be a term of its own; make it a synonym of the metric it means (`average_order_value`), or define that metric first.
- **The Business terms button** shows only on installs with the semantic layer.
- Two shared models with a term of the same name both list on the page, but the catalog's glossary keeps one of them. Keep names unique across shared projects.

Settings

Setting	Chart value (`deploy/helm/storscale-platform/values.yaml`)	Compose (`stack/compose.yaml`, `stack/.env`)	Default
How many `semanticd` pods	`replicas.semanticd`	one container	1
The semantic layer platformd calls	set by the chart	`SEMANTIC_URL` on platformd	`http://semanticd:8000`; empty turns off Metrics, SQL, Flows, Agents and Apps tabs and Business terms
How often the catalog's glossary is synced	not a chart value	`METADATA_EVERY` on catalog-sync	60 seconds
How often Superset's datasets follow the models	not a chart value	`SYNC_EVERY` on superset-sync	60 seconds

Errors you may meet

Error	What it means	What to do
"only those who may change retail's metrics (its editors) change its semantic model"	You lack `change-metrics` on the project	Ask an editor, or for a role that gives it
"not YAML: mapping values are not allowed here"	A value with `:` in it isn't quoted (often a term's definition)	Put the value in double quotes
"MetricFlow can't read it: ..."	A document isn't the shape MetricFlow expects	Check the field names against MetricFlow's documentation

Error	What it means	What to do
"terms: X isn't an entity, metric or dimension of this model"	A term names nothing the model defines	Name the metric, entity or dimension it means; make the word a synonym
"test X: the model has no metric Y"	A test names a metric that isn't there (often after a rename)	Rename it in the test too
"the company's model has these already, so build on them rather than defining them again: …"	Your model defines a name the shared model it uses has	Remove yours, and use theirs
"it would break the models of marketing (…)"	A change to a shared model breaks a project built on it	Change both: the user's model first, or keep the old name
"MetricFlow can't answer that: …"	The question names a metric or dimension that doesn't fit	Pick from the Metrics tab's list; dimensions are `entity__dimension`
"you're not a member of retail"	The semantic layer answers the project's members only	Be made a member
"The definitions don't compile: …" (banner)	The saved model no longer compiles, often after a shared model changed	Fix the model; the reason names what's wrong

7. Dashboards, apps and assistants

The semantic model (Metrics and business terms) defines revenue once; this chapter is about the places people read it. Superset's dashboards, on datasets the platform makes from the models; apps, small pages a project's builders declare for its readers; and assistants (Claude Code, a company's own) that ask the platform over MCP. Each of them asks as the person using it, so each shows what that person's rules allow. The last section is the platform's API reference, for those who'd rather call it themselves. It's for analysts who build charts and apps, engineers who connect assistants, and the administrators who register them.

In this chapter:

- Dashboards (Superset)
- Superset's datasets made from the models
- Apps
- Assistants over MCP
- Registering an assistant
- The API reference
- Settings

Dashboards (Superset)

What it's for. Charts and dashboards for the people who read the numbers, built in a tool made for it (Apache Superset), on the same tables and the same definitions as everything else, with each person's rules applied to what each chart shows them.

Who can do it. Superset lets in the members of two Keycloak groups and turns each into Superset roles at every sign-in:

Keycloak group	Superset roles	What that gives
`analysts`	Gamma, `sql_lab`, Lakehouse SQL	SQL Lab and the lakehouse database, and what Superset's Gamma role allows; not making datasets
`engineers`	Alpha, `sql_lab`, Lakehouse SQL	The same, with Superset's Alpha role: making datasets, charts and dashboards

Anyone in neither group is refused at sign-in. Superset's own roles decide the rest (who may edit whose dashboard); the platform doesn't change them.

Set up

- **A project's Dashboards tab**, or **Tools → Dashboards**: Superset's dashboard list, in the platform's frame, signed in with your Keycloak session (no second sign-in). **Open in a new tab ↗** on the frame's bar opens it on its own; **Reload** reloads it.
- **Tools → SQL**: Superset's SQL Lab, for SQL against the lakehouse.
- Superset is served at the platform's `/dashboards/`.

Build a chart as Superset's documentation says: **Datasets**, pick one (those the platform made are called `<model> (semantic)`, below), make a chart, save it to a dashboard.

How it behaves

- Superset reaches Trino through one database, **Lakehouse (Trino)**, as the service user `superset` with impersonation on: every query runs in Trino **as the person viewing the chart**, so Ranger's policies, row filters and masks apply, and Ranger's audit log names them. As alice, a chart of revenue by region shows the EU only. The database doesn't allow DML.
- Charts are Superset's: the platform keeps no copy of them and never changes them.

See Lab 7.

Limits and gotchas

- **Two groups only.** The groups Superset lets in (`analysts`, `engineers`) and their roles are fixed in `stack/superset/superset_config.py`; another group's people can't sign in to Superset, whatever their projects.
- **Dashboards aren't per project.** The Dashboards tab opens Superset's own list; what a chart shows is decided by Trino's rules for the viewer, not by which project's tab it was opened from.
- **Power BI** on the same definitions: not yet.

Superset's datasets made from the models

What it's for. A chart built on raw SQL brings back the three revenues. The platform makes a Superset dataset from each semantic model, with its metrics already defined, so a chart's `revenue` is the semantic layer's revenue, and follows when the definition changes.

Who can do it. Nobody sets it up: a service beside Superset, `superset-sync`, does it for every project's model. Anyone who may use Superset may chart on them.

How it behaves

Every minute, `superset-sync` reads every project's model and keeps one Superset dataset per semantic model:

Part	What it is	For `retail`'s model `orders` (**the example**)
Name	`<semantic model> (semantic)`, in the table's schema	`orders (semantic)`, schema `retail`
Columns	Each dimension, named `<primary entity>__<dimension>` as MetricFlow names it (time dimensions marked as times); each measure, under its own name; through each foreign entity, the dimensions of the model where that entity is primary, as `<entity>__<dimension>`	`order__region`, `order__channel`, `order__ordered_on`; `amount_sum`, `orders`, `refunds`, `customers`
Metrics	Each simple metric of a measure of this model, as its aggregate; each ratio of two such metrics, as one over the other	`revenue` = `SUM("amount_sum")`, `order_count`, `refund_count`, `customer_count`, `average_order_value` = `CAST(SUM("amount_sum") AS DOUBLE) / NULLIF(SUM("orders"), 0)`
Description	The model's description, then "From the semantic model of retail, kept by the platform: don't change it here." and, when some of its metrics can't be said in a dataset, "Ask the semantic layer for ..." naming them	

The dataset's SQL is a view over the model's table (and the tables it joins to), so its numbers are read in Trino as the viewer, like any other chart.

- **Kept in step.** A changed model changes its dataset within a minute (`SYNC_EVERY`, 60 seconds). A model removed, or renamed, removes the dataset made from it. Datasets people make themselves are never touched.
- **Shared models.** A project that builds on a shared model (sharing) gets datasets for its own semantic models; its foreign entities join to the shared models' too, and ratios of the shared metrics are defined where both measures are the dataset's own model's.

Limits and gotchas

- **Don't edit a platform dataset.** The next change to its model replaces its SQL, columns and metrics. Make a dataset of your own (as an engineer) for anything else.
- **Only simple metrics and ratios of them.** A metric whose measures aren't all the model's own (a ratio across models), or any other kind (derived, cumulative, conversion), stays the semantic layer's: ask it on the Metrics tab, in an app, or through an assistant. The dataset's description names those metrics.
- **A renamed semantic model is a new dataset.** Charts on the old one lose it: keep a model's name once charts use it.
- `metric_time` **isn't a column.** Group a dataset by the model's own time dimension (`order__ordered_on`) and Superset's time grain.

Apps

What it's for. The regional leads need one number by region, not Superset. An app is a small page of the project's, declared rather than built: a metric (or a few) by a dimension, as bars, a line, one number or a table; a table's rows; or questions answered from a knowledge bank. Its data is fetched as whoever opens it, so one app serves every audience: alice sees the EU, bob every region.

Who can do it. Making, changing and removing apps takes the permission `build` (editors have it). Every member of the project opens them; they're on the project's **Apps** tab and on each member's **Home**, under **Apps**.

Set up

On the project → **Apps** → **Make an app**:

Field	What it is
Name	Lowercase letters, digits and dashes, starting with a letter, up to 41 characters (`revenue-by-region`)
Title	What the page and the card say (default: the name)
What it's for	A line under the title
What it shows	**Metrics, by a dimension**; **A table's rows**; or **Questions, answered from a knowledge bank** (offered when the project has a knowledge bank and an LLM budget)
Metrics	(metrics) The metrics, as chips: each is its own chart, on its own scale
By	(metrics) A dimension, or **nothing (the total)**
As	(metrics) **Bars**, **A line**, **One number** or **A table**
Table, **Columns (none: all)**	(table) The table, and the columns shown
Knowledge bank	(questions) The bank its answers come from
Filters	(metrics, table) **Add a filter**: a dimension (or a column), a label, and the values to pick from (none: the viewer types one)

Press **Make it**. **Remove** on an app's card takes it out.

In the project's spec, the same apps are `apps`:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "members": [{"group": "engineers", "role": "editor"}, {"group": "analysts", "role": "reader"}],
    "tables": {},
    "llm": {"budget": 20},
    "knowledge": [{"name": "reviews", "tables": [{"table": "uploads_reviews", "column": "text", "id":
      "review_id"}]}],
    "apps": [
      {"name": "revenue-by-region", "title": "Revenue by region", "kind": "metric", "metrics":
        ["revenue"], "by": "order__region",
       "show": "bar", "filters": [{"field": "order__region", "label": "Region", "values": ["EU",
         "US", "APAC"]}]},
      {"name": "revenue-total", "title": "Revenue", "kind": "metric", "metrics": ["revenue"], "show":
        "number"},
      {"name": "orders", "title": "The orders", "kind": "table", "table": "uploads_orders",
       "columns": ["order_id", "region", "amount"], "filters": [{"field": "region", "label":
         "Region"}], "limit": 200},
      {"name": "ask-reviews", "title": "What customers say", "kind": "qa", "bank": "reviews"}
    ]
  }
}
```

Field	Kinds	What it is
`name`	all	Required: `^[a-z][a-z0-9-]{0,40}$`
`title`, `description`	all	What the page says
`kind`	all	`metric`, `table` or `qa`
`metrics`	metric	At least one: the project's metrics, or the shared ones it builds on
`by`	metric	A dimension (`order__region`, `metric_time__month`); required for `bar` and `line`
`show`	metric	`table`, `bar`, `line` or `number` (one metric, by nothing); a table app shows `table`
`table`, `columns`	table	One of the project's tables, by its bare name, and the columns shown (none: all)
`filters`	metric, table	Each `{field, label, values}`: a dimension (metric) or a column (table) equal to the value the viewer picks; `values` up to 200 characters each
`bank`	qa	One of the project's knowledge banks (`knowledge`)
`limit`	metric, table	Rows shown, at most; default 500, up to 5000

Through the API:

Method and path	What it does	Who
`GET /api/projects/{name}/apps`	The project's apps, and whether you may make them	Members
`PUT /api/projects/{name}/apps/{app}`	Makes an app, or changes one: the app as JSON (the fields above, without `name`)	`build`
`DELETE /api/projects/{name}/apps/{app}`	Takes it out	`build`
`POST /api/projects/{name}/apps/{app}/data`	Its rows, as you: `{"filters": {"order__region": "EU"}}`	Members
`POST /api/projects/{name}/apps/{app}/ask`	A question app's answer, as you: `{"question": "..."}`	Members

```
curl -s -X PUT http://storscale.localhost:8800/api/projects/retail/apps/revenue-by-region \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"title": "Revenue by region", "kind": "metric", "metrics": ["revenue"], "by": "order__region",
    "show": "bar"}'
```

How it behaves

- **Fetched as the viewer.** platformd fetches the data with the viewer's own token. A metric app asks the semantic layer (the filters become conditions on the dimensions, the rows ordered by `by`); a table app runs `SELECT <columns> FROM <table> WHERE CAST(<column> AS varchar) = '<value>'` in Trino. Ranger's rules apply either way.
- **The page.** Filters above the chart, applied with **Show**; **As a table** / **As a chart** switches a metric app's view; a number shows large, with its metric's name. "Nothing to show" when the rules or filters leave no rows.
- **A question app** embeds the question through the gateway (the install's embedding model, on the project's key), finds the five closest passages in the bank's table (`knowledge_<bank>`), read as the person asking, and has the model answer from those passages alone, citing them by number, or say they don't answer it. The answer lists the passages under **From:**, each with its source and row. It spends the project's LLM budget, under the guardrail (AI).
- **Audited.** The platform's audit log records `app.saved` (made or changed, with its kind), `app.removed`, and each question asked (`app.asked`, with how many passages and what it spent).

See Lab 7, step 4, and Lab 18 for the knowledge bank.

Limits and gotchas

- The form's **Table** list is the project's own tables, as the catalog has them: a table made a moment ago shows once the catalog has read it (a few minutes).
- A filter with `values` refuses any other value: "Region is one of EU, US, APAC".
- A metric app needs the project's tables ("a metric app needs the project's tables, where its metrics are"); a bar or line needs `by` ("a bar chart is by a dimension (by)"); a number is one metric by nothing ("a number is one metric, by nothing").
- A question app needs a bank of the project's (`a question app answers from one of the project's knowledge banks (bank): "x" isn't one`) and an LLM budget ("give the project an LLM budget (llm.budget)"); asked before the bank is built, it says "the bank reviews isn't built yet: build it on the Agents tab's Knowledge card".
- Apps aren't shared outside the project: a person sees an app only as a member of its project.

Assistants over MCP

What it's for. The CEO asks in a chat window instead of waiting for Thursday's deck. The platform's MCP server lets an assistant (Claude Code, Claude, a company's own) answer from the same definitions, as the person using it: it finds the business terms, the metrics and the data products, asks for the numbers under that person's rules, and says what it used. It can't see what the person can't, and every query is audited with both their names.

Who can do it. Anyone signed in, through an assistant the platform knows: the realm's built-in client `storscale-agent` (for Claude, IDEs and scripts), or one an administrator registered (below). The answers are the person's: carol, a member of no project, gets no projects and no answers.

Set up

The MCP server is at `<the platform's address>/mcp` (streamable HTTP). With Claude Code:

```
claude mcp add --transport http storscale http://storscale.localhost:8800/mcp
```

Claude Code finds the sign-in from the server (an OAuth resource server whose metadata names Keycloak); a browser opens on Keycloak's sign-in; sign in as yourself. The project's **Metrics** tab shows the command with your install's address, on its **Agents** card.

Other clients sign in the same way: OAuth's authorization code with PKCE, through the public client `storscale-agent`, whose redirect addresses are `http://localhost:*`, `http://127.0.0.1:*` and `https://claude.ai/api/mcp/auth_callback`. A client that needs another redirect address is registered as an assistant of its own.

The tools

Tool	Arguments	What it does
`search_terms`	`question`	The business terms a question is about, by their names, labels and synonyms; with a note to say so when none matches
`list_terms`		Every business term: definition, owner, synonyms, tables, the projects using it
`describe_term`	`term`	One term, by name or synonym
`list_projects`		The projects you're a member of, your role in each, and whether each has metrics
`list_metrics`	`project`	A project's metrics, each with its description and dimensions
`query_metrics`	`project`, `metrics`, `group_by`, `where`, `order_by`, `limit` (100)	Metrics computed as you; with `used`: what it used
`explain_query`	`project`, `metrics`, `group_by`, `where`	The SQL a query would run, without running it
`list_products`		The data products, certified first: producer, owner, version, terms, contract, consumers, certification
`describe_product`	`project`, `product`	One data product
`query_product`	`project`, `product`, `columns`, `where`, `limit` (100)	Rows of a product's table, as you, under the producer's rules
`list_runs`	`project`, `limit` (20, up to 200)	A project's pipelines and their latest runs, and the repo's files Airflow couldn't load
`list_alerts`	`project`	A project's open alerts: failed runs and checks, stale tables, broken contracts, budgets
`search_documents`	`project`, `bank`, `query`, `k` (5)	A knowledge bank searched by meaning, each passage with its source and score
`list_tools`	`project`	A project's own tools you may call, and its agents you may ask
`call_tool`	`project`, `tool`, `arguments`	Calls one of a project's own tools
`ask_agent`	`project`, `agent`, `task`	Asks one of a project's agents to do a task; it may take minutes

The server's instructions tell an assistant to start from `search_terms`, then the projects, metrics and dimensions, then query; to name the definitions, terms and products each answer used; and to say so when no term, metric or product covers a question rather than guess. The project tools and agents (`list_tools`, `call_tool`, `ask_agent`) are

AI's.

Each `query_metrics` answer carries `used`: the metrics' definitions (and the project each is defined in, with a link to its Metrics tab), the terms on what it asked for, and the data products behind the tables it read. Each `query_product` answer carries the product and its terms.

How it behaves

- **Signed in as the person.** The server accepts only Keycloak tokens of the realm `lakehouse` whose audience names the MCP server's address and Trino (`lakehouse`); any other is refused with 401. The person's projects are those whose members list them.
- **Asked as the person.** A query runs in Trino with the person's own token, so their row filters and masks apply. Its leading comment names the client and the person (`agent=storscale-agent user=alice`), and Ranger's audit log keeps it: the project's **Audit** tab lists the assistant's queries with both names. `query_product` runs the same way; a person who is neither a reader of the producer nor of a consuming project is refused, and told where their project can ask for the product.
- **A project's own agents** (clients `agent-<project>-<name>`, AI) get only the tools their registration lists, and none until approved: "the agent may use: nothing yet (not approved)". Assistants and `storscale-agent` get every tool.
- **What's open to everyone signed in:** the terms and the list of data products (what exists is no secret; reading it is).

See Lab 8.

Limits and gotchas

- An answer has at most 1000 rows.
- The Metrics tab's **Agents** card lists four tools (`list_projects`, `list_metrics`, `query_metrics`, `explain_query`); the server has all sixteen above.
- MCP's dynamic client registration (a client registering itself) isn't offered: the company approves its assistants.
- A tool's refusal reaches the assistant in words ("you're not a member of retail", "no business term is called ...; list_terms has them all"), which it can tell the person.

Registering an assistant

What it's for. An assistant the company runs (DevAssist, say) should sign people in as itself, so the audit log says "DevAssist, for alice" and the company can switch it off. Registering it gives it a Keycloak client of its own.

Who can do it. Administrators.

Set up

Administration → Assistants → Register an assistant:

Field	What it is
Name	3 to 40 lowercase letters, digits and dashes, starting with a letter (`devassist`)
What it is (optional)	A description
Redirect URIs, space-separated	Where Keycloak may send the person back after they sign in: `https://` addresses, or `http://localhost:*` / `http://127.0.0.1` for one on the person's own machine; a trailing `*` is allowed

Press **Register**. The table lists each assistant with its **Client ID** (`assistant-<name>`), the **MCP server** address to give it, its redirect URIs, and who registered it when; **Remove** takes it away.

Through the API, administrators:

Method and path	What it does
`GET /api/assistants`	The registered assistants, each with `clientId` and `mcpUrl`
`PUT /api/assistants/{name}`	Registers one, or changes it: `{"description": "...", "redirectUris": ["..."]}`
`DELETE /api/assistants/{name}`	Removes it

```
curl -s -X PUT http://storscale.localhost:8800/api/assistants/devassist \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"description": "The company coding assistant", "redirectUris":
    ["https://devassist.example.com/callback"]}'
```

How it behaves

- platformd keeps the registration in its store (`assistants/<name>.json`); the operator, on its next pass, makes the Keycloak client `assistant-<name>` as `storscale-agent` is made: public, PKCE (S256), the person's groups in its tokens, and tokens for the MCP server and Trino. The assistant then signs people in with OAuth's authorization code flow and calls `<platform>/mcp`.
- The client's ID is the agent's name everywhere: in each query's comment and Ranger's audit log.
- Removing it deletes the client: it signs no one in from then on, and the tokens it had end within Keycloak's token lifetime.
- The platform's audit log records `assistant.register` (with the redirect URIs) and `assistant.remove`.

Limits and gotchas

- "an http redirect URI is for localhost only; elsewhere it's https"; "an assistant needs at least one redirect URI".
- Changing an assistant's redirect URIs updates its client; its other settings stay as the operator made them.
- An install without a file store can't register assistants: the page says so, and the API answers "this install has no file store for assistants".

The API reference

What it's for. Everything the web app does is platformd's API, and the same calls serve scripts, the Python package and the MCP server. The reference lists every route, so a developer can call them without reading the code.

Who can do it. Anyone may read the reference (it says what exists, not what's in it). Each call runs as whoever signed in, under their projects' rules.

Using it

- **On the platform:** the account menu (your name, top right) → **API reference**. Routes are grouped by what they're about (apps, products, semantic, ...), with a search box; each opens to its description and its **Operation ID**.
- **As a document:** `GET /api/openapi.json`, OpenAPI 3.1, for any client generator. It's made from platformd's routes and their handlers' comments, and a test keeps it current, so a route can't be added without it.

- **Signing in:** `Authorization: Bearer <token>`, a Keycloak access token of the realm `lakehouse` (a person's, through `storscale-agent`, or a service account's); or the web app's session. Every change (any method but `GET`) also sends `X-Platform-Request: 1`.
- **Errors** are JSON: `{"error": "<why>", "request": "<id>"}`, with the same ID in the `X-Request-ID` header; quote it to an administrator (Troubleshooting).
- **From Python**, the `storscale` package: `p.call("<operation ID>", body, **path_parameters)` calls any route; `p.operations()` lists them (Operating the platform).

```
import storscale

p = storscale.Platform("http://storscale.localhost:8800")   # reads STORSCALE_TOKEN, or signs you in
  by a device code
p.call("post_projects_name_semantic_query", {"metrics": ["revenue"], "group_by": ["order__region"]},
  name="retail")
```

Settings

Setting	Chart value (`deploy/helm/storscale-platform/values.yaml`)	Compose	Default
A base map for Superset's map charts	`superset.mapboxApiKey`	`MAPBOX_API_KEY`	empty: map layers on no base map
Superset's pods	`replicas.superset`	one container	1
Superset's Keycloak client secret, session key, database and Trino passwords	made by the chart	`SUPERSET_OAUTH_SECRET`, `SUPERSET_SECRET_KEY`, `SUPERSET_DB_PASSWORD`, `SUPERSET_TRINO_PASSWORD` in `stack/.env`	change the examples' values
How often datasets follow the models	not a chart value	`SYNC_EVERY` on `superset-sync`	60 seconds
`semanticd`'s pods (the MCP server)	`replicas.semanticd`	one container	1
The MCP server's address	the platform's address + `/mcp`	`MCP_URL` on semanticd	`http://storscale.localhost:8800/mcp` locally

8. Flows

A flow is a pipeline built on a canvas: steps joined into a graph, from one or more tables to one output table. Each step is a dbt model in the project's repo, so what an analyst builds on the canvas is code an engineer can read, review and change by hand, and the canvas keeps working after the change. This chapter is for the analysts who build flows, the engineers who review and change them, and the administrators who set up what they need (a repo, the semantic layer, the LLM gateway).

In this chapter:

- What a flow is
- Building a flow on the canvas
- The step kinds
- The prepare step and its processors
- The model step (Ask a model)
- Previews and what they cost
- Saving a flow as a merge request
- Code steps: changing a flow by hand
- The flows API
- Settings
- Limits and gotchas

What a flow is

What it's for. An analyst needs a table that doesn't exist yet: orders joined to campaign responses, filtered, grouped, with a computed column. A flow builds it from steps picked from dropdowns, shows each step's columns and SQL before anything runs, previews each step on a sample, and saves the result as dbt models in the project's repo, on a branch with a merge request. Merged, it's one of the project's pipelines, built on its schedule with its checks.

Where it lives. A flow named `campaign_results` is a folder of the project's repo:

File	What it is
`<path>/dbt/models/flows/campaign_results/<model>.sql`	One dbt model per step. A step's model is named `<flow>__<step>` (`campaign_results__filter_2`), ephemeral; the output step's is named after its table, materialized as a table
`<path>/dbt/models/flows/campaign_results/sources.yml`	The tables the flow's inputs read, as dbt sources (one per catalog and namespace, named `<flow>__<catalog>_<namespace>`)
`<path>/dbt/models/flows/campaign_results/schema.yml`	Each model's description and place on the canvas (`meta.storscale`: step, x, y), and the output's checks as dbt tests
`<path>/flow_campaign_results.py`	The Airflow DAG `flow_campaign_results`, which builds the flow on its schedule
`<path>/dbt/dbt_project.yml`	Written by the first save if the repo has none: `profile: storscale`, `model-paths: [models]`

`<path>` is the project's `pipelines.path` (the DAGs' folder, `dags` in the examples). A model the canvas wrote starts with its step's settings as YAML in a comment, and a hash of the SQL under it:

```
-- storscale-step:
--   kind: filter
--   inputs:
--     - input
--   where: converted
--   sql: 3f1c0e9a2b7d4c55
{{ config(materialized='ephemeral') }}
select *
from {{ ref('campaign_results__input') }}
where converted
```

The canvas reads the folder back the same way: a model whose header still matches its SQL is a step; one changed by hand, or written without a header, is a code step.

Who can do it.

What	Who
Open the Flows tab, open a flow, see each step's columns and SQL, preview a step	Any member of the project (an administrator counts as an editor). A preview runs as you, under your rules
Start a flow, add, change and connect steps on the canvas	The permission `build`: editors, and custom roles that have it (Roles). Others see the canvas read-only; through the API, any member may compile and preview a flow they send
Save a flow (as a merge request)	`build`
Run a model step's question for every row (`POST .../steps/{step}/run`)	`build`, the project's pipelines, administrators
Merge the merge request	Whoever GitLab lets merge into the branch the pipelines follow
Start the flow's DAG by hand	`run-pipelines` (Pipelines and the project's repo)

What it needs. Building and previewing need only the platform: the catalog (Nessie, for the inputs' columns) and the semantic layer (semanticd, which runs previews in Trino as you). Saving needs the project's repo: `pipelines.repo` with an `https://` address, `pipelines.credentials.secretRef` naming a connection of kind `git`, and preferably `pipelines.ref` (Pipelines and the project's repo says how). Without it, the Flows tab says "Flows can be built and previewed here; saving one needs the project's pipelines repo (...)", with the reason.

See Lab 9 in the lab guide, and Lab 10 for saving.

Building a flow on the canvas

Set up

1. Open the project's **Flows** tab. It lists the flows in the repo (the folders under `models/flows/` on the branch the pipelines follow, and any saved on a branch of their own and not merged yet), each with its branch and a link to its merge request. The same tab holds the project's **Uploads** (Getting data in).
2. Under **Start a flow**, type a name (`campaign_results`) and press **New flow**. The name is lowercase letters, digits and underscores, starting with a letter, at most 41 characters: it names the dbt models and, by default, the output table.
3. **Add a step...** offers the kinds below. A new step is placed beside the selected one and takes it as its input when it takes one. Drag from a node's right edge to another's left edge to connect them; select a connection or a node and press Delete to remove it. **Arrange** lays the steps out left to right as the data flows, each input just before the step that reads it; a flow whose steps were saved without places on the canvas (dbt models written by hand) opens arranged. Saving keeps the places.

4. Select a node to see its settings on the right. Every table, column, step and metric is picked from a dropdown, so what you can choose is what exists. Inputs offer the project's tables, and the tables of other projects' data products it consumes, under **Products of other projects**.
5. Each change is compiled again a moment later (about 0.4 seconds): every node shows its column count, "can't compile" in red, or "columns unknown". The errors are listed above the canvas, one per failing step.
6. Optionally, type a schedule in the box beside **Save as merge request**: cron's five fields, UTC (`0 6 * * 1`, Mondays at 06:00). Empty means the DAG runs only when started.
7. Press **Preview** on a selected step to see its rows (Previews), or **SQL** to read its model.
8. Press **Save as merge request** (Saving).

How it behaves

- **Compiling** works out each step's columns and their Trino types from its inputs', and its SQL, before anything runs. An input's columns come from the catalog. A metric step's come from the semantic layer, asked as you. A code step's come from Trino, which the platform asks as you (it runs the step's query with no rows).
- A step that can't be compiled has an error and no columns; the steps after it say "its input `<step>` has an error".
- **Reading the folder back**: opening a flow reads its folder from its own branch (`storscale/flow-<name>`) while that branch exists, else from the branch the pipelines follow.
- Nothing is written anywhere until you save; closing the canvas without saving loses the change.

The step kinds

Each step kind, its label on the canvas, its inputs, and its settings as JSON (the field names of the flow's JSON, the `step` objects in the flows API). Every step also has `id` (its name: lowercase letters, digits and underscores, starting with a letter), `kind`, `inputs` (step IDs), an optional `description`, and `x`, `y` (its place).

Kind	Canvas label	Inputs	Settings
`input`	Input	none	`table`; `namespace` (another project's, for a product it consumes; default the project's); `catalog` (default the project's, `iceberg`)
`select`	Select and rename	1	`columns`: `[{name, as}]`, the columns kept, in order, each renamed when `as` is set
`retype`	Retype	1	`columns`: `[{name, type}]`, each cast to a Trino type
`filter`	Filter	1	`where`: a Trino condition
`join`	Join	2 (left, right)	`join`: `inner` (default), `left` or `full`; `on`: `[{left, right}]`
`union`	Union	2 or more	`distinct`: true for `union`, false (default) for `union all`
`dedupe`	Deduplicate	1	`by`: the columns that make rows the same (none: whole rows); `order`: `[{column, desc}]`, which row to keep first
`group`	Group by	1	`by`; `aggregates`: `[{function, column, as}]`
`pivot`	Pivot	1	`by` (the rows); `column` (whose values become columns); `values` (those values); `value`; `function`
`unpivot`	Unpivot	1	`unpivot` (columns that become rows); `name` (the name column, default `name`); `value` (the value column, default `value`)
`window`	Window	1	`partition`; `order`; `windows`: `[{function, column, as, offset}]`
`prepare`	Prepare	1	`processors`: `[{column, do, args, as}]` (below)

Kind	Canvas label	Inputs	Settings
`formula`	Formula	1	`columns`: `[{name, expression, type}]`
`llm`	Ask a model	1	`column`, `mode`, `values`, `prompt`, `name`, `bank`, `limit` (below)
`metric`	Metric	none	`metrics`; `by` (dimensions); `where`
`output`	Output	1	`table` (the table it makes; default the flow's name on the canvas); `checks`: `[{column, test, values}]`
`code`	Code	0 or more (its refs)	`sql`: the model as written (below)

What each does, and what it refuses:

- **input** reads `select * from` the table, through the flow's dbt source. The table must be the project's, or a data product's table whose `consumers` name this project (Data products); otherwise "`<namespace>.<table>` isn't the project's, nor a product it consumes". Its catalog must be the lakehouse's: "an input is a table of the lakehouse (iceberg), not of ...".
- **select** keeps at least one column ("a select step keeps at least one column"); two kept columns can't share a name. The canvas's **Keep all** fills in every input column.
- **retype** casts columns. The type is one of `varchar`, `char`, `varbinary`, `tinyint`, `smallint`, `integer`, `bigint`, `real`, `double`, `decimal`, `boolean`, `date`, `time`, `timestamp`, `json`, `uuid`, with a size or precision where Trino takes one (`decimal(10,2)`, `timestamp(6)`, `timestamp(3) with time zone`). Anything else: "`"x"` isn't a Trino type".
- **filter** keeps the rows where the condition holds (`not test and amount > 0`).
- **join** joins two inputs on pairs of columns. Columns whose types can't be compared are refused before anything runs: "join on filter.customer_id (bigint) and input_2.responded_on (date): their types differ; retype one first". Numbers compare with numbers, text with text, timestamps with timestamps of the same zone-ness. A right-hand column whose name the left has is renamed with `_right` (`customer_id_right`).
- **union** stacks two or more inputs. Every input must have the first's columns, by name, of compatible types, and no others: "union: input x has 6 columns, y has 5; select the same columns in each first".
- **dedupe** without `by` is `select distinct *`; with `by`, keeps one row per `by` value, the first by `order` (or by the `by` columns when no order is given).
- **group** groups by `by` (none: one row for the whole input) with at least one aggregate: `sum`, `count`, `min`, `max`, `avg`, `count_distinct`. `count` without a column is `count(*)`, named `count`; otherwise an aggregate without `as` is named `<function>_<column>` (`sum_amount`). `sum` and `avg` need a number column. Sums of integers are `bigint`, of `decimal(p,s)` `decimal(38,s)`; averages of integers are `double`.
- **pivot** makes one column per listed value of `column`, each `function` (default `sum`; `sum`, `count`, `min`, `max`, `avg`) of `value`, grouped by `by`. Trino needs the values named: "pivot: say which values of region become columns (Trino needs them named)". A value becomes a column name in lower case with other characters as underscores (`North America` → `north_america`; one starting with a digit gets `v_`).
- **unpivot** turns the listed columns into rows of two new columns, their names and their values. The listed columns must share a type: "unpivot: ... retype them to one type first".
- **window** adds columns computed over a window (`partition` by, `order` by): `rank`, `dense_rank`, `row_number`, `running_sum`, `sum`, `avg`, `min`, `max`, `count`, `lag`, `lead`. Each needs a name (`as`). `rank`, `dense_rank`, `row_number`, `running_sum`, `lag` and `lead` need an order: "window: rank needs an order". `lag` and `lead` look `offset` rows away (default 1).
- **formula** computes columns from Trino expressions (`revenue / budget`). A `type` casts the result and tells the steps after what it is; without one, the column's type is unknown to the canvas. A formula named as an existing column replaces it.

- **metric** asks the semantic layer for a metric query's SQL (its metrics, grouped by `by`, filtered by `where`, in MetricFlow's filter syntax: `{{ Dimension('order__region') }} = 'EU'`), so a flow's revenue is the company's (Metrics and business terms). It needs at least one metric, and the semantic layer: "the semantic layer isn't set up, which gives a metric step its SQL".
- **output** is the table the flow makes, with dbt tests on its columns: `not_null`, `unique`, `accepted_values` (with `values`). A flow has exactly one output; saving refuses a flow without one: "a flow is saved with an output step: the table it makes".

An expression in a filter or a formula is one expression, nothing more: it can't contain `;`, `--`, `/*`, `{{` or `{%` ("an expression can't contain "--"").

A flow's shape is checked first: names, each step's number of inputs ("step join (join): takes 2 inputs, not 1"; "takes at least two inputs" for a union; "takes no input" for an input or a metric), inputs that exist, one output, and no circle ("the steps go round in a circle: a -> b -> a").

The prepare step and its processors

What it's for. The cleaning that every file needs (trim, case, parse dates and numbers, fill nulls, standardize values) without writing SQL. A prepare step applies named processors to columns, in order; each is compiled to Trino SQL, and each argument is a value, never SQL.

Set up. Add a step → Prepare, then **Add a processor** for each change: pick the column, the processor (grouped as Text, Nulls, Numbers, Dates, Flags and values), its arguments, and optionally a new column's name ("as"). A processor changes its column in place; with "as", it makes a new column and leaves the original alone. Later processors see the column as the earlier ones left it.

```
{"id": "clean", "kind": "prepare", "inputs": ["orders"], "x": 440, "y": 0,
 "processors": [
   {"column": "region", "do": "trim"},
   {"column": "region", "do": "upper"},
   {"column": "region", "do": "standardize", "args": ["EUROPE=EU, USA=US"]},
   {"column": "amount", "do": "round", "args": ["2"]},
   {"column": "ordered_on", "do": "days_since", "as": "days_ago"}
 ]}
```

The processors (42), their arguments in order (optional ones marked), and the column's type after ("same" keeps it; "text" makes it `varchar`). A processor that works on text reads a non-text column as text.

Group	do	Label	Arguments	Type after
text	`trim`	Trim spaces		text
text	`upper`	Upper case		text
text	`lower`	Lower case		text
text	`title`	Title Case		text
text	`squeeze`	Squeeze spaces (one between words, none around)		text
text	`left`	Keep the first characters	how many (number)	text
text	`right`	Keep the last characters	how many (number)	text
text	`substring`	Substring	from (1 is the first) (number); length (number, optional)	text
text	`replace`	Replace text	find (text); replace with (text, optional)	text

Group	do	Label	Arguments	Type after
text	regex_replace	Replace by pattern	pattern; replace with (text, optional)	text
text	extract	Extract by pattern	pattern; group (0 is the whole match) (number, optional)	text
text	split	Split, and keep a part	separator (text); part (1 is the first) (number)	text
text	prefix	Add a prefix	prefix (text)	text
text	suffix	Add a suffix	suffix (text)	text
text	pad_left	Pad on the left	to length (number); with (text)	text
text	pad_right	Pad on the right	to length (number); with (text)	text
text	length	Length		bigint
text	hash	Hash (SHA-256)		text
nulls	fill	Fill nulls with a value	value	same
nulls	null_if	Null when it equals a value	value	same
nulls	empty_to_null	Null when empty or blank		text
nulls	null_unless	Null unless it matches a pattern	pattern	text
numbers	parse_number	Parse as a number (digits, sign and point kept)		double
numbers	round	Round	decimals (number)	same
numbers	abs	Absolute value		same
numbers	floor	Round down		same
numbers	ceil	Round up		same
numbers	clamp	Clamp between two values	lowest (number); highest (number)	same
numbers	bucket	Bucket (1 to n, between two values)	lowest; highest; buckets (numbers)	bigint
numbers	scale	Multiply by	factor (number)	same
numbers	to_text	To text		text
dates	parse_date	Parse as a date	format (`%Y-%m-%d`; empty for ISO) (optional)	date
dates	parse_timestamp	Parse as a timestamp	format (`%Y-%m-%d %H:%i:%s`; empty for ISO) (optional)	timestamp (3)
dates	format_date	Format a date as text	format (`%Y-%m-%d`)	text
dates	date_part	A part of a date	part (unit)	bigint
dates	truncate_date	Truncate a date	to (unit)	same
dates	add_days	Add days (negative to subtract)	days (number)	same
dates	days_since	Days since (to today)		bigint
flags	to_boolean	To true or false (yes, y, 1, true; no, n, 0, false)		boolean

Group	do	Label	Arguments	Type after
flags	not_null_flag	True when set		boolean
flags	is_in	True when one of	values, comma-separated (list)	boolean
flags	standardize	Standardize values (from=to, comma-separated)	from=to, comma-separated (pairs)	same

How arguments are read:

- A **number** must parse as one: "Round: decimals is a number, not "two"".
- A **unit** is one of `year`, `quarter`, `month`, `week`, `day`, `hour`.
- **pairs** are `from=to`, comma-separated: "Standardize values (from=to, comma-separated): each pair is from=to; "EU" isn't".
- A **pattern** is a Trino (Java) regular expression; a **format** is Trino's `date_parse` format.
- `fill`, `null_if`, `is_in` and `standardize` read their values as the column's type: a number as a number, a date column's value as `date '…'`, a timestamp's as `timestamp '…'`, anything else as text.
- `parse_date`, `parse_timestamp` and `parse_number` give null for a value they can't parse, rather than failing the step.
- A required argument left empty: "Round needs decimals". Too many: "Round takes 1 argument(s), not 2".

Limits. A prepare step applies at least one processor ("a prepare step applies at least one processor"). A processor on a column the input hasn't: "Trim spaces: no column region in the step's input". An unknown processor: "processor 2: no processor "titlecase"". What a processor can't say is a formula's job.

The model step (Ask a model)

What it's for. A free-text column (a review, a complaint) becomes a labelled one: each row's value is sent to a model through the platform's LLM gateway, on the project's key and budget, under the guardrail, and the answer is a new column. Or each value is searched in one of the project's knowledge banks, and the closest passage is the new column.

Who can do it. Those with `build` add one on the canvas; any member may preview it, and the preview spends the project's budget, recorded for whoever previewed. Saving needs `build`; the full run is the flow's DAG, as the project's pipelines.

What it needs. The project's gateway key: the project has an LLM budget (`llm.budget` in its spec), and the operator keeps its key as the connection `<project>-llm` (AI). Without it: "the project has no gateway key: give it an LLM budget (llm.budget in its spec) and the operator makes one". The `nearest` mode also needs a built knowledge bank.

Set up

Add a step → Ask a model, then:

Field on the canvas	JSON	What
Column	column	The column asked about
What to do	mode	`classify` (Classify into labels), `extract` (Extract), `summarize` (Summarize), `nearest` (Nearest passage in a knowledge bank)

Field on the canvas	JSON	What
Labels, comma-separated	`values`	`classify`: at least two labels, each a word or a few
What to extract / Instructions (optional)	`prompt`	`extract`: required, what to extract; `classify` and `summarize`: optional guidance
Knowledge bank	`bank`	`nearest`: the bank to search
Answer column	`name`	The new column's name; `nearest` adds `<name>_source` too
Rows a run, at most	`limit`	1 to 1000; empty or 0 means 1000

```
{"id": "topic", "kind": "llm", "inputs": ["masked"], "x": 660, "y": 0,
 "column": "text", "mode": "classify", "values": ["delivery", "sizing", "quality", "price",
   "service"],
 "prompt": "Delivery is about shipping and packaging; sizing about fit; quality about the product
   failing.",
 "name": "topic", "limit": 1000}
```

The modes

Mode	What the model is asked	The answer column
`classify`	"Classify each item below as exactly one of: <labels>." then your instructions	One of the labels (matched without regard to case), or null when the answer isn't one (counted as unread)
`extract`	"For each item below, extract what is asked." then your prompt	The text extracted, or null when there's nothing
`summarize`	"Summarize each item below in one sentence." then your instructions	One sentence
`nearest`	Not a question: each value is embedded through the gateway (the model named `embedding`) and the bank's closest passage found in Trino by cosine similarity	The passage, and `<name>_source`: where it's from (`table.column:ref`, or `files:<folder>:<key>`)

The platform asks in batches of 20 rows: one question per batch, the values numbered, each cut at 2,000 characters, and the model told to answer with a JSON array of strings, one per item, in order. A batch whose answer isn't such an array leaves that batch's answers empty and says so ("rows 21 to 40: the answer has 19 items for 20 values; their answers are empty"). Both answer columns are `varchar`.

How it behaves: when the model is asked

The model is asked in exactly two cases:

1. **You preview the model step itself** (select it and press **Preview**). The platform runs the steps before it, as you, on a sample, takes the first 50 rows (fewer if the preview asks for fewer), and asks the model about them: at most three questions of twenty rows. The result shows the rows with the answer column, and a note: "The model was asked 3 question(s) about these rows, 0.0042 spent, 1 answer(s) unread." The spend goes on the project's key, recorded for you.
2. **The flow runs.** The DAG the canvas writes asks first (one Airflow task per model step, `llm_<step>`), then builds the models with dbt. The task calls `POST /api/projects/{name}/flows/{flow}/steps/{step}/run` as the project's pipelines: the platform reads the step's input in full (no sample) up to the step's `limit`, asks the model about every row, and writes the rows with the answers as a table of the project's, `<flow>__<step>` (`review_topics__topic`), replacing the last run's. The step's dbt model reads that table. The audit log records `flow.llm`: "120 rows asked in 6 questions, 0.0310 spent, 0 answers unread". The catalog gets the table's lineage from the flow's inputs.

Nothing else asks the model. Compiling doesn't. **Previewing a later step doesn't**: in a preview of any step after a model step, the model step's answer column is there but empty (null), and the preview says "the column topic of the step topic is empty in this preview: preview that step to ask the model". Opening, saving and reading the flow don't ask either.

Limits

- The guardrail: a question carrying what looks like personal data or a secret is refused, and the whole preview or run fails, naming what was found: "asking the model: rows 1 to 20: the gateway refused (HTTP 400): the question holds an email address, which may not leave the platform; it wasn't sent to the model (the gateway's guardrail) (mask such values with a prepare step first)". Put a prepare step before the model step (`regex_replace` the addresses to `[email]`). See Lab 18.
- The budget: a question the gateway refuses because the project's budget is spent fails the run the same way.
- At most 1,000 rows a run, and the run **keeps only the rows it asked about**: the step's table, and every step after it, has at most `limit` rows. Filter before the model step: "an llm step reads at most 1000 rows a run (filter first)".
- A model step's input is computed as a preview computes it, so a second model step that reads the first one's answer gets it empty: don't chain one model step's answer into another's question.
- The answer column's name must be new: "the input already has a column topic".
- `nearest`: "the project has no knowledge bank manuals", or "the bank manuals isn't built yet: build it from the project's Knowledge card" (AI).

Previews and what they cost

What it's for. Seeing a step's rows before anything is saved, cheaply, under your own rules.

How it behaves.

- **Preview** runs one query: the steps up to the selected one as `with` clauses, then the first rows of that step. It goes to Trino through the semantic layer, signed in as you, so your readers' row filters and masks apply: a reader sees what a reader may (Access and protection).
- **Inputs are sampled.** An input of more than 50,000 rows is read with `tablesample bernoulli`, at the percentage that gives about 50,000 rows (never below 0.01%); smaller inputs are read whole. The note under the rows names the inputs that were sampled: "A sample of uploads_orders." A join or a group on a sample is a sample's answer, not the table's.
- The canvas shows 100 rows; the API takes `limit` up to 1,000 (more, or none, is 100).
- **SQL** shows the step's model as it will be saved, or, after a preview, the query the preview ran.
- A code step is previewed with its `config` removed and its `ref` and `source` calls turned into the preview's own names; other Jinja can't run there: "step x uses Jinja the canvas can't run (only ref, source and config); a preview can't show it".
- A metric step's SQL is the semantic layer's, asked as you: "its SQL isn't known yet (sign in, so the semantic layer can be asked as you)" when there's no one to ask as.

What it costs. A plain preview is one Trino query on about 50,000 rows of each input, as you (it counts against your project's limits, Operating the platform). A preview of a model step also spends the project's LLM budget: up to three questions about 50 rows. Previewing other steps never asks the model. Previews are logged by platformd (who, which step, how many rows, and a model step's cost), not recorded in the audit log.

Saving a flow as a merge request

Set up

Press **Save as merge request** (enabled when the flow compiles with no errors and the project has a repo). The canvas then shows "on storscale/flow-campaign-results" and a **merge request** link.

How it behaves

1. The flow is compiled again; a step that can't be compiled stops the save: "a step can't be compiled: filter_2: ..." (HTTP 422, with every error).
2. The platform commits, with the repo's token, on the branch `storscale/flow-<name>` (underscores become dashes), made from the branch the pipelines follow (`pipelines.ref`, or the repo's default branch in GitLab when no ref is set):
   - each step's model with its header (a code step as written), `sources.yml` and `schema.yml`;
   - the DAG `<path>/flow_<name>.py`;
   - `<path>/dbt/dbt_project.yml`, if there isn't one;
   - the deletion of models of steps no longer in the flow. The commit message is "Flow campaign_results: 10 steps into campaign_results", built on the canvas by whoever saved.
3. It opens a merge request "Flow campaign_results" into the branch the pipelines follow, or updates the open one. Saving again updates the same branch and merge request.
4. The audit log records `flow.saved` ("campaign_results: 10 steps into campaign_results, on storscale/flow-campaign-results"). A save refused for want of `build` is recorded as a refusal: "the project's editors save flows in it" (HTTP 403).

The merge request is reviewed like any other change: with reviews on, its dbt models are built on a data branch and the merge request is told how it went (merge-request reviews). Merged, the project's Airflow loads the DAG within about a minute.

The DAG. `flow_<name>`, tagged `flow`, one run at a time, on the flow's schedule (UTC; none: by hand). It builds with the platform's dbt helper, `dbt.build(project="retail", path="dbt", select="flows.campaign_results")`: `dbt build` on the flow's models only, in Trino, as the project's pipelines, its checks run as dbt tests, its artifacts sent to the catalog (so the table appears with its column lineage). With model steps, their `llm.run` tasks run first:

```
# dags/flow_review_topics.py, as the canvas writes it (abridged)
from airflow.sdk import dag
from storscale import dbt, llm

@dag(dag_id="flow_review_topics", schedule="0 7 * * 1", start_date=pendulum.datetime(2026, 1, 1,
  tz="UTC"), catchup=False,
     max_active_runs=1, tags=["flow"])
def flow_review_topics():
    asked = [llm.run(project="retail", flow="review_topics", step="topic")]
    asked >> dbt.build(project="retail", path="dbt", select="flows.review_topics")
```

A failed run is one of the project's alerts, and the Pipelines tab shows why (Pipelines and the project's repo).

Code steps: changing a flow by hand

What it's for. What the canvas can't say. An engineer opens the merge request (or the repo), changes a model's SQL, and pushes. The canvas shows that step as **Code** from then on, and the rest as steps.

How it behaves.

- A model is a step while the hash in its header matches the SQL under it. Change the SQL (or remove the header) and it's a code step: its SQL is kept exactly as written, and the canvas never rewrites it.
- A code step's inputs are its `{{ ref('…') }}` calls to models of the same folder. A ref to a model outside the folder stops the flow from opening: "model x refers to y, which isn't in the flow's folder: the canvas reads only its own models".
- A code step's columns are what Trino says its query returns, asked as you when the flow is compiled. Until then it shows "columns unknown", and so do the steps after it.
- A code step named without the flow's prefix and not marked `ephemeral` in its `config` is the flow's output, written by hand.
- On the canvas a code step's SQL can be edited in its settings; **Add a step** doesn't offer the kind. Through the API, a step `{"kind": "code", "sql": "…"}` is accepted.

See Lab 10, step 1.

The flows API

All routes take the web app's session or a Keycloak token (`Authorization: Bearer …`); with a session, a change also needs the header `X-Platform-Request: 1`. A flow is sent as `{"flow": {name, schedule, steps}}`.

Method and path	What	Who
`GET /api/projects/{name}/flows`	The flows: `{flows: [{name, branch, mergeRequest}], dbtPath, target}`, or `{flows: [], why}` without a repo	Members
`GET /api/projects/{name}/flows/{flow}`	One flow read from the repo and compiled: `{flow, order, steps, errors, branch, mergeRequest, dbtPath, processors}`	Members
`POST /api/projects/{name}/flows/compile`	Compile a flow as sent: each step's `model`, `columns`, `sql`, `error`, `unknown`; nothing runs	Members
`POST /api/projects/{name}/flows/preview`	`{flow, step, limit}`: a step's rows on a sample, as you: `{sql, columns, types, rows, seconds, sampled, notes}`; a model step's adds `cost`, `batches`, `unread`	Members
`PUT /api/projects/{name}/flows/{flow}`	Save as a merge request: `{branch, mergeRequest, new, files, dag}`	`build`
`POST /api/projects/{name}/flows/{flow}/steps/{step}/run`	Ask the model about a saved flow's model step, every row up to its limit, and write its table: `{table, rows, batches, cost, unread, warnings}`	`build`, the project's pipelines, administrators

A complete flow, as Lab 9 builds it, saved with `PUT /api/projects/retail/flows/campaign_results`:

```
{"flow": {"name": "campaign_results", "schedule": "0 6 * * 1", "steps": [
  {"id": "input", "kind": "input", "table": "uploads_campaign_responses", "x": 30, "y": 30},
  {"id": "filter", "kind": "filter", "inputs": ["input"], "where": "converted", "x": 220, "y": 30},
  {"id": "input_2", "kind": "input", "table": "uploads_orders", "x": 30, "y": 120},
  {"id": "join", "kind": "join", "inputs": ["filter", "input_2"], "join": "inner",
   "on": [{"left": "customer_id", "right": "customer_id"}], "x": 410, "y": 30},
  {"id": "filter_2", "kind": "filter", "inputs": ["join"],
   "where": "not test and not refunded and ordered_on >= responded_on and ordered_on <=
     date_add('day', 30, responded_on)", "x": 600, "y": 30},
  {"id": "input_3", "kind": "input", "table": "uploads_campaigns", "x": 410, "y": 120},
  {"id": "join_2", "kind": "join", "inputs": ["filter_2", "input_3"], "join": "inner",
   "on": [{"left": "campaign_id", "right": "campaign_id"}], "x": 790, "y": 30},
  {"id": "group", "kind": "group", "inputs": ["join_2"], "by": ["name", "budget"],
   "aggregates": [{"function": "sum", "column": "amount", "as": "revenue"},
                  {"function": "count_distinct", "column": "customer_id", "as": "buyers"},
                  {"function": "count", "as": "orders"}], "x": 980, "y": 30},
  {"id": "formula", "kind": "formula", "inputs": ["group"],
   "columns": [{"name": "roi", "expression": "revenue / budget", "type": "double"}], "x": 1170, "y":
     30},
  {"id": "prepare", "kind": "prepare", "inputs": ["formula"],
   "processors": [{"column": "roi", "do": "round", "args": ["2"]}], "x": 1360, "y": 30},
  {"id": "out", "kind": "output", "inputs": ["prepare"], "table": "campaign_results",
   "checks": [{"column": "name", "test": "not_null"}, {"column": "name", "test": "unique"}], "x":
     1550, "y": 30}
]}}
```

A preview of one step of it: `POST /api/projects/retail/flows/preview` with `{"flow": {…}, "step": "join", "limit": 100}`.

Settings

Setting	Chart value (default)	Compose	What it changes
The semantic layer	always installed (semanticd)	`SEMANTIC_URL` on platformd	Runs every preview and a metric step's SQL; without it: "the semantic layer isn't set up (no SEMANTIC_URL), which runs the previews"
The LLM gateway	`llm.gateway` (`stub`)	`stub`	`stub` answers without a model (a lab's install), `litellm` asks the company's models, `""` gives no keys, so model steps can't run (AI)
The guardrail	`llm.guardrails` (`true`), `llm.forbiddenTerms` (`[]`)	the same	What a model step's questions may not carry
The branch a repo is followed on	`pipelines.defaultRef` (`main`)	`PIPELINES_DEFAULT_REF` on the operator (`main`)	Which branch Airflow builds flows from when the project sets no `pipelines.ref` (see the gotcha below)

The preview's sample (about 50,000 rows an input), a model step's preview (50 rows), its batch (20 rows) and its run (1,000 rows at most) are fixed in the code, not settings.

Limits and gotchas

- **Saving needs an https repo with a token.** A repo over SSH runs pipelines, but the canvas can't commit to it: "the project's repo needs an https address and a token (its connection) for the platform to commit to it". The connection needs the token in its `password`: "the repo's connection has no token". The token needs GitLab's `api` scope and the Developer role; a weaker one gets HTTP 403 from GitLab's API.

- **Set `pipelines.ref`.** Without it, the canvas saves into the repo's default branch in GitLab, while Airflow follows the install's default (`main`) and reviews watch merge requests into `main`. A repo whose default branch is another (`master`) then has flows merged where nothing builds them. Name the branch in `pipelines.ref`.
- **Delete the flow's branch when its merge request merges** (GitLab's "Delete source branch"). While `storscale/flow-<name>` exists, the canvas opens the flow from it, not from the branch the pipelines follow, so a later change made on that branch in the repo isn't what the canvas shows.
- **Previews are samples.** A large input is read at about 50,000 rows, so a join's or a group's preview numbers aren't the table's. The note names the sampled inputs.
- **One output.** A flow makes one table; a second output step: "a flow has one output step; this one has 2".
- **Names.** A flow's, a step's and a new column's name are lowercase letters, digits and underscores, starting with a letter. The name in the address must be the flow's: "the flow's name is lowercase letters, digits and underscores, and the one in the address".
- **Schedule.** Cron's five fields, UTC: "a schedule as cron's five fields (UTC): 0 6 * * 1, or none".
- **Inputs from other projects** are only the tables of data products the project consumes.
- **No Python step yet**, and no Spark: every step is Trino SQL (or a model's answers written as a table).
- **Unsaved work is lost** when you leave the canvas.

See Troubleshooting for more of the errors, and Lab 18 for a model step end to end.

9. Pipelines and the project's repo

A project's pipelines are Airflow DAGs in a git repo of its own, run as the project's pipelines service account. The repo also holds its dbt models, the flows built on the canvas, its scheduled notebooks, and the project itself as code. This chapter is for the engineers who write pipelines and dbt models, the analysts who schedule notebooks, and the administrators who connect a project's repo and keep the company's shared libraries.

In this chapter:

- Connecting the project's repo
- The project's Airflow
- DAGs in the repo
- Running pipelines and reading run logs
- dbt in a pipeline
- Merge-request reviews on data branches
- Scheduled notebooks
- The company's shared libraries
- The pipelines API
- Settings
- Limits and gotchas

Connecting the project's repo

What it's for. Pipelines written as code, reviewed and versioned in git, and run by the platform: push a DAG to the repo and it's one of the project's pipelines within a minute, running as the project's service account, never as a person. The same repo is where the canvas saves flows, where notebooks are scheduled, where merge requests are reviewed on their own copy of the data, and where the project is released from.

Who can do it.

What	Who
Give the project the repo's token (a connection)	The permission `connections` (editors)
Name the repo in the project's spec (the **Edit** page)	Administrators
Start runs	`run-pipelines` (editors)
See the Pipelines tab, its runs and their logs	The project's members

Set up

1. A token in GitLab. In the repo, **Settings → Access tokens**: a project access token with the `api` scope and the **Developer** role. The platform reads the repo with it, commits flows and scheduled notebooks on branches of their own, opens merge requests, and reports reviews on them; the project's Airflow clones with it. A token with only `read_repository`, or a Guest's or Reporter's role, is refused by GitLab's API (HTTP 403). (A project made from a template gets such a token made for it, Projects.)

2. The connection. On the project's **Connections** tab, add a connection of kind **A git token**:

Field	Key	What
User (the token's name)	`username`	The token's name in GitLab
Token (GitLab: the api scope, the Developer role)	`password`	The token. Required
GitLab webhook's token	`webhookToken`	Optional: the secret a GitLab webhook sends (below)

Its name starts with the project's and a dash: `retail-git`. For a repo over SSH, the kind is **An SSH key** (`sshPrivateKey`, and `sshPrivateKeyPassphrase` if it has one); see the limits below. Through the API:

```
{"name": "retail-git", "kind": "git", "values": {"username": "storscale-retail", "password":
  "glpat-xxxxxxxxxxxxxxxxxxxx"}}
```

sent as `POST /api/projects/retail/connections` (Getting data in has the Connections tab in full).

3. The spec. As an administrator, open the project's **Edit** page and tick **Pipelines**:

Field on the page	Spec field	What
Keycloak client whose service account runs them	`pipelines.serviceAccount`	Required. The shared Airflow's is `airflow-pipelines`. A project with an Airflow of its own (on Kubernetes, with a repo) runs as a service account the operator makes for it, `project-<name>-pipelines`, whatever this says
Repo (https or ssh address, no credentials in it)	`pipelines.repo`	`https://gitlab.example.com/harbor/retail-pipelines.git`, or an ssh address (`ssh://…` or `git@host:group/repo.git`)
Folder of the DAGs	`pipelines.path`	The folder in the repo Airflow reads (`dags`); default the whole repo
Branch (or tag)	`pipelines.ref`	**The branch the pipelines follow. Set it.** Default: the install's own (`main`), see below
Connection with its token or key	`pipelines.credentials.secretRef`	The connection of step 2
GitLab address (the server's)	`pipelines.reviews.gitlab`	`https://gitlab.example.com`: the server's address, not the repo's
Project path (empty: the repo's own)	`pipelines.reviews.project`	The repo's path in GitLab, as `group/repo`; leave it empty
dbt folder	`pipelines.reviews.dbt`	The dbt project's folder in the repo: `dags/dbt` when the DAGs are in `dags`

`pipelines.memory` (the project's Airflow's memory, `2Gi` to `64Gi`, as `3Gi` or `2560Mi`) is set in the JSON box. The same in the spec:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "description": "Harbor Outfitters' retail team.",
    "members": [
      {"group": "engineers", "role": "editor"},
      {"user": "alice", "role": "editor"}
    ],
    "tables": {},
    "pipelines": {
      "serviceAccount": "airflow-pipelines",
      "repo": "https://gitlab.example.com/harbor/retail-pipelines.git",
      "path": "dags",
      "ref": "main",
      "credentials": {"secretRef": "retail-git"},
      "memory": "3Gi",
      "reviews": {"gitlab": "https://gitlab.example.com", "dbt": "dags/dbt"}
    }
  }
}
```

How it behaves

- **Within a minute** the operator gives Airflow the repo: an Airflow connection `git_<project>` (dashes as underscores) with the token, and a DAG bundle `project-<project>` that follows `pipelines.ref` in `pipelines.path`. On Kubernetes the project gets an Airflow of its own (below).
- Airflow fetches the branch every 60 seconds. Each pipeline and each run shows the commit it came from.
- The platform's own uses of the repo (the canvas, scheduling a notebook) talk to GitLab's API at `pipelines.reviews.gitlab` when reviews are set, else at the repo address's scheme and host, with the connection's `password`.
- Removing `pipelines.repo` removes the Airflow connection and the project's bundle, and with them its DAGs.

Which branch. Three things follow a branch of the repo, and they agree only when `pipelines.ref` is set:

What	Follows
Airflow (the DAGs that run)	`pipelines.ref`, else the install's `pipelines.defaultRef` (`main`)
The canvas and scheduled notebooks (where merge requests go)	`pipelines.ref`, else the repo's default branch in GitLab
Merge-request reviews (which merge requests are reviewed)	`pipelines.ref`, else `main`

A repo whose default branch isn't `main` (an old `master`) and no `ref` set: flows and notebooks are merged into `master`, while Airflow looks for a `main` the repo doesn't have. Name the branch.

See Lab 10 in the lab guide.

The project's Airflow

What it's for. A project's DAG code is code: it shouldn't be able to read another project's connections or the platform's secrets. On Kubernetes, each project with a repo gets an Airflow of its own.

How it behaves.

	On Kubernetes (the chart)	In Compose
Where a project's DAGs run	Its own Airflow, `project-<name>-airflow`: one pod with the API server, the scheduler (LocalExecutor, four tasks at a time) and the DAG processor, its own Postgres database (`airflow_<name>`), its own keys	The shared Airflow
Its DAGs	Its repo's bundle, and the platform's own DAGs (merge-request reviews, plans, deliveries)	Its repo's bundle, in the shared Airflow beside the platform's and other projects'
Runs as	`project-<name>-pipelines`, a Keycloak service account that's a member of this project alone	`pipelines.serviceAccount` (`airflow-pipelines`)
Its UI	**Open in Airflow** on the Pipelines tab opens `/airflow/<name>/` in a tab of its own	**Open in Airflow** opens the shared Airflow (**Tools → Pipelines**)
Memory	`pipelines.memory`, else the chart's `pipelines.memory` (`3Gi`)	the shared Airflow's

A project's imports (Getting data in) run in the platform's Airflow even when the project has its own, and are listed on its Pipelines tab beside its own pipelines.

DAGs in the repo

What it's for. Any Airflow DAG the project needs, written in Python, reviewed in git.

Set up

Push a DAG file into `pipelines.path`. Airflow reads only files that mention both "airflow" and "dag" (its safe mode), so a file of helpers without them is skipped. A minimal one:

```
# dags/orders_daily.py: an Airflow DAG
import pendulum
from airflow.sdk import dag, task


@dag(dag_id="orders_daily", schedule="0 5 * * *", start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
  catchup=False)
def orders_daily():
    @task
    def hello():
        print("retail's pipelines, as the project's service account")
    hello()


orders_daily()
```

How it behaves

- The project's pipelines are the DAGs of its bundle (`project-<name>`), and those of the platform's own DAGs tagged `project:<name>`. A tag `project:<name>` on a DAG from another project's repo counts for nothing.
- Its tasks run as the project's pipelines: in Trino, Buckets and the catalog they may do what the project's pipelines may (Roles).
- **A file that doesn't load** is shown on the Pipelines tab: "A file in the repo didn't load", with the file and the end of its error. The pipeline stays at its last good version.
- **Two repos, one DAG ID.** Airflow keeps one DAG per ID. The Pipelines tab warns: "Another repo has a DAG with this ID too (marketing). Airflow keeps only one at a time, so this one may show the other's code. Give one of them a new `dag_id`."

Running pipelines and reading run logs

Set up

Open the project's **Pipelines** tab. On the left, the project's pipelines, each with its last run's state. For the one selected:

- where it's from ("From the repo, at `3f1c0e9`", or "The platform's own, in Airflow's DAGs folder"), paused or not;
- **Recent runs**, up to twelve, as coloured marks;
- **Latest run**: its tasks on one time axis, its state, how long it took, who started it;
- a table of runs: state, commit, started, took, and **By** (who started it, or "schedule");
- for a run that ended, **log** (or **why it failed**): what each task logged, its last try, in the words of what ran (dbt's, dlt's, the notebook's);
- for a scheduled notebook's run, **notebook**: the executed notebook (below).

Run now (with `run-pipelines`) starts a run. "Started run `manual__…`. It runs as the project's pipelines service account." whoever pressed it; the run records who asked (`requested_by`), and the audit log records `pipeline.started`.

The tab polls Airflow every five seconds.

Limits

- Without `run-pipelines`: "The project's editors can start runs."; through the API, "only the project's editors run its pipelines" (HTTP 403).
- A pipeline that isn't the project's can't be started from it: "no pipeline "x" in project retail".
- A pipeline whose latest run failed is one of the project's alerts ("orders_daily's latest run failed"), sent where the project's `notify` says (Data quality).
- Pausing, clearing and backfilling are Airflow's: **Open in Airflow**.

dbt in a pipeline

What it's for. SQL transformations kept as dbt models in the project's repo, with their tests, built in Trino on the project's Iceberg tables as the project's pipelines, their models, columns, tests and lineage shown in the catalog.

Set up

Put a dbt project in the repo beside the DAGs, `dags/dbt/` (the canvas's flows go in the same one, under `models/flows/`). Its `dbt_project.yml` names the profile `storscale` and holds no settings or secrets:

```
# dags/dbt/dbt_project.yml
name: retail
version: '1.0'
config-version: 2
profile: storscale
model-paths: [models]
```

Then a DAG that builds it with the platform's helper:

```
# dags/retail_models.py: an Airflow DAG
import pendulum
from airflow.sdk import dag

from storscale import dbt


@dag(dag_id="retail_models", schedule="@daily", start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
  catchup=False)
def retail_models():
    dbt.build(project="retail", path="dbt")   # the dbt project beside this DAG file


retail_models()
```

`storscale.dbt.build(project, path="dbt", schema=None, catalog="iceberg", task_id="dbt_build", select=None)` is one task:

Argument	What
`project`	The project's name
`path`	The dbt project's folder, relative to the DAG file (default `dbt`)
`schema`	The project's namespace (default its name, dashes as underscores)
`catalog`	The Trino catalog (default `iceberg`)
`task_id`	The task's ID (default `dbt_build`)
`select`	dbt's selection (`flows.campaign_results`, `tag:daily`); default everything

How it behaves

1. dbt runs in a virtualenv of its own (`/opt/dbt`: dbt-core and dbt-trino), signed in to Trino with a Keycloak token of the project's service account (the Airflow connection `keycloak_pipelines`, which the platform keeps): Ranger's rules are the pipelines', and its audit names the account.
2. The profile is written for the run: Trino's address and certificate, the catalog, the project's namespace, four threads.
3. With a `packages.yml`, the project is copied to a folder of the run's and `dbt deps` installs its packages there (below).
4. `dbt build` runs the models and their tests; then `dbt docs generate`, whatever came of the build.
5. The run's artifacts (`manifest.json`, `catalog.json`, `run_results.json`) go to the platform, as the same account (`POST /api/projects/{name}/dbt/artifacts`, the project's pipelines only; recorded as `dbt.artifacts`). The catalog reads them: the models, their columns, their lineage column by column, and their tests' results. The last manifest is also what merge-request reviews compare a change with.
6. A failed build fails the task: "dbt build failed (exit 1): its log above says which model or test".

dbt packages and the company libraries

A `packages.yml` beside `dbt_project.yml` is installed with `dbt deps` at each run (the repo's checkout is read-only, so the project is copied first). A package that names one of the company's shared libraries by its repo's address is replaced by the library's files at the ref the administrators gave it, fetched through the platform, so the project holds no token for it:

```
# dags/dbt/packages.yml
packages:
  - git: "https://gitlab.example.com/data/libraries/company-dbt.git"   # a shared library: fetched
    through the platform
  - package: dbt-labs/dbt_utils                                        # any other package: as dbt
    deps gets it
    version: 1.3.0
```

- The address is matched without credentials, `.git` or a trailing slash, in any case. Administration → Templates shows each library's address in the column **In packages.yml**.
- A library's `revision` is ignored, saying so in the log ("the library's ref is the administrators' (revision v2 ignored)"); a `subdirectory` is kept.
- The run's log names the libraries used: "storscale.dbt: the company's libraries: company-dbt".
- A failed `dbt deps`: "dbt deps failed (exit 2): its log above says which package".

Merge-request reviews on data branches

What it's for. A change to the project's dbt models is built and tested on its own copy of the data before it merges, and the merge request says what it would do: which models built, their rows against production's, which rows differ, which metrics move, which tests fail, and whether a data product's contract would break. Production is untouched.

Who can do it. An administrator turns it on (`pipelines.reviews`). Anyone who can open a merge request in the repo gets it reviewed.

Set up

`pipelines.reviews` with `gitlab` (the GitLab server's address) and `dbt` (the dbt project's folder in the repo), on an `https://` repo with a token (`credentials`). Optionally, a GitLab webhook so a review starts at once: in the repo, **Settings → Webhooks**, the address `https://<platform>/api/webhooks/gitlab/<project>`, the secret token the same as the connection's `webhookToken`, and the events **Merge request events** (and **Push events**). Without a webhook the operator finds new merge requests and commits at its next pass, within a minute or two.

How it behaves

1. Each pass (every minute), the operator lists the repo's open merge requests into the branch the pipelines follow (`pipelines.ref`, else `main`).
2. For each, a **data branch**: a Nessie branch `<project>-mr-<iid>` (`retail-mr-7`), made from the catalog's main as it is when the merge request is first seen, and a Trino catalog on it, `<project>_mr_<iid>` (`retail_mr_7`).
3. At each new commit of the merge request, it starts a run of the platform's DAG `storscale_review` in the project's Airflow, and sets the commit's status `storscale/dbt` to pending.
4. The run checks the repo out at that commit, fetches production's last manifest, and runs `dbt build --select state:modified+` (the models the change touches and those downstream; everything when production has no manifest yet), with the project's packages, in the branch's catalog, as the project's pipelines.
5. It tells the merge request, in one note it updates at each commit ("DataWorks: dbt on the data branch"):
   - each model built, with its rows on the branch and in production, and a sample of the rows that differ;
   - the project's metrics whose values differ on the branch from production's (up to twenty metrics);
   - each failed model or test, with its message;

- with `storscale/semantic.yaml` in the commit, the semantic model's tests run on the branch, those that fail and those whose answers changed; their own status `storscale/semantic`;
- each data product whose contracted columns the branch's table would lose, and who reads it.

6 The status `storscale/dbt` ends `success` ("3 models built, their tests passing") or `failed` (naming what failed). The review's own run is in the project's Airflow (**Open in Airflow**), not on the Pipelines tab.

7 When the merge request is merged or closed, its catalog and its Nessie branch are removed. Production's own runs build what was merged.

The webhook route answers only for a project with reviews, and only with the right token: "the webhook's token isn't the project's (webhookToken)" (HTTP 401). Other GitLab events are ignored.

Limits

- GitLab only: the reviews use GitLab's merge requests, statuses and notes.
- The reviews' settings are refused when they can't work: "reviews: gitlab "gitlab.example.com": GitLab's address, https://...", "reviews: GitLab's API needs the repo's credentials (a token, for an https repo)", "reviews: dbt "": the dbt project's folder in the repo", "reviews: project "x/y" isn't the repo's path, "harbor/retail-pipelines": leave it empty for the repo's own".
- A data branch holds the data as it was when the merge request was first seen; a long-open merge request reviews against an older copy.
- Only dbt models are built on the branch, not the project's other DAGs.

See Lab 10, step 2.

Scheduled notebooks

What it's for. An analyst's notebook runs on a schedule as a pipeline, reviewed like any other change: no notebook running from a laptop, no copy pasted into a DAG.

Who can do it. Someone with `build` in a project that has an https repo with a token.

Set up

1 In JupyterLab (**Tools → Notebooks**), press **Schedule** on the notebook's toolbar, or **Schedule in a project...** on a notebook's menu in the file browser.

2 In **Schedule this notebook**, pick the **Project** (those you may schedule in) and **When (cron, UTC)**: `0 5 * * *` is every day at 05:00; `0 6 * * 1` is Mondays at 06:00. Press **Schedule**.

3 "Scheduled, for review": the merge request's link. Merge it.

How it behaves

- The notebook is sent with its outputs cleared (what it computed may be data no repo should hold).
- The platform commits two files on the branch `storscale/schedule-<name>`, from the branch the pipelines follow: the notebook, `<path>/notebooks/<name>.ipynb`, and its DAG, `<path>/notebook_<name>.py`, five lines:

```
from storscale import notebook

notebook.schedule(project="retail", name="call_list", schedule="0 6 * * 1", owner="alice")
```

`<name>` is the file's name in lower case, other characters as underscores (`Call List.ipynb` → `call_list`).

- It opens a merge request "Schedule call_list.ipynb", or updates the open one: scheduling again updates the same branch. The audit log records `notebook.scheduled`.
- Merged, the DAG `notebook_<name>` runs the notebook with papermill, in a virtualenv with the notebook image's packages, as the project's pipelines: `buckets_lake` signs in as the pipelines' service account.
- Each run's executed notebook (and its HTML) is kept by the platform, linked from the run on the Pipelines tab (**notebook**), and recorded as `notebook.ran`. A failed run says which cell failed and why, and is one of the project's alerts.

Limits

- No project to pick: "You aren't an editor of a project with a repo, where a notebook can be scheduled."
- The schedule is cron's five fields, UTC: "a schedule as cron's five fields (UTC): 0 5 * * *".
- The notebook runs with the image's packages; a package of the project's own isn't installed.

See Lab 11, and Notebooks and models.

The company's shared libraries

What it's for. Code kept once and used by every project without copying it: dbt macros and generic tests, Python modules. The administrators name a library's repo in the install's GitLab and a ref; projects use it by name, and the platform reads it for whoever asks, so no project holds a token for it. A new ref reaches every project at its next run.

Who can do it. Administrators add, change and remove libraries. Anyone the realm signed in (a person, a project's pipelines, an agent) may read them.

What it needs. The install's GitLab (`templates.gitlab` in the chart; Settings). Without it, Administration → Templates says "Libraries need the install's GitLab (templates.gitlab)."

Set up

1 **Administration → Templates → Shared libraries → Add a library**:

Field	JSON	What
Name	`name` (from the address)	Lower-case letters, digits and dashes: `company-dbt`
What it's for	`description`	Required
Its repo in GitLab	`repo`	Its path, as `group/repo`: `data/libraries/company-dbt`
Ref	`ref`	A tag (best: it doesn't move), a branch or a commit: `v1.0.0`

The platform checks the repo has that ref, with the install's token. **Change** moves a library to another ref.

2 Use it:

- **In a dbt project**: name it in `packages.yml` by its address, `- git: "https://gitlab.example.com/data/libraries/company-dbt.git"` (above).
- **In a pipeline**: `storscale.library.use(name, path="")` puts the library's files (its folder `path` in the repo, default its top) on `sys.path`:

```
from airflow.sdk import task
from storscale import library


@task
def score():
    library.use("company-utils")          # its repo's files on sys.path, at its ref
    from company_utils import clean
    ...
```

`library.libraries()` lists them; `library.fetch(name)` returns the folder.

- **In a notebook**: `buckets_lake.library("company-utils")`, then import its modules.

Through the API (administrators): `PUT /api/libraries/company-dbt` with

```
{"description": "The company's dbt macros and checks", "repo": "data/libraries/company-dbt", "ref":
  "v1.0.0"}
```

How it behaves

- Each use asks the platform which commit the library's ref names now, and fetches the files at that commit as a tar archive (the install's token reads GitLab). Each commit is fetched once per Airflow worker or notebook server (a cache under `/tmp`).
- The audit log records `library.saved` (with the repo, the ref and its commit, and what it was) and `library.removed`.
- A removed library can't be fetched any more: "no library x (Administration → Templates lists them)" (HTTP 404).

Limits

- Libraries live in the install's GitLab, read with the install's token: the token must be able to read the library's repo.
- `repo: the library's path in GitLab, as group/repo`; `ref: a tag, a branch or a commit (a tag is best: it doesn't move)`; "the library's repo data/x at v9: ..." when GitLab hasn't it.
- A library at a branch changes under every project when the branch moves: use tags.

The pipelines API

Method and path	What	Who
`GET /api/projects/{name}/pipelines`	The pipelines, their recent runs and the newest run's tasks; `canRun`; the repo (`url`, `path`, `ref`) and the files Airflow couldn't load; `airflowPath` for a project's own Airflow	Members
`POST /api/projects/{name}/pipelines/{pipeline}/runs`	Start a run: `{run}`	`run-pipelines`
`GET /api/projects/{name}/pipelines/{pipeline}/runs/{run}/log`	What its tasks logged: `{log}`	Members
`POST /api/projects/{name}/dbt/artifacts`	A dbt run's artifacts, for the catalog	The project's pipelines

Method and path	What	Who
`GET /api/projects/{name}/dbt/artifacts/{artifact}`	The last run's `manifest` (for reviews)	The project's pipelines
`POST /api/projects/{name}/notebooks/schedules`	`{path, schedule, notebook}`: schedule a notebook (JupyterLab calls it)	`build`
`GET /api/projects/{name}/notebooks/runs/{dag}/{run}`	A run's executed notebook: `?format=html` (default) or `ipynb`	Members
`POST /api/webhooks/gitlab/{name}`	A GitLab webhook's merge request or push event	GitLab, with `webhookToken`
`GET /api/libraries`, `GET /api/libraries/{library}`, `GET /api/libraries/{library}/archive`	The libraries; one with the commit its ref names; its files (`?commit=`)	Anyone signed in to the realm
`PUT /api/libraries/{library}`, `DELETE /api/libraries/{library}`	Add or change, remove a library	Administrators

With a session, a change needs the header `X-Platform-Request: 1`; a Keycloak token (`Authorization: Bearer …`) works too.

Settings

Setting	Chart value (default)	Compose	What it changes
The branch a repo is followed on when its project names none	`pipelines.defaultRef` (`main`)	`PIPELINES_DEFAULT_REF` on the operator (default `main`)	Airflow's branch for a project without `pipelines.ref`
A project's Airflow's memory	`pipelines.memory` (`3Gi`)	n/a (the shared Airflow)	When the project sets no `pipelines.memory`
The git hosts' SSH keys	`pipelines.gitKnownHosts` (`""`; `--set-file pipelines.gitKnownHosts=known_hosts`)	`GIT_KNOWN_HOSTS`	SSH repos checked against them; without, a host's key is taken the first time it's seen
A project's own Airflow	always on Kubernetes (`PROJECT_AIRFLOWS=true`)	off: one shared Airflow	Where a project's DAGs run
The install's GitLab, for templates and libraries	`templates.gitlab.url`, `.group`, `.tokenSecret` (a Secret with key `token`)	`GITLAB_URL`, `GITLAB_GROUP`, `GITLAB_TOKEN` in `stack/.env`	Shared libraries and project templates; a new token is read at each use

Limits and gotchas

- **Set** `pipelines.ref` to the branch: see which branch.
- **An SSH repo** runs pipelines (with `sshPrivateKey`; a key with a passphrase can't be checked out for reviews or plans), but the canvas, scheduled notebooks and reviews need an `https://` repo with a token.
- **No credentials in the address**: "repo: no user or token in the address; name a Secret in credentials". An ssh repo without credentials: "an ssh repo needs credentials: a Secret with sshPrivateKey".
- **path, ref, credentials, memory and reviews need a repo**: setting them without `pipelines.repo` is refused.
- **The connection must be the project's**: one named in `secretRef` that isn't among its connections stops the operator: "no secret "retail-git" (the repo's credentials) among the project's connections"; one without a token: "secret "retail-git" has no password (a token) for an https repo".

- **The token's scope**: `api` and Developer. Merge requests and statuses fail with HTTP 403 otherwise.
- **The Pipelines tab says "every few minutes"**; Airflow in fact fetches the branch every 60 seconds.
- **DAG IDs are global** to an Airflow: in Compose, every project shares one, so give DAGs IDs that name the project.

See Troubleshooting for the errors by feature, and History and releases for releasing the project itself from the same repo.

10. Notebooks and machine learning

This chapter is for analysts and data scientists who work in notebooks, and for the editors and administrators who put their models to use. It covers JupyterLab and VS Code in the platform, the `buckets_lake` module that signs a notebook in as you (Buckets, SQL, MLflow, the company's libraries), Spark across many machines, each project's own MLflow workspace, registering and approving model versions, model cards, scoring a table on a schedule with drift checks, serving a model as an API, how models appear in the catalog's lineage, scheduled notebooks, and the four notebook templates.

In this chapter:

- JupyterLab and VS Code
- buckets_lake: the notebook signed in as you
- Distributed Spark
- MLflow, a workspace per project
- Registering and approving model versions
- Model cards
- Scoring on a schedule, and drift
- Serving a model as an API
- Models in the catalog's lineage
- Scheduled notebooks
- The notebook templates: tables, churn, propensity, forecast
- Settings
- Errors you may meet

Lab 11 in the lab guide walks through most of this end to end: Lab 11: Customers worth a call.

JupyterLab and VS Code

What it's for. A notebook server of your own, inside the platform, signed in as you: no keys to paste, the same rules as everywhere else, and your files kept between sessions. VS Code runs in the browser on the same files for those who prefer an editor.

Who can do it. Anyone in the Keycloak groups `analysts` or `engineers` may sign in to JupyterHub; members of `engineers` are the hub's administrators. Of the example people, alice (analysts) and bob (engineers) can; carol (no groups) can't. These groups are set in `stack/jupyterhub/jupyterhub_config.py` (`allowed_groups`, `admin_groups`), not by a chart value.

Set up

Nothing to set up per project. To open it:

1. **Tools → Notebooks**, or a project's **Notebooks** tab (shown when the install has notebooks). JupyterLab opens in the platform's frame, at the platform's `/notebooks/`, signed in through Keycloak.
2. Your files are in `work/` (`/home/jovyan/work`). They live on a volume of your own that outlives the server: on Kubernetes a persistent volume claim `notebooks-<you>` (size `notebooks.storage`, default `1Gi`); in Compose a Docker volume `storscale-notebooks-<you>`.

3 For VS Code, open JupyterLab's launcher and pick **VS Code**: code-server at your server's `/vscode/`, on the same `work/` folder, with no sign-in of its own (the hub has signed you in).

How it behaves

- Each person gets a server of their own: a container (Compose) or a pod (Kubernetes). It starts within two minutes (the hub's start timeout is 120 seconds).
- The hub keeps your Keycloak tokens, encrypted, and refreshes them when they're more than two minutes old, and before each start. `buckets_lake` asks the hub for the current one, so a notebook is always signed in as you.
- A server serves every project you're in: its CPU and memory are yours, not a project's (`notebooks.cpuLimit`, `notebooks.memoryLimit`).
- The image has, beside JupyterLab 4.5: boto3, requests, pandas, pyarrow, scikit-learn, statsmodels, scipy, LightGBM, XGBoost, FLAML, MLflow (the skinny client), skops, geopandas and networkx.
- Only the platform's own pages may frame the hub and the notebook servers.

Limits and gotchas

- Notebook servers are a person's, so a project's limits (Projects) don't cap them; the install's `notebooks.*` settings do.
- `geopandas` and `networkx` are in the notebook image but not in the environment a scheduled notebook runs in (see Scheduled notebooks).
- Someone outside `analysts` and `engineers` is refused at the hub's sign-in, whatever projects they're in.

---

buckets_lake: the notebook signed in as you

What it's for. One module, already on the path, that turns your sign-in into everything a notebook needs: Buckets (S3) credentials, SQL through the platform under your rules, the project's MLflow, and the company's shared libraries. No key lives in the notebook.

Who can do it. Anyone signed in to JupyterLab. What each call may reach is what you may reach: your Buckets policies follow your Keycloak groups, and SQL runs as you under the project's rules (Access and protection).

The calls

Call	What it gives
`buckets_lake.username()`	Your name, as Keycloak gives it (`preferred_username`).
`buckets_lake.token()`	Your current Keycloak access token, from the hub.
`buckets_lake.session()`	A boto3 session whose credentials renew themselves.
`buckets_lake.s3()`	A boto3 S3 client for Buckets, as you.
`buckets_lake.sql(project, query)`	A query's answer as a pandas DataFrame, run in Trino through the platform's SQL route, as you.
`buckets_lake.tables()`	The tables `sql()` has read in this kernel, as `catalog.schema.table`, sorted.
`buckets_lake.mlflow(project)`	The `mlflow` module, pointed at the project's MLflow workspace, signed in as you.
`buckets_lake.library(name, path="")`	One of the company's shared libraries put on `sys.path`; returns its folder.
`buckets_lake.spark(tables=None, create=None)`	A Spark session on the tables you read whole (Kubernetes, the Distributed Spark profile): see Distributed Spark.

s3() and session()

```
import io, pandas as pd, buckets_lake
s3 = buckets_lake.s3()
orders = pd.read_csv(io.BytesIO(s3.get_object(Bucket="retail",
  Key="landing/uploads/orders.csv")["Body"].read()))
s3.put_object(Bucket="home", Key=f"{buckets_lake.username()}/notes.txt", Body=b"draft")
```

The token is exchanged with Buckets' STS (`AssumeRoleWithWebIdentity`, role `arn:minio:iam:::role/jupyterhub`) for credentials that last an hour and renew themselves. Your private prefix is `home/<username>/`. A project's bucket is named after the project unless its spec says otherwise (`files.bucket`).

sql()

```
orders = buckets_lake.sql("retail", "SELECT region, sum(amount) AS revenue FROM uploads_orders GROUP
  BY region")
```

- The query uses the project's own table names (`FROM uploads_orders`, not `iceberg.retail.uploads_orders`), and runs as you, under the project's rules: a reader's row filters and masks apply, as on the **SQL** tab.
- Types come back as pandas types: integers as `int64` (`float64` when there are nulls), reals and decimals as `float64`, booleans as `bool` (`boolean` with nulls), dates and timestamps as datetimes. A timestamp with a time zone pandas can't read stays text.
- At most 100,000 rows. A full answer warns: "the answer has 100000 rows, the most the platform's SQL route gives: there may be more". Aggregate or sample in the query.
- A refusal raises with the platform's reason: `PermissionError` when it's who you are (401 or 403), `RuntimeError` otherwise, worded "the platform refused the query on retail (403): ...".
- **What it read is kept for lineage.** After the answer, `sql()` asks Trino for the query's `EXPLAIN (TYPE IO, FORMAT JSON)` and records the tables it read. `buckets_lake.tables()` lists them. On MLflow runs they become the tag `lake.tables` (see MLflow). A query `EXPLAIN` can't describe is simply not recorded; the answer still stands.

mlflow()

```
mlflow = buckets_lake.mlflow("retail")
mlflow.set_experiment("churn")
```

Sets `MLFLOW_TRACKING_URI` to the platform's `/mlflow/retail` and `MLFLOW_TRACKING_TOKEN` to your token, renewed every two minutes. It also registers a run-context provider, so each run started from then on is tagged `lake.tables` with what `sql()` read before it; and `sql()` run inside a run updates the tag.

library()

```
where = buckets_lake.library("harbor-utils")          # the library at the ref the administrators
  gave it
from harbor_utils import cleaning
```

The platform reads the library from GitLab for you (`GET /api/libraries/{library}` names the commit; `GET /api/libraries/{library}/archive?commit=...` gives its files), so no GitLab token is needed in the notebook. Each commit is fetched once into a cache under `/tmp/storscale-libraries`. `path` names a folder inside the library to put on `sys.path` instead of its top. Libraries are kept on **Administration → Templates**; see Pipelines and the project's repo for defining them. A failure raises `RuntimeError("library <name>: HTTP <status> ...")`.

The pipelines' variant

A scheduled notebook, and an agent's notebook, run in Airflow with a `buckets_lake` of the same calls (`s3()`, `session()`, `username()`, `token()`, `sql()`, `tables()`, `mlflow()`, `library()`), so a notebook that ran by hand runs on a schedule unchanged. The differences:

	In JupyterLab	On a schedule
Signed in as	you	the project's pipelines' service account, or the agent's client for an agent's run
`username()`	your name	`STORSCALE_RUN_AS`: `service-account-<client>`
Token	from the hub	client credentials from Keycloak, fetched for each call
STS role	`jupyterhub`	`airflow`
`agent()`, `ask()`	absent	present: for agents only (see AI)

The project's pipelines read and write all of the project's tables, so a notebook that worked under your reader's rules may see more rows on a schedule; one that worked as an editor may see fewer as an agent, which reads as a reader does.

Limits and gotchas

- `sql()` posts to the platform's SQL route; it doesn't connect to Trino itself, so the 100,000-row limit holds whatever the client.
- `lake.tables` holds at most 8,000 characters (MLflow's limit for a tag's value); tables past that are left out.
- The `storscale.*` tags are the platform's: name your own tags anything else.

Distributed Spark

What it's for. Work too big for one notebook: Spark's DataFrames and SQL on the platform's tables, run across executors that start as a job needs them and stop when it's done. The notebook stays where you write the code; the work happens on other machines.

Who can do it. Anyone who may sign in to JupyterLab, on an install on Kubernetes (Compose has no Spark). Spark reads the tables' files itself, so Ranger can't filter rows or mask columns for it: a session gets only the tables **you read whole**, with no row filter or column mask applying to you, and writes only those you may insert into and delete from. alice, an analyst who reads `sales.orders` under the EU filter and the card mask, gets no Spark access to it; bob, its editor, reads and writes it.

Set up

1. When your notebook starts, the hub asks which server you want. **Distributed Spark** is picked unless you pick **Notebook only** (an install may make the notebook alone the default). To switch later: **File → Hub Control Panel → Stop My Server**, then **Start My Server**.
2. In a notebook:

```
import buckets_lake
spark = buckets_lake.spark()
orders = spark.table("sales.orders")          # iceberg.sales.orders, as Trino names it
orders.groupBy("region").sum("amount").show()
buckets_lake.spark_tables                      # what it got: {"sales.orders": "read and write"}
buckets_lake.spark_skipped                     # and what it didn't, and why
```

3 To write a new table, have the platform make it first: `buckets_lake.spark(create={"sales.order_counts": df.schema})` (a DataFrame's schema, or `{column: Trino type}`). Trino makes it, as you, so only where you may make tables; then `df.writeTo("sales.order_counts").append()`. An existing table you may change is written the same way.

How it behaves

- Spark's driver runs beside your notebook, in a container of its own; the executors are pods in the install's Spark namespace, up to `notebooks.spark.executors.max` of them, let go after `idleTimeout` with nothing to do. Stopping your server stops them too.
- Each `spark()` asks the platform for credentials: it asks Trino, as you, which tables a plain `SELECT *` gives you whole, and gets Buckets credentials for those tables' files alone, for 12 hours (`credentialHours`). A new session picks up tables made or shared with you since.
- The session's catalog is the platform's, not Nessie itself: it lists and loads only the session's tables, commits only to those you may change, and stays on the catalog's main branch.
- Every grant and every commit is in the audit log (`spark.credentials`, `spark.create`, `spark.commit`).

Limits and gotchas

- A table with a row filter or mask for you stays Trino's: read it with `buckets_lake.sql()`.
- One session takes about thirty tables (Buckets' policy limit). With more, name the ones you need: `buckets_lake.spark(tables=["sales.orders", "sales.payroll"])`.
- **Spark's own** `MERGE INTO`**,** `UPDATE` **and** `DELETE FROM` **fail** with `[INTERNAL_ERROR] The Spark SQL phase planning failed` (Spark 4.1 with Iceberg 1.12, as of 0.42.0): change rows in SQL, through Trino (`buckets_lake.sql()`), or rewrite the table from Spark (`writeTo(...).overwritePartitions()`, as the tables template does). `append()` and `overwritePartitions()` work.
- Spark's own `CREATE TABLE`, `DROP TABLE` and `ALTER TABLE ... RENAME` are refused ("a Spark session doesn't make, drop or rename tables"): make a table with `create=`, and drop or rename one in SQL, through Trino.
- A job running past the credentials' hours loses its access: start a new session.
- The notebook has no Kubernetes access; only Spark's driver may start pods, and only in the Spark namespace.

MLflow, a workspace per project

What it's for. One MLflow tracking server and model registry for the install, with a workspace per project: experiments, runs, logged models, registered models and artifacts are the project's, and no other project sees them. Each run is stamped with who ran it and the data it read, so a model can always be traced to its data.

Who can do it.

Who	May
The project's members (readers included)	Read its experiments, runs and models.
Members with the permission `models` (editors) and the project's pipelines	Track runs, log and register models.
Administrators, signed in	As an editor.
Anyone else	Nothing: "not a member of the project".

Set up

Nothing per project: the workspace is made the first time anyone in the project reaches it. From a notebook, use `buckets_lake.mlflow("<project>")`. From anywhere else (a laptop, a CI job), point MLflow's own client at the platform:

```
export MLFLOW_TRACKING_URI=https://<platform>/mlflow/retail
export MLFLOW_TRACKING_TOKEN=<a Keycloak access token>
```

The project's **Models** tab shows this address under **From a notebook**.

```
import buckets_lake
mlflow = buckets_lake.mlflow("retail")      # the project's workspace, as you
mlflow.set_experiment("churn")
with mlflow.start_run():                    # stamped with you and the data's commit
    ...                                     # train; mlflow.log_metric("auc", auc)
    mlflow.sklearn.log_model(model, name="model", registered_model_name="churn")
```

How it behaves

- Only platformd reaches MLflow, at `/mlflow/<project>/...`. It signs the caller in (the web app's session, or a Keycloak token as `MLFLOW_TRACKING_TOKEN`), checks they're in the project, and names the project's workspace on every request, whatever the caller sent: a request naming another workspace still lands in the project's.
- Only MLflow's tracking, registry and artifact routes go through (`/api/2.0|3.0/mlflow/{experiments,runs,metrics,registered-models,model-versions,logged-models,artifacts,datasets,traces}` and `/mlflow-artifacts/artifacts`). MLflow's own web UI is not served; the **Models** tab is the platform's view.
- **Stamps on every run.** When a run is created, the platform adds `storscale.user` (who ran it) and `storscale.data_commit` (the catalog's Nessie `main` commit at that moment: every table as it was). With `buckets_lake.sql()`, the run also carries `lake.tables`. Together they say exactly which data the model learned from; read it again with **As of** on the SQL tab (History and releases).
- **Stamps on every version.** When a model version is created, the platform adds `storscale.registered_by`.
- Artifacts are kept in Buckets (the bucket `mlflow`), proxied through MLflow.
- Deleting a project deletes its workspace: first its experiments' artifacts, then the workspace with its experiments, runs and models.

Audit log: `mlflow.run` ("a run started") for each run created, and `model.registered` for each registered model or version created, under the person or pipelines that did it; a refused write is `mlflow.write`.

Limits and gotchas

- You can't set, change or delete a `storscale.*` tag, or the alias `approved`, through MLflow. The platform refuses with 403: "the storscale.* tags are the platform's: a version is approved on the project's Models tab".
- A reader's write (a new run, a registration) is refused: "the project's readers read its experiments and models; its editors and pipelines write them".
- An agent is a reader of its project: it reads the project's experiments and models but doesn't log or register.
- A route other than tracking, registry or artifacts answers 404: "not one of MLflow's routes the platform serves (tracking, the registry, artifacts)".
- An install without MLflow answers 503 "MLflow isn't set up here (no MLFLOW_URL)", and the Models tab says "MLflow isn't set up on this install."

Registering and approving model versions

What it's for. A model reaches a pipeline or an API only once someone other than its author has looked at it. Registering makes a numbered version; approving it, by a second editor, points the alias `approved` at it, and that's the version scorings and servings load.

Who can do it. Registering: members with `models` (editors) and the project's pipelines. Approving: members with `approve` (editors), never the person who registered that version. Administrators approve as editors, under the same rule.

Set up

1. **Register.** In MLflow, `registered_model_name` on `log_model` (or `mlflow.register_model(...)`) makes a new version of a registered model in the project's workspace. On the project's **Models** tab, the model's card lists the version with **Registered** (when) and **By** (you).
2. **Approve.** Another editor opens **Models** and presses **Approve** on the version's row. To the person who registered it, the row says "another editor approves it" instead of a button. The model's heading then shows "version N approved".

API:

```
curl -s -X POST https://<platform>/api/projects/retail/models/churn/versions/2/approve \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1"
# {"approved": true, "by": "bob", "at": "2026-10-24T09:12:03Z"}
```

`GET /api/projects/{name}/models` lists the registered models (each with `name`, `description`, `approved`, and its `versions` with `version`, `runId`, `source`, `status`, `created`, `registeredBy`, `approvedBy`, `approvedAt`), the experiments, `canApprove`, and `trackingUri`.

How it behaves

- Approval sets two tags on the version, `storscale.approved_by` and `storscale.approved_at`, and moves the registered model's alias `approved` to it. Only the platform can set these.
- Approving a newer version moves the alias: the older version stays loadable by its number (`models:/churn/1`) but is no longer what pipelines load (`models:/churn@approved`).
- Scorings pick up the new version at their next run; servings within about a minute.
- **Audit log:** `model.approved` with "registered by <name>"; a refused self-approval is recorded as `model.approved` with "approving their own".

Limits and gotchas

- Approving your own version: 403 "a model version is approved by an editor other than the one who registered it". Have a second editor in every project that registers models.
- A version or model that doesn't exist: 404 "no such model version: ...".
- Approving through MLflow's own API (the alias or the tags) is refused (see above).
- There is no "unapprove": approve another version to move the alias.

Model cards

What it's for. A short, human description of what a model is for, what it learned from, how it was judged and where it shouldn't be used, kept with the model and shown in the catalog.

Who can do it. Members with `models` (editors). The **Write its card** button shows to editors.

Set up

1. On **Models**, on the model's card, press **Write its card** (or **Edit its card**).
2. Write it. The placeholder suggests four lines: *Purpose*, *Data*, *Metrics*, *Limits*.
3. **Save the card**.

It's the registered model's description in MLflow, written through the project's MLflow route (`PATCH /mlflow/<project>/api/2.0/mlflow/registered-models/update` with `{"name": ..., "description": ...}`), so `mlflow.MlflowClient().update_registered_model(name, description=...)` from a notebook does the same.

How it behaves

The card shows under the model's name on **Models**, and is the model's description in the catalog (see Models in the catalog's lineage). A model without one says "No card yet: its purpose, data, metrics and limits."

Scoring on a schedule, and drift

What it's for. Putting an approved model to work: every night (or on any schedule) its approved version scores a table's rows into a table of the project's, as the project's pipelines. Each run's inputs are compared with what the model saw on its first run, so a model drifting away from its data says so before its scores go wrong.

Who can do it. Scorings are declared in the project's spec: administrators on the **Edit** page, or a release of the project's repo (History and releases). They run as the project's pipelines. The project's members see them on **Models**.

What it needs. The project has `tables` and `pipelines`, and the model has an approved version.

Set up

Add `scoring` to the spec (**Edit** → the JSON box):

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "members": [
      {"group": "engineers", "role": "editor"},
      {"user": "alice", "role": "editor"}
    ],
    "tables": {},
    "pipelines": {"serviceAccount": "airflow-pipelines"},
    "scoring": [
      {
        "name": "churn-nightly",
        "model": "churn",
        "table": "customer_features",
        "id": "customer_id",
        "columns": ["orders", "revenue", "tenure_days"],
        "output": "customer_scores",
        "schedule": "0 3 * * *",
        "drift": 0.2,
        "maxRows": 100000
      }
    ]
  }
}
```

Field	Required	What
`name`	yes	Lowercase letters, digits and dashes, starting with a letter, up to 41 characters.
`model`	yes	A registered model of the project's MLflow workspace, by name. Its approved version is used.
`table`	yes	The input: one of the project's tables.
`id`	yes	The column that names each row; it's carried to the output so scores join back.
`columns`	no	The model's inputs, by name. None: every column of `table` but `id`.
`output`	yes	The table the scores are written to, made afresh each run. Not the input; no two scorings write the same output.
`schedule`	yes	Cron's five fields, UTC.
`drift`	no	The population stability index past which a column has drifted. Default `0.2`; 0 to 10.
`maxRows`	no	The most rows read. Default 100,000; up to 5,000,000.

There is no API to add a scoring by itself; it's part of the spec (`PUT /api/projects/{name}`, administrators). Reading them: `GET /api/projects/{name}/scoring`.

How it behaves

- Within a minute of saving, the **Pipelines** tab lists the scoring's DAG, `scoring_<project>_<name>` with dashes as underscores (`scoring_retail_churn_nightly`), tagged `scoring`. It runs in the project's own Airflow if it has one, else the shared one.
- **Each run**, as the project's pipelines, in a Python environment of its own (MLflow, scikit-learn, LightGBM, XGBoost, FLAML, skops, pandas):
  1. loads `models:/<model>@approved` through the project's MLflow;
  2. reads `SELECT * FROM <table> LIMIT <maxRows>` through Trino;
  3. predicts on `columns`. A score per row: when the model answers several columns (a classifier's probabilities), the last one, the positive class's of two;

4. writes `output` afresh (`CREATE OR REPLACE TABLE`) with four columns: the `id` column (as `varchar`), `score` (`double`, or `varchar` when the scores aren't numbers), `model_version` and `scored_at` (`timestamp(3)`, UTC);
5. works out each input column's drift and records the run with the platform.

- **Drift.** A version's first run is its reference: each input column's profile (a number's shares over ten quantile bins of that run; a text column's ten commonest values and "other"), kept by the platform. Every later run of the same version is profiled over the same bins and compared by the population stability index (PSI). A newly approved version gets a new reference from its first run.
- **Alerts.** When the latest run's worst column is past the threshold, the project's alerts say "churn-nightly: its inputs have drifted (revenue, PSI 0.85, past 0.20)", with "version 2 of churn, the run of ..., against the version's first run; retrain, or check the data", linked to **Models**. Alerts go where the project's alerts go (Data quality).
- **The Scoring card** on **Models** lists each scoring: its DAG, model and the version of its last run, input → output, schedule, last run (rows and the scores' mean), and the worst column's PSI, green or red against the threshold.
- **Metrics.** Prometheus scrapes `storscale_scoring_drift{project,scoring,version,column}` (each column's PSI on the latest run) and `storscale_scoring_rows{project,scoring}`, so Grafana can draw them.
- The platform keeps a scoring's 60 most recent runs (in its bucket, `scoring/<project>/<name>/`).

Settings

None of its own. The scoring runs in the project's Airflow (`pipelines.memory`, default `3Gi`).

Limits and gotchas

- **Only the first `maxRows` rows are scored**, in whatever order Trino reads them. A table bigger than `maxRows` is scored in part, with no warning: raise `maxRows` or split the table.
- The model must load in the scoring's environment: scikit-learn, LightGBM, XGBoost and FLAML models do; a model needing a library the environment doesn't have (statsmodels, for one) fails to load. The churn and propensity templates restrict FLAML to the kinds a scoring can load for this reason.
- The model is given exactly `columns` as a DataFrame: names and types must match what it was trained on. Log an `input_example` so MLflow records the signature.
- The scoring scores the table as it is: rebuild the features table (a pipeline of its own) before the scoring runs.
- No version approved: the run fails at loading `models:/<model>@approved`.
- Refusals at save time: "scoring: a model scores the project's tables as its pipelines: the project needs tables and pipelines"; "scoring: churn-nightly: the output is a table of its own, not its input"; "scoring: two write customer_scores"; "scoring: churn-nightly: schedule "...": five cron fields, as 0 4 * * *"; "drift 12: a PSI, as 0.2 (more is a bigger shift)"; "maxRows ...: up to five million".

Serving a model as an API

What it's for. A score on demand: the web shop asks for one customer's churn chance as they sign in, rather than waiting for the night's run. A served model answers at the platform's address, signed in, each call audited, and moves to a newly approved version by itself.

Who can do it. Servings are declared in the project's spec (administrators on **Edit**, or a release). Calling one: any member of the project (readers, editors, custom roles) and its agents, signed in with a Keycloak token or the web app's session. Not members are refused.

Set up

Add `serving` to the spec:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "members": [
      {"group": "engineers", "role": "editor"},
      {"user": "alice", "role": "editor"}
    ],
    "tables": {},
    "serving": [
      {"name": "churn", "model": "churn", "memory": "1Gi", "maxRows": 1000}
    ]
  }
}
```

Field	Required	What
`name`	yes	Up to 16 lowercase letters, digits and dashes, starting with a letter. `model-<project>-<name>` must fit in 63 characters.
`model`	yes	A registered model of the project's, by name (letters, digits, `_ . -`). Its approved version is served.
`memory`	no	Its server's memory on Kubernetes, from `256Mi` to `16Gi`. Default the install's `serving.memory` (`1Gi`).
`maxRows`	no	The most rows a call may score. Default 1,000; up to 100,000.

Call it:

```
curl -s https://<platform>/api/projects/retail/serving/churn/predict \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"dataframe_records": [{"orders": 1, "revenue": 40.0, "tenure_days": 300}]}'
# {"predictions": [[0.31, 0.69]], "model": "churn", "version": "2"}
```

The body is MLflow's: `{"dataframe_records": [...]}`, `{"dataframe_split": {"columns": [...], "data": [...]}}`, `{"instances": [...]}` or `{"inputs": ...}`. The answer is `{"predictions": [...], "model": ..., "version": ...}`: the version that answered. A model logged with `pyfunc_predict_fn="predict_proba"` answers each class's chance.

`GET /api/projects/{name}/serving` lists the project's servings, each with `maxRows`, `url` (where to post), `version` (served now) or `error` (why none is).

How it behaves

- **On Kubernetes**, each serving gets a server of its own: a Deployment, Service and Secret named `model-<project>-<name>`, signed in to Keycloak as a client of its own (`serving.<project>.<name>`) whose service account reads the project as its readers do, and nothing else's. With the chart's `networkPolicies.enabled`, a network policy keeps everyone but platformd out. **In Compose**, one shared server serves every project's servings, signed in as the pipelines' account, which the operator makes a reader of each project it serves.
- The server loads `models:/<model>@approved` through the project's MLflow, and **looks again every 60 seconds**: a newly approved version answers within about a minute, at the same address, nobody restarting anything.
- **Models → Serving** shows each serving, the version it's answering with (or why not: "starting", "no version of churn is approved", "its server isn't answering (starting?)"), its rows a call, and the address, with a `curl` example.

- A project's calls run at most eight at a time across its servings.
- **Audit log:** each call is `model.called` with "churn version 2, 1 rows" (who, the version, how many rows; never the rows). Each refusal is `model.call` with the reason: not a member, the project's calls at their limit, or the model server's error.

Settings

Setting	Default	What
`serving.memory` (chart)	`1Gi`	A serving's server's memory when its spec names none.
`SERVING_URL` (Compose, platformd)	`http://serving:8080`	Where the model server answers. Empty: models aren't served.
`SERVING_REFRESH_SECONDS` (the model server)	`60`	How often it looks for a newly approved version.

Limits and gotchas

Answer	Why	What to do
403 "not a member of the project"	The caller isn't one of the project's people or agents.	Add them to the project, or call as someone who is.
429 "the project's models answer 8 calls at a time: try again in a moment"	Eight of the project's calls are already in flight.	Retry after the `Retry-After` second.
413 "a call is up to 32 MB"	The body is too big.	Send fewer rows a call.
413 "1500 rows: a call scores up to 1000"	Over the serving's `maxRows`.	Split the call, or raise `maxRows`.
503 "no version of churn is approved"	Nothing to serve yet.	Approve a version on **Models**.
400 "ValueError: ..." with the model and version	The model refused the rows, most often their columns or types.	Send the columns the model was trained on.
502 "the model's server isn't answering (starting?)"	The server is starting or down.	Wait a minute; on Kubernetes check the pod `model-<project>-<name>`.
503 "models aren't served here (no SERVING_URL)"	The install doesn't serve models.	An administrator sets it up.

- A person calls only with a role that gives `call-models` (the built-in reader and editor do): "your role in retail (looker) doesn't let you call its models (call-models)" (HTTP 403), the refusal in the audit log. The project's own accounts (its pipelines, its agents) call as members.
- A served model's version moves the moment another is approved: tell its callers, or read `version` in each answer.

Models in the catalog's lineage

What it's for. Finding a model where people already look for data: the catalog shows each registered model between the tables it learned from and the tables it scores into, with its card and approval, so "what does this model read, and who approved it?" has an answer.

Who can do it. Nothing to do: catalog-sync keeps it. Anyone who can open the catalog (**Tools → Catalog**) sees it.

How it behaves

- Every round of its metadata ingest (every `METADATA_EVERY` seconds, default 60), catalog-sync asks platformd for every project's registered models and scorings (`GET /internal/catalog/models`, its own service account only) and puts each model in the ML model service `storscale-mlflow`, as `<project>__<model>` (`retail__churn`).
- **Its page:** the card as its description, then the version approved, who registered and who approved it, the training run (who ran it, its parameters, the algorithm as autologging names it) and the catalog's Nessie commit when the run started. A model without a card says "No card yet: it's written on the project's Models tab." The page links back to the project's **Models** tab.
- **Upstream:** the tables the approved version's run read (its `lake.tables`), each at its Iceberg snapshot at the run's data commit, and each scoring's input table.
- **Downstream:** each scoring's output table, the edge saying "Its scores of customer_features, by the approved version: the scoring churn-nightly (0 3 * * *), written afresh each run."
- Edges no longer drawn (a scoring removed, a version trained on other tables) are pruned; models no longer registered are removed.

Limits and gotchas

- Only a version that's approved appears with its run and tables; a model with no approved version appears with its card alone.
- Training tables come from `lake.tables`, which only `buckets_lake.sql()` records. A model trained on data read another way (`s3()`, a CSV) has no training tables in the lineage; its run's `storscale.data_commit` still names the data.
- Up to 100 of a run's tables are given to the catalog.

Scheduled notebooks

What it's for. An analyst's notebook becomes a project's pipeline with one button, as a reviewed change in the project's repo rather than a job on someone's laptop. Each run keeps its executed notebook, outputs and all.

Who can do it. Members with the permission `build` (editors) of a project that has a repo the platform can commit to: an `https://` repo and its token (the repo's connection). The pipelines and git side is in Pipelines and the project's repo.

Set up

1. In JupyterLab, open the notebook and press **Schedule** on its toolbar (or right-click it in the file browser: **Schedule in a project...**, also in the command palette).
2. Pick the **Project** (the projects you edit; those without a usable repo are listed greyed out with why) and **When (cron, UTC)**, default `0 5 * * *`.
3. **Schedule**. The dialog "Scheduled, for review" links the merge request.
4. Review and merge it in GitLab. Within a minute of the merge, the **Pipelines** tab lists `notebook_<name>`.

API (what the button calls, as you): `GET /api/notebooks/projects` lists the projects you may schedule in; `POST /api/projects/{name}/notebooks/schedules` with `{"path": "call_list.ipynb", "schedule": "0 6 * * 1", "notebook": {...}}` commits it and answers with `dag`, `branch`, `commit`, `mergeRequest`, `iid` and `new`.

How it behaves

- The notebook goes with its outputs cleared: what it computed may be data no repo should hold.
- The platform commits two files to the branch `storscale/schedule-<name>`: the notebook as `<dags path>/notebooks/<name>.ipynb`, and a five-line DAG file `<dags path>/notebook_<name>.py` that calls `notebook.schedule(project=..., name=..., schedule=..., owner=...)`. `<name>` is the file's name lowercased, other characters as _, up to 50 characters. Scheduling the same notebook again updates the same branch and merge request.
- **Each run** executes the notebook with papermill as the project's pipelines, with `STORSCALE_PROJECT` set to the project (the templates read it). The executed notebook and its HTML go to the platform, linked from the run on **Pipelines** (**notebook**): the HTML is shown with no script running, or download the `.ipynb` (`GET /api/projects/{name}/notebooks/runs/{dag}/{run}?format=html|ipynb`, to the project's members).
- A failed run says which cell failed and why ("cell 3 failed: KeyError: ..."), the project's alerts say so, and the kept notebook shows the cell and its error.
- **Audit log:** `notebook.scheduled` (the path, schedule, commit, branch and merge request), and `notebook.ran` for each run ("ran" or "failed: ...").

Limits and gotchas

- "the project's editors schedule notebooks in it" (403): you aren't an editor there.
- "the project's repo needs an https address and a token (its connection) for the platform to commit to it": the repo is SSH, or has no token. "the repo's connection has no token": the connection is empty.
- "a schedule as cron's five fields (UTC): 0 5 * * *": the schedule isn't valid.
- The scheduled environment has the notebook image's analysis and model libraries but not geopandas or networkx.
- A scheduled notebook has no gateway key: `buckets_lake.ask()` is for agents (AI).
- It runs as the pipelines, not as you: it may read more than you can, and writes only where the pipelines may (all tables; the bucket's `landing/`).

See Lab 11, steps 1 to 5.

The notebook templates: tables, churn, propensity, forecast

What it's for. Worked starts: one on reading and changing tables, and three for the models most teams want, each ending in a registered version ready for approval. Change the parameters, run top to bottom.

Who can do it. Anyone in JupyterLab. Changing tables (and making the features table) needs `write-tables` in the project; registering needs `models`; the tables template's Spark part needs the Distributed Spark profile.

Where they are

Each server start puts them in `work/templates/`: `tables.ipynb`, `churn.ipynb`, `propensity.ipynb`, `forecast.ipynb`. A template you've changed, or deleted, is yours: only one still as it was last put there is updated when the image is.

Template	Model	Judged by	Examples use
churn.ipynb	A classifier: each customer's chance of stopping. FLAML picks among LightGBM, XGBoost, random forest, extra trees and L1 logistic regression.	ROC AUC on a held-out quarter	customer_features; label from uploads_customers and uploads_orders (no order since 1 September)
propensity.ipynb	A classifier: each customer's chance of responding to a campaign. Same estimators.	ROC AUC on a held-out quarter	customer_campaign_features; label from uploads_campaign_responses (campaign 6)
forecast.ipynb	A daily series for the next HORIZON days. FLAML tries ARIMA, SARIMAX, Holt-Winters and tree models on the calendar.	Mean absolute error on the last HORIZON days, beside a naive forecast (mae_naive)	Revenue a day from uploads_orders

Reading and changing tables: tables.ipynb

The same work done two ways, side by side, on the guide's retail project: it reads uploads_orders and makes and changes its own table, customer_tiers (and, from Spark, customer_tier_summary).

- **SQL, through Trino** (buckets_lake.sql): read and aggregate; make a table (CREATE TABLE IF NOT EXISTS ... AS SELECT ... WITH NO DATA); INSERT, UPDATE, DELETE; MERGE to update the rows that are there and insert the rest; a few rows from pandas as VALUES; the table's history (its changes, as the History tab lists them) and reading it as it was ("customer_tiers#<commit>"). Row filters and masks apply; answers have at most 100,000 rows.
- **Spark** (buckets_lake.spark): a session on the tables named, with a new table made by create=; read with DataFrames; replace a table's rows (writeTo(...).overwritePartitions()), and change rows by rewriting the table from what it should hold. Only tables you read whole.

Change PROJECT, ORDERS and the two tables' names in the first code cell for your own. Its last cell, commented out, drops the two tables.

Use one of the model templates

1. Save a copy (**File → Save Notebook As...**).
2. Change the first code cell's parameters: PROJECT (it reads STORSCALE_PROJECT first, so a scheduled run uses its project), the features table and ID, COLUMNS, the LABEL and the LABELS query (or SERIES, DATE, VALUE, HORIZON for the forecast), MODEL, EXPERIMENT, and TIME_BUDGET (seconds FLAML may spend; default 60).
3. For churn and propensity, make the features table once: adapt FEATURES_SQL and run it (uncomment the last line of its cell) or on the **SQL** tab. One table, two uses: the model learns from it and the scoring scores it, a row per ID, its inputs as numbers, no label. The inputs describe each row *before* the period the label looks at.
4. Run top to bottom. The data comes through buckets_lake.sql() (as you, at most 100,000 rows: aggregate in SQL). The run logs its parameters, metrics and model in the project's MLflow and registers a new version of MODEL.
5. Then, as each template's **Next** section says: write the card, have another editor approve, and for churn and propensity add a scoring. The templates give it:

```
{"scoring": [{"name": "churn-nightly", "model": "churn", "table": "customer_features", "id":
  "customer_id",
              "output": "customer_churn_scores", "schedule": "0 3 * * *", "drift": 0.2}]}
```

Limits and gotchas

- The classifiers assert their inputs are numbers: "the model takes numbers: in customer_features, CAST these or make a 0/1 column per value: [...]".
- The forecast template asserts three times `HORIZON` days of history at least.
- **The forecast model is the whole FLAML AutoML object, saved with cloudpickle, and loads where FLAML and statsmodels are: in notebooks.** Use it from a notebook, as its **Next** section shows (`mlflow.pyfunc.load_model("models:/revenue_forecast@approved")`), and write the forecast to the landing area for import; don't point a scoring or serving at it.
- Propensity wants hundreds of rows with a few dozen responders at least, or the model is noise.

Settings

Setting (chart)	Default	Compose	What
`notebooks.memoryLimit`	`1G`	fixed `1G` in `jupyterhub_config.py`	Each person's notebook server's memory.
`notebooks.cpuLimit`	`2`	2 (the hub reads `NOTEBOOK_CPUS`, which `compose.yaml` doesn't set)	Each person's notebook server's CPUs.
`notebooks.storage`	`1Gi`	a Docker volume, no size	Each person's `work/` volume.
`notebooks.prePull`	`true`	n/a	The notebook image (over a gigabyte) pulled onto every node ahead of time and at each upgrade (a DaemonSet, `notebook-prepull`), so a first notebook doesn't wait on the download.
`notebooks.startTimeout`	`600`	`600`	Seconds a notebook server may take to start, its image's pull included, before JupyterHub gives up.
`notebooks.spark.enabled`	`true`	n/a	Distributed Spark, a profile people pick as their notebook starts.
`notebooks.spark.default`	`true`	n/a	Distributed Spark picked unless people pick **Notebook only**; `false`: the notebook alone is.
`notebooks.spark.driver.memory`	`2G`	n/a	Spark's driver beside the notebook (at least `1G`); `driver.requestMemory` asks for less.
`notebooks.spark.executors.*`	`max: 2`, `cores: 2`, `memory: 4g`, `idleTimeout: 120s`	n/a	Each person's executors at once, their CPUs and memory, and how long one idles before it's let go.
`notebooks.spark.credentialHours`	`12`	n/a	How long a session's credentials last.
`storage.jupyterhub`	`1Gi`		The hub's own database volume.
`placement.jupyterhub`	`{}`		A node selector for the hub.
`images.mlflow`	`platform-mlflow`	`mlflow` service	MLflow's image.
`serving.memory`	`1Gi`	one shared `serving` service	A serving's server's memory.

platformd reaches MLflow at `MLFLOW_URL` (Compose: `http://mlflow:5000`) and the model server at `SERVING_URL`; MLflow keeps its database in `mlflow-db` and its artifacts in Buckets. Operating the platform has the rest.

Errors you may meet

Where	The words	What to do
`sql()`	`PermissionError: the platform refused the query on retail (403): ...`	You may not read that: check your role and the readers' rules.
`sql()`	warning "the answer has 100000 rows ..."	Aggregate or sample in SQL.
`spark()`	`RuntimeError: this notebook has no Spark: start it with the Distributed Spark profile ...`	Restart your server with that profile.
`spark()`	`PermissionError: there's no table you read whole, so Spark has nothing to read ...`	Use `sql()`; `spark_skipped` says why each was left out.
`spark()`	413 "your N tables don't fit in one session's credentials ..."	Name the tables: `spark(tables=[...])`.
Spark	`ForbiddenException: sales.orders isn't one of this session's tables ...`	It has a row filter or mask for you (`spark_skipped`), or the session named other tables.
Spark	`ForbiddenException: a Spark session doesn't make, drop or rename tables ...`	Use `spark(create=...)`, or Trino.
Spark	`NotAuthorizedException: ... the session's grant has expired`	Start a new session: `buckets_lake.spark()`.
`spark(create=...)`	`PermissionError: Trino didn't make <table> as you: ...`	Make it where you're an editor.
`mlflow()`	403 "not a member of the project"	Use a project you're in.
MLflow write	403 "the project's readers read its experiments and models; its editors and pipelines write them"	Ask for `models`, or have an editor run it.
MLflow tag	403 "the storscale.* tags are the platform's ..."	Use another tag name; approve on **Models**.
Approve	403 "a model version is approved by an editor other than the one who registered it"	Have another editor approve.
Approve	403 "the project's editors approve its models (it takes the permission approve)"	Ask for `approve`.
`library()`	`RuntimeError: library <name>: HTTP 404 ...`	Check the name on **Administration → Templates**.
Schedule	"You aren't an editor of a project with a repo, where a notebook can be scheduled."	Become an editor, or connect a repo (Pipelines).
Scoring run	fails loading `models:/churn@approved`	Approve a version.
Predict	503 "no version of churn is approved"	Approve a version; wait a minute.

More in Troubleshooting.

11. Data products

A data product is one of a project's tables published for other projects, with an owner, a version and a promise (its contract): the columns it keeps, how fresh it stays, its checks passing. Another project's editors ask for it; the publisher's editors approve; the consuming project then reads the table where it is, under the publisher's rules, with no copy to go stale. The governance group certifies what it has checked, and a broken promise is the publisher's alert, naming who depends on it. This chapter is for the engineers who publish tables, the analysts whose projects ask for them, and the governance group that certifies them.

In this chapter:

- Publishing a data product
- The contract, and broken promises
- Asking for a product
- Approving, declining and removing consumers
- Reading a product in place
- Certifying
- Changing and removing a product
- Where products show
- Settings
- Errors you may meet

Who does what, in one table:

Action	Who	Where
Publish, change, remove a product	Those who may write the project's tables (`write-tables`: editors, and roles that give it)	Data products → **Publish a data product**, a card's **Change** and **Remove**
Ask for a product for a project	That project's editors (the permission `build`)	A card's **Ask for it**
Approve or decline a request; remove a consumer	The publishing project's editors (`build`)	The card's **Approve**, **Decline**, **Remove** `<project>`
Certify, or take certification away	The governance group (the platform permission `certify-products`; administrators always)	The card's **Certify** / **Take certification away**
See the products	Anyone signed in	The top bar's **Data products**
Read a product's rows	The publisher's members, and the members of the projects it was approved for	SQL, notebooks, pipelines, metrics, assistants

Publishing a data product

What it's for. Marketing wants retail's orders. Copying the table gives marketing a second table that drifts from the first and escapes retail's rules. Publishing it says: this table is for others, this person answers for it, and this is what we promise.

Who can do it. Anyone who may write the project's tables: its editors, and members whose role gives `write-tables`.

Set up

On the page. Open the top bar's **Data products** and press **Publish a data product** (it shows when you may write some project's tables). Fill in the form:

Field	What it is
From the project	The project publishing it: those whose tables you may write
Its table	One of the project's tables, picked from the catalog's list (or typed, when the catalog has none yet)
Name	The product's name: lowercase letters, digits and underscores (`orders`). Picking a table suggests one
Version	Its version, as `1` or `2.3` (default `1`)
What it is	A description
Who answers for it	The owner: a user name (default: you)
The business terms it holds	Tick the company's terms the table holds (shown when there are any: Business terms)
What it promises its consumers	Tick the columns it keeps (picked from the table's columns); **Changed at least every** (default `24h`); **The project's checks on its table pass**

Press **Publish it**. The page lists the product as a card: its name and project, **not certified**, its version, its table and owner, what it promises, the terms it holds, and "Read by: no other project yet".

In the project's spec, the same product is an entry of `products` (the form writes it there):

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "members": [{"group": "engineers", "role": "editor"}, {"group": "analysts", "role": "reader"}],
    "tables": {
      "readers": {"rowFilters": [{"table": "uploads_orders", "filter": "region = 'EU'"}]}
    },
    "products": [
      {
        "name": "orders",
        "table": "uploads_orders",
        "description": "Harbor Outfitters' orders, net of test rows",
        "owner": "bob",
        "version": "1",
        "terms": ["revenue"],
        "contract": {
          "columns": ["order_id", "customer_id", "ordered_on", "region", "amount"],
          "within": "24h",
          "checks": true
        }
      }
    ]
  }
}
```

Field	What it is
`name`	Required: lowercase letters, digits and underscores, starting with a letter or underscore, up to 63 characters; unique in the project
`table`	Required: one of the project's tables, by its bare name. The project must have `tables`
`owner`	Required: a person's user name
`version`	Required: up to 20 characters (`1`, `2.3`)
`description`	What it is

Field	What it is
`terms`	The company's business terms it holds, by name
`contract.columns`	The columns it keeps, with these names
`contract.within`	It changes at least this often: `30m`, `24h`, `7d`
`contract.checks`	`true`: the project's checks on its table pass (Data quality)
`consumers`, `certification`	The platform's: who reads it, and who certified what. Whatever a save says, they're kept as they were

A project kept as code has the same in `storscale/project.yaml`, applied by a release (History and releases).

Through the API, for those who may write the project's tables:

Method and path	What it does
`PUT /api/projects/{name}/products/{product}`	Publishes a table as a product, or changes its promise. Body: `table`, `description`, `owner`, `version`, `terms`, `contract`
`DELETE /api/projects/{name}/products/{product}`	Takes it back, once no other project reads it

```
curl -s -X PUT http://storscale.localhost:8800/api/projects/retail/products/orders \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"table": "uploads_orders", "description": "Harbor Outfitters orders, net of test rows",
    "owner": "bob",
       "version": "1", "terms": ["revenue"],
       "contract": {"columns": ["order_id", "customer_id", "ordered_on", "region", "amount"],
         "within": "24h"}}'
```

The answer is the product as saved. An unknown field is refused ("a data product, as JSON: json: unknown field ..."); `consumers` and `certification` in the body are ignored.

How it behaves

- The product is saved in the project's spec, checked as any spec is; the operator is asked to apply the project at once.
- The platform's audit log records `product.saved`: "published: table uploads_orders, version 1", or "changed: ...".
- Within a few minutes the catalog (OpenMetadata) has it as a data product in the domain **DataWorks**, named `orders (retail)`, with its table as its asset and its owner when the catalog knows that user.

See Lab 12, step 1.

Limits and gotchas

- Picking a table suggests its name without `uploads_` (`uploads_campaign_responses` becomes `campaign_responses`). A product's name takes no dashes: `products: name "campaign-responses": an identifier`.
- **Names aren't checked against the table.** A table that doesn't exist yet, or a contracted column it doesn't have, is accepted; the contract's check then reports the columns missing (below).
- **A product's name and project can't be changed** on the form: remove it and publish it again (once no project reads it).
- A product is one table. Several tables are several products.

The contract, and broken promises

What it's for. A consumer builds on the columns it reads. The contract says which columns will stay, how fresh the table stays, and whether its checks pass; the platform watches each, and tells the publisher, naming who's let down, when one breaks.

Who can do it. The contract is part of the product: those who publish it set it.

How it behaves

Once a minute, the platform looks at each product with a contract and raises an alert on the publishing project for each promise broken:

Promise	Broken when	The alert says
`columns`	A contracted column isn't in the table (renamed or dropped)	"the product orders breaks its contract: region gone from uploads_orders"
`within`	The table hasn't changed within the window (by its last commit)	"the product orders breaks its contract: uploads_orders not changed within 24h"
`checks`	One of the project's checks on the table is failing	"the product orders breaks its contract: a check on uploads_orders is failing"

Each alert names the consumers it lets down ("its consumers: marketing", or "no consumers yet") and links to the table. The alerts are the project's own: on its **Overview**'s **Alerts** card, sent where the project's `notify` says, muted as other alerts are (Data quality), and listed to assistants by `list_alerts`.

In merge requests. With a GitLab repo, a merge request whose dbt build leaves a product's table without a contracted column fails its review: the review's note says "the product `orders` (uploads_orders): its contract's columns region would be gone; read by marketing", and its status is failed (Pipelines and the project's repo).

See Lab 12, step 7.

Limits and gotchas

- A broken contract doesn't stop anyone reading the product, nor take away its certification: it's an alert for the publisher to act on.
- The column check compares names only: a column whose type changed keeps its promise as far as the platform can tell.
- A compatible change (a new column) needs nothing; a breaking one is best published as a new version, or a new product, while consumers move.

Asking for a product

What it's for. A project that needs another's table asks for it, saying why, and the publisher decides. Nothing is read until they approve.

Who can do it. The editors of the project that will read it (the permission `build` there, or an administrator). The **Ask for it** button shows for projects where you're an editor.

Set up

On **Data products**, press **Ask for it** on the product's card. Pick **For the project** (your projects that don't read it yet), write **What it's for** ("The regional forecast"), and press **Ask**.

Through the API: `POST /api/products/{project}/{product}/requests`, with the producing project and product in the path:

```
curl -s -X POST http://storscale.localhost:8800/api/products/retail/orders/requests \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"consumer": "marketing", "why": "Campaign results by region"}'
```

How it behaves

- The request waits in the platform's store until the publisher's editors answer; asking again replaces it.
- The publisher's editors see it on the card, under **Asked for**: the project, who asked, and why. The **Governance** page lists it among what waits for an approval.
- The platform's audit log records `product.requested`, "for marketing".

See Lab 12, step 2.

Limits and gotchas

- "the project to read it in (consumer), not its own": a project doesn't ask for its own product.
- "the editors of marketing ask for it": you must be able to build in the consuming project.
- "marketing reads it already": it's a consumer already.

Approving, declining and removing consumers

What it's for. The publisher decides who reads its table, and can take that back.

Who can do it. The publishing project's editors (the permission `build` there), and administrators.

Set up

On the product's card, under **Asked for**, press **Approve** or **Decline**. Under the card's line "Read by: ...", **Remove** `<project>` (confirmed with "Stop marketing reading orders?") takes a consumer away.

Through the API:

Method and path	What it does
`POST /api/products/{project}/{product}/requests/{consumer}/approve`	Approves the request: the consumer reads it
`POST /api/products/{project}/{product}/requests/{consumer}/decline`	Declines it
`DELETE /api/products/{project}/{product}/consumers/{consumer}`	Takes the consumer's reading away

How it behaves

- Approving adds the project to the product's `consumers` in the publisher's spec and has the operator apply it: within a minute the consumer's people read the table (below). Removing a consumer takes it out, and their reads are refused once it's applied, within a minute.
- The request is gone once answered, either way.
- The platform's audit log records `product.approved` and `product.declined` ("for marketing"), and `product.consumer-removed`.

Limits and gotchas

- "the editors of retail answer requests for its products" / "... decide who reads its products": you need `build` on the publisher.
- "no request from marketing": it was answered already, or never made.
- "marketing doesn't read it": it isn't a consumer.

Reading a product in place

What it's for. The consumer reads the publisher's own table: one table, always current, under the publisher's rules.

Who can do it. The consuming project's readers, editors and pipelines, and its members whose custom role reads (a role giving `read`, `read-all` or `write-tables`), plus the publisher's own members as always.

How it behaves

The operator gives the consuming project's groups, in Ranger, `select` on the product's table and nothing else of the publisher's (and `use` and `show` on its namespace, to find it). The publisher's readers' rules on that table, its row filters and masks, apply to the consumers too, in the same Ranger policies:

- A consumer who isn't also a member of the publisher sees what the publisher's readers see: with retail's row filter `region = 'EU'`, marketing's people read the EU rows only.
- A consumer who is also an editor of the publisher (alice, in the labs) reads it as an editor: every row.

The table keeps its own name. From the consuming project's **SQL** tab, a notebook, a pipeline or a flow:

```
SELECT region, count(*) AS orders, round(sum(amount), 2) AS amount
FROM iceberg.retail.uploads_orders
WHERE NOT test
GROUP BY region
```

Other ways in:

- **Metrics on the publisher's definitions.** If the publisher's model is shared and the consumer's spec has `"semantic": {"uses": ["retail"]}`, the consumer's model builds on retail's metrics; the questions read the product's table as the person asking (Sharing a model).
- **Assistants.** `query_product` reads a product's rows as the person; someone who isn't a reader of the publisher or of a consumer is refused, and told the product's page where their project can ask for it (Assistants).
- **Flows.** The canvas's **Input** step lists the project's tables and the products it reads (Flows).

See Lab 12, steps 3, 4 and 8.

Limits and gotchas

- **Read only.** Consumers can't write the table.
- **Only the product's table.** Being a consumer of `orders` reads nothing else of retail's.
- The publisher's rules are those for its readers on that table (`tables.readers`) and the install's protection tags (Access and protection): a column retail masks for its readers is masked for its consumers.

Certifying

What it's for. Anyone can publish a table; the governance group says which ones it has checked. A certified product is marked everywhere it shows and listed first. The certification is of what was checked: the table, the version and the contract. A change to any of them lapses it.

Who can do it. The governance group: members of the Keycloak groups given a platform role with `certify-products`. The built-in platform role **governance** gives it (and the Governance page) to the groups in `products.certifierGroups`, by default the platform's administrators; an administrator can define other platform roles that give it (Roles). Administrators may always certify.

Set up

On the product's card, **Certify**. On a certified one, **Take certification away**.

Through the API: `POST /api/products/{project}/{product}/certification` certifies it as it is now; `DELETE /api/products/{project}/{product}/certification` takes it away.

How it behaves

- The certification records who certified it, when, and a hash of the product's table, version and contract.
- The card shows **certified**, "Certified by bob" and the date; certified products list first on the page, in search, in the Governance page's products and in assistants' `list_products`.
- The catalog marks the product's table **Certification.Gold** (for a year by the catalog's clock), and removes the mark when the product is no longer certified.
- **A changed promise lapses it.** When the table, version or contract changes (on the form, the API, the spec or a release), the hash no longer matches: the product shows **not certified**, the catalog's mark goes, and the governance group certifies again. Its consumers keep reading it. The description, owner and terms can change without lapsing it.
- The platform's audit log records `product.certified` and `product.uncertified` ("version 1, table uploads_orders"); a save that lapsed one says so in its `product.saved` record ("its certification lapsed (the promise changed)").

See Lab 12, steps 5 and 6.

Limits and gotchas

- "the governance group (a platform role with certify-products) certifies products": you're not in it.
- A broken contract (a column gone) doesn't lapse the certification by itself; the alert does the telling.

Changing and removing a product

What it's for. Promises change: a column added to the contract, a new version. And a product nobody reads any more can go.

Who can do it. Those who may write the project's tables.

Set up

Change on the card opens the form with the product's values (its project and name fixed). Change what you need and press **Save**. On a certified product the form warns: "A changed promise lapses its certification: the governance group certifies it again."

Remove (confirmed with "Take orders back? No other project reads it.") takes the product back. It's off while any project reads it, saying which ("marketing read it: remove them first").

Through the API: `PUT /api/projects/{name}/products/{product}` with the whole product, as when publishing; `DELETE /api/projects/{name}/products/{product}`.

How it behaves

- A change keeps who reads it: its consumers stay, and keep reading.
- A changed table, version or contract lapses a certification (Certifying).
- Removing takes the product out of the spec, out of the catalog's data products, and the platform's audit log records `product.removed`.

Limits and gotchas

- **A product another project reads can't be removed:** "marketing read it: take them off its consumers first" (409). Remove the consumers first, then the product.
- That rule is the Data products page's and its API's. An administrator who takes the product out of the spec's JSON on the project's **Edit** page removes it whatever reads it, and its consumers lose their reading: use the page.
- Changing `table` points the product (and its consumers) at another table; that's a changed promise, so a certification lapses.

Where products show

Where	What
Data products (top bar)	Every project's products, to anyone signed in, certified first, refreshed every 30 seconds; requests only to the publisher's editors
The top bar's search	Products after business terms, certified first ("Product ✓"); picking one opens its card
The catalog (OpenMetadata)	The domain **DataWorks**: one data product per product, its table as its asset; a certified one's table marked Certification.Gold
Governance page	Every product with its owner, certification and consumers; requests waiting for an editor (Operating the platform)
Assistants	`list_products`, `describe_product`, `query_product`; answers from metrics cite the products behind the tables they read
A business term's page	The terms ticked on a product link to their pages

Through the API: `GET /api/products`, to anyone signed in: every product with its project, `certified`, `consumers`, `requests` (for its editors) and `canApprove`; and `canCertify` for the caller.

Settings

Setting	Chart value (`deploy/helm/storscale-platform/values.yaml`)	Compose	Default
Who certifies data products	`products.certifierGroups` (Keycloak groups, comma-separated)	`PRODUCT_CERTIFIER_GROUPS` on platformd	empty: the platform's administrators' groups
How often catalog-sync updates the catalog's data products	not a chart value	`METADATA_EVERY` on catalog-sync	60 seconds

Errors you may meet

Error	What it means	What to do
"the project's editors publish its data products (it takes the permission write-tables)"	You may not write that project's tables	Ask an editor, or for a role with `write-tables`
`products: name "x-y": an identifier`	The name has dashes or capitals	Use lowercase letters, digits and underscores
"products: orders: its owner is a person (their user name)"	The owner is empty or not a user name	Give a user name
"products: orders: a version, as 1 or 2.3"	No version, or longer than 20 characters	Give one
"products: orders: a product is one of the project's tables, and it has none (tables)"	The project has no `tables`	Give the project tables first (Projects)
`products: orders: contract: within "1 day": as 30m, 24h or 7d`	The freshness window isn't a duration the platform reads	Write `24h` or `1d`
"the editors of marketing ask for it"	You can't build in the project you asked for	Ask as one of its editors
"marketing read it: take them off its consumers first"	Removing a product a project reads	Remove the consumer first
"the governance group (a platform role with certify-products) certifies products"	You may not certify	Ask the governance group
"Access Denied: ..." on `iceberg.retail.uploads_orders`	Your project isn't a consumer yet, or the operator hasn't applied the approval	Ask for the product; wait a minute after approval

12. Deliveries to partners

A delivery sends files to a client or partner on a schedule: a table of the project's, or a query on its tables, written as CSV, Parquet or JSON lines, split into a file per value if asked, compressed and PGP-encrypted if asked, with a manifest beside them, to an SFTP server or an S3 bucket. It runs only once someone other than who changed it has approved it, and a delivery of people's data needs a data-protection approver too. This chapter is for the project's editors who set deliveries up, the people who approve them, and the administrators and privacy team who answer for them.

In this chapter:

- What a delivery is for
- Who can do what
- Targets: where files go
- PGP: encrypting for the recipient
- Declaring a delivery
- Formats and file names
- Checks before sending
- Approvals
- Runs and manifests
- The delivery log
- Settings
- Limits and gotchas

See Lab 14 in the lab guide for a delivery set up, approved and run end to end, and Privacy for searching what was sent about a person.

What a delivery is for

Partners and clients want files: last week's campaign results every Monday, the orders they brought in, in their own bucket or on their SFTP server. Without a platform this is a script on someone's machine or a spreadsheet in an email, with nobody approving what leaves and no record of what was sent.

A delivery is declared once in the project's spec: what it sends, in what shape, where and when. It runs as one of the project's pipelines, sends nothing until approved, sends nothing when a check fails, writes a manifest with each file's checksum, and leaves a record of every run in the delivery log. When it carries people's data, the catalog as it read the tables is kept, so the log can be searched later for what was sent about one person.

Who can do what

Action	Who
Declare or change a delivery (the project's spec, on the **Edit** page or `PUT /api/projects/{name}`)	The platform's administrators. A release applied from the project's repo can change it too (History and releases).
Add the target and PGP connections (the **Connections** tab)	The project's editors, or a role with the `connections` permission
See the **Deliveries** tab, its deliveries and their runs	The project's members, and administrators

Action	Who
Approve as an editor	The project's editors, a role with `approve`, or an administrator; never the person who last changed what the delivery sends
Approve as a data-protection approver (needed for people's data)	Anyone with the platform permission `protect-personal`: the built-in platform role `data-protection` (its groups are the install's `deliveries.approverGroups`), a platform role an administrator defines with `protect-personal`, or an administrator
Start a run now (**Pipelines** → **Run now**)	Whoever may run the project's pipelines: its editors, a role with `run-pipelines`
Search the delivery log by a person's ID	Data-protection approvers and administrators (Privacy)

On the lab install, bob (engineers) is an administrator, so he is also a data-protection approver; on Compose the analysts (alice) are data-protection approvers too, so the example has one besides bob.

Targets: where files go

A delivery's `target` is the name of one of the project's connections: either **An SFTP server** (kind `sftp`) or **An S3 bucket** (kind `s3`). The connection holds the address and the credentials; the spec only names it.

Set up

1. Open the project → **Connections** → **Add a connection**.
2. **Name**: the project's name, a dash, then your own: `retail-acme-bucket`. A connection added on the page must start with the project's name and a dash, in lower-case letters, digits and dashes.
3. **Kind**: **An S3 bucket** or **An SFTP server**, then fill the fields below and press **Add**. Secret fields are written once and never shown again; **New values** on the card replaces them (a secret left empty stays as it was).

Kind	Field (key)	Required	Notes
`s3`	Endpoint (`endpoint`)	no	Empty means AWS
	Region (`region`)	no	Default `us-east-1`
	Bucket (`bucket`)	yes	
	Prefix (`prefix`)	no	Files go under it: `from-harbor/harbor_campaigns_2026-10-26.csv`
	Access key (`accessKey`)	yes	
	Secret key (`secretKey`)	yes, secret	
`sftp`	Host (`host`)	yes	
	Port (`port`)	no	Default 22
	User (`username`)	yes	
	Password (`password`)	one of these two, secret	
	Private key (`sshPrivateKey`)		Takes precedence over the password when both are given
	The server's host key (`hostKey`)	yes	As a `known_hosts` line, or its last two fields: `ssh-ed25519 AAAA...`
	Folder (`path`)	no	Default the login folder

The same through the API (`POST` adds, `PUT` gives new values):

```
curl -X POST "$PLATFORM/api/projects/retail/connections" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"name": "retail-acme-bucket", "kind": "s3",
       "values": {"endpoint": "https://s3.acme.example", "bucket": "acme", "prefix": "from-harbor",
                  "accessKey": "AKIA...", "secretKey": "..."}}'
```

`PUT /api/projects/{name}/connections/{connection}` takes `{"kind": ..., "values": {...}}` to change one; `DELETE` removes one that nothing names (the page's **Remove** is disabled while a delivery uses it).

How it behaves

- The platform operator copies the target's values (and the PGP key, if any) into an Airflow connection of the platform's own, `delivery_<project>_<delivery>`, in the Airflow the project's pipelines run in (its own on Kubernetes when it has a repo, else the shared one). The run reads them from its environment; they never appear in its arguments or its log.
- An S3 target is written with path-style addressing, one object per file at `<prefix>/<name>`.
- An SFTP target's host key is checked before anything is sent. A different key stops the run: `the SFTP server's key isn't the one given (<type>): nothing sent`. That one isn't retried.
- Removing a delivery, or the project, removes its Airflow connection, and the target's credentials with it.

PGP: encrypting for the recipient

Set up

1. On **Connections**, **Add a connection** of kind **A recipient's PGP key** (`pgp`), named for example `retail-acme-pgp`, with the recipient's armored public key in **Public key (armored)** (`pgpPublicKey`).
2. Name it in the delivery's `pgp`: `"pgp": "retail-acme-pgp"`.

How it behaves

Each file (after compression, when `gzip` is on) is encrypted for that key with GnuPG, as binary OpenPGP, trusting the key as given. The manifest is not encrypted: it names the files, their rows, sizes and checksums, and holds no data. The platform doesn't add `.pgp` to the names; put it in `files` yourself (`acme_{date}_{part}.csv.gz.pgp`).

A connection without `pgpPublicKey` puts the project's `deliveries` part in error: `delivery <name>: the connection "<pgp>" has no pgpPublicKey`. A key GnuPG can't import fails the run with `the recipient's key: ...`.

Declaring a delivery

Set up

Deliveries are part of the project's spec. As an administrator, open the project → **Edit** → **JSON**, add a `deliveries` list, and press **Save**. The project needs `tables` (a delivery sends them; without them the spec is refused), and should have `pipelines`, since a delivery runs as the project's pipelines.

Field	Required	What it is
`name`	yes	1 to 20 lower-case letters, digits and underscores, starting with a letter; unique in the project
`table`	one of `table`, `query`	One of the project's tables

Field	Required	What it is
`query`		One `SELECT` (or `WITH ... SELECT`) on the project's tables, with no `;`
`format`	yes	`csv`, `parquet` or `jsonl`
`gzip`	no	`true` compresses each file
`files`	yes	The files' names: must hold `{date}`, and `{part}` when split; no folders
`splitBy`	no	A column: a file per value; goes with `{part}` in `files`
`schedule`	yes	Cron's five fields, in UTC: `0 6 * * 1` is Mondays at 06:00
`target`	yes	A connection of kind `sftp` or `s3`
`pgp`	no	A connection of kind `pgp`
`checks`	no	What must hold before anything is sent: see Checks
`changedBy`, `approval`	never by hand	The platform's own: who last changed what the delivery sends, and its approval. Whatever is written there is replaced by what the platform had.

A complete spec for Harbor Outfitters' `retail` project with two deliveries, one of a table and one of a query on a people's table:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "description": "Harbor Outfitters: orders, customers, campaigns",
    "members": [
      {"group": "engineers", "role": "editor"},
      {"group": "analysts", "role": "reader"}
    ],
    "tables": {
      "people": [
        {"table": "uploads_customers", "column": "customer_id"},
        {"table": "uploads_orders", "column": "customer_id"}
      ]
    },
    "files": {},
    "pipelines": {"serviceAccount": "airflow-pipelines"},
    "deliveries": [
      {
        "name": "campaign_results",
        "table": "campaign_results",
        "format": "csv",
        "files": "harbor_campaigns_{date}.csv",
        "schedule": "0 6 * * 1",
        "target": "retail-acme-bucket",
        "checks": {"minRows": 1, "within": "7d"}
      },
      {
        "name": "partner_orders",
        "query": "SELECT order_id, customer_id, ordered_on, amount, region FROM uploads_orders WHERE
          channel = 'partner' AND NOT test",
        "format": "csv",
        "gzip": true,
        "files": "harbor_partner_orders_{date}_{part}.csv.gz.pgp",
        "splitBy": "region",
        "schedule": "0 6 * * 1",
        "target": "retail-acme-sftp",
        "pgp": "retail-acme-pgp",
        "checks": {"minRows": 1, "maxRows": 100000, "parts": ["EU", "NA", "APAC"]}
      }
    ]
  }
}
```

The same spec is saved with `PUT /api/projects/retail` (an administrator's token, header `X-Platform-Request: 1`). `GET /api/projects/{name}/deliveries` lists the project's deliveries as the Deliveries tab shows them.

How it behaves

- Saving records the administrator as `changedBy` for each delivery that is new or whose sends changed (see Approvals), and the audit log gets `project.saved` with the parts of the spec that changed.
- Within a minute the operator applies the project. Its **Overview** → **Applied to** card lists a `deliveries` part: green when every delivery's target (and key) was found and handed to Airflow; red with the reason when not.
- The **Deliveries** tab appears once the project has a delivery. Each card shows what it sends, its format, gzip and PGP, the split column, target, schedule and file names, the query if any, its personal columns (amber), who changed it, who approved it, and what it still needs.
- Only approved deliveries become pipelines. Within a few minutes of approval, `delivery_<project>_<name>` (dashes in the project's name become underscores: `delivery_retail_campaign_results`) appears on the **Pipelines** tab, tagged `project:<project>` and `delivery`.

Formats and file names

format	What's written
csv	A header row of the columns, then the rows; a null is written as an empty field
jsonl	One JSON object per line, column to value; dates and other values JSON doesn't have are written as text
parquet	One Parquet file, its types from the rows

The files' names come from `files`:

- `{date}` is the run's date, `YYYY-MM-DD` (Airflow's date for the run).
- `{part}` is the `splitBy` column's value. Each distinct value gets its own file, holding that value's rows; files are made in the order of the values. A `splitBy` column that isn't in what's sent fails the run: `splitBy <column>: no such column`.
- `gzip: true` compresses each file (the name isn't changed; add `.gz` yourself).
- A manifest goes beside the files: `manifest_<delivery>_<date>.json`.

For `harbor_partner_orders_{date}_{part}.csv.gz.pgp`, split by region, the Monday 26 October run writes `harbor_partner_orders_2026-10-26_EU.csv.gz.pgp`, `..._NA.csv.gz.pgp`, `..._APAC.csv.gz.pgp`, and `manifest_partner_orders_2026-10-26.json`.

Checks before sending

Set up

Under the delivery's `checks`:

Field	Meaning
minRows	At least this many rows
maxRows	At most this many (0 or absent: no upper limit); not below `minRows`
within	The table changed within `30m`, `24h` or `7d` (its latest snapshot's commit). Only for a delivery of a `table`; a query's is accepted and ignored.
parts	Every one of these values of `splitBy` is there; needs `splitBy`

```
{"checks": {"minRows": 1, "maxRows": 100000, "within": "24h", "parts": ["EU", "NA", "APAC"]}}
```

How it behaves

The checks run first, as the project's pipelines in Trino. If any fails, nothing is sent: the run log says `checks failed, nothing sent:` and each problem (`12 rows, fewer than 100`, `campaign_results not changed within 24h (last ...)`, `parts missing: APAC`), the run fails, and the project's alerts say the pipeline's latest run failed, where its `notify` sends them (Projects). The table's own quality checks (Data quality) are separate: a delivery doesn't run them.

Approvals

What needs approving

An approval covers what a delivery sends and where: its `table` or `query`, `splitBy`, `format`, `target` and `pgp`. Changing any of these makes the delivery unapproved again, names whoever saved the change as `changedBy`, and takes its pipeline away until it's approved again. Changing `schedule`, `files`, `gzip` or `checks` keeps the approval.

Who must approve:

1. **An editor of the project other than** `changedBy`: the project's editors, a role with `approve`, or an administrator.
2. **A data-protection approver other than** `changedBy`, when the delivery sends personal columns. A delivery's personal columns are:
   - the columns the project masks for its readers (`tables.readers.masks`) of its table, or named in its query;
   - the person's column of a people's table (`tables.people`): the whole table when the delivery sends it, or, for a query, when the query names both the table and its person's column.

The card lists what's still needed in words, for example `Needs: an editor of the project other than bob; a data-protection approver (engineers, analysts).`

Set up

On the **Deliveries** tab, a delivery that isn't approved has an **Approve** button for those with the `approve` permission. Press it. The answer says whether the delivery is now approved, and if not, who's still needed.

A person who is both (an editor and a data-protection approver, as an administrator is) gives both approvals with one press. A data-protection approver who is only a reader of the project doesn't see the button; they approve through the API:

```
curl -X POST "$PLATFORM/api/projects/retail/deliveries/partner_orders/approve" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1"
```

```
{"approved": false, "needs": ["an editor of the project other than bob"]}
```

They must be able to see the project (a member in any role, or an administrator); otherwise the answer is `no such project`.

How it behaves

- Each approval is recorded on the delivery's `approval`: the hash of what it sends, who approved as editors (`by`), who as data-protection approvers (`protection`), when, and whether it's complete (`done`).
- The audit log records `delivery.approved` with what it sends and its personal columns (`sends a query (csv) to retail-acme-sftp; personal: uploads_orders.customer_id`). A refused approval by the one who changed it is recorded as refused (`changed it`).
- Once complete, the operator lists it as approved, and the pipeline appears within a few minutes. It starts from its approval: a delivery approved on a Thursday sends next Monday's, not last Monday's (no catch-up), and one run at a time.
- The **Governance** page lists every unapproved delivery across projects under **Awaiting approval**, with who it waits for (Operating).

Runs and manifests

Set up

Nothing to set: an approved delivery runs on its schedule. To run it now, open **Pipelines**, choose `delivery_<project>_<name>` and press **Run now**, or `POST /api/projects/{name}/pipelines/{pipeline}/runs`.

How it behaves

Each run, in order:

1. **Checks**, as above. A failing one sends nothing.
2. **Reads** the rows through Trino as the project's pipelines (their Keycloak token, client tag `project:<project>`, so the query counts against the project's limits and usage).
3. **Writes** the files: rendered in the format, split, gzipped and encrypted as asked.
4. **Writes a manifest**: the delivery, project, date, total rows, the table's latest snapshot ID (for a delivery of a table; a query's is empty), and each file's name, rows, bytes and SHA-256 (of the bytes as sent: compressed and encrypted).
5. **Sends** the files and the manifest to the target, trying three times (10 then 20 seconds apart); `sending failed (...); again in 10s` is logged between tries.
6. **Records** the run with the platform (the delivery log), and for a delivery of personal columns, platformd tags the catalog's commit the run read at (see The delivery log).

The run's log, on **Pipelines** → the run → **log** (or **why it failed**), has lines such as:

```
delivery retail/partner_orders: sent 4 files, 212 rows, to sftp://acme@sftp.acme.example/inbox
delivery retail/partner_orders: recorded: HTTP 200
```

A partner checks a file against the manifest:

```
sha256sum harbor_partner_orders_2026-10-26_EU.csv.gz.pgp
jq '.files[] | select(.name=="harbor_partner_orders_2026-10-26_EU.csv.gz.pgp") | .sha256'
  manifest_partner_orders_2026-10-26.json
```

The delivery log

How it behaves

- Each run is kept by platformd in its own bucket as `deliveries/<project>/<delivery>/<time>.json`: the manifest, where it went (`s3://acme/from-harbor`, `sftp://user@host/path`), when the tables were read, what it sent (table or query), the approval it ran under, and a summary.
- The audit log has `delivery.sent` with the summary (`4 files, 212 rows, to sftp://...`).
- The **Deliveries** tab shows each delivery's last ten runs: when, rows, the files with their sizes (hover a file for its SHA-256), and where.
- For a delivery with personal columns, the run also records `kept`: the Nessie commit on `main` at the moment it read the tables, and a tag on it, `Delivery.<project>.<delivery>.<time>`, kept until `deliveries.keepFor` (default seven years) has passed. Nightly maintenance removes those tags past it. This is what lets a data-protection approver search the log for one person's rows years later, from the tables as they were; the log itself holds no one's ID (Privacy).

The log is read through the project's deliveries:

```
curl "$PLATFORM/api/projects/retail/deliveries" -H "Authorization: Bearer $TOKEN"
```

Each delivery in the answer has `approved`, `personal`, `needs` and `runs`.

Settings

Chart value	Compose (`stack/compose.yaml`, platformd)	Default	What it does
`deliveries.approverGroups`	`DELIVERY_APPROVER_GROUPS`	empty: the administrators' groups (`PLATFORM_ADMIN_GROUPS`, `engineers`). Compose sets `engineers,analysts`.	Keycloak groups of the built-in platform role `data-protection` (comma-separated)
`deliveries.keepFor`	`DELIVERY_KEEP_FOR`	`7y`	How long each personal delivery's read is kept (a Nessie tag per run): years (`7y`) or days (`400d`). platformd refuses to start with another form.
`maintenance.at`	`MAINTENANCE_AT`	`02:00` (UTC)	When the nightly round removes delivery tags past `keepFor`

Administrators can also give `protect-personal` to more groups by defining a platform role on Administration → **Roles** (Roles).

Limits and gotchas

- **Who changed it can't approve it.** The administrator who saved a delivery (or planned the release that brought it) is its `changedBy`; another editor approves. `someone other than who changed what it sends approves it`.
- **People's data needs two approvals.** A delivery that sends a people's table, its person's column in a query, or a masked column waits for a data-protection approver as well as an editor.
- **A query is matched by words.** A query's personal columns are found by the column and table names it contains. `SELECT *` from a people's table through a view or a CTE named otherwise may not be recognised: name the table and its person's column in the query.
- **One broken target stops them all.** If any delivery's target or key connection is missing, the project's `deliveries` part is red (`delivery partner_orders: no connection "retail-acme-sftp" among the project's`) and none of the project's deliveries has a pipeline until it's fixed.
- **Connection names.** A connection added on the page must start with the project's name and a dash: `retail-acme-bucket`, not `acme-bucket`.
- **The Deliveries tab shows only when the project has a delivery.** For a project without one, open `/p/<project>/deliveries` directly to reach the privacy search and erasure.
- **Not yet:** fixed-width files, Azure Blob and Google Cloud Storage targets, and opening "the data as sent" from a run on the SQL page.
- **SFTP needs the host key.** Without `hostKey` the connection can't be saved (`The server's host key (as in known_hosts) is needed`), and a run against a connection made outside the platform without one stops: `the SFTP target has no hostKey`.
- **Checks don't include the table's quality checks.** Use `minRows`, `maxRows`, `within` and `parts`, or make the delivery read a table only a passing pipeline writes.

The errors you may meet, and what to do about each, are in Troubleshooting.

13. Privacy

This chapter is for the privacy team (the platform's data-protection approvers) and the administrators: how the platform knows where a person's data is, how to find what was sent about a person to partners, how to erase a person from a project's tables and the files its imports read, what the erasure record holds, how retention keeps less to erase, and what the tables' history still keeps after an erasure. The platform follows the CCPA, as amended by the CPRA; the deadline is an install setting.

In this chapter:

- The legal basis and the deadline
- Who can do what
- Where a person's data is
- What was sent about a person
- Erasure
- The erasure record
- What history keeps
- Retention
- Settings
- Limits and gotchas

See Lab 15 in the lab guide for customer 35 found, searched and erased, and Lab 4 for personal columns found by the catalog.

The legal basis and the deadline

The company follows the CCPA as amended by the CPRA (decided 2026-10-19):

- a request to delete is verified first, and answered within 45 days (one 45-day extension with notice is the law's, not the platform's);
- the recipients of the person's data are told, so the platform names the deliveries that carried it;
- the record of the request is kept at least 24 months. The platform never deletes erasure records.

The platform computes each erasure's due date as the request's date plus `erasure.deadlineDays` (default 45). It doesn't stop you erasing after it; it shows the due date in the record, on the Deliveries tab's Erasures list and on the Governance page.

Who can do what

Everything in this chapter takes the platform permission `protect-personal`. It comes with:

- the built-in platform role **data-protection**, given to the Keycloak groups in `deliveries.approverGroups` (default: the administrators' groups; Compose: `engineers,analysts`);
- any platform role an administrator defines on Administration → **Roles** with `protect-personal` (Roles);
- being an administrator.

Action	Route	Needs
Search what was sent about a person	`GET /api/projects/{name}/deliveries/search?person=<ID>`	`protect-personal`
Erase a person	`POST /api/projects/{name}/erasures`	`protect-personal`; to rewrite the landing files too, permission to write the project's files (its editors, a role with `write-files`)
See the erasure records	`GET /api/projects/{name}/erasures`	`protect-personal`
See every project's erasures	Administration → **Governance** (`GET /api/governance`)	`see-governance` (the data-protection role has it); its Erasures card lists them only to those with `protect-personal`

On the web app, the search and erasure live on the project's **Deliveries** tab, under **What was sent about a person**; you reach the project's tabs as a member of the project or as an administrator.

Where a person's data is

What it's for

Before anything can be found or erased, the platform has to know which tables hold people and which column says who. That declaration drives deliveries' approvals (Deliveries), the search below, and erasure.

Set up

As an administrator, on the project's **Edit** → **JSON**, add `tables.people`: each table holding people's data, and the column identifying a person in it.

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "members": [
      {"group": "engineers", "role": "editor"},
      {"group": "analysts", "role": "reader"}
    ],
    "tables": {
      "people": [
        {"table": "uploads_customers", "column": "customer_id"},
        {"table": "uploads_orders", "column": "customer_id"}
      ]
    },
    "files": {},
    "pipelines": {"serviceAccount": "airflow-pipelines"}
  }
}
```

Each table may be listed once; table and column are identifiers. `PUT /api/projects/retail` saves it.

How it behaves

- **The register.** Administration → **Governance** → **Readers' rules, by project** has a **People's data** column listing each project's `tables.people`, beside its masks and its tagged columns.
- **The columns.** The catalog's classifier tags personal columns (`personal`, `financial`, `health`) and masks them for everyone the install doesn't except; the **Data** page shows each column's protection. That's covered in Access and protection. `tables.people` is the declaration; the tags are what was found in the columns.

- **Who's in the data.** On the **SQL** tab, as someone who reads the tables unmasked (an editor): `SELECT * FROM uploads_customers WHERE customer_id = 35`.

What was sent about a person

What it's for

When a person asks what the company shared about them, or an erasure needs their data's recipients, the privacy team needs every file sent to a partner that carried that person's rows. The delivery log holds no one's ID; instead, each run of a delivery of personal data kept the catalog as it read the tables, and the search runs what that delivery sent again on the tables as they were, for that person's rows.

Set up

1. Open the project → **Deliveries**. Below the deliveries, data-protection approvers see **What was sent about a person**.
2. Type the person's ID in **Their ID** (`35`, `C-1042`) and press **Search**.

Through the API:

```
curl "$PLATFORM/api/projects/retail/deliveries/search?person=35" -H "Authorization: Bearer $TOKEN"
```

```
{
  "runs": [
    {"delivery": "partner_orders", "date": "2026-10-26", "sent": "2026-10-26T06:00:41Z",
     "target": "s3://acme/from-harbor", "approval": {"by": ["alice"], "protection": ["bob"], "at":
       "2026-10-22T09:14:03Z"},
     "rows": 3}
  ],
  "checked": 4,
  "notKept": 2
}
```

How it behaves

- For each run in the project's delivery log that kept its read (a Nessie tag at the commit it read), the search runs the delivery's table or query again at that commit, as the platform's maintenance account in Trino, and counts the rows whose person's column, as text, equals the ID.
- The page says how many runs sent the person's data, how many were checked, and how many couldn't be (`not personal, or from before reads were kept`). The table lists each run with the person's rows: the delivery, its date, where it went, how many of their rows, and who approved it.
- A run shows **unclear** (hover for why) when its rows couldn't be counted: `what it sent has no column identifying a person (tables.people)` (a delivery of a masked column, but not of a people's table), or `it couldn't be run again: ...` (for example, the tag has expired).
- The audit log records `deliveries.searched` with how many runs were checked and how many had the person's rows, never the ID. A refused search is recorded too (`not a data-protection approver`).

Erasure

What it's for

A person asks to be forgotten. The platform deletes their rows from every table the project declares as holding people, takes them out of the landing files those tables are imported from (so the next import doesn't bring them back), names the deliveries that carried their data, and keeps a record for the auditor that holds a hash of the person's ID and never the ID.

Set up

1. On the project's **Deliveries** tab, under **What was sent about a person**, type the ID in **Their ID** and press **Erase this person**.
2. The page asks **When did the person ask** (a date such as `2026-10-19`; empty means today), and **Who verified the request** (`the privacy tool`, or a person; empty means you).
3. Confirm: *Delete every row of 35 from this project's tables of people's data? This can't be undone, and is recorded.*

Through the API (the privacy tool's way in):

```
curl -X POST "$PLATFORM/api/projects/retail/erasures" \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"person": "35", "requestedOn": "2026-10-19", "verifiedBy": "the privacy tool"}'
```

Field	Required	Meaning
`person`	yes	The person's ID, as the people's tables hold it (up to 200 characters; spaces around it are dropped)
`requestedOn`	no	When the person asked, `YYYY-MM-DD`; default today. The due date counts from it.
`verifiedBy`	no	Who verified the request; default the caller

From Python, with the storscale package:

```
import storscale

p = storscale.Platform("https://storscale.example.com")
e = p.call("post_projects_name_erasures", {"person": "35", "requestedOn": "2026-10-19", "verifiedBy":
  "the privacy tool"}, name="retail")
print(e["id"], e["dueBy"], [(t["table"], t["rows"]) for t in e["tables"]])
```

How it behaves

In order, within seconds:

1. **What was sent about them.** The same search as above runs first, on the tables as each delivery read them (the deletion doesn't change that), and its runs go into the record.
2. **The tables.** For each table in `tables.people`, the platform counts the rows whose identifying column, as text, equals the ID, and deletes them (an Iceberg delete through Trino), as the platform's **maintenance** account in Trino, which reads and deletes unmasked. From then on, nobody's query finds them in the current tables.
3. **The landing files.** If the project has a files import (Getting data in) that reads files under `landing/uploads/` into one of its people's tables (an import `uploads` with a file table `orders` makes `uploads_orders`), each file in the project's bucket matching that file's name or glob is read and written again without the person's rows:
   - **CSV**: the header row is kept; the column is found by name (any case); rows whose value, trimmed, equals the ID are dropped; the rest are written back.
   - **JSON lines**: lines whose value for the column equals the ID (a number written as `35`, not `35.0`) are dropped.

- **Excel and Parquet** files aren't rewritten. Each is named in the record as not done: `a xlsx file isn't rewritten here: take the person's rows out of it by hand, or remove it` (or `a parquet file ...`).
- This is done **as the person erasing**, with their own sign-in to Buckets: the bucket's policies decide. The project's editors may write its files; anyone else gets each file named in the record with why (`writing it again: Access Denied...`), and the tables are erased all the same.
- A file without the person is listed with 0 rows and left as it was.

4 **Where the rows still are.** The record says how long the tables' history is kept (`historyKept`) and the date the erased rows age out of it (`agesOutBy`: now plus the history window), and names the project's tags (`closes`) that keep versions with the rows however old (What history keeps).

5 **The record** is written to the platform's bucket, `erasures/<project>/<id>.json`, and the audit log gets an `erasure` entry.

The page then says, for example: *Erased. 1 rows from uploads_customers, 8 rows from uploads_orders; and from the landing files the imports read, 1 rows from customers.csv, 8 rows from orders.csv, so the next import doesn't bring them back; 1 deliveries had carried their data. Due by 2026-12-03; recorded as 20261019T101502-3f2a9c, without the ID. The tables' old versions hold the rows until about 2026-10-26.* Anything not done is listed under **Not done:**.

The answer is HTTP 200 when everything was done, and HTTP 207 (Multi-Status) when something wasn't: the record is kept either way, with `errors` naming each table or file and why.

The erasure record

What it holds

Field	Meaning
`id`	When it ran, then six characters of the person's hash: `20261019T101502-3f2a9c`
`project`	The project
`personHash`	The first 16 hexadecimal digits of the SHA-256 of the ID. Never the ID: a second request for the same person is recognisable by it.
`requestedOn`, `dueBy`	When the person asked; that plus the deadline
`by`, `verifiedBy`, `at`	Who erased, who verified the request, when
`tables`	Each people's table, its column, and how many rows were deleted
`files`	Each landing file rewritten, its column, and how many rows were taken out
`deliveries`	The delivery runs that carried the person's rows (as the search returns them)
`deliveriesChecked`, `deliveriesNotCheckable`	How many runs were checked, and how many couldn't be
`historyKept`, `agesOutBy`	How long the tables' history is kept, and when the erased rows age out of it
`closes`	The project's tags, which keep their versions however old
`errors`	What couldn't be done, by table or file, and why (absent when everything was)

Where to read it

- **The Deliveries tab** → **Erasures** (data-protection approvers): each record's time, asked on, due by, what was deleted (table and file counts, "with errors" when there were some), how many deliveries, by whom, verified by whom. The card says *Each answered within 45 days of the request, as the law sets; the records are kept, without anyone's ID.*
- `GET /api/projects/{name}/erasures`: the records, newest first, with `deadlineDays`.
- **Administration → Governance** → **Erasures**: every project's, with its due date, who, and **erased** or how many tables or files were left.
- **The Audit tab**: an `erasure` entry, by the person who erased, whose detail names the record and counts, and never the ID: `20261019T101502-3f2a9c: 9 rows from 2 tables and 9 from 2 landing files, 1 deliveries had carried the person's data; verified by the privacy tool; due 2026-12-03`. A refused request is recorded as refused (`not a data-protection approver`).

What history keeps

An erasure deletes the rows from the tables as they are now. Older versions of the data still hold them, for as long as the platform keeps those versions:

Where	How long	What the record says
The tables' history (every version, for reading a table as it was: History and releases)	The install's history window, `maintenance.history.keep` (default `P7D`, a week). Nessie's garbage collection then deletes versions older than that and the files no branch or tag needs.	`historyKept` and `agesOutBy`
The project's tags (a month-end close, `close-2026-09`)	Until each tag's `keepUntil`, however old	`closes` names the project's tags (all of them: it doesn't check which hold the person)
Each run of a delivery of personal data	Its read is tagged (`Delivery.<project>.<delivery>.<time>`) for `deliveries.keepFor`, default seven years: the search above depends on it	Not listed; the `deliveries` field names the runs
Backups (Kubernetes)	The newest `backup.keep` nightly backups (default 14), and the data copy if `backup.data` is on	Not listed

The record makes these visible rather than silent; what a close or a delivery's kept read may keep is for the privacy team and Finance to decide. Taking a tag away early is an editor's **Remove** on the **Tags** tab (History and releases).

Retention

What it's for

Rows that age out never need erasing. Retention deletes a table's rows older than a set age, every night.

Set up

As an administrator, on **Edit** → **JSON**, add `retention` under `spec.tables` (here beside `people`):

```
{
  "tables": {
    "people": [
      {"table": "uploads_customers", "column": "customer_id"},
      {"table": "uploads_orders", "column": "customer_id"}
    ],
    "retention": [
      {"table": "uploads_campaign_responses", "column": "responded_on", "keep": "730d"},
      {"table": "uploads_orders", "column": "ordered_on", "keep": "7y"}
    ]
  }
}
```

Field	Meaning
`table`	One of the project's tables
`column`	The date or timestamp column that dates its rows
`keep`	Days or years: `730d`, `2y` (a year counts as 365 days; 1 to 99999)

How it behaves

- Each night at `maintenance.at` (02:00 UTC by default), table maintenance runs `DELETE FROM <table> WHERE <column> < <now minus keep>` as the maintenance account, before compacting the table's files. A project editor can start a round now: **Data** → a table → **Storage** → **Run now**.
- The deleted rows stay in the tables' history until the history window passes, like an erasure's.
- A project whose `tables.maintenance.paused` is `true` has no retention either.
- A failed retention step shows on the table's Storage tab and as the project's alert (`<table>: its maintenance failed`).

Settings

Chart value	Compose (platformd's environment)	Default	What it does
`erasure.deadlineDays`	`ERASURE_DEADLINE_DAYS`	`45`	Days from the request to the due date
`deliveries.approverGroups`	`DELIVERY_APPROVER_GROUPS`	the administrators' groups (Compose: `engineers,analysts`)	Who has the built-in `data-protection` role
`deliveries.keepFor`	`DELIVERY_KEEP_FOR`	`7y`	How long each personal delivery's read is kept, for the search
`maintenance.history.keep`	`NESSIE_GC_KEEP` in `stack/.env` (platformd reads it as `MAINTENANCE_KEEP_HISTORY`)	`P7D` (Compose's lab value: `PT2M`)	The tables' history window: when erased rows age out
`maintenance.at`	`MAINTENANCE_AT`	`02:00`	When retention runs
`audit.keep`	`AUDIT_KEEP`	`365d`	How long the platform's audit entries (the `erasure` entry among them) are kept

Limits and gotchas

- **The ID is matched as text, exactly.** `CAST(column AS varchar) = '35'`. `035`, `C-1042` with a different case, or a number stored with decimals won't match. Search first: the counts in the answer say what matched.
- **Only declared tables.** An erasure reaches `tables.people` and the landing files feeding them. Tables built from them (a dbt model, a flow's output, a model's training data), other projects' copies, files outside `landing/uploads/`, and data branches are not erased yet. Declare every table that holds people, or rebuild derived tables after an erasure.
- **No people's tables, no erasure.** `the project declares no tables of people's data (tables.people), so there's nothing to erase from`.
- **Landing files need write access.** A data-protection approver who isn't an editor of the project erases the tables, but each landing file is named in the record as not rewritten. Have an editor with `protect-personal` (or an administrator) erase, or rewrite those files afterwards; otherwise the next import brings the person back.
- **Excel and Parquet landing files** are named, not rewritten: take the rows out by hand, upload the file again, and run the import.
- **The search and erasure need table maintenance set up.** Without the maintenance account (`MAINTENANCE_TRINO_PASSWORD`) and the platform's files: `erasing needs the platform's files and its Trino account (maintenance)`.
- **Runs from before reads were kept**, and runs of deliveries without personal columns, can't be searched: they're counted as `notKept`.
- **The Deliveries tab only shows when the project has a delivery.** To search or erase in a project without one, open `/p/<project>/deliveries` directly, or use the API.
- **Not yet:** erasing through lineage (what was made from the people's tables), the declared files' prefixes beyond the imports' landing files, and the privacy tool's own sign-in (use a service account or a person's token today).

14. AI

This chapter is for the editors who give a project models, agents and knowledge banks, and for the administrators who run the gateway they all go through. It covers the LLM gateway (LiteLLM, or the platform's stand-in), budgets and the keys made from them, the guardrail and the install's forbidden terms, the record of model calls, agents (registration, approval, tools, agents that run a notebook, agents that run a prompt, agents asking agents and calling the project's own tools), and knowledge banks (tables and files, building, searching, on a schedule). Flows' model step is in Flows; assistants such as Claude Code over MCP are in Dashboards, apps and assistants.

In this chapter:

- The LLM gateway
- Budgets and keys
- The guardrail and forbidden terms
- Model calls
- Asking a model on the project's key
- Agents
- Agents with a notebook
- Agents with a prompt
- The project's own tools, and agents asking agents
- Knowledge banks
- Settings
- Errors you may meet

The labs: Lab 16: Models through the gateway, on a budget, Lab 17: An agent of the project's own and Lab 18: What customers say, sorted by a model.

The LLM gateway

What it's for. Every call the platform makes to a language model goes through one gateway the company controls: a key per project and per agent, each with a monthly budget, a guardrail that stops what may not leave, and a record of every call. Nobody pastes a provider's key into a notebook.

Who can do it. Administrators choose and configure the gateway at install time. Projects then use it through budgets (Budgets and keys).

The two gateways

`llm.gateway`	What runs	For
`stub` (the chart's default)	The platform's stand-in (`stack/tools/llm_stub.py`, in the tools image).	CI, labs and installs without a provider.
`litellm`	LiteLLM (`llm.litellm.image`, default `ghcr.io/berriai/litellm:v1.106.0`) on the models listed in `llm.litellm.models`, with the provider's key from the Secret `storscale-llm` (key `api_key`).	A company install.
`""`	Nothing.	No keys are made, no budgets kept; the Agents tab has no gateway card.

Either answers inside the install at `http://llm:4000`, with the master key `LLM_MASTER_KEY` (in the Secret `storscale-env`; in Compose, `stack/.env`, default `sk-storscale-local`). platformd reads spend with it and the operator makes keys with it. Compose always runs the stand-in.

The stand-in speaks the little of LiteLLM's API the platform uses (`/key/generate`, `/key/delete`, `/key/info`, `/key/list`, `/chat/completions`, `/embeddings`, `/spend/logs`). It answers a question by repeating it ("You asked: ..."), charges 0.01 per 100 characters asked, embeds text by hashing its words into 256 dimensions (so a search finds texts sharing its words, not meaning), and enforces budgets and the guardrail as LiteLLM does. It also answers a flow's model step and a semantic test's question in the shapes they expect, so they run in CI.

LiteLLM. Its keys and spend are kept in a database `litellm` on the platform's Postgres, made on first start (the master key is its password). The guardrail is loaded from `llm_guard.py` beside its config. Its models are LiteLLM's `model_list`:

```
llm:
  gateway: litellm
  guardrails: true
  forbiddenTerms: ["Project Falcon", "/acme-\\d+/"]
  litellm:
    models:
      - model_name: default
        litellm_params:
          model: anthropic/claude-opus-5
          api_base: https://<resource>.openai.azure.com/anthropic
          api_key: os.environ/LLM_PROVIDER_KEY
      - model_name: embedding
        litellm_params:
          model: azure/text-embedding-3-small
          api_base: https://<resource>.cognitiveservices.azure.com
          api_version: "2023-05-15"
          api_key: os.environ/LLM_PROVIDER_KEY
```

The platform asks for the model `default` unless told another name, and for `embedding` when it embeds (knowledge banks, the canvas's nearest-passage step). Anthropic's models on Microsoft Foundry answer the Anthropic Messages API at `https://<resource>.openai.azure.com/anthropic`, which LiteLLM's `anthropic` provider speaks: `model: anthropic/<deployment>`, that address as `api_base`.

How it behaves

- A change to the models, `guardrails`, `forbiddenTerms` or the guardrail's code restarts the gateway (a checksum on its pod).
- The provider's key is read from the Secret with any trailing newline or spaces trimmed.
- The gateway records each call (who asked, the model, tokens, cost, time), never the question or the answer.

Limits and gotchas

- Without a model named `embedding` in `llm.litellm.models`, knowledge banks can't be built on LiteLLM (it's commented out in the chart's defaults). The stand-in embeds without one.
- The stand-in's answers are the question repeated: good for checking keys, budgets and the guardrail, not for judging a prompt.
- The stand-in keeps keys in a file in an `emptyDir` on Kubernetes (`/state/keys.json`) and in memory in Compose: a restart loses its keys and their spend. The operator makes keys again for every budget it finds missing at the gateway, so spend starts again from zero.

Budgets and keys

What it's for. A spending limit per project and per agent, enforced by the gateway: at the budget, the gateway stops answering that key rather than the bill growing. The keys are made and kept by the platform; nobody handles them.

Who can do it. A project's budget is in its spec: administrators on **Edit**, or a release (History and releases). An agent's budget is part of its registration (editors, with `build`), approved with the agent. Any member sees the spend on the **Agents** tab.

Set up

A project's budget, in USD a month, up to a million: on the project's **Edit** page, **LLM budget** → **USD a month** (empty: no key), and **Save**. In its JSON it's `llm.budget`:

```
{
  "apiVersion": "platform.storscale.io/v1alpha1",
  "kind": "Project",
  "metadata": {"name": "retail"},
  "spec": {
    "members": [
      {"group": "engineers", "role": "editor"},
      {"user": "alice", "role": "editor"}
    ],
    "tables": {},
    "llm": {"budget": 20}
  }
}
```

An agent's budget: the **LLM budget (USD a month)** field when registering it, or `budget` in its JSON (Agents). 0 or none: no key.

How it behaves

- **The project's key.** Within a minute of saving, the operator asks the gateway for a key with alias `storscale/<project>`, `max_budget` the budget and a `budget_duration` of `1mo` (spend starts again on the 1st of each month, UTC), and keeps it as the project's connection `<project>-llm` (kind `llm`, added by `platform-operator`), holding `llmKey`, `llmUrl` and `llmBudget`. On Kubernetes, connections are Secrets (`retail-llm`). The **Connections** tab lists it; the key itself is shown nowhere.
- **An agent's key.** Once the agent is approved and has a budget, the operator makes a key with alias `storscale/<project>/agent/<name>` and keeps it beside the agent's secret in its connection `<project>-agent-<name>`. The agent spends against its own budget, not the project's.
- **What uses the project's key**: a flow's model step (Flows), a question app (Apps), a semantic test asked in words (Metrics), knowledge banks (building and searching), and `POST /api/agent/projects/{name}/ask`. Each call names who asked (the gateway's `user`).
- **A changed budget** means a new key: the old one is deleted at the gateway, and a program holding it must read the connection again. Spend starts again on the new key.
- **No budget any more** (the project's taken out, the agent's set to 0, the agent revoked or unapproved): its key is deleted. Keys of the project's that nothing wants any more are swept at each pass.
- **The spend.** The **Agents** tab's **The LLM gateway** card shows each key: "retail: 0.04 of 0.05 spent", with a green, amber (80%) or red (100%) share. An agent's row shows "0.00 of 0.50 USD a month at the gateway". `GET /api/projects/{name}/llm` gives the same: `gateway`, and `uses` (each `who`, `budget`, `spend`, `key`, `share`, `error`).

- **Alerts.** At 80%: "the project has used 80% of its LLM budget (0.04 of 0.05)". At 100%: "the project is at its LLM budget (0.05 of 0.05): the gateway answers it no more this month", with "Raise the budget on the project's settings, or wait for the month." An agent's say "the agent nightly-summary ...". They link to the Agents tab and go where the project's alerts go (Data quality).
- **At the budget** the gateway answers 422 ("Budget has been exceeded!"): the refusal is the gateway's, the same with LiteLLM or the stand-in.
- Model calls also count toward the project's `cost.budget`, if it has one (Operating the platform).
- **Audit log:** a budget change is a `project.saved` entry; an agent's budget is in its `agent.registered` and `agent.approved` entries.

Limits and gotchas

- `"llm: budget 2000000: USD a month, up to a million"` and `"agents: <name>: budget ...: USD a month, up to a million"` are refused at save.
- "A month" is the calendar month: LiteLLM sets the spend back to nothing at midnight UTC on the 1st. The stand-in doesn't reset spend at all; its keys are made afresh only when a budget changes or it restarts.
- A knowledge bank needs a project budget ("knowledge: a bank is embedded on the project's gateway key: give the project an LLM budget (llm.budget)"); an agent with a prompt needs an agent budget.

See Lab 16.

The guardrail and forbidden terms

What it's for. What the platform masks in tables mustn't reach a model through a prompt either, nor may the platform's secrets or the company's own codewords. The guardrail looks at every question before the model does and refuses it, naming what it found and never the value.

Who can do it. Administrators turn it on or off and keep the forbidden terms (install settings). It applies to everyone and every key.

What it refuses

A question (a chat's messages, or text sent to be embedded) is refused when it holds:

Kind	How it's found
a card number	13 to 19 digits (with spaces or dashes) that pass the Luhn check
a national ID	`123-45-6789`
a bank account number	an IBAN that passes its mod-97 check
an email address	`name@domain.tld`
a phone number	ten to fifteen digits written in groups (`+1 (415) 555-0123`, `0171 234 5678`)
a gateway key, a GitLab token, a GitHub token, an AWS key, a signed token (JWT), a bearer token, a private key	the platform's secrets by shape
a secret by its name	`LLM_MASTER_KEY=...`, `STORSCALE_LLM_KEY`, `STORSCALE_AGENT_SECRET`, or any `*PASSWORD`, `*SECRET`, `*TOKEN`, `*API_KEY` with a value of eight characters or more beside it
a term the company keeps in	the install's forbidden terms

The refusal is HTTP 400, type `guardrail`: "the question holds a card number and an email address, which may not leave the platform; it wasn't sent to the model (the gateway's guardrail)". A refused question costs nothing.

Set up

```
llm:
  guardrails: true                                  # false: everything goes through
  forbiddenTerms: ["Project Falcon", "/acme-\\d+/"]
```

A word or phrase matches whole, in any case. One between slashes is a pattern (case-insensitive); a pattern that isn't valid is skipped, not the whole list. In Compose: `LLM_GUARDRAILS` (default `true`) and `LLM_FORBIDDEN_TERMS`, one term a line (default `Project Falcon`, which the test suites ask with).

How it behaves

- The same module (`stack/tools/llm_guard.py`) runs in LiteLLM (a pre-call guardrail, `llm_guard.StorScaleGuardrail`, on by default for every key) and in the stand-in, so the two refuse the same things.
- `buckets_lake.ask()` raises it as "the gateway refused the question: ..."; the agents' runner as "the gateway refused: ..."; a knowledge bank's build stops at the chunk, adding "(mask such values in the source first: a prepare step, or a view)".

Limits and gotchas

- The guardrail reads text. Mask personal columns before they reach a model step or a bank: a **Prepare** step's **Replace by pattern** on the canvas, or a view (Flows, Lab 18 step 3).
- Patterns over-match on purpose: a long run of grouped digits may read as a phone number. Reword, or keep such values out of prompts.
- With `guardrails: false`, nothing is checked, forbidden terms included.

Model calls

What it's for. Who asked what of the models, how much it cost and how long it took, without keeping the questions or answers.

Who can do it. The project's editors and administrators. The **Model calls** card shows to members with `build` once a key exists.

Set up

On the **Agents** tab, the **Model calls** card: pick **last 1 day**, **last 7 days** or **last 31 days**.

API: `GET /api/projects/{name}/llm/calls?days=7` (1 to 31, default 7).

How it behaves

- It reads the gateway's own spend logs for each of the project's keys (the project's and each agent's).
- The card says the totals (calls, tokens, USD), the top five by who asked and by model, and the latest 50 calls: **When**, **Key** ("the project" or the agent), **Who asked**, **Model** (chat or embeddings), **Tokens**, **Cost**, **Seconds**, and a red pill for a call that didn't succeed. The API gives up to 200 calls, and totals `byDay`, `byUser`, `byModel` and `byWho`.

- "As the gateway records them: the question and the answer are never kept."

Limits and gotchas

- The API checks the built-in role: a custom role with `build` sees the card but is refused, 403 "the project's editors see its model calls".
- 503 "the platform has no LLM gateway" when `llm.gateway` is empty.

Asking a model on the project's key

What it's for. One question to the model on the project's key, without the key leaving the platform. semanticd's tests in words use it; a script can too.

Who can do it. Any member of the project, by their own Keycloak token (a reader included), and its agents.

```
curl -s -X POST https://<platform>/api/agent/projects/retail/ask \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"prompt": "Name three reasons a customer returns a jacket."}'
# {"answer": "...", "cost": 0.0123}
```

How it behaves. The gateway's guardrail and the project's budget apply; a refusal comes back as it came ("the gateway refused (HTTP 422): Budget has been exceeded! ..."), status 422. **Audit log:** `llm.asked`, with what it cost. A project without a key: 422 "the project has no gateway key: give it an LLM budget (llm.budget in its spec) and the operator makes one".

The project's gateway as OpenAI's API

What it's for. An application or a script that already speaks OpenAI's API (its SDKs, LangChain, any OpenAI-compatible client) uses the project's model through the platform: system prompts, a conversation's turns, tools, streaming, embeddings. The key stays in the platform; the caller signs in as itself.

Who can do it. Any member of the project, by their own DataWorks token (a reader included); the project's service account; and a person's token that the project's service account got on their behalf (Projects).

Set up

The project needs an LLM budget (Budgets and keys). Then point an OpenAI client at the project, with a DataWorks token as its API key:

```
from openai import OpenAI
client = OpenAI(base_url="https://<platform>/api/projects/retail/llm/v1", api_key=token)   # a
  DataWorks access token
reply = client.chat.completions.create(
    model="default",
    messages=[{"role": "system", "content": "Answer in one line."},
              {"role": "user", "content": "Why do customers return jackets?"}])
print(reply.choices[0].message.content)
```

Method and path, under `/api/projects/{name}/llm/v1`	OpenAI's	Default model
`POST /chat/completions`	Chat completions, `stream: true` too (server-sent events, ending `data: [DONE]`)	`default`

Method and path, under `/api/projects/{name}/llm/v1`	OpenAI's	Default model
`POST /embeddings`	Embeddings	`embedding`
`GET /models`	The models the project's key may ask for	

How it behaves

- **As OpenAI's API, to the letter:** the request goes to the gateway as it came, on the project's key, and its answer (or its refusal) comes back as it came: `choices`, `usage`, `finish_reason`, tool calls, the stream's chunks. Only two fields are the platform's: `model` when it's missing (`default`, or `embedding`), and `user`, which is always the caller (whatever the request says), so the spend on the project's **Model calls** is theirs.
- **The project's budget and the guardrail hold:** past the budget, 422 "Budget has been exceeded!"; a card number, an email address, a key or a term the company keeps in, 400, naming the kind and never the value.
- **Each call** is in the platform's audit log as `llm.asked` on the project, with the model and its cost (`x-litellm-response-cost` is passed back too); a streamed call's cost is on Model calls. The question and the answer are never kept.
- **Its own refusals** are OpenAI's shape too, `{"error": {"message": ..., "type": ..., "code": ...}}`: 422 "the project has no gateway key: give it an LLM budget ...", 400 for a body that isn't JSON, 404 for another route.

Limits and gotchas

- **Don't send** `temperature` (nor `top_p`) to `default` on Foundry: Claude Opus 5 through LiteLLM takes only its own. Other models may.
- The token is a DataWorks token, which lasts minutes: a long-lived client gets a fresh one (the Python package renews its own).
- 403 "<who> isn't a member of <project>"; a person's token got on their behalf is for its project alone.
- The project's single-prompt route, `POST /api/agent/projects/{name}/ask` (`{"prompt": ...}` → `{"answer", "cost"}`), stays for one-shot questions.

Agents

What it's for. A program of the project's own (a nightly summary, a triage bot) that reads the project's data with an identity of its own, only the tools it was given, an owner who answers for it, approval by a second person before it may sign in, and a switch that stops it at once. The audit log names it.

Who can do it.

Action	Who
Register, change, revoke	Members with `build` (editors).
Approve	Members with `approve` (editors), never the one who registered it.
See	The project's members, on the **Agents** tab.

The **Agents** tab shows when the project has tables and the install has the semantic service.

An agent is a reader of its project. Once approved, its Keycloak client's service account (`service-account-agent-<project>-<name>`) is put in the project's readers' group. The readers' rules hold for it as for any reader: only the tables readers may read, their row filters and masks, in Trino, in the MCP server's queries, in `buckets_lake.sql()`. It reads the project's MLflow experiments and models but doesn't log or register; it may call the project's served models and search its knowledge banks. It is a member of its own project only.

Set up

1. **Agents → Register an agent.**
2. **Name** (lowercase letters, digits and dashes, starting with a letter, up to 31 characters) and **What it's for**.
3. **Tools it may use**: pick from the chips. The MCP server's tools, the project's own tools (`tool:<name>`) and its agents with a prompt (`agent:<project>/<name>`). None picked means every tool a reader may use.
4. For an agent the platform runs: a **Schedule** (cron, UTC), and a **Notebook** (a path in the project's repo) or, in its place, **what it's to do** in words (a prompt). Give it an **LLM budget (USD a month)** if it asks a model; a prompt needs one.
5. **Register**. The owner is you. The card says "awaiting approval".
6. Another editor presses **Approve**. The page shows, once: "nightly-summary is approved. Its client is agent-retail-nightly-summary, its secret … (shown once: give it to the agent now), and it talks to https://<platform>/mcp with a token from Keycloak (client credentials)." Give the secret to the program now; it's not shown again.
7. Within a minute the operator makes its Keycloak client, and its gateway key if it has a budget.

Revoke removes it at once (after a confirm: "Revoke nightly-summary? It signs in no more.").

API (signed in, with `X-Platform-Request: 1`):

```
curl -s -X PUT https://<platform>/api/projects/retail/agents/nightly-summary \
  -H "Authorization: Bearer $TOKEN" -H "X-Platform-Request: 1" -H "Content-Type: application/json" \
  -d '{"description": "sums the day for the team", "tools": ["list_metrics", "query_metrics",
    "list_alerts"], "budget": 0.5}'
curl -s -X POST https://<platform>/api/projects/retail/agents/nightly-summary/approve -H
  "Authorization: Bearer $OTHER_EDITORS_TOKEN" -H "X-Platform-Request: 1"
curl -s -X DELETE https://<platform>/api/projects/retail/agents/nightly-summary -H "Authorization:
  Bearer $TOKEN" -H "X-Platform-Request: 1"
```

`GET /api/projects/{name}/agents` lists them (each with `clientId`, `connection`, `approved`, `mcpUrl`, and `dag` once it runs), the MCP tools, the `grants` the project offers (`tool:` and `agent:`), `canEdit` and `canApprove`. The approval's answer carries `clientId`, `secret`, `mcpUrl` and "The secret is shown once: give it to the agent now. The operator makes its client within a minute."

The agent's fields, as the PUT body or an entry of the spec's `agents`:

Field	What
`name`	Taken from the address on the PUT.
`description`	What it's for. Changing it doesn't need approval again.
`owner`	A person, who answers for it. Default: who registers it.
`tools`	MCP tool names, `tool:<name>`, `agent:<project>/<name>`. None: all the MCP server's.
`schedule`	Cron, UTC, for an agent the platform runs.
`notebook`	A relative path to an `.ipynb` in the project's repo, under the pipelines' path, as `notebooks/nightly.ipynb`.
`prompt`	What it's to do, in words, up to 8,000 characters; in place of a notebook.
`budget`	USD a month at the gateway, up to a million. 0: no key.
`registeredBy`, `registeredAt`, `approvedBy`, `approvedAt`	The platform's, kept as they were on each save; never set by hand.

Signing in as the agent

The program gets a token from Keycloak with the client's credentials, and calls the MCP server at `https://<platform>/mcp` with it:

```
curl -s https://<platform>/sso/realms/lakehouse/protocol/openid-connect/token \
  -d grant_type=client_credentials -d client_id=agent-retail-nightly-summary -d client_secret=$SECRET
```

The client's tokens carry the MCP server's audience. The same token reads `GET /api/agent/projects/{name}/runs` and `/alerts` (what `list_runs` and `list_alerts` use).

The MCP server's tools an agent may be given

Tool	What
`list_projects`	The projects it's a member of, its role, whether each has metrics.
`list_metrics`, `query_metrics`, `explain_query`	A project's metrics, their values (as the agent, under its rules), the SQL a query would run.
`list_terms`, `describe_term`, `search_terms`	The company's business terms.
`list_products`, `describe_product`, `query_product`	Data products.
`list_runs`	The project's pipelines and latest runs, and the repo's files Airflow couldn't load. Read-only.
`list_alerts`	The project's open alerts. Read-only.
`search_documents`	A knowledge bank searched by meaning.
`list_tools`, `call_tool`, `ask_agent`	The project's own tools and agents (below). Given with a `tool:` or `agent:` grant.

Dashboards, apps and assistants describes each tool's arguments and answers.

How it behaves

- **Approval covers** the owner, tools, schedule, notebook, prompt and budget. Changing any of them takes the approval away until another editor approves again; the operator removes its client meanwhile, so it signs in no more. Approving again makes a new secret.
- **Before approval** it has no client: it can't sign in. A tool it wasn't given is refused, naming what it may use: "the agent may use: list_metrics, query_metrics".
- **Revoked**, it leaves the spec, its connection (secret and key) is deleted, its key goes at the gateway, and the operator removes its client at its next pass: its tokens are refused.
- **Audit log:** `agent.registered` ("registered ..." or "changed ...", with the owner, tools and schedule), `agent.approved`, `agent.revoked`, `agent.ran` for each scheduled run, and every query it makes under its own name (Ranger's audit names `agent-retail-nightly-summary`). A refused self-approval is recorded as `agent.approved` with "approving their own".

Limits and gotchas

- Approving your own: 403 "an agent is approved by an editor other than the one who registered it".
- Missing permission: "the project's editors register its agents (it takes the permission build)"; "the project's editors approve its agents (it takes the permission approve)".
- Refused at save: "agents: name ...: lowercase letters, digits and dashes, starting with a letter"; "agents: x: owner ... isn't a person's name"; "agents: x: tool y: no tool "y" (one of ...; or agent:<project>/<name>, or

tool:<name> of the project's tools)"; "agents: x: a notebook or a prompt, not both: ..."; "agents: x: a prompt is done by a model through the gateway: give the agent a budget"; "agents: x: a schedule needs something to run: a notebook (a path in the project's repo) or a prompt"; "agents: x: a notebook runs from the project's repo (pipelines.repo), which the project doesn't have".

- Register agents on the Agents tab or through the PUT route. An agent a release's `project.yaml` brings is recorded as registered by whoever planned the release, so someone else approves it; one the project had keeps its approval unless what approval covers changed. An agent added directly in the spec's JSON on **Edit** has no registrant recorded, so the approval's two-person check has nobody to hold the approver against.
- The secret is shown once. Lost: change something approval covers and have it approved again, or revoke and register again.
- An agent sees what a reader sees. If the readers' rules hide a table, the agent can't read it; give readers that table (or a view of it) rather than making the agent something more.

See Lab 17, steps 1 to 5.

Agents with a notebook

What it's for. An agent the platform runs on a schedule: a notebook from the project's repo, executed as the agent, with the MCP server and the gateway at hand.

Who can do it. As for agents. The project needs a repo (`pipelines.repo`); see Pipelines and the project's repo.

Set up

1. Put the notebook in the repo under the pipelines' path, say `notebooks/nightly_summary.ipynb`:

```
import buckets_lake
a = buckets_lake.agent()                                    # the MCP server, as the agent
revenue = a.call("query_metrics", project="retail", metrics=["revenue"],
  group_by=["order__region"])
night = a.call("list_runs", project="retail", limit=10)
alerts = a.call("list_alerts", project="retail")
print(buckets_lake.ask(f"Two sentences for the team's chat: revenue by region {revenue}; runs
  {night['runs']}; alerts {alerts['issues']}"))
```

2. Register the agent with its tools, a **Schedule**, the **Notebook** path and a **budget** if it calls `ask()`. Have it approved.

How it behaves

- Once approved, it's a pipeline of the project's: `agent_<project>_<name>` (dashes as underscores), tagged `agent`, on the **Pipelines** tab within a minute or two, starting from its approval (no catch-up). The Agents tab names the pipeline.
- Each run checks the repo out at the pipelines' ref and runs the notebook with papermill, signed in as the agent: its client and gateway key come from an Airflow connection the operator keeps (`agent_<project>_<name>`), put in the kernel's environment, never in the logs.
- In the kernel, `buckets_lake` is the pipelines' variant signed in as the agent: `username()` is `service-account-agent-<project>-<name>`, and two more calls:

Call	What
`buckets_lake.agent()`	The MCP server, as the agent. `.tools()` lists the tools it may use; `.call(tool, **args)` gives a tool's result, and raises `PermissionError` naming what it may use when refused.
`buckets_lake.ask(prompt, model="default", system=None)`	A model's answer through the gateway, on the agent's key and budget.

- The executed notebook, outputs and all, is kept and linked from the run on **Pipelines**; **Audit** has `agent.ran` ("ran" or "failed: cell N failed: ...").

Limits and gotchas

- `agent()` outside an agent's run: "not an agent's run: no STORSCALE_MCP_URL (an agent's notebook runs from its schedule)".
- `ask()` without a budget: "no gateway key: give the agent a budget (its registration) and the operator makes one". At the budget: "the agent is at its LLM budget: the gateway answers it no more this month". Refused by the guardrail: "the gateway refused the question: ...".
- The notebook missing from the repo: "the agent's notebook notebooks/nightly_summary.ipynb isn't in the repo at main".
- Runs aren't retried: a failure waits for the next schedule, or **Run now**.

See Lab 17, step 6.

Agents with a prompt

What it's for. An agent with no code: say what it's to do, give it tools and a budget, and the platform's runner does it on the schedule, as the agent.

Who can do it. As for agents. No repo needed.

Set up

Register the agent with its tools, a **Schedule**, a **budget**, and in **Or, in place of a notebook, what it's to do**, the task: *Each morning, read the open alerts and the failed runs, and write a two-sentence summary for the team.* Have it approved.

```
{
  "name": "triage",
  "description": "sums up the open alerts",
  "owner": "bob",
  "tools": ["list_alerts", "list_runs"],
  "schedule": "0 7 * * *",
  "prompt": "Each morning, read the open alerts and the failed runs, and write a two-sentence summary
    for the team.",
  "budget": 0.5
}
```

How it behaves

- Once approved it's a pipeline, `agent_<project>_<name>`, in the project's own Airflow or the shared one.
- Each run signs in as the agent, lists the tools the MCP server gives it, and asks the gateway's `default` model on the agent's key, giving it those tools and a system message ("You are the agent triage of the project retail: ...

Use the tools to do the task; each answers as you, under the project's rules. When you're done, answer with your report alone."). Each tool the model asks for is called as the agent and its answer (up to 20,000 characters) given back, until the model answers with its report: at most 8 rounds.

- The run's record is the transcript, kept as a notebook and linked from the run: the task, each tool call with its arguments and answer (marked "(refused)" when it was), and the report. The report is also in the run's log.
- **Audit log:** `agent.ran`, under the agent.

Limits and gotchas

- "no report after 8 rounds of tools": the model kept calling tools. Narrow the task or the tools.
- "the gateway refused: ..." stops the run: the budget, or the guardrail.
- The prompt is the agent's instructions for every run; a different task is a different agent (or ask it through `ask_agent`, below).

See Lab 17, step 7.

The project's own tools, and agents asking agents

What it's for. An agent sometimes needs the company's own systems (the ticketing API) or another agent's skill. A project declares an API's operation as a tool, signed in by one of its connections, so the agent uses the API without ever seeing its token; and an agent with a prompt can be given to another agent as a tool.

Who can do it. Tools are declared in the spec (administrators on **Edit**, or a release). Calling a tool: agents given it (`tool:<name>`), and members with `build`. Asking an agent: agents given it (`agent:<project>/<name>`), and any member of its project.

Set up a tool

1. On **Connections**, add an **API token** connection (its `token`), say `retail-tickets` (Getting data in).
2. Declare the tool in the spec:

```
{
  "tools": [
    {
      "name": "open_tickets",
      "description": "The open support tickets about an order.",
      "url": "https://tickets.example.com/api",
      "method": "GET",
      "path": "/orders/{order_id}/tickets",
      "connection": "retail-tickets",
      "parameters": [
        {"name": "order_id", "in": "path", "type": "integer", "required": true, "description": "The
          order's number"}
      ]
    }
  ]
}
```

Field	What
`name`	Lowercase letters, digits and underscores, starting with a letter, up to 41.
`description`	Required: it's how an agent knows when to use it.
`url`	The API's address, `https` or `http`, no query or credentials.

Field	What
`method`	`GET`, `POST`, `PUT`, `PATCH` or `DELETE`.
`path`	Under the URL, starting with `/`; path parameters in braces.
`connection`	One of the project's connections; its `token` is sent as `Authorization: Bearer`. None: no sign-in.
`parameters`	Each `name`, `in` (`path`, `query` or `body`), `type` (`string` default, `integer`, `number`, `boolean`), `description`, `required`. Every `{name}` in the path needs a `path` parameter. A `GET` or `DELETE` has no body.

3 Give it to an agent: `tool:open_tickets` in its tools. Give another agent with a prompt: `agent:retail/triage`. Either grant also gives `list_tools`, and `call_tool` or `ask_agent`.

How it behaves

- `list_tools(project)` lists the tools the caller may call (with their parameters) and the agents it may ask. `GET /api/agent/projects/{name}/tools`.
- `call_tool(project, tool, arguments)`: the platform builds the request (path parameters escaped, query parameters added, body parameters as a JSON object), adds the connection's token, and calls the API within 30 seconds, following no redirects. The answer is `{"tool", "status", "answer"}`: JSON as it came, or text cut to 20,000 characters. `POST /api/agent/projects/{name}/tools/{tool}/call` with `{"arguments": {...}}`.
- `ask_agent(project, agent, task)`: platformd runs the asked agent's loop as that agent (its client, its key, its tools less `ask_agent`, so asking can't loop) with the task given, for up to ten minutes, and answers `{"agent", "report", "steps", "cost"}`. Only an approved agent with a prompt and a budget can be asked. `POST /api/agent/projects/{name}/agents/{agent}/ask` with `{"task": "..."}`.
- **Audit log:** `tool.called` ("GET /orders/1042/tickets: HTTP 200 in 0.3s", or "failed: ...") under the caller; `agent.asked` ("asked as service-account-agent-retail-triage by service-account-agent-retail-weekly: 2 tool calls, 0.0300 spent"). Refusals: `tool.call`, `agent.ask`.

Limits and gotchas

- "the project's editors, and the agents given it (tool:open_tickets), call its tools" (403).
- "an agent asks only the agents it's given (agent:retail/triage)" (403); an agent can't ask itself.
- "the project has no agent triage to ask: an approved agent with a prompt and a budget" (404).
- "the tool's connection retail-tickets isn't kept yet: add it on the Connections tab (an API token)" (422).
- "the tool needs order_id", "order_id is a whole number", "the tool has no parameter x" (400): the arguments don't fit.
- An `agent:` grant may name another project's agent, but the MCP server checks the asking agent is a member of that project, and an agent is a member of its own project only: in practice an agent asks its own project's agents.
- The answer goes to the model: an API that returns personal data hands it to the model's context. Choose operations that return what the agent needs.

See Lab 17, step 8.

Knowledge banks

What it's for. A project's text, searched by meaning: what customers wrote, the care guide, the returns page. Each passage is embedded through the gateway and kept as a table of the project's, so a person, an assistant, an agent or a canvas step can find the passages closest to a question, under the project's rules, with where each came from.

Who can do it.

Action	Who
Declare, change, remove a bank	Members with `build` (editors).
Build it	Members with `build`, administrators, and the project's pipelines (on a schedule).
Search it	Any member (readers included), and agents with `search_documents`, each under the project's rules on the bank's table.

What it needs. A project LLM budget (it's embedded on the project's key); `files` for a folder of the bucket; `pipelines` for a schedule.

Set up

On the **Agents** tab, the **Knowledge banks** card: **Add a bank**.

1. **Name** (lowercase letters, digits and underscores, up to 50) and **What it holds**.
2. **Tables**: a table, its **text column**, and its **id column** (cited with each passage). **Add a table** for more.
3. **And/or a folder of the project's bucket**: its `.txt`, `.md` (`.markdown`) and `.html` (`.htm`) files, as `landing/uploads/`.
4. **Build it afresh on a schedule** (cron, UTC), or leave empty to build by hand.
5. **Add**, then **Build**. Nothing is embedded until it's built. The card says "reviews: 90 passages embedded, 0.0009 spent".

Or in the spec:

```
{
  "llm": {"budget": 5},
  "files": {},
  "pipelines": {"serviceAccount": "airflow-pipelines"},
  "knowledge": [
    {
      "name": "reviews",
      "description": "What customers wrote",
      "tables": [{"table": "uploads_reviews", "column": "text", "id": "review_id"}]
    },
    {
      "name": "manuals",
      "description": "The care guide and the returns page",
      "files": [{"prefix": "landing/uploads/"}],
      "schedule": "0 3 * * *"
    }
  ]
}
```

API: `GET /api/projects/{name}/knowledge`; `PUT /api/projects/{name}/knowledge/{bank}` with the bank's JSON (no `name`: it's the address); `DELETE` the same; `POST .../{bank}/build`; `POST .../{bank}/search` with `{"query": "the zip broke", "k": 5}`.

How it behaves

- **Building**, as the caller (the editor, or the project's pipelines): each table's `id` and text are read through Trino (up to 20,000 rows a source, nulls skipped), and each file under the folder (up to 2,000 files, 4 MB each; an HTML page's scripts, styles and head dropped and its tags taken out). Each text is cut into chunks of about 1,200 characters, embedded 64 at a time on the project's key (the model `embedding`), and written afresh as the table `knowledge_<bank>` in the project's namespace (`source`, `ref`, `chunk`, `text`, `embedding`). It's an ordinary table of the project's, under its rules. The answer says the table, `chunks`, `files`, `skipped` (files of another kind or too big) and `cost`.
- **Searching**: the question is embedded the same way, and the bank's table is ranked in Trino by cosine similarity, as the person or agent asking. Each hit is `source` (`table.column`, or `files:<folder>`), `ref` (the row's id, or the file's key), `text` and `score` (1 the closest). `k` is 5 by default, up to 50. The card's search box does it; an assistant or agent uses `search_documents(project, bank, query, k)`.
- **On the canvas**, an **Ask a model** step's mode **Nearest passage in a knowledge bank** adds each row's nearest passage and its source as two columns (Flows).
- **On a schedule**, the bank is a pipeline of the project's, `knowledge_<project>_<bank>`, tagged `knowledge`, that asks the platform to build it afresh as the project's pipelines (one retry).
- **Removing** a bank takes it out of the spec; its table `knowledge_<bank>` stays until someone drops it.
- **Audit log:** `knowledge.declared` (its sources and schedule), `knowledge.built` ("90 chunks (0 files), 0.000900 spent"), `knowledge.removed` ("its table knowledge_reviews kept"); a refused build is `knowledge.build`.

Limits and gotchas

- **The bank's table carries the text of every row its builder could read.** Readers' row filters and masks on the source table don't follow the passages into `knowledge_<bank>`: give the bank's table its own readers' rule (Access and protection), or build it from a view of what readers may see. If the project lists the tables readers may read, add `knowledge_<bank>` there for readers and agents to search it.
- A source holding an email address, a card number or another masked value stops the build at the guardrail: "embedding chunks 1 to 64: the gateway refused (HTTP 400): the question holds an email address, ... (mask such values in the source first: a prepare step, or a view)".
- "the bank isn't built yet: build it from the project's Knowledge card": search before build.
- "the project has no gateway key yet: give it an LLM budget (llm.budget) and the operator makes one".
- A bank is built afresh each time, not added to: a big source is embedded, and paid for, in full on each build.
- PDFs aren't read: convert them to text or HTML first.
- On the stand-in, search matches shared words, not meaning; LiteLLM's embedding model finds meaning.
- Refused at save: "knowledge 1: a bank's name is lowercase letters, digits and underscores"; "knowledge x: names at least one table's text column, or a folder of files"; "knowledge x, folder 1: a folder of the bucket, ending in /, as landing/docs/"; "knowledge x: files are read from the project's bucket, which it doesn't have (files)"; "knowledge x: on a schedule, the project's pipelines build it: give the project pipelines (pipelines.serviceAccount)".

See Lab 18, steps 6 and 7.

Settings

Chart value	Default	Compose	What
`llm.gateway`	`stub`	the `llm` service runs the stand-in	`stub`, `litellm`, or `""` for none.
`llm.guardrails`	`true`	`LLM_GUARDRAILS` (`true`)	The guardrail on or off.

Chart value	Default	Compose	What
`llm.forbiddenTerms`	`[]`	`LLM_FORBIDDEN_TERMS` (`Project Falcon`, one a line)	The install's words and `/patterns/` that may not leave.
`llm.litellm.image`	`ghcr.io/berriai/litellm:v1.106.0`		LiteLLM's image.
`llm.litellm.models`	`default`: `anthropic/claude-opus-5`		LiteLLM's `model_list`: `default`, and `embedding` for banks.
Secret `storscale-llm`, key `api_key`			The provider's key, as `LLM_PROVIDER_KEY`.
`LLM_MASTER_KEY` (Secret `storscale-env`)		`stack/.env`: `sk-storscale-local`	The gateway's master key, held by platformd and the operator.

platformd reaches the gateway at `LLM_GATEWAY_URL` (`http://llm:4000`) with `LLM_GATEWAY_MASTER_KEY`. The MCP server agents use is the semantic service, at the platform's `/mcp`.

Errors you may meet

The words	Where	What to do
"Budget has been exceeded!" (HTTP 422)	the gateway	Raise the budget (a new key is made), or wait for the 1st of the month (UTC).
"the question holds ..., which may not leave the platform; it wasn't sent to the model (the gateway's guardrail)"	the gateway	Take the value out, or mask the column first.
"the project has no gateway key: give it an LLM budget (llm.budget in its spec) and the operator makes one"	ask, banks	An administrator adds `llm.budget`.
"no gateway key: give the agent a budget (its registration) and the operator makes one"	`buckets_lake.ask()`	Give the agent a budget; it's approved again.
"the agent may use: ..."	an agent's tool call	Give it the tool (approved again), or use one it has.
"an agent is approved by an editor other than the one who registered it"	Approve	Another editor approves.
"the project's editors see its model calls"	Model calls	Use the built-in editor role.
"no report after 8 rounds of tools"	a prompt agent	Narrow its task or tools.
"the bank isn't built yet: build it from the project's Knowledge card"	search	Press **Build**.
"the agent triage's client or gateway key isn't kept yet"	`ask_agent`	Wait a minute after approval; check its budget.

More in Troubleshooting.

15. History and releases

Two ways of keeping things true over time. **History**: every change to a project's tables is a commit of the catalog, so a table can be read as it was at any moment the install still keeps, and a tag keeps a moment (a month-end close) for as long as the project says. **Releases**: the project itself (its members, rules, checks, products and metrics) kept as code in its repo, planned from a tag by one person and applied by another, and rolled back by applying an earlier one. This chapter is for analysts and finance users who need last month's numbers again, the editors who make tags and plan releases, and the administrators who apply them and set how much history the install keeps.

In this chapter:

- A table's history
- Reading the tables as they were (As of)
- Tags: moments kept, such as a close
- Nightly maintenance and what history is kept
- The project as code
- Planning and applying a release
- Rolling back
- The API
- Settings
- Limits and gotchas

A table's history

What it's for. Seeing what happened to a table: each change, when, what it did (rows added and removed), who wrote it, and which tags hold it; and reading it as it was before any of them.

Who can do it. Any member of the project. Reading a change as it was runs as you, under your rules: a reader who sees the EU's rows and masked card numbers today sees the same in last month's data.

Set up

Open **Data**, pick a table, and its **History** tab. It lists the table's changes, newest first:

Column	What
When	The commit's time, and its hash (the first twelve characters)
What	What the change did: its operation (`append`, `overwrite`, `delete`), `+N rows`, `−N rows`, and the rows the table had then; the project's tags on that commit, as blue pills
Who	Who wrote to the table then, from the audit log; – when it can't tell
	Read it as it was, or "past the history kept"

Read it as it was opens the SQL tab with `SELECT * FROM "uploads_orders#<commit>" LIMIT 100`: the table named at that commit. Change the query as you like; it runs as you.

How it behaves

- The catalog (Nessie) keeps every table's changes as commits on its `main` branch; a table is read at one by its name and the commit, `"uploads_orders#3f1c0e9a2b7d4c55…"` (quoted, because of the `#`). Trino passes the name to Nessie, and Ranger's policies name it too, so row filters and masks hold.
- **A table's first entry is its creation**, which holds no rows (it reads as a change with 0 rows then). For an imported table, **the import's first load is the entry after it**: read that one to see the table as the import left it. See Lab 19.
- The tab shows up to the 50 newest changes, looking back over at most 5,000 of the catalog's commits (the catalog is shared by every project, so on a busy install an old table's first changes may be further back than that). Rows and who wrote are filled in for the 25 newest changes still kept.
- **Who** is the user Trino's audit (Ranger) records writing to the table (an insert, update, delete, merge, create or truncate) from three minutes before the commit to one minute after: a person, or a project's pipelines service account.
- A change older than the history the install keeps, and not held by a tag, says "past the history kept"; its data files may be gone (what is kept). The newest change is always readable.
- The tab's note says how far back: "The data as it was is kept 168h back, and as long as a tag holds it."

Reading the tables as they were (As of)

What it's for. Reproducing a number from last month exactly: every table of a query read as of a tag, a time or a commit.

Who can do it. Any member, on the SQL tab, under their own rules.

Set up

1. On the **SQL** tab, write the query with the project's own table names: `SELECT round(sum(amount), 2) FROM uploads_orders WHERE NOT test`.
2. In the box under it (**As of: now, a tag, 2026-09-30, a commit**), type the moment and press **Read the tables then** (or Enter). The query's project tables are rewritten to that commit (`FROM "uploads_orders#<commit>"`), and the moment is shown: "as of the tag close-2026-09: commit 3f1c0e9a2b7d".
3. Run it. **As of** `now` (or an empty box) puts the names back.

A moment is one of:

You type	It means
`now` (or nothing)	The catalog's main as it is
A tag of the project's: `close-2026-09`	The commit the tag keeps
A commit: 16 to 64 hex characters	That commit
A time: `2026-09-30T18:00:00Z`, `2026-09-30T18:00` or `2026-09-30 18:00` (UTC)	The commit of main current at that moment
A date: `2026-09-30`	As it was at the day's end, 23:59:59 UTC

Through the API, `GET /api/projects/retail/asof?at=close-2026-09` answers `{commit, time, label}`; a notebook or a script names the table at that commit itself: `SELECT … FROM "uploads_orders#<commit>"` through `buckets_lake.sql()` (Notebooks and models).

Limits

- A moment it can't read: ""last week": now, one of the project's tags, a time (2026-09-30 or 2026-09-30T18:00:00Z) or a commit".
- A time before the history kept: "the catalog's history kept starts after 2026-09-01 00:00 UTC".
- Only `FROM` and `JOIN` of the project's own tables are rewritten; another project's product named in the query is read as it is now.
- There is no restore button: to put a table back, read it as it was and write it with SQL (`CREATE TABLE … AS SELECT … FROM "uploads_orders#<commit>"`), which needs `write-tables`.

Tags: moments kept, such as a close

What it's for. A month-end close, the data a model learned from, the state at an audit: every table of the project as it was at a moment, kept for as long as you say (seven years for a close), whatever the install's history window does to the rest.

Who can do it. Seeing the tags: any member. Making and removing them: the permission `release` (editors).

Set up

On the **Tags** tab, **Make a tag**:

Field	What
Name	Lower-case letters, digits and dashes: `close-2026-09`
At	A moment, as **As of** takes it: `now`, `2026-09-30`, a commit (default `now`)
Keep for	Days or years: `90d`, `7y` (default `7y`)
Note	What it is, at most 500 characters: "September's close, signed off by Finance"

Press **Make the tag**. The list shows each tag's name, its commit, **Kept until**, the note and who made it, and "being applied" until the catalog has it. **Remove** takes it away.

The same in the spec, `tables.tags` (the Tags tab writes it; `by` is who made it):

```
{"tables": {"tags": [
  {"name": "close-2026-09", "commit":
    "3f1c0e9a2b7d4c55e6f708192a3b4c5d6e7f8091a2b3c4d5e6f708192a3b4c5d",
   "keepUntil": "2033-10-03", "note": "September's close, signed off by Finance", "by": "bob"}
]}}
```

Through the API: `POST /api/projects/retail/tags` with

```
{"name": "close-2026-09", "at": "now", "keep": "7y", "note": "September's close"}
```

and `DELETE /api/projects/retail/tags/close-2026-09`.

How it behaves

- A tag names the catalog's commit on main at that moment: every table of the project, as it was. The platform saves it in the project's spec; within a minute the operator makes the Nessie tag `<project>.<name>` (`retail.close-2026-09`) at that commit, and removes those no longer listed.
- Read a table at a tag with **As of** on the SQL tab, or from its History, where the tag shows on the change it keeps.

- Nessie's GC keeps whatever a tag points to, however old, so a tagged moment stays readable past the history window.
- Each night, before the tables are kept, tags past their **Kept until** are removed from every project (below); asking for maintenance by hand does the same.
- The audit log records `tag.made` (its commit, the moment, until when, the note), `tag.removed`, and `tag.expired` (by `storscale-platform`).
- Deliveries of personal data keep the catalog as each run read it the same way, with tags of their own (Deliveries); what an erasure means for tagged closes is in Privacy.

Limits

- Without `release`: "the project's editors make and remove its tags" (HTTP 403). A project without tables: "the project has no tables" (409).
- A name taken: "the project has a tag close-2026-09 already" (409). A bad name: "tables: tags: "Close Sept" isn't a tag's name (lower case letters, digits and -)".
- **Keep for** as `90d` or `7y`: "keep: as 90d or 7y".
- A tag holds every table's data files at that commit: a long-kept tag on fast-changing tables keeps their old files in Buckets.

See Lab 19.

Nightly maintenance and what history is kept

What it's for. Keeping the tables fast and their storage in check: small files compacted each night, rows past their retention deleted, and history older than the install's window removed, except what tags hold.

Who can do it. It runs by itself. Asking for a project's maintenance now: `run-pipelines` (editors). Pausing it: an administrator, in the spec. Setting the window: the install's operators.

How it behaves

Each night at `maintenance.at` (02:00 UTC by default), platformd:

1. removes the projects' tags past their keep, and the deliveries' tags past `deliveries.keepFor`;
2. queues every project, and keeps them one at a time, as Trino's `maintenance` account. For each table of the project:
   - **retention** first, when `tables.retention` names the table: `DELETE` the rows whose date column is older than its keep (Privacy);
   - **optimize**: `ALTER TABLE … EXECUTE optimize`, small files compacted into larger ones. Each step may take up to 30 minutes; a failed step ends that table's maintenance and says why.

Later (`maintenance.history.schedule`, 03:30 UTC by default), **Nessie's GC** deletes, across the install, the tables' history older than `maintenance.history.keep` (`P7D`, a week, by default) and the files no branch or tag needs. Iceberg's own snapshot expiry is off: Nessie keeps a table's history as its commits, shared by every project.

So, of a table's past: the last week (by default) is readable at any moment; older moments are readable only where a tag (a close, a delivery's read, a merge request's data branch) holds them; the rest is gone.

Set up

- **Run now**: **Data** → a table → **Storage**, **Run now** (all of the project's tables, one at a time). The button says "Waiting its turn…" while queued and "Keeping the tables…" while it runs. The Storage tab shows the table's data files, rows, and what the last maintenance did (the files and bytes before and after, and how long it took) or why it failed. The audit log records `maintenance.asked`.
- **Pausing** a project's maintenance (an administrator, in the spec):

```
{"tables": {"maintenance": {"paused": true}}}
```

- A table whose last maintenance failed is one of the project's alerts. Prometheus has `platform_table_maintenance_last_run_seconds`, `platform_table_maintenance_failed` and `platform_table_data_files`, per project and table.

Limits

- Paused, or no tables: "the project's tables have no maintenance (none, or paused)" (409).
- Without the maintenance account (`MAINTENANCE_TRINO_PASSWORD`), there's no maintenance: "Table maintenance isn't set up on this install."
- Maintenance remembers its last runs only since platformd last started: "Not kept since the platform last started."

The project as code

What it's for. What a project is (its members, readers' rules, checks, imports, products, deliveries, agents, metrics) kept in its repo, reviewed like code, and the same on every install apart from what differs per install. Production then runs a tag of the repo, not whatever someone last typed on the Edit page.

Set up

In the project's repo (the one in `pipelines.repo`), a folder `storscale/`:

```
storscale/
  project.yaml              # the whole project: apiVersion, kind, metadata (its name) and spec
  semantic.yaml             # the semantic model: metrics, terms and tests (optional)
  targets/development.yaml  # what differs on the install whose target is development (optional)
  targets/production.yaml   # ... and on production
```

`project.yaml` **is the whole project**, as the Edit page's JSON box has it, in YAML or JSON: `apiVersion` (`platform.storscale.io/v1alpha1`), `kind` (`Project`), `metadata` with its `name`, and `spec`. Applying it replaces the project's spec, so whatever it leaves out, the project loses: its `pipelines` (and with them the repo itself), its imports, its tags. Start from the project's spec as it is (the Edit page's JSON box) and keep it whole.

```
# storscale/project.yaml
apiVersion: platform.storscale.io/v1alpha1
kind: Project
metadata:
  name: retail
spec:
  description: Harbor Outfitters' retail team.
  members:
    - {group: engineers, role: editor}
    - {user: alice, role: editor}
  tables:
    freshness:
      - {table: uploads_orders, within: 24h}
  pipelines:
    serviceAccount: airflow-pipelines
    repo: https://gitlab.example.com/harbor/retail-pipelines.git
    path: dags
    ref: main
    credentials: {secretRef: retail-git}
    reviews: {gitlab: "https://gitlab.example.com", dbt: dags/dbt}
```

A target's file is laid over `project.yaml` for the install whose target it is: maps are merged, anything else (a list, a value) is replaced whole. So a target that changes one member replaces the whole `members` list.

```
# storscale/targets/production.yaml
spec:
  description: Harbor Outfitters' retail team (production).
  pipelines:
    ref: production
```

The install's target is `install.target` in the chart (`production`), `INSTALL_TARGET` in Compose (`development`). The Releases tab names it: "and `targets/production.yaml` for this install (production)".

`semantic.yaml` is the project's semantic model as the Metrics tab keeps it (Metrics and business terms). Without one, a release leaves the model as it is.

Planning and applying a release

What it's for. A change to the project reaches it on purpose: someone plans a tag of the repo and sees exactly what it would change; someone else applies it.

Who can do it.

What	Who
See the plans and the release	The project's members
Plan a ref	`release` (editors)
Apply a plan	An administrator who isn't the one who planned it

What it needs. The project's repo (`pipelines.repo`), the platform's files (its bucket) and Airflow.

Set up

1. Tag the repo (`git tag v1.0 && git push origin v1.0`).
2. On the project's **Releases** tab, type the ref in **A release to plan** (`v1.0`, a branch such as `main`, or a commit) and press **Plan**. The plan shows "checking out…", then **ready** with what would change, or **failed** with why.

3 An administrator other than the planner presses **Apply** on the ready plan. The card at the top then says "Released: v1.0 (3f1c0e9a2b7d), planned by alice, applied by bob" and when.

How it behaves

1 **Plan.** platformd records the plan (`fetching`) and starts a run of the platform's DAG `storscale_plan` in the project's Airflow. The run checks the repo out at the ref with the project's git connection, and sends back the commit and the folder's files (`project.yaml`, `semantic.yaml`, `targets/*.yaml`, each up to 1 MiB), as the project's pipelines. The audit log records `release.planned`.

2 platformd composes the spec (`project.yaml`, with `targets/<this install's target>.yaml` laid over it), checks it as any spec is checked, checks `semantic.yaml` with the semantic layer, and lists what would change against the project as it is:

A change	Reads
A member added or removed	`member added: carol as reader`, `member removed: group analysts as reader`
A part of the spec changed (two levels deep)	`changed: description`, `changed: tables.readers`, `changed: pipelines.ref`
A delivery whose sending changes	`delivery acme-weekly: what it sends changes, so it needs approving again`
A metric	`metric added: region_rows`, `metric changed: revenue`, `metric removed: region_count`
The model changed, its metrics not	`semantic model changed (its metrics as they were)`

Or "No changes."

3 **Apply.** The plan applies only if the project is still as it was when planned (its spec and semantic model). The composed spec is saved as if edited by hand (checked again), keeping what only the platform sets: deliveries' approvals (a delivery whose sending changed needs approving again, Deliveries), and each product's consumers and certification (Data products). `semantic.yaml`, when there is one, becomes the project's model. The operator applies the spec within a minute, as any change.

4 The release is recorded: the plan becomes **applied**, with who applied it and when, and the audit log records `release.applied` with the ref, its commit, the target, who planned it and the changes.

The Releases tab lists the 30 newest plans; each keeps its ref, commit, target, who planned it and when, its state, and its changes.

Limits

- Planning without `release`: "the project's editors plan its releases" (403). Without a repo: "the project has no repo (pipelines.repo)". A ref that can't be one: "a ref of the repo: a tag (v1.3), a branch, or a commit".
- A ref the repo hasn't: the plan **fails**, saying so ("checking out v9: ...").
- `project.yaml` missing: "no storscale/project.yaml at v1.0". Naming another project: "project.yaml names project "sales", not "retail"". A field the spec doesn't have: "not a project: json: unknown field "readers"". Any of the spec's own checks fails the plan with its words.
- A plan whose run never answers fails after 15 minutes: "no answer from its run (storscale_plan in the project's Airflow)".
- Applying your own plan: "someone other than who planned it applies it" (403, recorded as a refusal). Not an administrator: refused. A plan not ready: "the plan is failed" (409).
- The project changed since the plan (someone saved the Edit page, or another release was applied): "the project has changed since it was planned: plan it again" (409). Plan again from the repo, so the repo stays the truth.

See Lab 20.

Rolling back

What it's for. Going back to the project as an earlier release had it, when a release turns out wrong.

How. Plan the earlier ref (`v1.0`) and have an administrator apply it, as any release. Its spec and metrics come back; the plan lists what that undoes ("member removed: carol as reader", "metric removed: region_rows").

What it doesn't do. The tables' data written since stays, with its history: a rollback is of the project's definition, not of its data. To read the data as it was before the bad release, use As of at the earlier release's time, or a tag made before it. The DAGs and dbt models are what the branch `pipelines.ref` names holds, not part of the release: revert them in git.

The API

Method and path	What	Who
`GET /api/projects/{name}/history?table=<table>`	A table's changes, newest first: `{table, changes: [{hash, time, message, author, keys, stats, tags, who, kept, query}], keepHistory}`	Members
`GET /api/projects/{name}/asof?at=<moment>`	The commit a moment stands for: `{commit, time, label}`	Members
`GET /api/projects/{name}/tags`	The tags: `{tags: [{name, commit, keepUntil, note, by, nessie, inPlace}], keepHistory}`	Members
`POST /api/projects/{name}/tags`	Make one: `{name, at, keep, note}`	`release`
`DELETE /api/projects/{name}/tags/{tag}`	Remove one	`release`
`GET /api/projects/{name}/maintenance`	The tables as maintenance keeps them: `{enabled, at, state, paused, keepHistory, tables}`	Members
`POST /api/projects/{name}/maintenance`	Ask for the project's maintenance now: `{state}` (202)	`run-pipelines`
`GET /api/projects/{name}/plans`	The plans (30 newest), the release, the install's `target`, `repo`, `canPlan`, `canApply`	Members
`POST /api/projects/{name}/plans`	Plan a ref: `{"ref": "v1.0"}`	`release`
`POST /api/projects/{name}/plans/{plan}/apply`	Apply a ready plan	An administrator other than the planner
`POST /api/projects/{name}/plans/{plan}/files`	The repo's `storscale/` folder at the plan's ref (the DAG `storscale_plan` sends it)	The project's pipelines

With a session, a change needs the header `X-Platform-Request: 1`; a Keycloak token (`Authorization: Bearer …`) works too.

Settings

Setting	Chart value (default)	Compose	What it changes
When the tables are kept each night	`maintenance.at` (`"02:00"`, UTC)	`MAINTENANCE_AT` on platformd (default `02:00`)	Compaction, retention, and tags' expiry
How much history is kept	`maintenance.history.keep` (`P7D`)	`NESSIE_GC_KEEP` in `stack/.env` (`PT2M` as shipped, for the tests: set `P7D` or more for real use)	The window in which any moment can be read; an ISO 8601 duration
When Nessie's GC runs	`maintenance.history.schedule` (`"30 3 * * *"`)	daily at 03:30 UTC, or every `NESSIE_GC_EVERY` seconds (`180` as shipped; leave it empty for real use)	When old history is deleted
The maintenance account	`MAINTENANCE_TRINO_PASSWORD` in the Secret `storscale-env`	`MAINTENANCE_TRINO_PASSWORD` in `stack/.env`	Without it, no maintenance
This install's target	`install.target` (`production`)	`INSTALL_TARGET: development` on platformd	Which `storscale/targets/<target>.yaml` a plan lays over `project.yaml`
How long deliveries' reads are kept	`deliveries.keepFor` (`7y`)	`DELIVERY_KEEP_FOR` on platformd (default `7y`)	Their tags' keep (Deliveries)

Limits and gotchas

- **A history window shorter than you think.** The shipped Compose `.env` keeps two minutes of history, collected every three minutes, so the test suites can watch history go. On an install for real use, set `NESSIE_GC_KEEP=P7D` (or more) and leave `NESSIE_GC_EVERY` empty. Anything that must outlive the window needs a tag.
- **The window is install-wide.** It isn't per project or per table; a project keeps longer only by tagging.
- **A release replaces the whole spec.** `project.yaml` must hold everything the project has, `pipelines` included. What the platform sets is kept: the Tags tab's tags (a tag `project.yaml` names too stays as the platform made it), agents' registrations and approvals (a new agent registered by who planned it), data products' consumers and certification, and deliveries' approvals.
- **Lists in a target replace, maps merge.** A target's `members` is the whole list.
- **A hand edit invalidates a plan.** Edit the repo, not the Edit page, once a project is released from code.
- **Rollback is of the definition, not of the data.**
- **Reading as it was is under today's rules.** Masked stays masked and filtered stays filtered, whatever the rules were then.

See Troubleshooting, Pipelines and the project's repo for the repo itself, and Operating the platform for backups, which are a different thing from history.

16. Roles

Roles say what people may do. This chapter is for the platform's administrators, who define roles, and for anyone who needs to know exactly what a role gives in Trino, Buckets, Airflow and the platform's own pages. It covers the two built-in project roles (editor and reader) as the platform's permissions, defining roles of the company's own from those permissions, giving them in a project, the platform roles given to Keycloak groups across every project, and what a change to a role reaches, and when.

In this chapter:

- Two kinds of role
- The built-in project roles
- The project permissions
- Defining a project role
- Giving a project role
- How a project role is applied
- Reading tables: readers' rules, read-all and editors
- Platform roles
- Changing and removing a role
- Settings
- Limits and gotchas

Two kinds of role

	Project role	Platform role
Given	by a project, to its members (people and Keycloak groups), in its spec	on Administration → Roles, to Keycloak groups
Reaches	that project only	every project, and the pages across them
Built in	`editor`, `reader`	`finance`, `governance`, `data-protection`, `table-adoption` (from the install's settings)
Made of	the 17 project permissions	the 4 platform permissions
Defined by the company	yes: **Define a project role**	yes: **Define a platform role**

An administrator (in `PLATFORM_ADMIN_GROUPS`, `engineers` by default) has every permission of both kinds, in every project, on the platform's pages and API. What an administrator reads in Trino is still what their groups are given there.

The built-in project roles

What they're for

Most teams need two roles: those who build (editors) and those who read (readers). The platform defines both as sets of its permissions, so a role of the company's own is made of the same parts.

Role	Permissions	In short
`editor`	all 17	Makes and changes the project's tables, files, pipelines, metrics and models
`reader`	`read`, `read-files`, `call-models`	Reads what the project shows readers, and calls its models

The project's pipelines (`pipelines`), its approved agents (`agent`) and its served models' accounts (`serving`) are members of the platform's own, never given by hand: the pipelines read and write the tables and the bucket's `landing/`; agents and servings read as readers do.

Administration → Roles lists editor and reader as cards marked **built in**, with what each gives in words. They can't be changed or removed.

The project permissions

Seventeen, in the order the Roles page lists them. Some imply others: a role with `write-tables` reads every table as it is (`read-all`), and so reads (`read`); a role with `write-files` reads the files. The Roles page marks an implied permission "(comes with another)".

Permission	The Roles page says	Where it holds
`read`	Read the tables the project shows readers, as readers see them (its row filters and masks)	Trino (Ranger): the tables in `tables.readers.tables` (or all), under the readers' row filters and masks and the tags' masks; a source with `readers: true`
`read-all`	Read every table of the project as it is: no readers' row filters or masks, and tagged columns unmasked	Trino (Ranger): every table, unfiltered and unmasked; every source; the data branches of its merge requests
`write-tables`	Make, change and drop the project's tables	Trino (Ranger): everything in the namespace; publishing its data products
`read-files`	Read the project's files (its bucket)	Buckets: list and read the bucket
`write-files`	Write the project's files, and upload to its landing area	Buckets: read, write, delete; uploads to the landing area (the Flows canvas's uploads)
`run-pipelines`	Start and clear the project's pipelines' runs, and its tables' maintenance	The Pipelines tab's runs; asking for table maintenance; the project's own Airflow as `Op` (else `Viewer`)
`change-metrics`	Change the project's metrics (its semantic model) and run their tests	semanticd: saving the semantic model
`build`	Build the project's flows, apps, knowledge banks and agents, and schedule notebooks	Flows, apps, knowledge banks, agents (registering, revoking), scheduled notebooks, calling the project's tools, asking for and answering requests for data products
`models`	Track runs and register models in the project's MLflow	MLflow: writing (reading is every member's); scorings' references and runs
`call-models`	Call the project's served models	`POST /api/projects/{name}/serving/{serving}/predict`
`approve`	Approve agents, model versions and deliveries (never one's own)	Approving agents, model versions, deliveries
`classify`	Confirm or remove the catalog's tags on the project's columns	The Data page's classification

Permission	The Roles page says	Where it holds
`connections`	Give and change the project's credentials (its connections)	The Connections tab; the Imports tab
`release`	Plan and apply releases, make tags, and restore from history	The Releases and Tags tabs
`see-usage`	See the project's usage and cost	The Usage tab
`see-audit`	See the project's audit log	The Audit tab
`mute-alerts`	Mute the project's alerts for a while	**Mute** on the Overview's Alerts

A page or a route that needs a permission the person lacks refuses in words that name it, as "the project's editors change its connections (it takes the permission connections)", and the tab or button that needs it isn't shown.

Defining a project role

What it's for

Teams need roles between editor and reader: a data steward who reads every table unmasked and confirms the catalog's tags but changes nothing; an auditor who reads everything and the audit log; an analyst who runs pipelines but doesn't build. An administrator defines each once, for every project to give.

Who can do it

Administrators. Anyone signed in sees the roles (`GET /api/roles`); anyone else who tries to define one is refused: "only administrators change projects".

Set up: on the page

1. Open **Administration** → **Roles** → **Define a project role**.
2. **Name**: 2 to 24 lowercase letters, digits and dashes, starting with a letter: `auditor`. Fixed once defined.
3. **What it's for**: up to 300 characters: "Reads every table as it is and the audit log; changes nothing".
4. **What it may do**: tick the permissions. `read` is ticked to start with. For the auditor: `read-all`, `read-files`, `see-audit`.
5. Press **Define it**. The role's card lists what it gives, `read` among them "(comes with another)", and "No project gives it." until one does.

Set up: the API

`PUT /api/roles/{role}`, with `X-Platform-Request: 1`:

```
{
  "description": "Reads every table as it is and the audit log; changes nothing",
  "permissions": ["read-all", "read-files", "see-audit"]
}
```

Field	What it is
`description`	Up to 300 characters. Optional.

Field	What it is
`permissions`	At least one of the 17 project permissions
`scope`	Leave it out for a project role (`"platform"` for a platform role)

The name comes from the path. It answers with the role, what it grants (`granted`, the implied permissions with it) and the projects giving it. The audit log records `role.saved` with "permissions read-all, read-files, see-audit".

The definitions are kept in the platform's bucket (`roles/roles.json`), not in any project.

Limits and gotchas

What's refused (400 unless said):

Message	Why
"a role's name is 2 to 24 lowercase letters, digits and dashes, starting with a letter: "Auditor" isn't"	The name
"editor is the platform's own"	`editor`, `reader`, `pipelines`, `agent`, `serving`, `admin`, `editors`, `readers`
"finance is the install's (its settings' groups): define another"	A built-in platform role's name
"a role gives at least one permission"	None ticked
"a description of up to 300 characters"	Too long
"no permission "write" for a project's role: they're [read read-all ...]"	Not one of the 17
"a project's role is given in a project's spec, not to groups here"	`groups` on a project role
"auditor is a project's role: remove it first to make it another kind" (409)	Changing a role's scope
"a role, as JSON: its description and permissions"	The body isn't JSON
"the platform keeps no files here (no STORE_URL)" (503)	No platform bucket

Giving a project role

Set up

A project gives a defined role exactly as it gives editor and reader: on the project's Edit page, **Members**, the role offered beside editor and reader, for a Keycloak group or a person. In the spec:

```
"members": [
  {"group": "engineers", "role": "editor"},
  {"group": "analysts", "role": "reader"},
  {"user": "alice", "role": "auditor"}
]
```

See Projects.

How it behaves

- **A role must be defined before a project gives it.** Saving a project that gives an undefined role is refused (422): "role steward: reader, editor, or a role defined on Administration → Roles (none is called that)". A platform role can't be given in a project: the same refusal.
- **One role each, the one that gives the most.** Someone given several roles in a project (through groups and by name) has one: an editor's above every other, a reader's below every other, and between them the role that grants more permissions. alice, a reader through `analysts` and an auditor by name, is an auditor.

- **Within a minute**, the project's **Access** tab lists her as `auditor`, with a card (titled "Auditor", its group `retail-role-auditor`) saying in words what the role gives; the Overview's "What you can do here" says "Your role here is auditor."; her header shows the role; the tabs and buttons follow its permissions (the Audit tab appears, for one).
- Administration → **Access** shows every person's role in every project.

How a project role is applied

The operator applies a project role in every system the built-in roles reach, from one mapping:

System	What the operator does
Keycloak	A group per role per project: `<project>-role-<role>`, holding exactly the people given the role (and who aren't given a higher one)
Ranger (Trino)	The same group, named in the project's policies by what the role's permissions give: the namespace, its tables, the readers' row filters and masks, the tags' masks, its sources, its merge requests' data branches, and the data products it consumes. See the next section
Buckets	A policy named as the group: read and write with `write-files`, read with `read-files`, none otherwise
The project's own Airflow (Kubernetes, a project with a repo)	The group as `Op` with `run-pipelines`, else `Viewer`
platformd and semanticd	Each member's permissions are recorded in the project's status; every page and route checks the one it needs

What doesn't follow a project role: Superset's own roles (who may build datasets, charts and dashboards) and the shared Airflow's pages follow the platform's groups (`analysts`, `engineers`). What a role *reads* holds in Superset all the same, since every query runs in Trino as the person. A project's pipelines are run from its Pipelines tab, which asks for `run-pipelines`.

A data product's consumers: when a project consumes another's product, its custom roles that read (`read`, `read-all` or `write-tables`) read the product too, as its readers and editors do: under the producing project's readers' rules.

Reading tables: readers' rules, read-all and editors

How a role reads a project's tables decides which of its Ranger policies name its group. There are three ways:

A role with	Reads	The readers' rules (`tables.readers`)	The tags' masks (protection)
`write-tables` (editors)	every table, and writes them	don't apply	lifted
`read-all`, not `write-tables`	every table, as it is	don't apply	lifted
`read`, not `read-all` (readers)	the tables readers may read	apply: their row filters and masks	apply
none of them	no table (not even the namespace's list)		

The project's pipelines read and write every table, and the readers' rules don't name them either.

Editors and read-all roles come first, unfiltered. Ranger keeps one row-filter policy and one mask policy for each table and column, and applies the first item of it that a person matches. A person can be in several groups at once:

alice may edit `retail` and be an analyst, and the analysts may be readers somewhere, or `retail`'s data may be read by another project whose groups she's in. So in each of the project's readers' row-filter and mask policies, the groups that read the tables as they are (its editors' and its read-all roles') come first, with no mask (`MASK_NONE`) and no row filter, then the readers' groups (and the consuming projects' groups) with the filter or mask. Whatever else alice is, an editor of `retail` reads `retail`'s tables whole.

"No row filter" for them is an empty one, which Ranger applies as none: Trino refuses to update a table that has any row filter on it (even `true`), so editors keep `UPDATE`, `DELETE` and `MERGE` on a filtered table.

Within one project, the operator puts each person in one of the project's groups only (see Giving a project role), so an editor of a project is never also in its readers' group.

For example, with `retail`'s readers' rule `uploads_orders: region = 'EU'` and a mask on `uploads_customers.signed_up`:

	`uploads_orders`	`uploads_customers.signed_up`
bob, an editor	every region	the date
alice, an auditor (`read-all`)	every region	the date
an analyst, a reader	`EU` only	the year only
a reader of `marketing`, which consumes `uploads_orders` as a data product	`EU` only	(not a product: not read)

The readers' rules themselves (which tables, which filters, which masks) are in Access and protection. The Governance page's **What readers see** lists, for each project, its readers and the custom roles that read as readers do (with `read` but not `read-all`), with the role named.

See Lab 13 for the readers' rules, and Lab 21 for a role that lifts them.

Platform roles

What they're for

Some people work across every project without being a member of any: finance reads every project's cost; the governance group certifies data products; the privacy office approves deliveries of personal data and answers erasures. A platform role gives such a group its permissions everywhere.

The platform permissions

Permission	The Roles page says	Where it holds
`see-all-usage`	See every project's usage and cost	Administration → Usage, or **Usage and cost** in the account menu; every project's Usage tab
`see-governance`	See the Governance page: every project's approvals, agents, rules, products and erasures	**Governance** in the account menu
`certify-products`	Certify data products, and take certification away	The Data products page
`protect-personal`	Approve deliveries that send personal columns, search what was sent about a person, and record erasures	Deliveries, the delivery log's search, erasures
`adopt-tables`	Adopt tables from another platform's catalog, take them over and give them back, in the projects one may write tables in	A project's Data page: **Adopt tables**, **Take over…**, **Give back…**, **Stop adopting** (Adopting tables); only with `write-tables` in the project too

The install's platform roles

Four are built in, made from the install's settings, and shown on the Roles page marked **the install's**:

Role	Permissions	Its groups (setting)	Chart value	Default
`finance`	`see-all-usage`	`USAGE_VIEWER_GROUPS`	`usage.viewerGroups`	the administrators' groups
`governance`	`certify-products`, `see-governance`	`PRODUCT_CERTIFIER_GROUPS`	`products.certifierGroups`	the administrators' groups
`data-protection`	`protect-personal`, `see-governance`	`DELIVERY_APPROVER_GROUPS`	`deliveries.approverGroups`	the administrators' groups
`table-adoption`	`adopt-tables`	`ADOPTION_GROUPS`	`adoption.groups` (a list)	`data-migration`, a group of its own: setup makes it in Keycloak, empty

Change their groups with the settings (`storscale upgrade --set products.certifierGroups=governance`), not on the page. `table-adoption` alone has a group of its own by default, not the administrators': adopting another platform's tables and taking them over is migration work, given to the few doing it (put them in `data-migration`). The administrators may anyway. Compose sets `DELIVERY_APPROVER_GROUPS: engineers,analysts`, so the example's alice is a data-protection approver there.

Who can do it

Administrators define, change and remove platform roles.

Set up: on the page

1. **Administration** → **Roles** → **Define a platform role**.
2. **Name**: `privacy-office`.
3. **What it's for**: "The privacy office: approves deliveries of personal data, answers erasure requests".
4. **Keycloak groups given it**: comma-separated, at least one: `privacy-office`. Everyone in them has it, across every project.
5. **What it may do**: tick `protect-personal` and `see-governance`.
6. **Define it**. The card says "Given to the Keycloak groups privacy-office."

Set up: the API

`PUT /api/roles/privacy-office`, with `X-Platform-Request: 1`:

```
{
  "description": "The privacy office: approves deliveries of personal data, answers erasure requests",
  "scope": "platform",
  "groups": ["privacy-office"],
  "permissions": ["protect-personal", "see-governance"]
}
```

How it behaves

- Someone may do what any platform role of any of their groups gives; an administrator, everything.
- platformd reads the defined platform roles again every half minute, so a new or changed one reaches its groups within half a minute (at once on the platformd that saved it). The account menu's links come with the page: reload it.

- A person's groups are those in their sign-in. Someone just added to the role's group in Keycloak signs out and in again to have it.
- The audit log records `role.saved` with the permissions and "given to groups privacy-office" (and, for a change, what they were).

Limits and gotchas

- **"a platform role is given to at least one Keycloak group"**; **"no permission "read" for a platform role: they're [see-all-usage see-governance certify-products protect-personal adopt-tables]"**; **""data protection" isn't a Keycloak group's name"**.
- A platform role is never a project's: a project can't give it to its members.
- A platform role gives nothing inside a project's tables: a privacy officer who must read personal data needs a project role too (or to be in a protection rule's `except`; see Access and protection).

Changing and removing a role

Set up

On a role's card, **Change** opens its form (name and kind fixed): change the description, the permissions, or (a platform role) its groups, and **Save**. **Remove** asks "Remove the role auditor?". The API: `PUT /api/roles/{role}` with the whole role again; `DELETE /api/roles/{role}` (with `X-Platform-Request: 1`), which answers 204.

How it behaves

- **A project role changed reaches every project giving it within a minute.** A project's applied state covers the permissions of the roles it gives, so the operator applies each such project again, without its spec changing: the groups' Ranger policies, Buckets policies, its Airflow's roles and the permissions recorded for each member. Take `read-all` from `auditor`, and alice reads `retail` as readers do again, in every project giving the role.
- **A platform role changed reaches its groups within half a minute.**
- The audit log records each change, with what it was: `role.saved`, "was read-all, read-files, see-audit; now read, read-files, see-audit"; and `role.removed`.
- **A project role that a project gives can't be removed** (409): "the projects retail give auditor: change their members first". The refusal is in the audit log too. On the page, **Remove** is greyed out, with "Given in retail" as its tip. Change those projects' members, then remove it.
- A platform role can be removed at any time; its groups lose what it gave within half a minute.
- If a role a project gives disappears all the same (its file edited by hand), its members get nothing, and the project says so under **Applied to**: "no role auditor: an administrator defines it (Administration → Roles); its members have no access meanwhile". The Access tab says "No role auditor is defined: its members have no access until an administrator defines it."

See Lab 21 in the lab guide, steps 7 to 10: an auditor defined, given, changed, and a platform role.

Settings

What	Chart value	Compose (platformd's environment)	Default
The administrators	not a value	`PLATFORM_ADMIN_GROUPS`	`engineers`
`finance`'s groups	`usage.viewerGroups` (a list)	`USAGE_VIEWER_GROUPS`	the administrators' groups

What	Chart value	Compose (platformd's environment)	Default
`governance`'s groups	`products.certifierGroups`	`PRODUCT_CERTIFIER_GROUPS`	the administrators' groups
`data-protection`'s groups	`deliveries.approverGroups`	`DELIVERY_APPROVER_GROUPS` (`engineers,analysts` in `stack/compose.yaml`)	the administrators' groups
`table-adoption`'s groups	`adoption.groups` (a list)	`ADOPTION_GROUPS`	`data-migration`

The defined roles are kept in the platform's bucket, `roles/roles.json` (`STORE_BUCKET`: `dataworks-platform`, or `storscale-platform` on an install made before 0.39.0 and on Compose): the operator and platformd both read it, and backups keep it with the bucket.

Limits and gotchas

- **Approving is never one's own.** `approve` lets someone approve agents, model versions and deliveries, but not one they registered or changed themselves, whatever their role: "someone other than who changed what it sends approves it".
- **A role with `write-tables` reads everything as it is.** There's no "writes but reads as a reader": writing a table implies reading all of it.
- **A role without `read` (or more) sees no tables at all**: not even the namespace's list. A role for running pipelines alone still needs `read` to see what they made.
- **Reading files and tables are separate.** `read-all` doesn't give the bucket: add `read-files`.
- **Asking for a data product takes `build`** in the asking project, and answering requests for one takes `build` in the producing project.
- **Superset's and the shared Airflow's own roles** follow the platform's groups, not project roles (see How a project role is applied).
- **The Roles page needs the platform's bucket.** Without `STORE_URL`, roles can't be defined: "the platform keeps no files here (no STORE_URL)".

17. Operating the platform

This chapter is for the people who run the platform: installing it on one machine with Docker Compose or on Kubernetes with its Helm chart, checking it and being told when something breaks, the audit log, the Governance and Usage pages, limits, table maintenance, backups and restore, upgrades, growing Buckets, the install's settings, and the platform's API and Python package. Most of it is for administrators (members of the `engineers` group on the example install); the Usage and Governance pages are also for finance, the governance group and the data-protection approvers.

In this chapter:

- Installing
- Checking an install: storscale test
- Health and the administrators' alerts
- The audit log
- Governance
- Usage and cost
- Limits
- Table maintenance
- External stores
- The platform's files on a company's store
- Backups and restore
- Upgrades
- Buckets capacity
- The install's settings
- The Python package and the API

See Lab 21 and Lab 22 in the lab guide.

Installing

What it's for

The platform runs the same way on one machine (Docker Compose, for trying it and for development) and on a Kubernetes cluster (the Helm chart, for real use). The `storscale` command starts, checks, upgrades, backs up and restores it.

Who can do it

Whoever runs Docker on the machine, or has `kubectl` and `helm` rights on the cluster's namespace. Nothing here goes through the platform's own sign-in.

On one machine (Docker Compose)

You need Docker with Compose 2.24 or later, about 10 GB of memory for Docker, and Go 1.26 or later to build the command.

```
go build -o bin/storscale ./cmd/storscale
bin/storscale up --dir stack      # this checkout's stack: builds the platform's images too
bin/storscale urls                # where each tool is
```

Then open `http://storscale.localhost:8800` and sign in as `alice`, `bob` or `carol`; their password is `LAKEHOUSE_USER_PASSWORD` in `stack/.env`, where every password is a local test value.

Command	What it does
`storscale up [--port N]`	Starts the platform (builds its images the first time) and waits for it; prints the tools' addresses
`storscale status`	The services (`docker compose ps --all`)
`storscale urls`	Where each tool is
`storscale logs [service ...]`	Follows the logs (the last 100 lines first)
`storscale test [suite ...]`	The end-to-end checks
`storscale down [--volumes]`	Stops it; `--volumes` also deletes its data
`storscale version`	Its version

`--dir DIR` uses the stack in a checkout instead of the one built into `storscale` (which runs the released images at its own version). The environment variables `STORSCALE_DOMAIN` and `STORSCALE_PORT` change the address (`storscale.localhost`, `8800`); with rootless Docker set `DOCKER_SOCK=/run/user/<uid>/docker.sock` so JupyterHub can start notebooks (`storscale up` warns when it's needed).

On a kind cluster of your own

`storscale up --k8s` makes a kind cluster called `storscale` and installs the chart as the release `storscale` in the namespace `storscale` (the published chart at `storscale`'s version, or `--version`; with `--dir stack`, the checkout's chart and images built from it). It needs `kind`, `kubectl` and `helm`. `status`, `urls`, `logs <service ...>` (required on kind), `test`, `backup`, `restore` and `down` take `--k8s` too; `down --k8s --volumes` deletes the cluster. `STORSCALE_K8S_VALUES` names more values files for the chart.

On a real cluster

It needs about 12 to 16 GB of memory, 4 or more CPUs, about 60 GB of disk, Helm, a default StorageClass (or `storage.storageClassName` and `buckets.storageClassName`), and for each project with a pipelines repo, its own Airflow's memory (`pipelines.memory`, 3Gi by default). With TLS at an ingress and a private CA, a values file such as:

```
global:
  domain: platform.example.com
  scheme: https
  port: 443
gateway:
  service: {type: ClusterIP}
  ingress:
    enabled: true
    className: nginx
    annotations:
      cert-manager.io/cluster-issuer: my-issuer
      nginx.ingress.kubernetes.io/proxy-body-size: "0"     # S3 uploads
    tls:
      - secretName: storscale-tls
        hosts: [platform.example.com, catalog.platform.example.com, access.platform.example.com,
          s3.platform.example.com]
trino:
  service: {type: ClusterIP}
```

```
helm install storscale oci://ghcr.io/storscale/charts/storscale-platform --version 0.42.1 \
  -n storscale --create-namespace -f platform.yaml --set-file global.ca.certs=root-ca.pem
kubectl -n storscale get pods,jobs,projects -w      # the first install takes 10 to 20 minutes
helm test storscale -n storscale --logs --timeout 90m
```

How it behaves and what to watch:

- The platform's four names, `domain` and `catalog.`, `access.`, `s3.` under it, must resolve to the ingress from people's browsers and to the gateway from inside the cluster (hairpin traffic through the load balancer, or a CoreDNS rewrite as `deploy/kind/dns.sh` does).
- Every secret is generated on install and kept on upgrade, in the Secret `storscale-env`; give one in `secrets:` to choose it. The example people's password: `kubectl -n storscale get secret storscale-env -o jsonpath='{.data.LAKEHOUSE_USER_PASSWORD}' | base64 -d`.
- Keycloak's realm is imported on the first install only; after that people and their sign-in are the organisation's. The example people (alice, bob, carol) and the sales project come with it unless `examples.users: false`, which is for an install people sign in to with their own accounts (`sso.entra`): set it before the first install, since it changes what's imported (people already in a realm stay; remove them in Keycloak). Name the catalog's administrators then too (`catalog.admins`), whom bob is otherwise.
- A Spark Operator another release installed owns the Spark CRDs: install with `--set spark-operator.enabled=false`.
- Upgrade with `storscale upgrade`, never a bare `helm upgrade` (Upgrades).

From a registry of your own (GitLab and Harbor)

A company that keeps its images in its own registry builds them there: `.gitlab-ci.yml` runs on the company's GitLab runner and pushes the same ten images as the public build (linux/amd64) to a Harbor project, and on each release tag the chart too, its `images.registry` set to that project. Pushes to the default branch push `<short sha>` and the branch's name; a tag `v<version>` pushes `<version>` and `latest`; another branch builds everything without pushing.

Set up, once, in the GitLab project's or its group's **Settings → CI/CD → Variables**:

Variable	What it is
`HARBOR_REGISTRY`	The registry's host, such as `harbor.example.internal`
`HARBOR_PROJECT`	The Harbor project for the images and the chart, with a folder in it if wanted (`acme/dataworks` puts them at `acme/dataworks/platformd` and so on)

Variable	What it is
`HARBOR_USER`, `HARBOR_PASSWORD`	A robot account of that project with push (and pull) permission; the password masked
`HARBOR_CA`	A **File** variable: the CA that signed the registry's certificate, when it isn't a public one

The runner is the one tagged in the file (`default: tags`). Jobs build with rootless BuildKit, so the runner needs no privileged pods, but it must reach the base images' registries (Docker Hub, quay.io, gcr.io) and, for the chart, `ghcr.io` and `kubeflow.github.io` (its dependencies). Install from it as from the public chart:

```
helm install storscale oci://harbor.example.internal/<project>/charts/storscale-platform --version
  0.42.1 \
  -n storscale --create-namespace -f platform.yaml --set-file global.ca.certs=root-ca.pem
```

An install from `ghcr.io` moves to Harbor at its next upgrade, its chart and images both:

```
storscale upgrade --context <cluster> --chart
  oci://harbor.example.internal/<project>/charts/storscale-platform \
  --version 0.42.1 --ca-file root-ca.pem --set images.registry=harbor.example.internal/<project>
    --dry-run
```

The `--set images.registry` is kept with the install's values, so later upgrades take Harbor's images too. The nodes pull them anonymously when the Harbor project is public; a private one needs an image pull Secret in the namespace.

Checking an install: storscale test

`storscale test` runs every end-to-end suite, or those named: `shell` (the web app, in a browser), `lakehouse`, `superset`, `airflow`, `jupyterhub`, `catalog`, `semantic`, `projects`, `monitoring`, and the rest in `stack/tests/` (`deliveries`, `erasure`, `health`, `limits`, `usage`, `audit`, ...). On Compose they run in the `test` service; with `--k8s` (or `storscale upgrade --test`) as `helm test`, the ConfigMap `storscale-tests` saying which. The lab guide is a suite too, run only when named (`storscale test --dir stack guide`), and it leaves what it makes.

A suite prints each check as it goes and fails the command if any failed: `failed: shell, the other suites`.

Health and the administrators' alerts

What it's for

The administrators need to know, before anyone tells them, that a service is down and what people can't do meanwhile; and to be told in their channel. Everyone else needs to know what isn't available just now.

Who can do it

Administration → **Health** is the administrators'. Every signed-in person gets the short form: the home page's notice *Some of the platform isn't available just now: ... Its administrators have been told.* Muting and the test alert are administrators only (`administrators only`).

Set up

Where alerts go is a Secret of the platform's own, `storscale-alerts`, read every half minute, so it changes without a restart:

Key	What
`teamsWebhook`	A Teams channel's Workflows webhook (the alert is an Adaptive Card)
`slackWebhook`	A Slack incoming webhook
`emailTo`	Addresses, comma-separated
`smtpServer`	`smtp.example.com:587` (STARTTLS when it offers it)
`smtpFrom`, `smtpUser`, `smtpPassword`	The sender and its sign-in

On Kubernetes:

```
kubectl -n storscale create secret generic storscale-alerts \
  --from-literal=teamsWebhook='https://prod-00.westeurope.logic.azure.com/workflows/...' \
  --from-literal=emailTo=platform-team@harbor.example \
  --from-literal=smtpServer=smtp.harbor.example:587 --from-literal=smtpFrom=storscale@harbor.example
```

On Compose the same keys go in a JSON object at `secrets/storscale-alerts.json` in the `storscale-platform` bucket.

Then on Administration → **Health**, **Where alerts go** says what's set, and **Send a test alert** sends one to each and says, per target, `sent` or why not.

How it behaves

- platformd checks every service every 15 seconds: the gateway, Keycloak and its database, Buckets, Nessie, Trino and its limits' database, Ranger and its database, the metrics and SQL service (semanticd), the project operator, Superset and its database, Airflow (its database, scheduler and DAG processor), JupyterHub, the catalog (OpenMetadata), its search and database, Prometheus and Grafana.
- The page lists each service: its state (**up**, **down**, **unknown**) and since when, **Why** (on Kubernetes, from its pods: `FailedMount: ...`, `CrashLoopBackOff (7 restarts)`), what it's **Used for**, and **Mute 4 h** / **Unmute**. The banner says *All N services are up*, or how many are down and what people can't use.
- **Other problems** lists the monitoring rules firing in Prometheus (Buckets' drives offline, capacity low, errors), and on Kubernetes setup steps that gave up, copies (change capture) that stopped, and overdue backups.
- **Alerts** go to every target when a service goes down (`Trino is down`, with why and what it affects) and comes back (`... is back`); when a monitoring rule starts and resolves; when a setup step fails; when backups are overdue (`Backups are overdue`, none succeeded for 26 hours); and once a day in the platform certificate's last 14 days (`The platform's certificate ends in N days`). Services going down and monitoring rules firing aren't sent in the first five minutes after platformd starts, while everything starts too.
- **Recent alerts** shows what was sent since platformd started. A muted service's changes aren't sent; muting is recorded in the audit log (`alert.muted`), as is a test alert (`alert.tested`).
- The checks are Prometheus metrics too (`platform_service_up`, on platformd's port 9102); Administration → **Monitoring** opens Grafana.
- The API: `GET /api/health` (everyone gets `affected`; administrators the whole), `POST /api/health/mute` with `{"service": "trino", "hours": 4}` (up to a week; 0 unmutes), `POST /api/health/test`.

Each project's own alerts (a failed pipeline run, a failing check, a stale table, a budget, a bucket near its size) go to that project's people where its `notify` says; see Projects and Data quality.

The audit log

What it's for

"Who read the customers' table last month, and who tried and was refused" is a search, not an investigation. The log holds two things: every access Trino allowed or refused (Ranger's audit), and everything people did in the platform (projects saved, approvals, uploads, erasures, model calls, sign-ins).

Who can do it

Where	Who
A project's **Audit** tab (`GET /api/projects/{name}/audit`)	Its editors, a role with `see-audit`, administrators. Holds the project's own records and every access to its tables.
Administration → **Audit** (`GET /api/audit`, `?project=` narrows it)	Administrators. Every project, and the platform's own records.
Export CSV (`?format=csv`, up to 1000 rows)	Administrators

Set up

Nothing to set: it's on in every install. On either page:

- **Person**: a user name; **Search**: a table, an action or words of a query; **Result**: *Allowed and refused*, *Refused only*, *Allowed only*; **Source**: *The platform and Trino*, *The platform's*, *Trino's*; **From** and **To**; press **Show**. **Older** pages back.
- API parameters: `user`, `q`, `result` (`allowed`, `denied`), `source` (`platform`, `trino`), `since`, `before` (RFC 3339), `limit` (default 100).

```
curl "$PLATFORM/api/projects/retail/audit?q=uploads_customers&result=denied&since=2026-10-01T00:00:00Z
  " \
  -H "Authorization: Bearer $TOKEN"
```

How it behaves

- Both kinds are kept in Ranger's Solr: the platform's records for `audit.keep` (default `365d`), Ranger's own for 90 days, on a volume of `audit.storage` (5Gi).
- Trino's entries appear within about a minute, with the policy and the reason Ranger gave.
- The platform's records name the person, the action and the resource (`project:retail`, `project:retail/partner_orders`), whether it was allowed, a detail, and the client's address. Actions include `project.saved` (with the parts of the spec changed), `project.deleted`, `connection.saved`, `connection.removed`, `upload`, `upload.removed`, `import.saved`, `import.reset`, `pipeline.started`, `flow.saved`, `tag.made`, `tag.removed`, `release.planned`, `release.applied`, `classification.*`, `product.*`, `delivery.approved`, `delivery.sent`, `deliveries.searched`, `erasure`, `agent.registered`, `agent.approved`, `agent.revoked`, `agent.asked`, `model.registered`, `model.approved`, `model.called`, `llm.asked`, `tool.called`, `knowledge.*`, `app.*`, `role.saved`, `role.removed`, `template.saved`, `library.saved`, `maintenance.asked`, `alert.muted`, `alert.tested`, `signed.in`, `signed.out`.
- Nothing in the log can be changed from the platform. A write to the log never holds a request up; if the log can't be written, platformd logs `audit` with the error.

Governance

What it's for

The audit log says what happened; the Governance page says where things stand now, across every project: what waits for someone's approval, the agents and their spend, the rules readers are under, the data products, and the erasures with their deadlines. It changes nothing; each row links to where the decision is made.

Who can do it

Anyone with the platform permission `see-governance`: administrators (Administration → **Governance**), and the built-in platform roles **governance** (`products.certifierGroups`) and **data-protection** (`deliveries.approverGroups`), who reach it from the account menu's **Governance**. `GET /api/governance`.

How it behaves

The page's cards:

Card	What
Awaiting approval	Agents not approved (waiting for *an editor of retail other than alice*), deliveries not approved (what they still need), requests to read a data product; with since when
Agents	Every project's agents: owner, tools, what runs on what schedule, the last run, spend against budget
What readers see	Each protection tag across the install: its mask and who's excepted
Readers' rules, by project	Readers, the only tables they see, row filters, masks, tagged columns, and the project's people's tables
Data products	Each product, owner, whether certified, consumers
Erasures	Each erasure with its due date, who, and **erased** or how many tables or files were left (only to those with `protect-personal`)

A summary line counts them: `3 awaiting approval; 4 agents (3 approved); 1 of 2 products certified; 1 erasures recorded`.

Usage and cost

What it's for

Finance wants each project's month in money; a project's editors want to know what drives it. The platform counts each project's Trino CPU time, what it keeps, and its model calls, and prices them by the install's rate card.

Who can do it

Where	Who
A project's **Usage** tab (`GET /api/projects/{name}/usage?month=2026-10`)	Its editors (`see-usage`), administrators, the install's usage viewers
Administration → **Usage**, or the account menu's **Usage and cost** (`GET /api/usage?month=2026-10`)	Administrators, and the platform role **finance** (`usage.viewerGroups`, permission `see-all-usage`)

Set up

- The install's rate card, `usage.rates` (chart) or `USAGE_RATES` (platformd's environment, JSON): `{"currency": "USD", "cpuHour": 0.05, "storageGBMonth": 0.023}` by default. Model calls are priced already, by the gateway.
- Who sees every project: `usage.viewerGroups` (`USAGE_VIEWER_GROUPS`, comma-separated); empty means the administrators.
- A project's monthly budget, in the rate card's currency, under its `spec`:

```
{"cost": {"budget": 50}}
```

How it behaves

- **Queries:** Trino reports each finished query to platformd; a query counts for the projects whose tables it read (split evenly when it read several), with its CPU time, the bytes it read and who ran it. Each platformd keeps its counts a minute apart.
- **Storage:** once a day, each table's data files (Iceberg's `$files`) and the project's bucket (Buckets' count) are sampled; a GB-month is priced from the samples.
- **Model calls:** what the gateway says the project's keys spent this month.
- The Usage tab shows this month against the last (**What**, **Used**, **Cost**, **Rate**), then **Who ran the queries**, **The costliest queries** (with their SQL) and **The largest tables**. The administrators' page lists every project's month: **Total**, **Queries**, **Storage**, **Model calls**, **Budget**, **Last month**.
- At 80% of a budget the project is alerted (`the project has used 80% of its monthly budget (...)`), and again past 100% (`... past its budget of 50.00`, *Nothing is stopped: see what drives it on the Usage tab.*). Limits stop things; budgets don't.
- Not yet: Spark isn't counted.

Limits

What it's for

One project, or one person, can't slow the others. Trino's resource groups cap how many queries run and wait at once and how much memory a project's queries take; a project's bucket can have a size limit.

Who can do it

The install's defaults are the chart's (`limits.*`); a project's own are on its spec, which administrators edit.

Set up

On the project's **Edit** → **JSON**, under `spec`:

```
{"limits": {"queries": 8, "queued": 100, "queryMemory": "4GB", "storage": "500Gi"}}
```

Field	Meaning	Allowed	Install default
`queries`	Trino queries running at once	1 to 100	`limits.project.queries`: 4
`queued`	Waiting; more are refused	up to 1000	`limits.project.queued`: 40
`queryMemory`	Trino memory for its queries	`4GB`, `512MB`, `50%`	`limits.project.queryMemory`: `50%`
`storage`	The project's bucket's size (needs `files`)	`500Gi`, `2Ti`	`limits.project.storage`: none

A person's own queries (`limits.person.queries` 4, `queued` 100) and the whole install's (`limits.total.queries` 20, `queued` 1000) are the chart's only. On Compose the operator uses the same defaults, or `LIMITS_PROJECT_*`, `LIMITS_PERSON_*` and `LIMITS_TOTAL_*` in its environment when they're set.

How it behaves

- The operator keeps Trino's resource groups (a group per project, per person, and one for the install) in a database Trino reads every second.
- A query counts against a project when one of its members names it (client tag `project:<name>`, which the SQL page, dashboards and metrics set) and for all the project's own pipelines; everything else counts against the person.
- A query past the queue is refused by Trino with `QUERY_QUEUE_FULL`, its message naming the group (`project-retail`), where it ran: on the SQL tab, in a dashboard, in a notebook.
- The bucket's size is a hard quota in Buckets. At 80% the project's people are alerted (`the bucket retail is 83% full (...)`), and when it's full (`... is full (...): new files are refused`); each resolves as the bucket empties.
- Notebooks are a person's: `notebooks.memoryLimit` (1G), `notebooks.cpuLimit` (2), `notebooks.storage` (1Gi).

Table maintenance

What it's for

Iceberg tables written often collect small files, and a table's history grows. Each night the platform compacts every project's tables, applies their retention, and lets the catalog's garbage collection delete history older than the window.

Who can do it

It runs by itself. A project's editors (`run-pipelines`) start a round now: **Data** → a table → **Storage** → **Run now** (`POST /api/projects/{name}/maintenance`).

Set up

Setting	Default	What
`maintenance.at` (`MAINTENANCE_AT`)	`02:00` UTC	When platformd starts the nightly round
`maintenance.history.keep`	`P7D`	How far back the tables' history goes (Nessie's GC). Compose: `NESSIE_GC_KEEP` in `stack/.env` (`PT2M` for the lab)
`maintenance.history.schedule`	`30 3 * * *`	When Nessie's GC runs. Compose: daily, or every `NESSIE_GC_EVERY` seconds
A project's `tables.maintenance.paused`	`false`	`true` skips the project (compaction and retention)

How it behaves

- One project at a time, as Trino's `maintenance` account: each table's retention first (`tables.retention`, Privacy), then `ALTER TABLE ... EXECUTE optimize`. Project tags past their `keepUntil` and delivery tags past `deliveries.keepFor` are removed at the start of the round.

- Nessie's GC then deletes history older than `history.keep`, and files no branch or tag needs; a tag (a close) keeps what it points to.
- Each table's **Storage** tab shows its files and snapshots, when it was last kept, and the step that failed if one did; a failed table is the project's alert (`<table>: its maintenance failed`). The page says *Table maintenance isn't set up on this install* when platformd has no maintenance account.

External stores

What it's for

Another platform's Iceberg tables (a Hive Metastore and its MinIO, say), which projects adopt where their files are while the move to this platform goes on (Adopted tables).

Who can do it

Whoever installs the chart. Who then adopts, takes over and gives back the store's tables is the chart's `adoption.groups` (default `[data-migration]`, a Keycloak group setup makes, empty): the platform role `table-adoption`, in the projects they edit, besides the administrators (Roles). Put the people doing the move in that group.

Set up

The chart's `externalStores`, one entry per store, and a Secret in the release's namespace with the store's key (`accessKey`, `secretKey`): a key that may **only read** the store's buckets. Nothing here writes to the store, and the key makes sure of it.

```
externalStores:
  - name: lakehouse                       # projects adopt from lakehouse.<schema>.<table>
    catalog:
      type: hive_metastore                # or rest (with uri and warehouse)
      uri: thrift://hive-metastore.lakehouse.svc.cluster.local:9083
    s3:
      endpoint: https://minio.lakehouse.svc.cluster.local
      region: us-east-1
      buckets: [warehouse]                # the buckets its tables' files are in
    secret: lakehouse-store
```

```
kubectl -n <namespace> create secret generic lakehouse-store \
  --from-literal=accessKey=<read-only key> --from-literal=secretKey=<its secret>
```

The store's TLS certificate must be one the install trusts (`global.ca`).

For projects to **take** adopted tables **over** once the other platform stops writing them (Taking a table over), give the store a second key, one that may also write its buckets:

```
    takeover:
      secret: lakehouse-store-write         # accessKey, secretKey: read and write on s3.buckets
```

The platform's own catalog then uses it for the store's buckets (Trino's S3 mapping, Nessie's per-bucket settings); the store's `ext_<name>` catalog keeps the read-only key, and the tables still adopted stay read-only by the projects' Ranger policies.

And for a takeover to **stop the other platform's writers**, and for tables to be **given back**, let the platform write the store's own catalog (with `takeover`):

```
    catalog:
      type: hive_metastore
      uri: thrift://hive-metastore.lakehouse.svc.cluster.local:9083
      write: true
```

Trino then has a second catalog for the store, `extw_<name>`, which only the table maintenance account may use (Ranger), for Iceberg's `unregister_table` (a takeover takes the table out of the store's catalog) and `register_table` (a give-back puts it back): they move no file. The store's catalog must accept those writes from Trino (a Hive Metastore without authentication, or one that lets it).

How it behaves

- Trino gets a catalog for each store, `ext_<name>`, read-only (`iceberg.security=read_only`), which only the platform operator may read (Ranger): it finds each adopted table's current metadata file there.
- The platform's `iceberg` catalog reads the store's buckets at the store's endpoint with its key (Trino's S3 security mapping, written at Trino's start from the Secret, never into a ConfigMap); every other bucket is read as before, in Buckets.
- Nessie reads the adopted tables' metadata there too (its per-bucket S3 settings, with the same key).
- Nessie's GC and the nightly maintenance never touch the store: GC reads and deletes in Buckets alone, and maintenance skips adopted tables.

The platform's files on a company's store

What it's for

Keeping the platform's files (its tables, projects' files, notebooks' homes, models) on an S3 store the company already runs (a MinIO), in place of the platform's own Buckets: one store for both while another platform's data moves to this one.

Who can do it

Whoever installs the chart, with an administrator's key for the store from the store's administrators.

Set up

An administrator's key for the store in a Secret in the release's namespace (`accessKey`, `secretKey`), then the chart's `storage.external`, with the platform's own Buckets off:

```
kubectl -n <namespace> create secret generic company-store \
  --from-literal=accessKey=<an administrator's key> --from-literal=secretKey=<its secret>
```

```
storage:
  external:
    endpoint: https://minio.lakehouse.svc.cluster.local   # its certificate one global.ca trusts
    secret: company-store
buckets-operator:
  enabled: false
```

On the store, the platform's Keycloak as an OpenID provider, so people's tokens get them keys for what their groups may (notebooks, the `s3.` address, the store's own console): its settings (here as MinIO's environment, or with `mc admin config set <alias> identity_openid ...`), then the store started afresh, once:

```
MINIO_IDENTITY_OPENID_CONFIG_URL=<the platform's
  URL>/sso/realms/lakehouse/.well-known/openid-configuration
MINIO_IDENTITY_OPENID_CLIENT_ID=lakehouse
MINIO_IDENTITY_OPENID_CLAIM_NAME=groups
MINIO_IDENTITY_OPENID_SCOPES=openid,profile,email
MINIO_IDENTITY_OPENID_DISPLAY_NAME=DataWorks
```

The scopes matter for the console's sign-in: without them MinIO asks for every scope Keycloak lists, `service_account` among them, which the lakehouse client doesn't have, and Keycloak refuses with `invalid_scope`. The display name is the console's button ("Login with SSO (DataWorks)"); without it the button reads "(_)", MinIO's name for its default provider.

A store signing people in through LDAP keeps doing so: LDAP and OpenID work side by side (MinIO RELEASE.2024-10-13 and later). The store reads its OpenID settings as it starts, so the platform's Keycloak must answer by then.

For people to sign in to the store's own console through the platform (its "Log in with SSO"), give the chart its address too, `storage.external.console` (`https://minio-console.example.com`, as the store's `MINIO_BROWSER_REDIRECT_URL` has it): the platform lets Keycloak's `lakehouse` client send people back to `<console>/oauth_callback`, at every install and upgrade. They see the buckets their groups' policies allow. The store must trust the platform's certificate (MinIO Operator: the tenant's `externalCaCertSecret`), since it fetches Keycloak's settings and keys itself.

Setup checks the store takes the platform's Keycloak tokens (STS), and stops if it doesn't. A store that's given its OpenID settings later (they restart it, which a shared store may need to plan for) installs with `storage.external.sts: false`: setup says so and goes on, and the platform's own services work as usual, with their keys. Only people's own credentials there wait, in notebooks and pipelines. Set it back to `true` once the store has them.

How it behaves

- Every service reaches the store at `buckets:9000`, as it does Buckets: a forwarder there (`store-forwarder`, platformd's image) passes each request to the endpoint as it came, so its signature holds.
- **A MinIO from before 2025** wants a `Content-MD5` on a multi-object delete, which AWS's SDKs since early 2025 replaced with a CRC32 checksum: such a MinIO refused them ("Missing required header for this request: Content-Md5"), and Nessie's GC deleted nothing there. The forwarder adds the body's `Content-MD5` to a delete that lacks one (it isn't among the headers the request's signature covers). Only the platform's own traffic goes through it: another client of that MinIO with a new SDK still needs MinIO upgraded.
- What the platform makes there it marks as its own: a bucket's tag `dataworks=owned`, a policy's ID `dataworks`. It changes nothing that isn't marked. A project whose bucket is already the store's (another's) is refused, naming the bucket (give the project another, `files.bucket`); so is one of its groups whose policy name is taken. Removing a project leaves the store's other policies as they are.
- Its own accounts there (Trino's, Nessie's, the backups', MLflow's, the copies', Prometheus's, its own) are named `dataworks-<name>`, apart from the store's others.
- The platform never restarts or reconfigures the store. If people's tokens don't get them keys (its OpenID settings missing), the install's setup stops and says so.
- Buckets capacity (below) is the store's administrators' to watch: its drives aren't the platform's.

Backups and restore

Backups are for Kubernetes installs.

What it's for

Losing a database (the projects' own records, Nessie's catalog, Keycloak, Ranger, Airflow, the catalog) or a whole cluster shouldn't lose the platform. Each night every database is dumped, the Projects and Secrets exported, and all of it packed and encrypted for a certificate whose key is kept off the cluster.

Set up

```
storscale backup keygen --out ~/storscale-backup-keys
storscale upgrade --context <cluster> --set-file
  backup.certificate=$HOME/storscale-backup-keys/storscale-backup-cert.pem
storscale backup --context <cluster>                    # one now; then nightly
```

`keygen` writes `storscale-backup-cert.pem` (for the chart) and `storscale-backup-key.pem` (for a restore; keep it somewhere other than the cluster: without it no backup can be restored, with it anyone can read them). It never overwrites a key.

Chart value	Default	What
`backup.certificate`	empty: backups off	The certificate backups are encrypted for
`backup.schedule`	`0 4 * * *` (UTC)	When the CronJob `storscale-backup` runs
`backup.keep`	`14`	How many backups are kept
`backup.bucket`	`dataworks-backups` (`storscale-backups` on an install made before 0.39.0)	Where
`backup.s3.endpoint`, `region`, `secretName`	empty: the platform's own Buckets	Another S3 endpoint, its credentials in a Secret with `accessKey` and `secretKey`. A real install should use one: a lost cluster takes its own Buckets with it.
`backup.data.enabled`, `schedule`, `allowInternal`	`false`, `30 4 * * *`, `false`	Every bucket copied to `backup.s3` nightly (what changed since); needs `backup.s3.endpoint`

How it behaves

- Administration → **Health** → **Backups** says when the last one succeeded and the schedule, or *Not set up: nothing is backed up*. If none succeeds for 26 hours, `Backups are overdue` is an alert and one of the page's other problems.
- `storscale restore --context <cluster> --key storscale-backup-key.pem` (the latest; `--backup NAME` another) checks the key is the certificate's, says what it will do and asks (`--yes` doesn't), then stops every Deployment, restores every database and the Projects and Secrets, and starts everything again. `--data` copies the buckets back from `backup.data`'s copy too. A restore that was cut off starts again from the replicas it saved.
- **A cluster lost:** with `backup.s3` elsewhere and `backup.data.enabled`, install the platform afresh on a new cluster with the same `backup.*` settings and certificate, then `storscale restore --context <new> --key <key> --into-new`: the data is copied back, every database restored, each database role given its password of then, the backup's Secrets put back, Buckets restarted, and the setup steps run again. CI checks each release this way.

Upgrades

Set up

```
storscale upgrade --context <cluster> --version 0.42.1 --dry-run   # check; show the Project
  definition's changes
storscale upgrade --context <cluster> --version 0.42.1             # then do it
```

Flag	What
`--version V`	The release to go to (default `storscale`'s own)
`--context C`, `--namespace N`, `--release R`	Which cluster, namespace (`storscale`) and Helm release (`storscale`)
`-f FILE`, `--set K=V`, `--set-file K=PATH`	Settings to change, over those it was installed with
`--dry-run`	Show the Project definition's changes (`kubectl diff`) and check the chart renders against the cluster; change nothing
`--yes`	Don't ask; `--test` then runs the end-to-end checks
`--force`	Even to an older release
`--dir DIR` (with `--image-tag V` on a cluster)	A checkout's chart instead
`--chart oci://...`	The chart from a registry of the company's own (Harbor's, as below) instead of `ghcr.io`'s
`--ca-file PEM`	The CA of that registry, when its certificate isn't a public CA's (the chart's pull and the image check)
`--timeout`	How long to wait for the services (60 minutes)

How it behaves

1. It checks the release isn't mid-upgrade (`another install, upgrade or rollback is under way`), isn't going backwards (`... is older than the installed ...`, unless `--force`), that Helm is 3.14 or later, and that the release's images are published where the install takes them from (`images.registry`: a `--set`, else the installed value, else the chart's).
2. It applies the release's **Project definition** (the CRD), which Helm never upgrades. Without it Kubernetes silently drops whatever fields the old definition lacks from every project.
3. It upgrades the chart keeping the installed values (`--reset-then-reuse-values`, so the new release's defaults apply to new settings), with the images pinned to the release.
4. It waits for every service to roll out and be ready, then prints `Upgraded storscale from 0.42.0 to 0.42.1 (revision N).` If anything fails it prints the `helm rollback` that undoes it.

The CHANGELOG's *Upgrading from ...* paragraph says what each release needs. A `--dry-run` to the version already installed changes nothing.

Buckets capacity

What it's for

Buckets holds every table's files and every project's bucket. Its usable space is its drives' total less parity: four drives at EC:2 hold half their size. When it fills, it's grown by bigger drives or another pool.

How you're told

- The administrators: the monitoring rules `BucketsCapacityLow` (15% free), `BucketsCapacityCritical` (5%) and `BucketsCapacityFullIn7Days` (at the last six hours' rate), delivered by platformd as alerts and listed under the Health page's other problems.
- A project's people: as its own bucket nears its `limits.storage` (Limits).

Bigger drives

The storage class must allow volume expansion (`kubectl get storageclass <class> -o jsonpath='{.allowVolumeExpansion}'` says `true`). Every drive of a pool grows together: an erasure set holds only as much as its smallest drive.

```
storscale upgrade --context <cluster> --version <installed> --set buckets.storage=40Gi --dry-run
storscale upgrade --context <cluster> --version <installed> --set buckets.storage=40Gi
kubectl -n storscale get pvc -l buckets.io/cluster=buckets   # within five minutes: 40Gi each
```

The chart keeps the drives' claim template as it was made (Kubernetes never lets a StatefulSet's change); the platform operator asks for the new size on each drive's claim within five minutes and logs each drive's state (`grown from 10Gi to 40Gi`, `growing to 40Gi (has 10Gi)`, `its file system grows to 40Gi when its server next starts`). If a claim's `FileSystemResizePending` condition stays, delete one Buckets pod at a time and wait for it to be ready before the next.

Another pool

When drives can't grow, or more servers are wanted: a whole set of servers and drives beside the first.

```
storscale upgrade ... --set 'buckets.pools[0].servers=1' --set 'buckets.pools[0].volumesPerServer=4' \
  --set 'buckets.pools[0].storage=100Gi'
```

Adding one restarts every Buckets server together (S3 is down for a minute or two); new objects go to the pool with the most free space; nothing moves existing objects unless someone runs `mc admin rebalance start`. Keep the pools in your values file: a later `-f` without them is refused.

What's refused

The chart refuses, before anything changes:

- `pool-0's drives are 40Gi each: they can grow, never shrink (its storage: 20Gi)`
- `pool-0 is 1 server(s) of 4 drives: a pool's shape can't change (add a pool to buckets.pools instead)`
- `pool-0's drives are on storage class "standard": a pool's class can't change (add a pool on the new one instead)`
- `Buckets has 2 pools and the values give 1: a pool can't be removed here, its data would go with it`

The install's settings

The chart's values that matter most, from `deploy/helm/storscale-platform/values.yaml`, with what Compose uses instead: environment variables of the service that reads them (platformd's unless said), set in `stack/compose.yaml` or `stack/.env`; one not set there takes the default. Change the chart's with `storscale upgrade --set ...` (or `-f`).

Chart value	Default	Compose	What it does
`global.domain`, `port`, `scheme`	`storscale.localhost`, `8800`, `http`	`STORSCALE_DOMAIN`, `STORSCALE_PORT`	The platform's one address
`global.ca.certs` / `configMap`	empty		Private CAs every service trusts
`sso.entra.enabled`, `tenantId`, `clientId`, `secretName`	off	`ENTRA_AUTHORITY`, `ENTRA_CLIENT_ID`, `ENTRA_CLIENT_SECRET` (setup's environment)	Sign-in with Entra ID, brokered by Keycloak (setup makes the identity provider `entra` from the tenant's discovery document). The app registration's redirect URI (Web) is `<platform URL>/sso/realms/lakehouse/broker/entra/endpoint`; its ID token carries `email`, `given_name`, `family_name` and `preferred_username`; its client secret is in the Secret `secretName` (key `client_secret`)
`sso.entra.roles`	`Analyst: [analysts]`, `Engineer: [engineers]`, `Administrator: [engineers]`	`ENTRA_ROLES`	The app roles (the token's `roles` claim) and the platform's groups each puts people in, at every sign-in
`sso.entra.displayName`	*Microsoft Entra ID*	`ENTRA_DISPLAY_NAME`	The button on Keycloak's sign-in page
`sso.entra.onBehalfOf.enabled`, `maxAssertionSeconds`	off, 5400	`ENTRA_ON_BEHALF_OF`, `ENTRA_ON_BEHALF_OF_MAX_SECONDS`	Applications people sign in to through Entra (a CRM) acting for them: the `entra` provider's JWT authorization grant, accepting the app registration's client ID as the audience, assertions without `jti`, valid up to `maxAssertionSeconds` ahead (Entra's last 60–90 minutes). In Entra the app registration exposes a scope (`access_as_user`) with v2 access tokens (`requestedAccessTokenVersion: 2`), and the application is granted delegated permission to it. A project's service account with `onBehalfOf` then exchanges the person's token (Projects)
`brand.name`, `title`, `logo`, `icon`	none: *DataWorks*	`BRAND_NAME`, `BRAND_TITLE`, `BRAND_LOGO`, `BRAND_ICON` (platformd's environment)	The top bar and the browser tab. The title is `<name> DataWorks` unless given (`brand.name: Acme` shows *Acme DataWorks*); the catalog's domain of data products takes it too
`gateway.ingress.*`, `gateway.trustedProxies`	off		The ingress and its TLS
`trino.memory`, `workers`, `workerMemory`	`2560Mi`, `0`		Trino's size
`buckets.servers`, `volumesPerServer`, `storage`, `storageClassName`, `pools`	`1`, `4`, `10Gi`, default, `[]`		Buckets' drives (Buckets capacity)

Chart value	Default	Compose	What it does
storage.*	postgres: 8Gi, nessie: 2Gi, …		The databases' volumes
pipelines.defaultRef, memory, gitKnownHosts	main, 3Gi, empty		Projects' repos and their Airflows
deliveries.approverGroups, keepFor	the administrators, 7y	DELIVERY_APPROVER_GROUPS (engineers,analysts), DELIVERY_KEEP_FOR	Deliveries
products.certifierGroups	the administrators	PRODUCT_CERTIFIER_GROUPS	Who certifies data products
erasure.deadlineDays	45	ERASURE_DEADLINE_DAYS	Privacy
protection.applySuggested, protection.tags	true; personal MASK_HASH, financial MASK, health MASK_NULL, none excepted	PROTECTION_APPLY_SUGGESTED, PROTECTION_TAGS (excepts engineers)	What the catalog's tags mean (Access and protection)
llm.gateway	stub	the llm service	stub (the stand-in), litellm, or empty (none) (AI)
llm.guardrails, llm.forbiddenTerms	true, []	LLM_GUARDRAILS, LLM_FORBIDDEN_TERMS (Project Falcon)	The guardrail
llm.litellm.models	default: anthropic/claude-opus-5		LiteLLM's model list; the provider's key in the Secret storscale-llm (key api_key)
usage.rates, usage.viewerGroups	{currency: USD, cpuHour: 0.05, storageGBMonth: 0.023}, []	USAGE_RATES, USAGE_VIEWER_GROUPS	Usage and cost
templates.gitlab.url, group, tokenSecret	empty	GITLAB_URL, GITLAB_GROUP, GITLAB_TOKEN	The company's GitLab for project templates and shared libraries (Projects)
install.target	production	INSTALL_TARGET (development)	Which of a project's storscale/targets/<target>.yaml this install is
monitoring.sampleTraffic	false	STORSCALE_SAMPLE_TRAFFIC (true)	Sample S3 traffic for the storage dashboards, on a fresh install with nothing else to show
serving.memory, copy.memory, copy.commitEvery	1Gi, 1Gi, 30		Model servers; change-capture copies
replicas.*	1 each		Pods of each stateless service
networkPolicies.enabled	true		Nothing reaches a pod unless it should
audit.storage, audit.keep	5Gi, 365d	AUDIT_KEEP	The audit log
backup.*	off		Backups and restore
maintenance.at, history.keep, history.schedule	02:00, P7D, 30 3 * * *	MAINTENANCE_AT, NESSIE_GC_KEEP, NESSIE_GC_EVERY	Table maintenance
limits.project.*, limits.person.*, limits.total.*	4/40/50%/none; 4/100; 20/1000	LIMITS_* (the operator's; same defaults)	Limits

Chart value	Default	Compose	What it does
`notebooks.memoryLimit`, `cpuLimit`, `storage`	`1G`, `2`, `1Gi`		Each person's notebook
`superset.mapboxApiKey`	empty	`MAPBOX_API_KEY`	A base map for map charts
`examples.users`	`true`		The example people (alice, bob, carol) in the realm Keycloak imports on the first install, and the sales project and its data (setup). `false`: neither, for real users only; setup then checks Trino as the maintenance account and the store's sign-in with the pipelines' service account
`storage.buckets.platform`, `backup.bucket`	`dataworks-platform`, `dataworks-backups`	The platform's own buckets (its projects and their status, the semantic models, dbt's artifacts; the backups). An install made before 0.39.0 keeps `storscale-platform` and `storscale-backups`: the install's config (`storscale-config`) remembers the names it has, and they never change under it. Name them only before the first install	
`catalog.admins`	empty (bob)		The catalog's administrators besides the platform operator's service account: usernames as people sign in (`jane.doe`)
`storage.external.sts`	`true`		The external store takes the platform's Keycloak tokens (STS), which setup checks. `false`: not yet; setup says so and goes on, and people get no credentials of their own there until it does

Who administers is `PLATFORM_ADMIN_GROUPS` (default `engineers`), and who may sign in at all `PLATFORM_GROUPS` (default `analysts,engineers`): platformd's environment; the chart has no value for them.

The Python package and the API

What it's for

Everything the web app does is an API call, so a script, a scheduled job or the company's privacy tool can do it too, as a person or a service account, under the same rules.

The API

- The account menu's **API reference** lists every route with what it does; `GET /api/openapi.json` is the same as OpenAPI, for any client generator.
- Sign in with a Keycloak access token of the realm `lakehouse`: `Authorization: Bearer <token>`.
- Every change (`POST`, `PUT`, `DELETE`) needs the header `X-Platform-Request: 1`; without it the answer is `403 missing X-Platform-Request`.

- Every answer carries `X-Request-ID`; every refusal or failure is JSON, `{"error": "...", "request": "<id>"}` (Troubleshooting).

The Python package

```
pip install "git+https://github.com/StorScale/dataworks#subdirectory=stack/python"
```

```
import storscale

p = storscale.Platform("https://storscale.example.com")   # or STORSCALE_URL; with no credentials, a
  device code to approve
p.projects()
p.sql("retail", "SELECT count(*) FROM uploads_orders")
run = p.start_run("retail", "delivery_retail_campaign_results")
p.wait_for_run("retail", "delivery_retail_campaign_results", run)
p.usage("retail")
p.call("get_projects_name_deliveries", name="retail")      # any route by its operation ID
```

It signs in, in this order, with `token=` (or `STORSCALE_TOKEN`), `client_id=` and `client_secret=` (or `STORSCALE_CLIENT_ID`, `STORSCALE_CLIENT_SECRET`: a service account), `username=` and `password=` (or `STORSCALE_USERNAME`, `STORSCALE_PASSWORD`, through the public client `storscale-agent`), or a device code: it shows a link and a code to approve in your own browser, where you sign in as you always do (Entra ID too, which has no password to give a script), for a notebook, a pod or a terminal without a browser. A refusal raises `storscale.Refused` (401, 403) or `storscale.Error`, with the platform's reason and the request's ID. A private CA: set `REQUESTS_CA_BUNDLE`.

18. Troubleshooting

This chapter collects the errors people meet, in the platform's own words, grouped by feature: what each means and what to do about it. It starts with the two things that make any error easier to chase, the request ID and the run log, then goes feature by feature. It's for everyone: the person who saw the error, and the administrator they hand it to.

In this chapter:

- Request IDs
- The run log
- The project's status: Applied to
- Signing in and access
- Projects and their spec
- Connections
- Uploads and imports
- Adopted tables
- Pipelines
- SQL, metrics and limits
- Access and protection
- The table catalog (Nessie)
- History, tags and maintenance
- Releases
- Flows
- Data products
- Deliveries
- Privacy: search and erasure
- AI: the gateway, agents, knowledge banks, apps
- Spark in notebooks
- Models
- Roles and templates
- Audit, usage, governance and health
- Installs and upgrades

Request IDs

Every request to the platform gets an ID. It comes back in the answer's `X-Request-ID` header, and every refusal or failure is answered the same way, as JSON with the ID in it:

```
{"error": "someone other than who changed what it sends approves it", "request": "9f3c1a0b7d2e4c55"}
```

platformd logs the same ID with the method, the path, the status and the error: a refusal (4xx) as `request refused`, a failure of the platform's (5xx) as `request failed`. So:

1. **On the web app**, the error shows in a red banner. To get its ID, open the browser's developer tools → Network, find the request that failed, and read `X-Request-ID` in its response headers (or `request` in its body).
2. **From a script**, the Python package puts it in the exception: `storscale.Error: the audit log isn't answering (HTTP 502, request 9f3c1a0b7d2e4c55)`, and on `err.request_id`. With curl, `-i` shows the header.
3. **Give your own:** send `X-Request-ID: <id>` (up to 64 printable characters, no spaces) and the platform uses it, so a privacy tool or a job can tie its own log to the platform's.
4. **An administrator finds it** in platformd's log: `storscale logs platformd | grep 9f3c1a0b7d2e4c55` on Compose, `kubectl -n storscale logs deploy/platformd | grep 9f3c1a0b7d2e4c55` on Kubernetes. The line has the ID, the method, the path, the status and the error; a refusal the platform records (a delivery approved by who changed it, a search by someone who may not) is in the audit log too, with the person.

A `401` answer is `{"login": "/auth/login"}`, not an error: the session ended, and the web app signs in again.

The run log

Pipelines (imports, deliveries, flows, dbt, scheduled notebooks, agents, knowledge banks, scorings) run in Airflow. When one fails:

1. Open the project → **Pipelines**, find the pipeline, and press **why it failed** beside the failed run (**log** beside any other).
2. The log is each task's last try, `=== <task> (try N)` then what it logged: dbt's, dlt's, the notebook's or the delivery's own words.
3. Through the API: `GET /api/projects/{name}/pipelines/{pipeline}/runs/{run}/log`, or `p.run_log("retail", "import_retail_uploads", run)` in Python.

A failed latest run is also one of the project's alerts (`<pipeline>'s latest run failed`), sent where its `notify` says.

A banner above the pipelines, *A file in the repo didn't load* (or *N files …*), means Airflow couldn't load a DAG file from the project's repo; it names the file and the error.

The project's status: Applied to

Within a minute of a save, the platform operator applies the project to every system. Its **Overview** shows the phase (**Ready**, **Error**, **Deleting**, or *Waiting for the operator*), and the **Applied to** card lists each system with a green or red dot and, when red, the operator's message. A red part doesn't undo the others; the operator tries again every few seconds and resyncs every minute.

Part	Message	Means	Do
`keycloak`	`no Keycloak group "<g>"`, `no Keycloak user "<u>"`	A member names a group or person the realm doesn't have	Fix the member, or make the group in Keycloak
`keycloak`	`Keycloak client "<id>" has no service account`	`pipelines.serviceAccount` names a client without one	Name the pipelines' client (`airflow-pipelines` on the example install)
`roles`	`no role <name>: an administrator defines it (Administration → Roles); its members have no access meanwhile`	A member's role was removed or never defined	Define it on Administration → Roles, or change the member's role
`ranger`	`waiting for Keycloak`	The keycloak part failed first	Fix that one

Part	Message	Means	Do
buckets	`bucket <b>'s size limit: ...`	Buckets refused the quota	Check `limits.storage`, and Buckets' health
trino	`source <s>: no connection "<c>"`, `source <s>: its connection needs a username and a password`	A source's connection is missing or incomplete	Add or fix it on **Connections**
trino	`source <s>: its url isn't a JDBC URL (jdbc:...)`, `... its url holds a password (or a line break): the password goes in password, which Trino never logs`	The connection's `url`	Put the password in its own field
trino	`source catalog <c>: ... (Trino may not see its connection yet: the next pass tries again)`	Trino hasn't taken the new catalog yet	Wait a minute
copy	`copying a source needs Kubernetes (Compose can't start its pods)`	Change-capture copies run only on Kubernetes	Query the source in place instead
copy	`copying <s>: copying needs its connection's host, username and password (a url alone won't do)`		Give the connection host, username and password
imports	`imports run in the platform's Airflow, and there's none here`	No Airflow on this install	
imports	`import <i>: no connection "<c>" among the project's`, `... its connection has no token`, `... its connection needs a host, username and password`, `... its port "<p>" isn't a number`	The import's connection	Fix it on **Connections**
airflow	`no secret "<s>" (the repo's credentials) among the project's connections`, `secret "<s>" has no password (a token) for an https repo`, `secret "<s>" has no sshPrivateKey for an ssh repo`	The repo's credentials	Add the connection `pipelines.credentials.secretRef` names
airflow	`its Airflow is starting`, `its Airflow is starting again, with its connections`	A project's own Airflow (Kubernetes) is coming up	Wait
deliveries	`delivery <d>: no connection "<c>" among the project's`, `delivery <d>: the connection "<c>" has no pgpPublicKey`	A delivery's target or key; while it's red, none of the project's deliveries runs	See Deliveries
agents	`agent <a>: its secret isn't kept yet`	A scheduled agent's secret (a connection the platform makes when the agent is approved) isn't there	Have the agent approved, by an editor other than who registered it
reviews	`merge request <n>: ...`, `the repo's credentials (<s>): ...`	Merge-request reviews couldn't reach GitLab or build the data branch	Check the repo's token (it needs GitLab's API)
serving	`serving <s>'s secret: ...`	A model server's client	Check the project's serving and its model version

Signing in and access

Message	Means	Do
`no access`	You're signed in but in none of the platform's groups (`PLATFORM_GROUPS`, `analysts,engineers` by default): carol, on the example install	An administrator adds you to a group in Keycloak
`no such project`	You aren't a member of it (and aren't an administrator), or it doesn't exist	Ask its administrator to add you; check the name
`not a member of the project`	The same, for routes a member's token calls (notebooks, MLflow, models)	
`<who> isn't a member of <project>`	An agent's or a script's token isn't a member	Add the person or service account
`missing X-Platform-Request`	A change sent without the header a cross-site form can't send	Scripts send `X-Platform-Request: 1` on every `POST`, `PUT` and `DELETE`; the Python package does
`projects aren't set up (no STORE_URL)`	platformd has no project store configured	An install problem: check platformd's environment

Projects and their spec

Saving a spec (the **Edit** page's **Save**, or `PUT /api/projects/{name}`) checks it all at once and lists every problem, separated by `;` . The most common:

Message	Do
`only administrators change projects`	Project specs are the administrators'; ask one, or change it through a release (History and releases)
`not a project: json: unknown field "<f>"`	A misspelt or misplaced field: the spec accepts only its own
`the project's name must be <name>`	The JSON's `metadata.name` differs from the project being saved
`name "<n>": 3 to 40 lowercase letters, digits and dashes, starting with a letter`	
`a project needs at least one member`	
`member 2: give a group or a user, not both`	
`member 1: role "<r>": reader, editor, or a role the platform's administrators define` / `role <r>: reader, editor, or a role defined on Administration → Roles (none is called that)`	Define the role first, or use reader or editor
`tables: people: <t> twice`, `tables: people: "<t>"."<c>": a table and its column identifying a person`	
`tables: retention: <t>: keep "<k>": as 730d or 2y`	
`tables: freshness: within "<w>": as 30m, 24h or 7d`	
`limits: queries <n>: from 1 to 100 at once`, `limits: queued <n>: up to 1000`, `limits: queryMemory "<m>": as 4GB, 512MB or 50%`, `limits: storage "<s>": as 500Gi or 2Ti`, `limits: storage needs files (a bucket)`	
`cost: budget <b>: a month, more than nothing and up to ten million`	
`llm: budget <b>: USD a month, up to a million`	

Message	Do
`notify: "<e>" isn't an email address`	
`the project store isn't answering`	platformd couldn't reach where projects are kept (Buckets, or Kubernetes): an administrator checks Health
A Kubernetes schema message	On Kubernetes, the cluster checked the spec against the Project definition; an old definition means the install wasn't upgraded with `storscale upgrade`

Connections

Message	Means	Do
`a connection's name starts with the project's and a dash (retail-dags), in lower-case letters, digits and dashes`	A new connection's name	Name it `retail-...`
`the project has a connection <c> already: give it new values`		Use **New values** (or `PUT`)
`kind: one of git, ssh, database, api, sftp, s3, pgp, teams`	An unknown kind	Pick one the page offers
`<Field> is needed`, `one of password, sshPrivateKey is needed`, `<key>: no such key for <Kind>`, `<key>: too long`	The values for the kind	Fill the required fields
`the project's editors change its connections (it takes the permission connections)`		Ask an editor, or a role with `connections`
`no such connection`		
Remove is greyed, *Used by ...*	The spec names it	Take it out of the spec first

Uploads and imports

Message	Means	Do
`the project's editors upload files to it`, `the project's editors remove its uploads`		Ask an editor, or a role with `write-files`
`a file, as multipart form data (file), up to 256 MB: ...`	The file is too big or the request malformed	Split the file, or put it in the bucket another way
`a file's name is letters, digits, dots, dashes and underscores: <name>`		Rename it
`Buckets: ...`	Buckets refused or failed	Check the project's bucket isn't full, and Health
`no connection <c> among the project's (its Connections tab)`	An import names a connection the project lacks	Add it first
`imports: <i>: a file of landing/uploads/ (no folders), ending in one of csv, xlsx, jsonl, parquet: "<f>"`		
`the project's editors reset its imports`		
`no Airflow here`		An install problem

A files import that fails with `Forbidden` in its run log reading the landing area: the project's pipelines can't read it. Since 0.31.0 an import gives the project pipelines; on an older project, add `pipelines` to its spec. A table whose dates were imported as text before 0.31.0 needs its import's **Reset** (the Imports tab) to take dates.

Adopted tables

Adopting another platform's tables, taking them over and giving them back (Getting data in).

Message	Means	Do
403 `adopting and taking over tables takes the platform's table-adoption role (adopt-tables): the group data-migration`	You edit the project, but aren't in the platform role `table-adoption`; the refusal is in the audit log. Editors without it don't see **Adopt tables**, **Take over…** or **Give back…**	An administrator puts you in the group the message names (`adoption.groups`; Roles); then sign out and in again
`no external store <s> (the chart's externalStores)`		An administrator sets the store up (Operating)
`the project already adopts <store>.<schema>.<table>, or a table by the name <t>`		Give it another name
`<t> was taken over: it's the project's own now, dropped as its other tables are`	**Stop adopting** a table taken over	Drop it in SQL, or give it back
`the store <s> takes no takeovers: its administrator gives it a key that may write its buckets first (the chart's externalStores[].takeover)`		An administrator sets `takeover.secret`
`<t> is still written there: its last change was ... Stop its writer first.`	The other platform wrote it within `TAKEOVER_QUIET` (a day)	Stop its jobs, and take it over a day later
`<t> is still written here: ... Stop its pipelines first.`, `the store has a table <t> again ...: rename one of them first`	Giving back	Stop the project's writers; rename one of the two tables
`the platform may not write the store <s>'s catalog (the chart's externalStores[].catalog.write), so nothing can be given back there`		An administrator sets `catalog.write`
`Access Denied` writing a table taken over, on 0.41.0 or before	Its project's Ranger policies kept it read-only after the takeover	Upgrade: the operator gives it the project's writers' policies

Pipelines

Message	Means	Do
`only the project's editors run its pipelines`		An editor, or a role with `run-pipelines`
`pipelines aren't set up (no AIRFLOW_URL)`		An install problem
A run that fails		Read the run log

SQL, metrics and limits

The SQL tab, metrics and assistants go through the metrics and SQL service (semanticd), which runs queries in Trino as you.

Message	Means	Do
`you're not a member of <project>`		

Message	Means	Do
`write a statement, one statement at a time`		One statement, no trailing second one
`Access Denied: ...` (Trino's)	Ranger refused: you may not read that table or column (a reader's tables list, a table outside your projects)	Check the project's **Access** tab; the refusal is in the Audit tab with Ranger's reason
A query refused with `QUERY_QUEUE_FULL`, naming a group such as `project-retail`	The project's (or your own) limit of waiting queries is reached	Wait, cancel what's running, or raise `limits.queued` (Operating)
`the semantic layer isn't answering`	semanticd is down or slow	Health page: *Metrics and SQL service*
`project <p> has no semantic model`		Write one on the Metrics tab
`only those who may change <p>'s metrics (its editors) change its semantic model`		A role with `change-metrics`
`ask for at least one metric, MetricFlow can't answer that: ...`		
`the company's model has these already, so build on them rather than defining them again: ...`	A shared model defines those names	Use the shared model's (Metrics and terms)
`it would break the models of <project> (<why>)`	Another project builds on this model	Keep the names it uses

Access and protection

Message	Means	Do
`the project's editors confirm or remove a column's tags`		A role with `classify`
`the catalog has no column <t>.<c>`	The catalog hasn't seen the column yet	Wait for catalog-sync, or check the name
`the catalog isn't set up, or the project has no tables`		
`the action is confirm or remove`		

A column you expect to read whole shows hashed or masked: it's tagged `personal`, `financial` or `health` and you're not excepted (Access and protection). The project's alerts say *<t>.<c> looks personal: the catalog tagged it, and it's masked* until someone confirms or removes the tag.

The table catalog (Nessie)

The table catalog answers the platform's own services alone (Trino, platformd, the operator, the copies, the imports and Nessie's GC), signed in as its catalog account. People, their notebooks, Spark's executors and the pipelines' code reach the tables through Trino, under Ranger's rules (Access and protection).

What you see	Means	Do
`401` from `http://nessie:19120/...`	A call with no token	Query through Trino instead: the **SQL** tab, `buckets_lake.sql()`, a JDBC client
`403`, `... is not allowed for role '<you>' ...` (Iceberg's REST API answers the same refusal as a `500`)	A call with a person's own token, or any account but the catalog's	The same: Trino, or Spark through `buckets_lake.spark()` for the tables you read whole

What you see	Means	Do
A call to Nessie from a notebook or a pipeline's pod hangs and times out (Kubernetes)	The chart's network policy lets only the platform's services reach it	The same
A script or tool of your own that called Nessie stopped working after 0.41.0		Give it the catalog account's token (client credentials, client `lakehouse-catalog`, secret `CATALOG_CLIENT_SECRET` in `storscale-env`), or have it go through Trino
A health check of Nessie fails after 0.41.0	Its health is on its management port now	Check `http://nessie:9000/q/health/ready`

History, tags and maintenance

Message	Means	Do
`the project's editors make and remove its tags`		A role with `release`
`the project has a tag <name> already`		Choose another name
`keep: as 90d or 7y`		
`want ?table=<one of the project's tables>`		
`the project's editors ask for its maintenance`		A role with `run-pipelines`
`the project's tables have no maintenance (none, or paused)`	`tables.maintenance.paused` is on, or there are no tables	
Storage tab: *Table maintenance isn't set up on this install*	platformd has no maintenance account	An install problem: `MAINTENANCE_TRINO_PASSWORD`
Alert `<table>: its maintenance failed`	Retention or compaction failed	The table's **Storage** tab says which step and why

Releases

Message	Means	Do
`the project has no repo (pipelines.repo)`		Connect a repo first
`a ref of the repo: a tag (v1.3), a branch, or a commit`		
`someone other than who planned it applies it`	Four eyes	Another editor applies
`the project has changed since it was planned: plan it again`		Plan again
`the plan is <state>`	It was applied or failed already	
`the spec is saved, the semantic model isn't: ...`	The release's semantic model couldn't be saved	The message says why; fix `semantic.yaml` in the repo and plan again

Flows

Message	Means	Do
`the project's editors save flows in it`		A role with `build`
`the flow's name is lowercase letters, digits and underscores, and the one in the address`		
`a flow is saved with an output step: the table it makes`		Add an output step
`a schedule as cron's five fields (UTC): 0 6 * * 1, or none`		
`step <s>: ...`	A step doesn't compile	The step's message says why
`GitLab refused the commit: ...,the merge request: ...`	Saving as a merge request failed	Check the repo's token
`the project's editors and its pipelines run a step's model`		

Data products

Message	Means	Do
`the editors of <consumer> ask for it`	Only the consuming project's editors ask	
`<consumer> reads it already`		
`the editors of <project> answer requests for its products`		
`the governance group (a platform role with certify-products) certifies products`		
`<consumers> read it: take them off its consumers first`	A product with consumers can't be taken back	
`no request from <consumer>,approve or decline`		

Deliveries

Where	Message	Means	Do
Approve	`someone other than who changed what it sends approves it`	You saved the change (or planned the release that brought it)	Another editor approves
Approve	`the project's editors (and data-protection approvers) approve deliveries`	You have neither `approve` nor `protect-personal`	
Approve	`no such delivery`		
Card	`Needs: an editor of the project other than <who>`	Waiting for an editor	
Card	`Needs: a data-protection approver (<groups>)`	It sends personal columns	Someone in those groups (or an administrator) approves; a data-protection approver who is only a reader approves through `POST .../approve`
Card	*not approved* again after a save	Its table, query, split, format, target or PGP key changed	Approve again

Where	Message	Means	Do
Spec	`deliveries: <d>: it sends the project's tables, and it has none (tables)`		
Spec	`deliveries: <d>: a table or a query, one of them`		
Spec	`deliveries: <d>: its query is one SELECT`	Not a `SELECT`/`WITH`, or has a `;`	
Spec	`deliveries: <d>: format "<f>": one of csv, parquet, jsonl`		
Spec	`deliveries: <d>: files "<f>": a name with {date} (and {part}, split), no folders`		
Spec	`deliveries: <d>: splitBy (a column) and {part} in the files' name go together`		
Spec	`deliveries: <d>: schedule "<s>": five cron fields, as 0 6 * * 1`		
Spec	`deliveries: <d>: its target and pgp are connections' names`		
Spec	`deliveries: <d>: checks: rows from minRows to maxRows`, `checks: within "<w>": as 30m, 24h, 7d`, `checks: parts need splitBy`		
Applied to	`delivery <d>: no connection "<c>" among the project's`	The target is missing; **no** delivery of the project runs until fixed	Add it on Connections
Applied to	`delivery <d>: the connection "<c>" has no pgpPublicKey`		Give the key connection the recipient's public key
Run log	`checks failed, nothing sent: <problems>`	A check failed (`N rows, fewer than M`, `... not changed within 24h (last ...)`, `parts missing: ...`)	Fix the data, or the check
Run log	`the SFTP target has no hostKey: ...`		Give the connection the server's host key
Run log	`the SFTP server's key isn't the one given (<type>): nothing sent`	The server's key changed, or the wrong one was given	Confirm the new key with the partner, then update `hostKey`
Run log	`the recipient's key: ...`	GnuPG couldn't import the PGP key	Paste the whole armored public key
Run log	`splitBy <c>: no such column`		The query must return the split column
Run log	`sending failed (...); again in 10s`	A transfer failed; it's tried three times	If all three fail, the run fails: check the target's address and credentials
Pipelines	No `delivery_...` pipeline	It isn't approved, or the project's `deliveries` part is red	Approve it; fix the part; allow a few minutes for Airflow to load it

Privacy: search and erasure

Message	Means	Do
`the data-protection approvers (a platform role with protect-personal) search the delivery log` / `... ask for erasures` / `... see erasures`		Be given a platform role with `protect-personal`, or be an administrator
`a person's ID (person=)`, `a person's ID (person)`	Empty, or over 200 characters	
`requestedOn as a date, 2026-10-19`		
`the project declares no tables of people's data (tables.people), so there's nothing to erase from`		Declare them (Privacy)
`no such project with tables`		
`erasing needs the platform's files and its Trino account (maintenance), searching needs ...`		An install problem: the maintenance account
`the erasure ran but its record couldn't be kept: ...`	The tables were erased; the record wasn't written	Note the answer's details, check Buckets, and erase again (it finds nothing more, and keeps a record)
HTTP 207 and **Not done:**	Something wasn't erased; the record names it	See below
Not done: `<table>: ...`	Trino couldn't count or delete in that table	Check the table exists and its column; erase again
Not done: `landing/uploads/<f>: a xlsx file isn't rewritten here: take the person's rows out of it by hand, or remove it` (or `a parquet file ...`)		Edit the file, upload it again, run the import
Not done: `landing/uploads/<f>: writing it again: Access Denied...`	You may not write the project's files	An editor with `protect-personal` (or an administrator) erases again, or rewrites the file
Not done: `landing/uploads/<f>: no column <c> in it`	The CSV hasn't the person's column	
Not done: `landing files: no token: sign in again`		Sign in again and erase again
Search: **unclear**, `what it sent has no column identifying a person (tables.people)`	The delivery sent masked columns but no people's table	
Search: **unclear**, `it couldn't be run again: ...`	The kept read is gone or the query no longer runs	
Search: *N not checkable*	Runs of deliveries without personal columns, or from before reads were kept	

AI: the gateway, agents, knowledge banks, apps

Message	Means	Do
`the project has no gateway key: give it an LLM budget (llm.budget in its spec) and the operator makes one`		Add `"llm": {"budget": 20}` (AI)
`the platform has no LLM gateway`	`llm.gateway` is empty	
`the project's editors see its model calls`		

Message	Means	Do
`an agent is approved by an editor other than the one who registered it`		
`the project has no agent <a> to ask: an approved agent with a prompt and a budget`		
`the agent <a>'s client or gateway key isn't kept yet`	Just approved	Wait a minute
`the project has no tool <t>`, `the project's editors, and the agents given it (tool:<t>), call its tools`		Give the agent the tool
`the tool's connection <c> isn't kept yet: add it on the Connections tab (an API token)`		
`the API didn't answer: ...`	The tool's API	
`the project has no knowledge bank <b>`		
`the project's editors and its pipelines build its knowledge banks`		
`embedding chunks N to M: ...`, `embedding the question: ...`	The gateway refused or failed to embed	The message says why
`a question app answers questions: ask it`, `only a question app is asked`		
`asking the model: ...`, `the model couldn't be asked: ...`	The gateway refused (the guardrail, a budget) or failed	The message says which

Spark in notebooks

Distributed Spark, on Kubernetes (Notebooks and models). A session gets only the tables you read whole, with no row filter or column mask for you, and writes only those you may change.

Where	Message	Do
`spark()`	`RuntimeError: this notebook has no Spark: start it with the Distributed Spark profile ...`	Restart your server with that profile (**File → Hub Control Panel**). Compose has no Spark
`spark()`	`PermissionError: there's no table you read whole, so Spark has nothing to read ...`	Use `sql()`; `buckets_lake.spark_skipped` says why each was left out
`spark()`	413 `your N tables don't fit in one session's credentials ...`	Name the tables: `spark(tables=[...])`
`spark()`	503 `this install gives Spark no credentials (no SPARK_S3_ACCESS_KEY or STORE_URL)`	An install problem: setup makes the Spark account
Spark	`ForbiddenException: sales.orders isn't one of this session's tables ...`	It has a row filter or mask for you (`spark_skipped`), or the session named other tables: read it with `sql()`
Spark	`ForbiddenException: a Spark session doesn't make, drop or rename tables ...`	Make it with `spark(create=...)`; drop or rename it in SQL
Spark	`[INTERNAL_ERROR] The Spark SQL phase planning failed with an internal error` on `MERGE INTO`, `UPDATE` or `DELETE FROM`	Spark's row-level SQL doesn't work here yet (0.42.0): change rows with `sql()`, through Trino, or rewrite the table from Spark with `writeTo(...).overwritePartitions()`
Spark	`NotAuthorizedException: ... the session's grant has expired`	Start a new session: `buckets_lake.spark()` (credentials last `credentialHours`, 12 by default)
`spark(create=...)`	`PermissionError: Trino didn't make <table> as you: ...`	Make it in a project where you may make tables

Models

Message	Means	Do
`MLflow isn't set up here (no MLFLOW_URL)`		
`the project's readers read its experiments and models; its editors and pipelines write them`		A role with `models`
`a model version is approved by an editor other than the one who registered it`		
`no such model version: ...`		
`the project serves no model as <s>`		
`the project's models answer N calls at a time: try again in a moment`		Retry
`a call is up to 32 MB`		
`the model's server isn't answering (starting?)`		Wait, then check the serving
`models aren't served here (no SERVING_URL)`		

Roles and templates

Message	Means	Do
`<r> is the install's (its settings' groups): define another`	finance, governance, data-protection and table-adoption are built in	
`<r> is a project's role: remove it first to make it another kind`		
`the projects <p> give <r>: change their members first`	A role in use can't be removed	
`the install has no GitLab for templates' repos (GITLAB_URL, GITLAB_GROUP, GITLAB_TOKEN): leave repo out`		Set `templates.gitlab` (Operating)
`there's a project <p> already`		
`no library <l> (Administration → Templates lists them)`		

Audit, usage, governance and health

Message	Means	Do
`a project's audit log is for its editors`		A role with `see-audit`
`the platform's audit log is its administrators'`		
`the audit log isn't answering`	Ranger's Solr is down	Health page
`the project's editors, and the install's usage viewers, see what it uses and costs`		A role with `see-usage`, or the finance platform role
`the install's usage viewers and its administrators see every project's usage`		
`month: as 2026-10`		
`the administrators, the products' certifiers and the data-protection approvers see the governance view`		
`administrators only, health checks are off`		

Message	Means	Do
`want {service, hours}: up to a week`		
Send a test alert says *No alert targets are set (the Secret storscale-alerts).*		Set the Secret (Operating)
An alert target answers with an error	The webhook or SMTP server refused	platformd logs `alert not sent` with the target and why

Installs and upgrades

Message	Means	Do
`the platform didn't start: see storscale status and storscale logs`	A Compose service didn't become healthy	`storscale status`, then `storscale logs <service>`
`Docker is rootless here, so notebooks need DOCKER_SOCK=/run/user/<uid>/docker.sock`		Set it
`no release "storscale" in the namespace "storscale" (context "..."): install it first`		
`the release storscale is pending-upgrade: another install, upgrade or rollback is under way (or was cut off; see helm history)`		Wait, or `helm rollback`
`<v> is older than the installed <w>: a downgrade isn't tested, ...`		`--force`, or `helm rollback`
`these images aren't published at <v>: ...`	The release's images aren't out yet	Wait for its images workflow
`upgrade needs Helm 3.14 or later`		
`the chart doesn't render: ...`	A value is wrong (the dry run caught it)	Fix the value
`upgraded, but ... (see kubectl get pods,jobs -n storscale; to undo: helm rollback ...)`	A service didn't become ready	Read the pods' state; roll back if needed
`pool-0's drives are ... they can grow, never shrink, ... a pool's shape can't change, ... a pool's class can't change, Buckets has N pools and the values give M`		See Buckets capacity
`backups aren't set up here: give the chart backup.certificate (storscale backup keygen makes one)`		
`this key isn't the one for the cluster's backup certificate: its backups can't be opened with it`		Find the matching key
`<file> exists: keygen never overwrites a key (backups made for it need it)`		Use another `--out`
`--data: the data isn't copied off this install (backup.data), so there's none to copy back`		
`the restore failed: ... (the platform was started again; its databases may be part restored: restore again)`		Restore again